Computer Architecture

Lecture 22: Virtual Memory

Konstantinos Kanellopoulos
Prof. Onur Mutlu

ETH Zürich

Fall 2025

5 December 2025

A Blueprint for Fundamentally Better Architectures

- Onur Mutlu,
  "Intelligent Architectures for Intelligent Computing Systems"
  *Invited Paper in Proceedings of the Design, Automation, and Test in Europe Conference (**DATE**)*, Virtual, February 2021.
  [Slides (pptx) (pdf)]
  [IEDM Tutorial Slides (pptx) (pdf)]
  [Short DATE Talk Video (11 minutes)]
  [Longer IEDM Tutorial Video (1 hr 51 minutes)]

Intelligent Architectures for Intelligent Computing Systems

Onur Mutlu
ETH Zurich
omutlu@gmail.com

Fundamentally Better Architectures

Data-centric

Data-driven

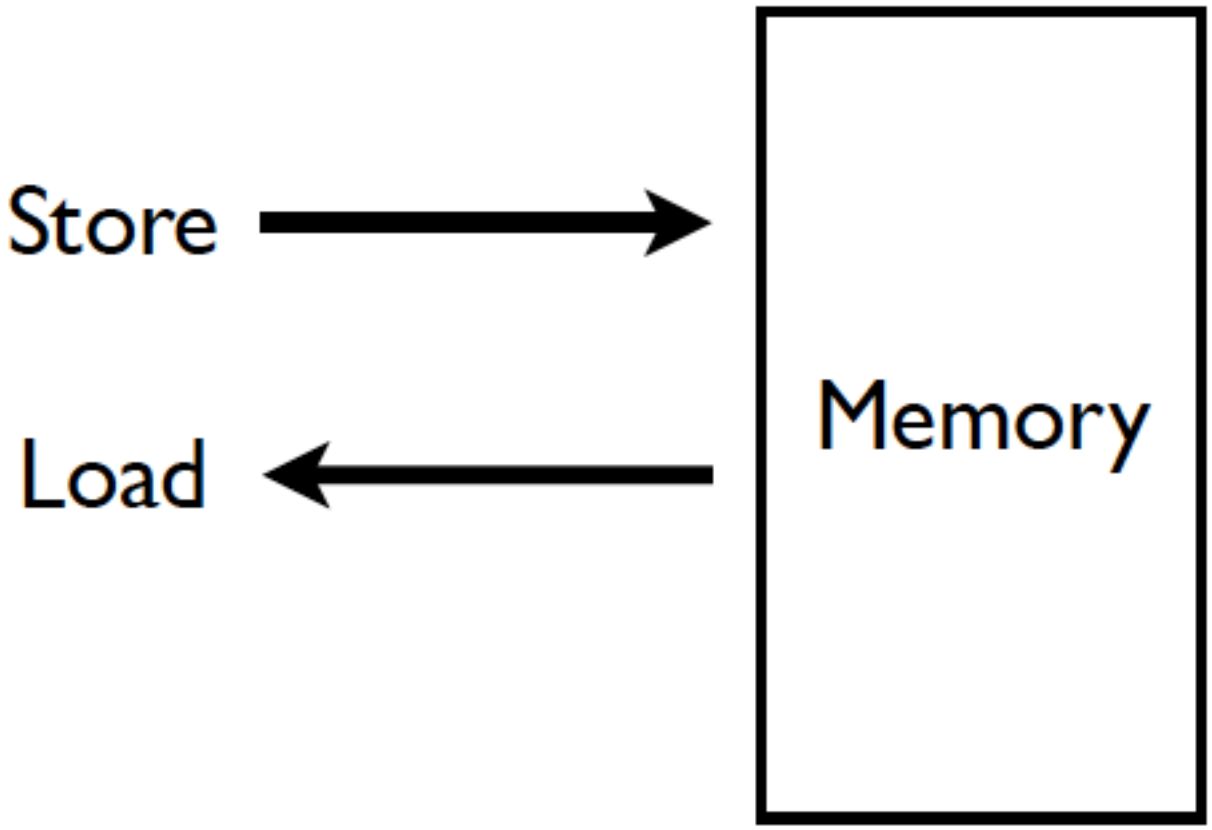
Data-aware

Readings

- Virtual Memory
- Recommended
  - H&H Chapter 8.4
  - Kim & Mutlu, "Memory Systems," Computing Handbook, 2014.
    - https://people.inf.ethz.ch/omutlu/pub/memory-systems-introduction_computing-handbook14.pdf
  - Jacob & Mudge, "Virtual Memory: Issues of Implementation," IEEE Computer, 1998.
  - Kanellopoulos et al. "Utopia: Fast and Efficient Address Translation via Hybrid Restrictive & Flexible Virtual-to-Physical Address Mappings" MICRO 2023
  - Kanellopoulos et al. "Virtuoso: Enabling Fast and Accurate Virtual Memory Research via an Imitation-based Operating System Simulation Methodology" ASPLOS 2025
  - Hajinazar et al. "The Virtual Block Interface: A Flexible Alternative to the Conventional Virtual Memory Framework" ISCA 2020

Virtual Memory

Memory (Programmer's View)



Ideal Memory

- Zero access time (latency)
- Infinite capacity
- Zero cost
- Infinite bandwidth (to support multiple accesses in parallel)

Abstraction: Virtual vs. Physical Memory

- Programmer sees virtual memory
  - Can assume the memory is "infinite"
- Reality: Physical memory size is much smaller than what the programmer assumes
- The system (system software + hardware, cooperatively) maps virtual memory addresses to physical memory
  - The system automatically manages the physical memory space transparently to the programmer

+ Programmer does not need to know the physical size of memory nor manage it → A small physical memory can appear as a huge one to the programmer → Life is easier for the programmer

-- More complex system software and architecture

A classic example of the programmer/(micro)architect tradeoff

Requires **indirection and mapping** between virtual and physical address spaces

REAL LIFE EXAMPLE

Warehouse



Warehouse

- Imagine a huge warehouse that stores thousands of items.
- Workers rely on **barcodes** and an **inventory database** rather than memorizing physical shelf locations.
- This warehouse resembles your **computer's virtual memory subsystem**.

Why do we need Virtual Memory?

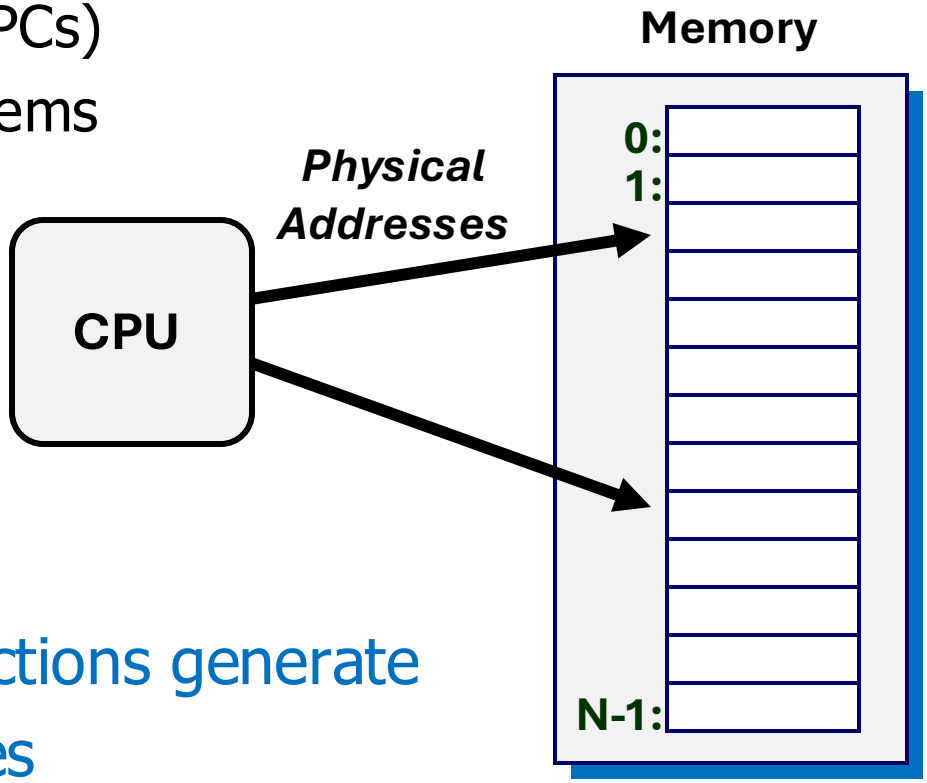
Benefits of Automatic Management of Memory

- Programmer does not deal with physical addresses
- Each process has its own
  - Virtual address space (very large)
  - Independent mapping of virtual→physical addresses

- Enables
  - Code and data to be located anywhere in physical memory (relocation and flexible location of data)
  - Isolation/separation of code and data of different processes in physical memory (protection and isolation)
  - Code and data sharing between multiple processes (sharing)

A System with Physical Memory Only

- Examples:
  - most early supercomputers
  - early personal computers (PCs)
  - many older embedded systems



CPU's **load or store** instructions generate **physical** memory addresses

The Problem

- Physical memory is of limited size (cost)
  - What if you need more?
  - Should the programmer be concerned about the size of code/data blocks fitting physical memory?
  - Should the programmer manage data movement from disk to physical memory?
- Multiple programs may need the physical memory
  - Should the programmer make sure all processes (different programs) can fit in physical memory?
  - Should the programmer ensure two processes do not unintentionally or incorrectly use the same physical memory portion?
- ISA can have an address space greater than the physical memory size
  - E.g., a 64-bit address space with byte addressability → 16 ExaBytes
  - What if you do not have enough physical memory?

Difficulties of **Direct Physical Addressing**

- Programmer needs to manage physical memory space
  - Inconvenient & difficult
  - More difficult when you have **multiple processes**
- Difficult to support code and data relocation
  - Addresses are directly specified in the program
- Difficult to support multiple processes (esp. concurrently)
  - Protection and isolation between multiple processes
  - Sharing of physical memory space without problems
- Difficult to support data/code sharing across processes
  - Different processes need to reference the same physical address

How does Virtual Memory work?

Virtual Memory

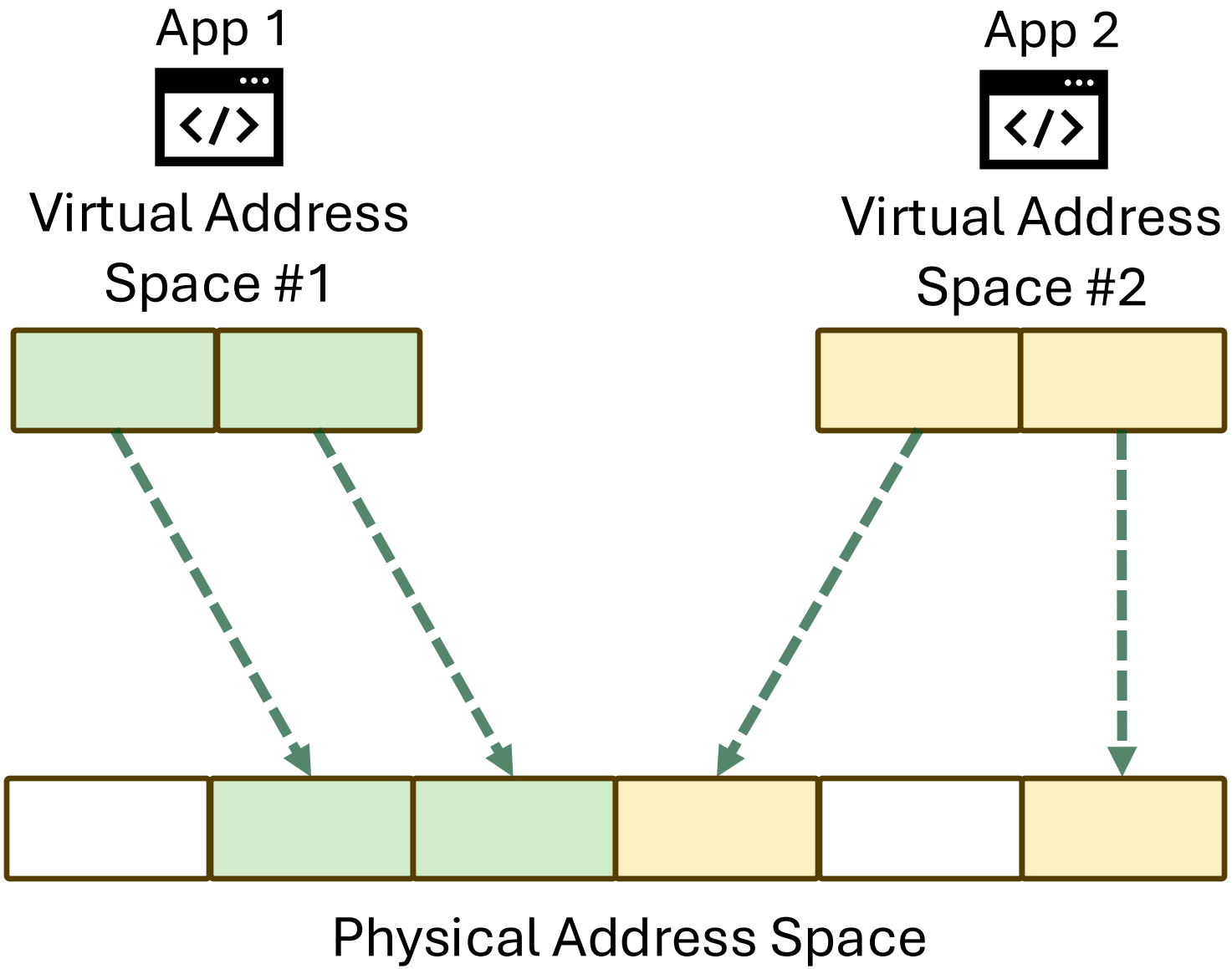
- Idea: Give each program the illusion of a large address space while having a small physical memory
  - So that the programmer does not worry about managing physical memory (within a process or across processes)

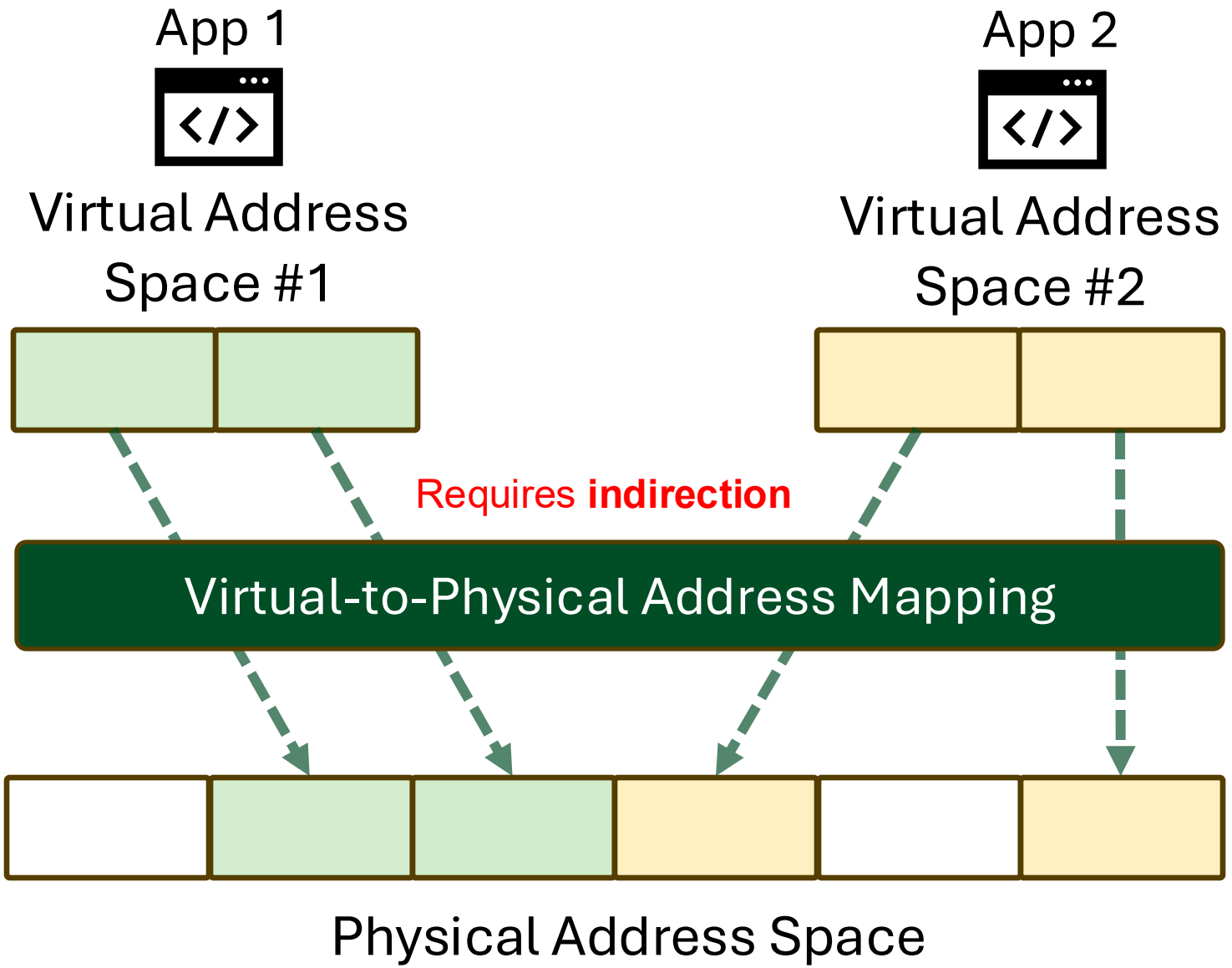
- Programmer can assume they have "infinite" amount of physical memory

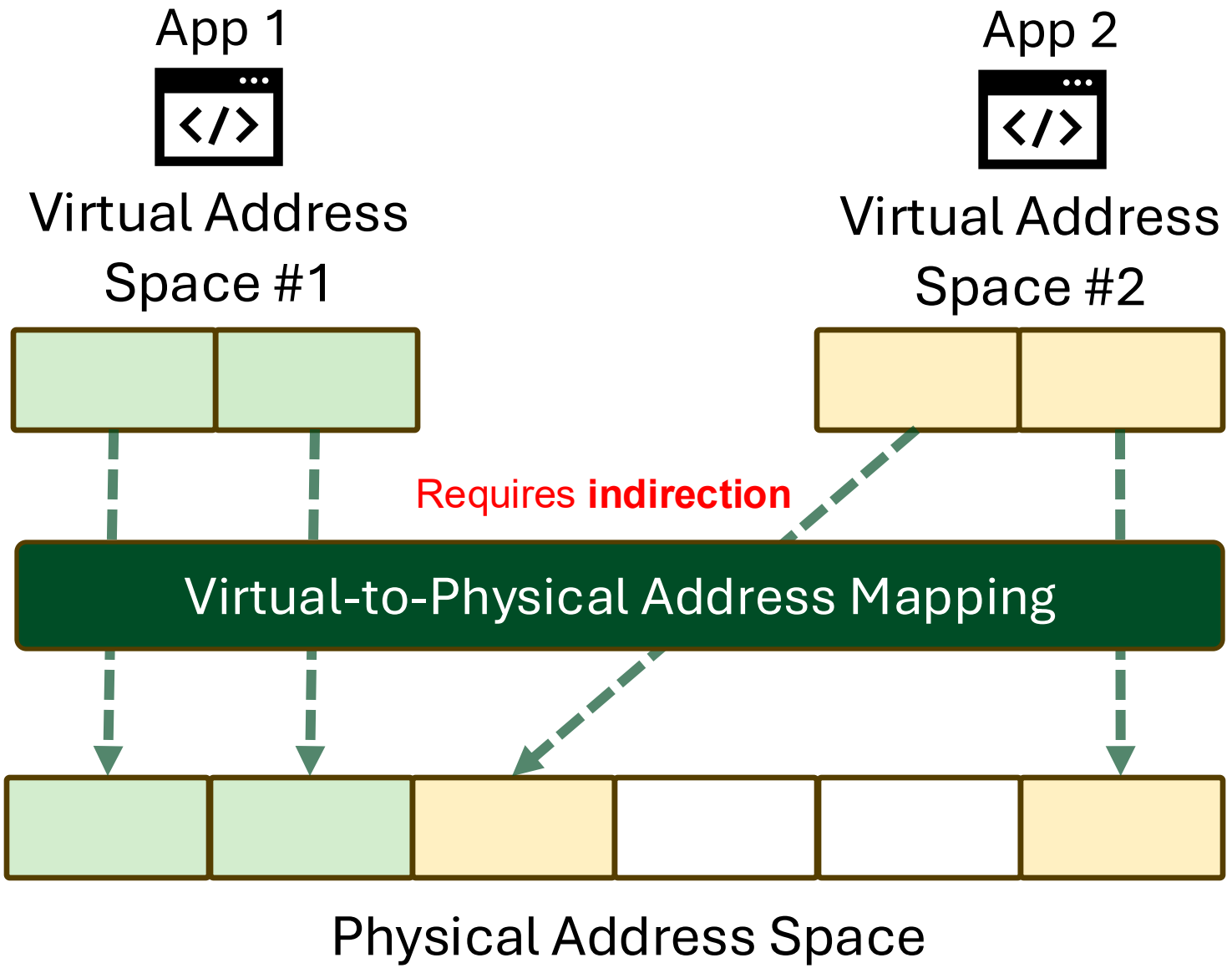
- Hardware and software cooperatively and automatically manage the physical memory space to provide the illusion
  - Illusion is maintained for each independent process

Basic Mechanism

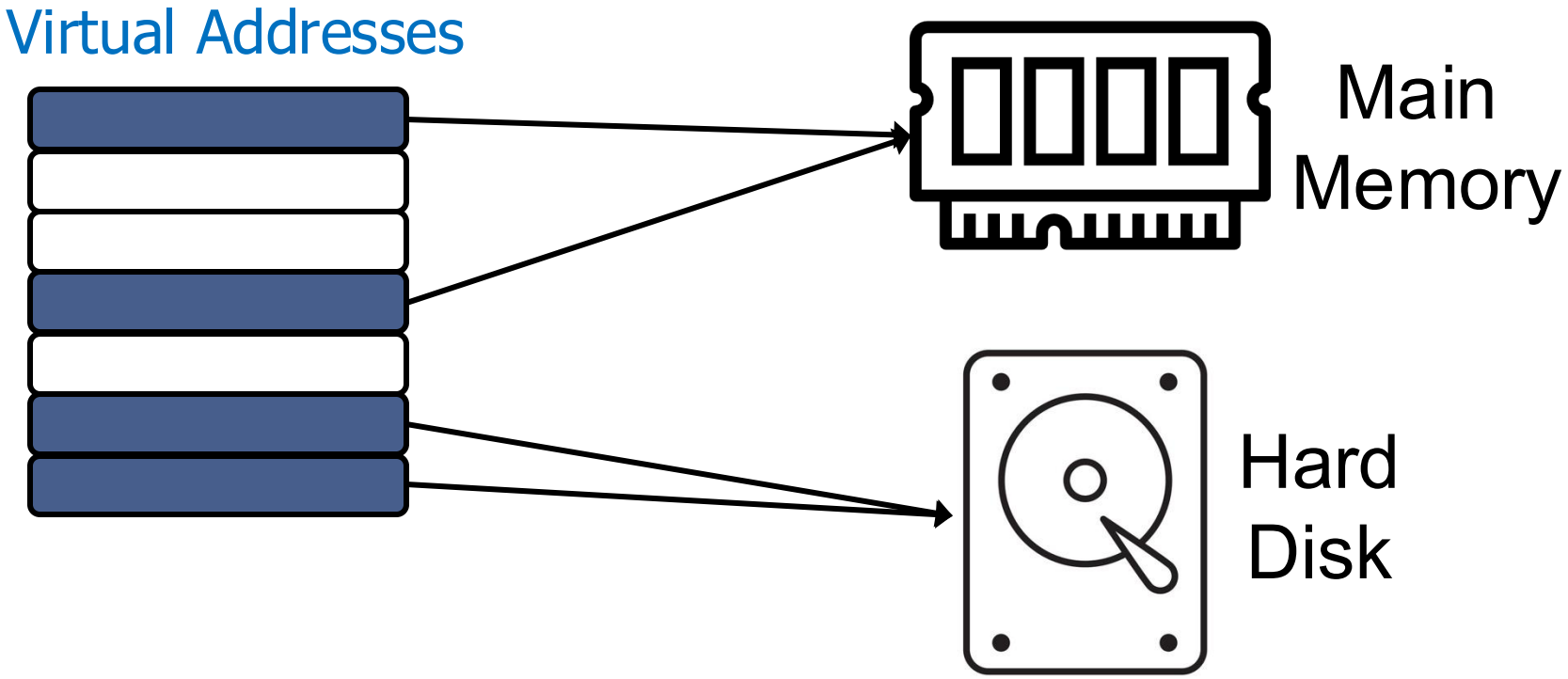
- Indirection and mapping (of addresses)
- Address generated by each instruction in a program is a "virtual address"
  - i.e., it is not the physical address used to address main memory
- An "address translation" mechanism maps this address to a "physical address"
  - Address translation mechanism can be implemented in hardware and/or software


App 1
App 2
Virtual Address Space #1
Virtual Address Space #2
Physical Address Space


App 1
App 2
Virtual Address Space #1
Virtual Address Space #2
Requires indirection
Virtual-to-Physical Address Mapping
Physical Address Space


App 1
App 2
Virtual Address Space #1
Virtual Address Space #2
Requires indirection
Virtual-to-Physical Address Mapping
Physical Address Space

Heterogeneity in Physical Data Storage



- Most accesses hit in physical memory
- Programmer is not concerned about whether main memory is full – data can be transparently stored in the disk

Modern Systems Use Physical Memory as a Cache

- Physical memory is a cache for pages stored on disk
  - In fact, it is a fully-associative cache in modern systems (a virtual page can potentially be mapped to any physical frame)

- Similar caching issues exist as we have covered earlier:
  - Placement: where and how to place/find a page in cache?
  - Replacement: what page to remove to make room in cache?
  - Granularity of management: large, small, uniform pages?
  - Write policy: what do we do about writes? Write back?

Real Life Example: Warehouse



Virtual Addresses = Barcodes

- Workers identify each item purely by scanning its barcode:
  - They don't know where it is stored.
  - They don't care where it is stored.
  - Barcodes provide a clean, unified namespace.
- This is exactly what **virtual addresses** are:
  - A program reads and writes data using virtual addresses.
  - The program never knows the *actual* physical location of that data.
- Virtual addresses are convenient, abstract identifiers — just like barcodes.

Physical memory = The Actual Warehouse Racks

- Behind the scenes, items must sit somewhere:
  - On a specific aisle
  - On a specific rack
  - On a specific shelf
  - Inside a specific bin

- These concrete storage spots represent **physical memory pages**.

- Virtual addresses (barcodes) do **not** reveal:
  - Which DRAM ranks/banks/rows the data is in
  - You only need that info when *actually accessing* the item (data)

Four Issues in Indirection and Mapping

- What is the mapping granularity?
  - Byte? Kilo-byte? Mega-byte? Giga-byte? …
  - Multiple granularities?
- Where and how to store the virtual→physical mappings?
  - Operating system data structures? Hardware? Cooperative?
- When to map a virtual address to a physical address?
  - When the virtual address is first referenced by the program
- What to do when physical address space is full?
  - Evict an unlikely-to-be-needed virtual address from physical memory

App 1



Virtual Address Space #1



App 2



Virtual Address Space #2





Physical Address Space (e.g., physical memory)

App 1



Virtual Address Space #1

Page	Page

(e.g., 4KB)

App 2



Virtual Address Space #2

Page	Page

Page	Page	Page	Page	Page

Physical Address Space (e.g., physical memory)

Virtual Pages, Physical Frames

- Virtual and Physical address space divided into pages
  - x86 / x86-64 (Intel & AMD)
    - 4KB – standard, 2MB, 1GB
  - POWER / PowerPC
    - 8 KB — standard, 64 KB, 512 KB, 4 MB, 32 MB
  - macOS/Apple Silicon
    - 16KB — standard

Trade-offs? We will find out soon

Virtual Memory Example

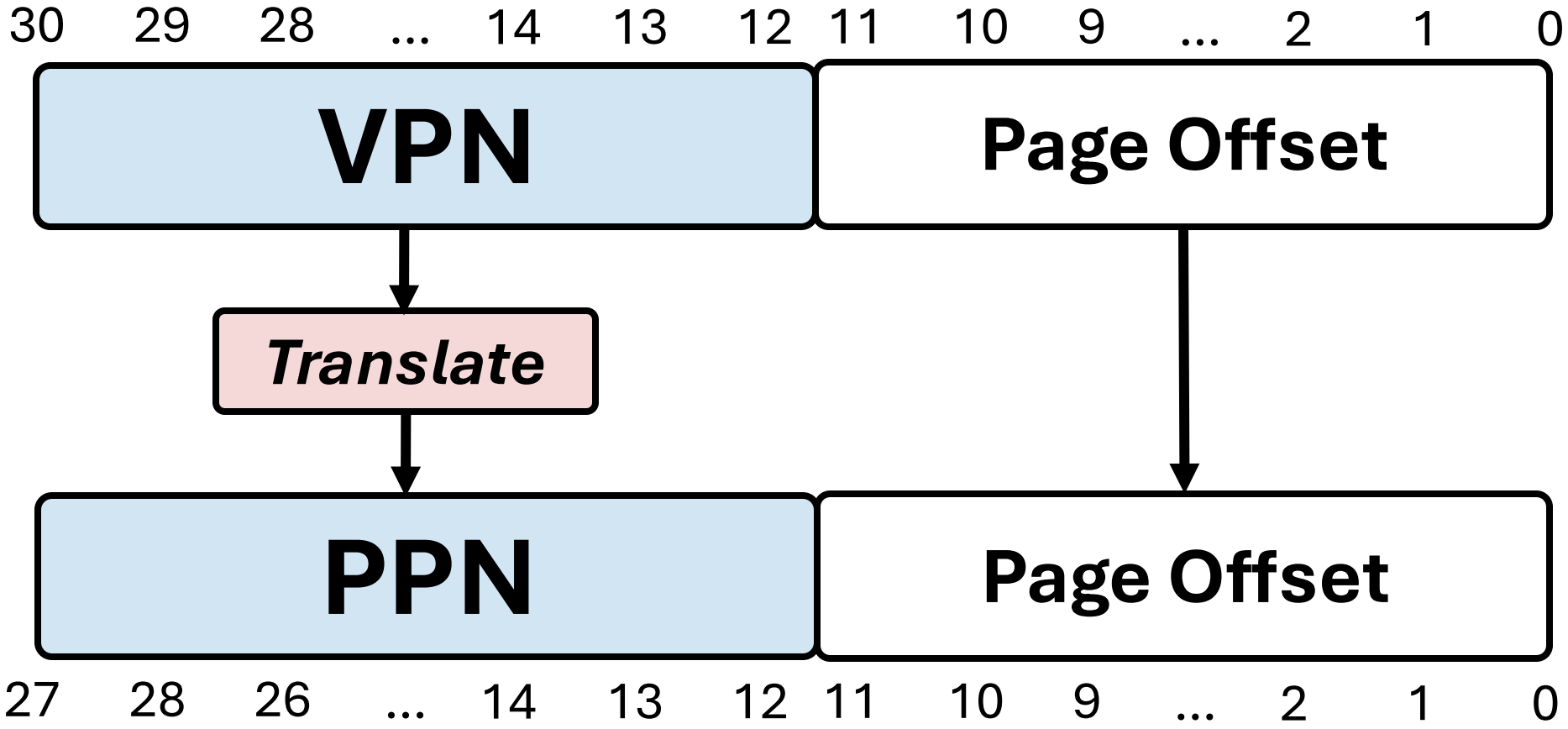
- System:
  - Virtual memory size: 2 GB = 2^{31} bytes
  - Physical memory size: 128 MB = 2^{27} bytes
  - Page size: 4 KB = 2^{12} bytes

Virtual Memory Example (Continued)

- System:
  - Virtual memory size: 2 GB = 2^{31} bytes
  - Physical memory size: 128 MB = 2^{27} bytes
  - Page size: 4 KB = 2^{12} bytes

- Organization:
  - Virtual address: **31** bits
  - Physical address: **27** bits
  - Page offset: **12** bits
  - # Virtual pages = $2^{31}/2^{12}$ = $\mathbf{2^{19}}$ (VPN = 19 bits)
  - # Physical pages = $2^{27}/2^{12}$ = $\mathbf{2^{15}}$ (PPN = 15 bits)

Discovering the Physical Address



Four Issues in Indirection and Mapping

- What is the mapping granularity?
  - Byte? Kilo-byte? Mega-byte? Giga-byte? …
  - Multiple granularities?

- Where and how to store the virtual→physical mappings?
  - Operating system data structures? Hardware? Cooperative?

- When to map a virtual address to a physical address?
  - When the virtual address is first referenced by the program

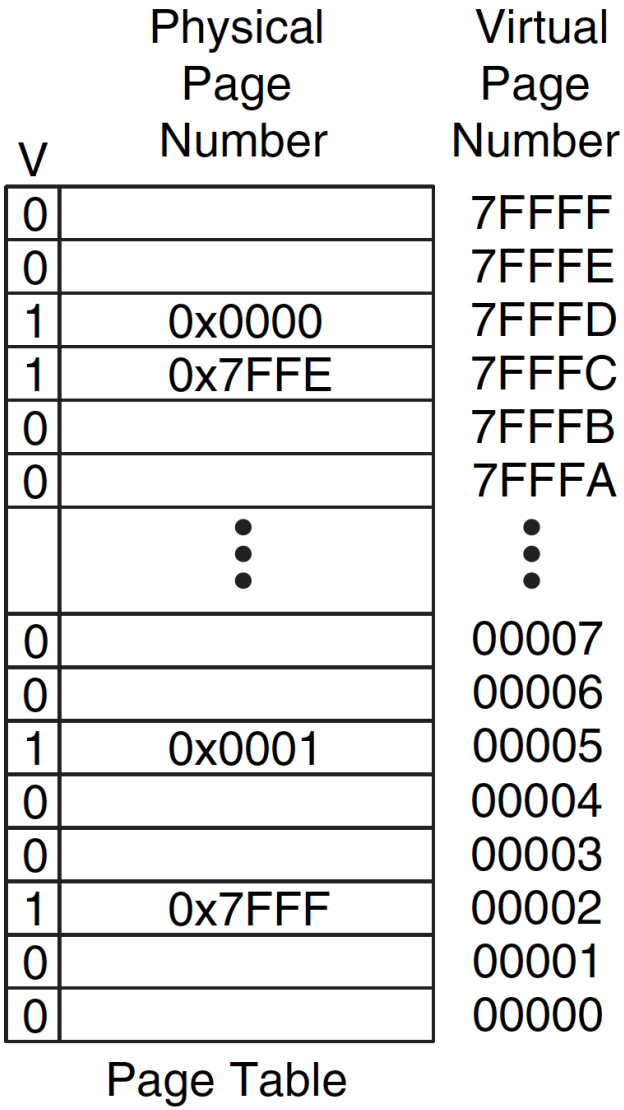
- What to do when physical address space is full?
  - Evict an unlikely-to-be-needed virtual address from physical memory

How Do We Translate Addresses?

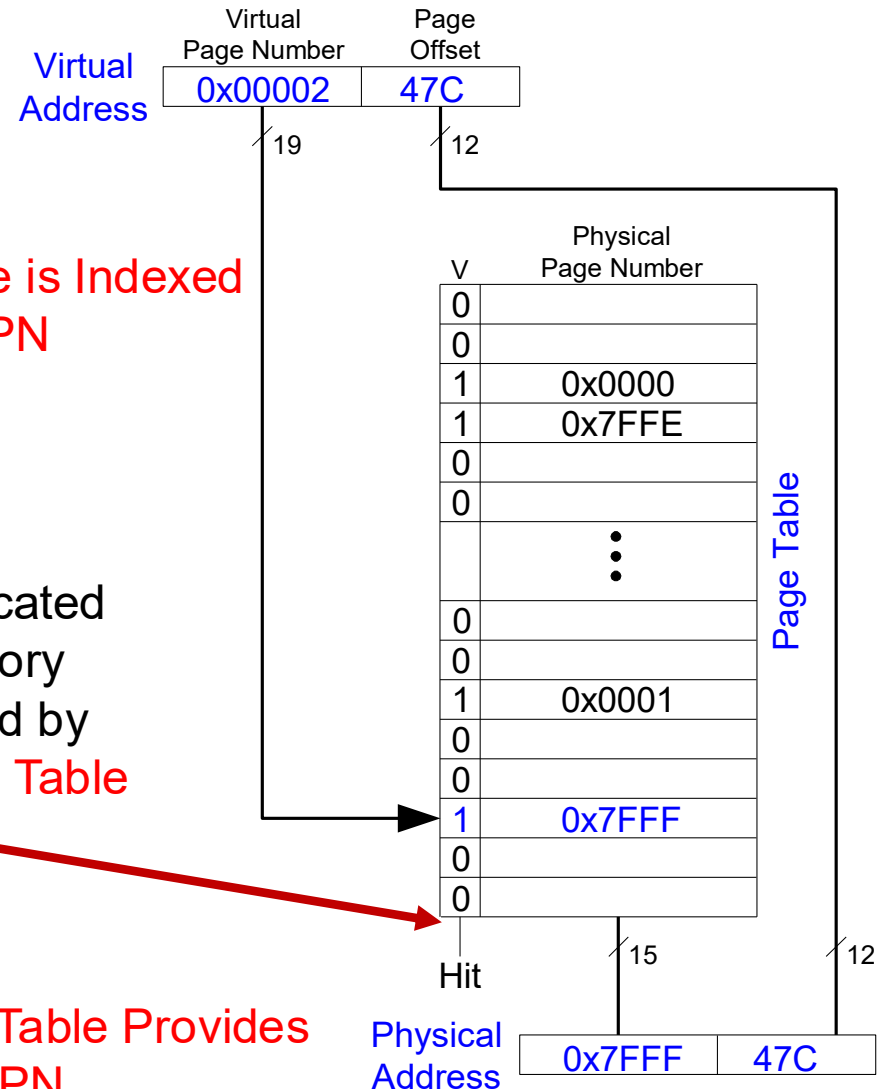
- **Page table**
  - Has entry for each virtual page

- Each page table entry has:
  - Valid bit: whether the virtual page is located in physical memory or not
  - Physical page number: where the virtual page is located in physical memory
  - (Replacement policy, dirty/modified, permission/access bits)

Page Table for Our Example (Continued)

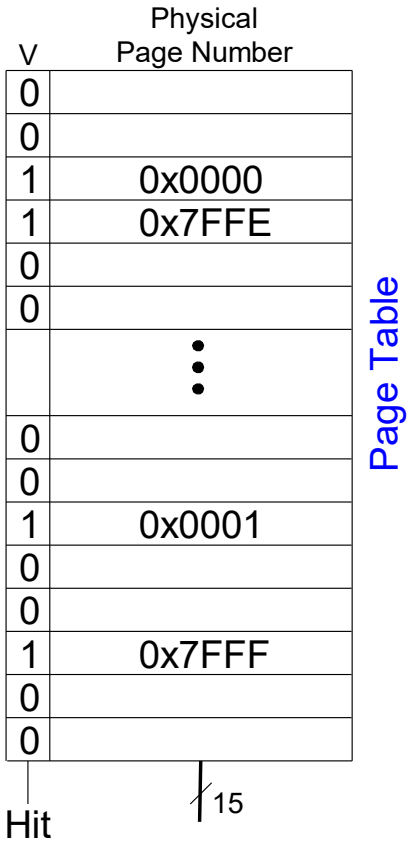


Page Table Address Translation Example



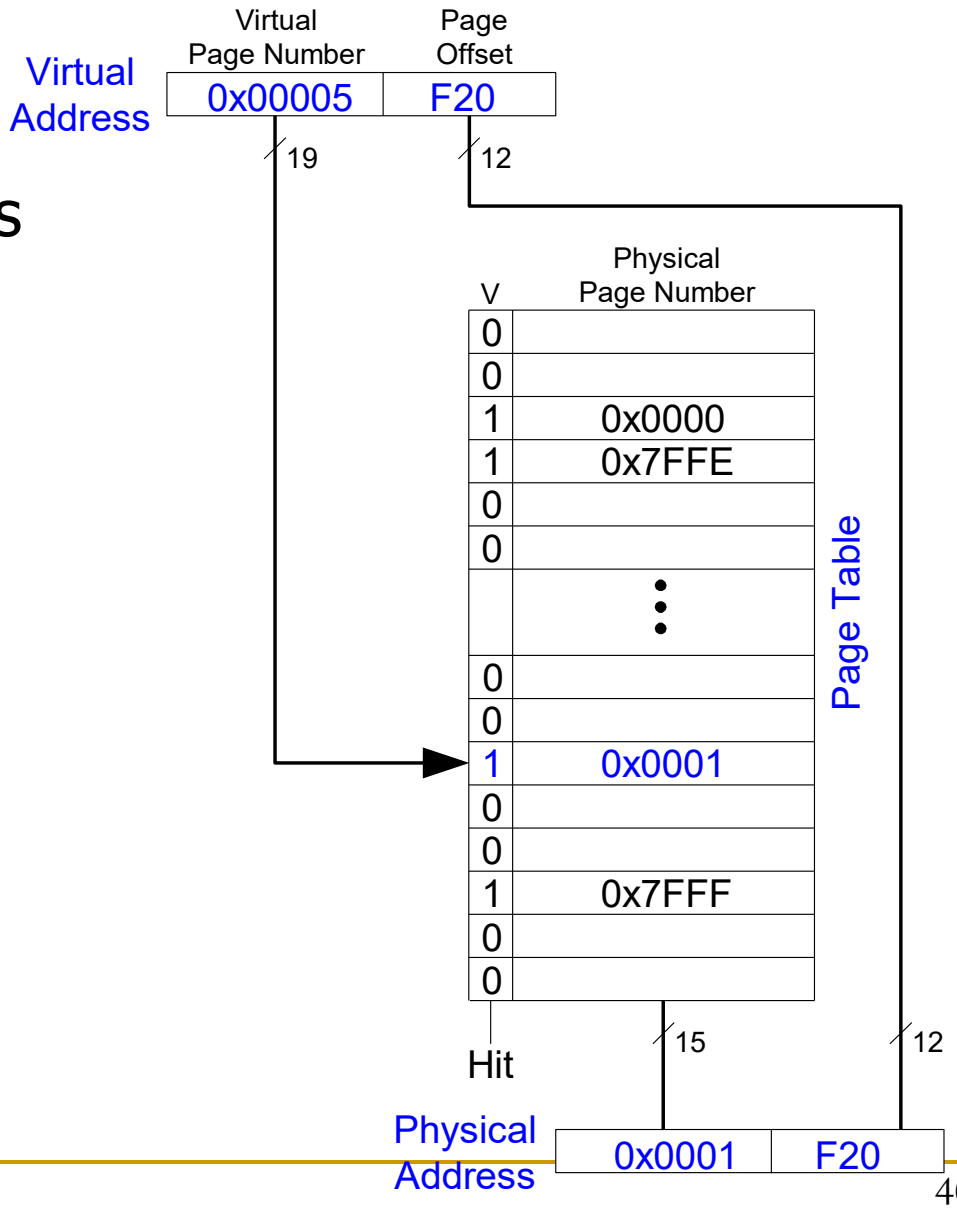
Page Table Address Translation Example 1

- What is the physical address of virtual address 0x5F20?
- We first need to find the page table entry containing the translation for the corresponding VPN
- Look up the PTE at the address
  - PTBR + VPN*PTE-size



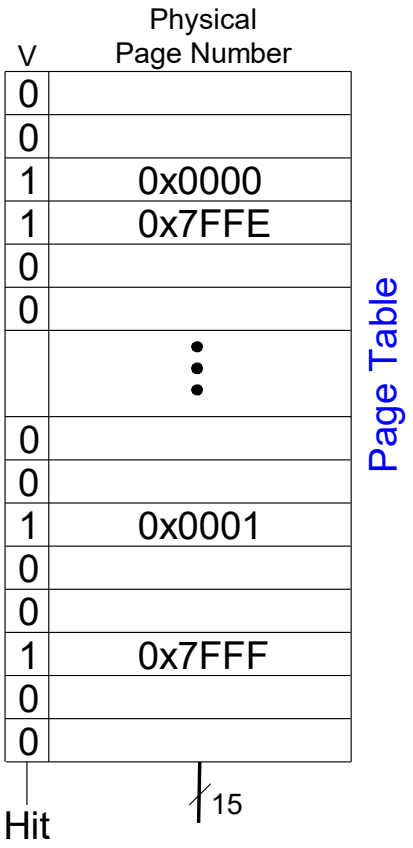
Page Table Address Translation Example 1

- What is the physical address of virtual address 0x5F20?
  - VPN = 5
  - Entry 5 in page table indicates VPN 5 is in physical page 1
  - Physical address is 0x1F20



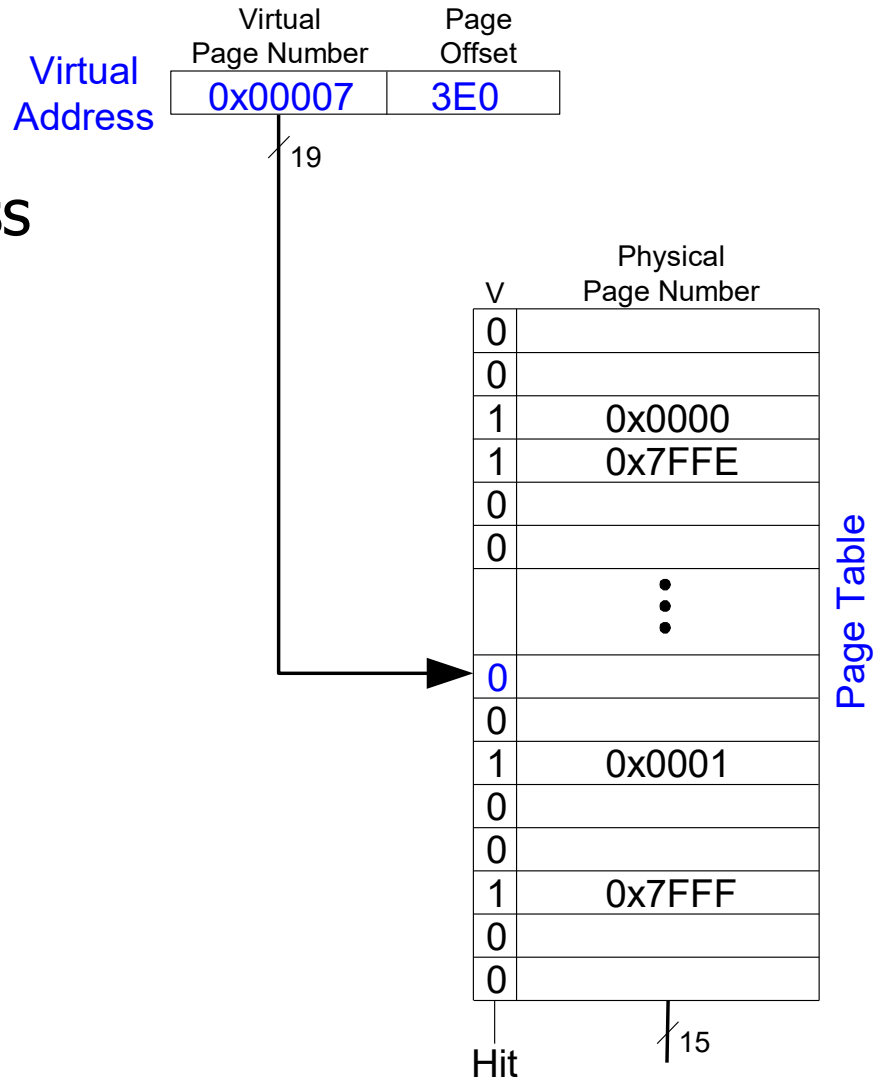
Page Table Address Translation Example 2

- What is the physical address of virtual address 0x73E0?

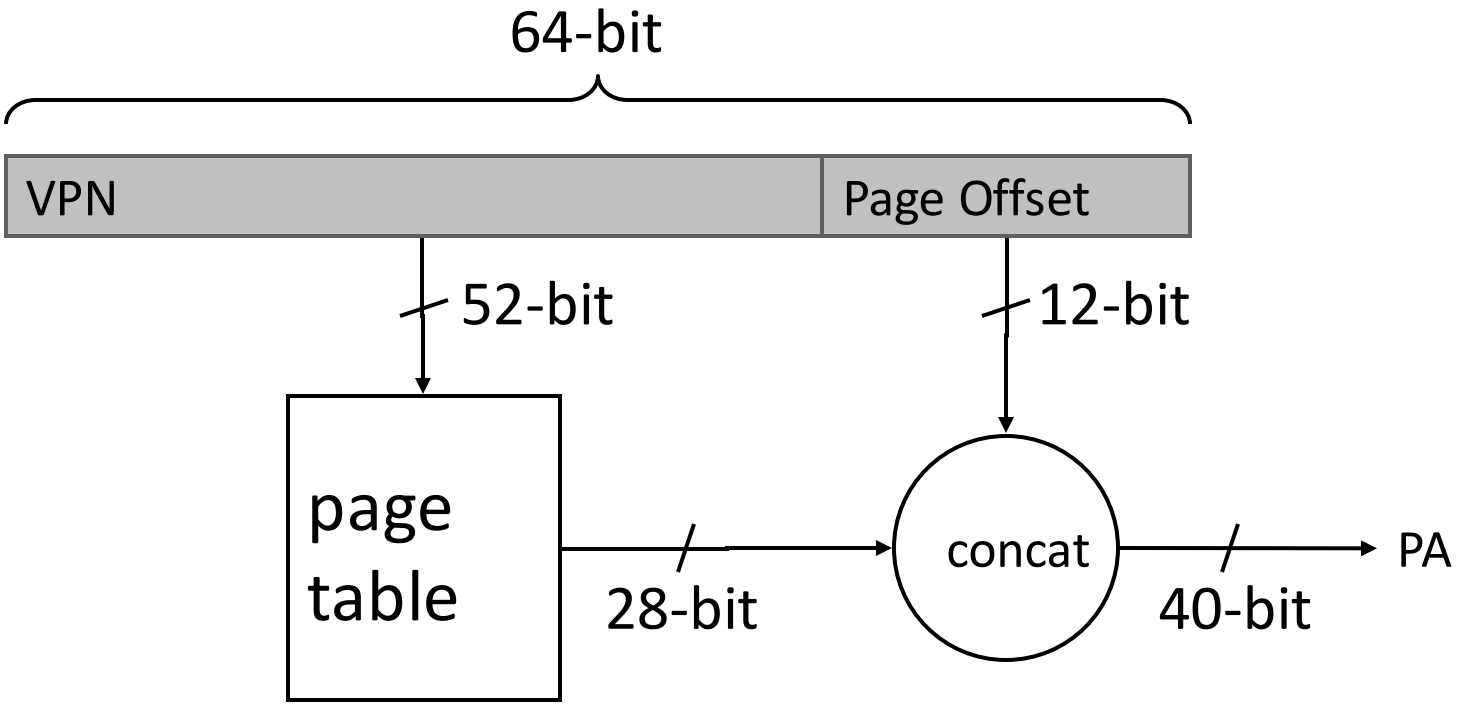


Page Table Address Translation Example 2

- What is the physical address of virtual address 0x73E0?
  - VPN = 7
  - Entry 7 in page table is invalid, so the page is not in physical memory



Issue: Page Table Size



- Suppose 64-bit VA and 40-bit PA, how large is the page table?
  - **2^{52} entries x ~4 bytes ≈ 2^{54} bytes**

  and that is for just one process!

  and the process may not be using the entire VM space!

Page Table Challenges (I)

- Challenge 1: Page table is large
  - at least part of it needs to allocated in physical memory
  - solution: multi-level (hierarchical) page tables

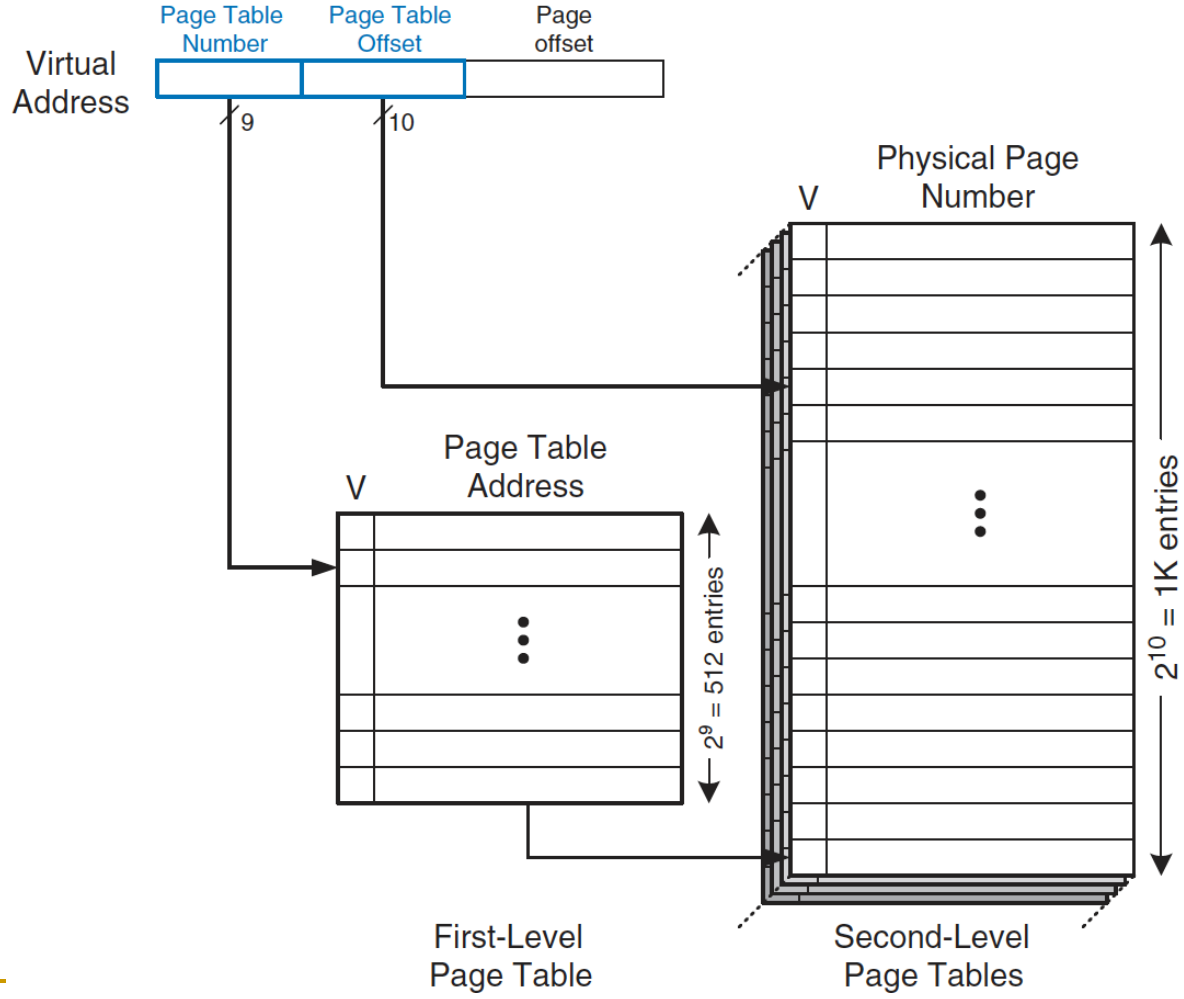
Multi-Level Page Tables

- Idea: Organize page table in a hierarchical manner such that only some parts of page table have to be allocated in physical memory

- Multi-level (hierarchical) page tables

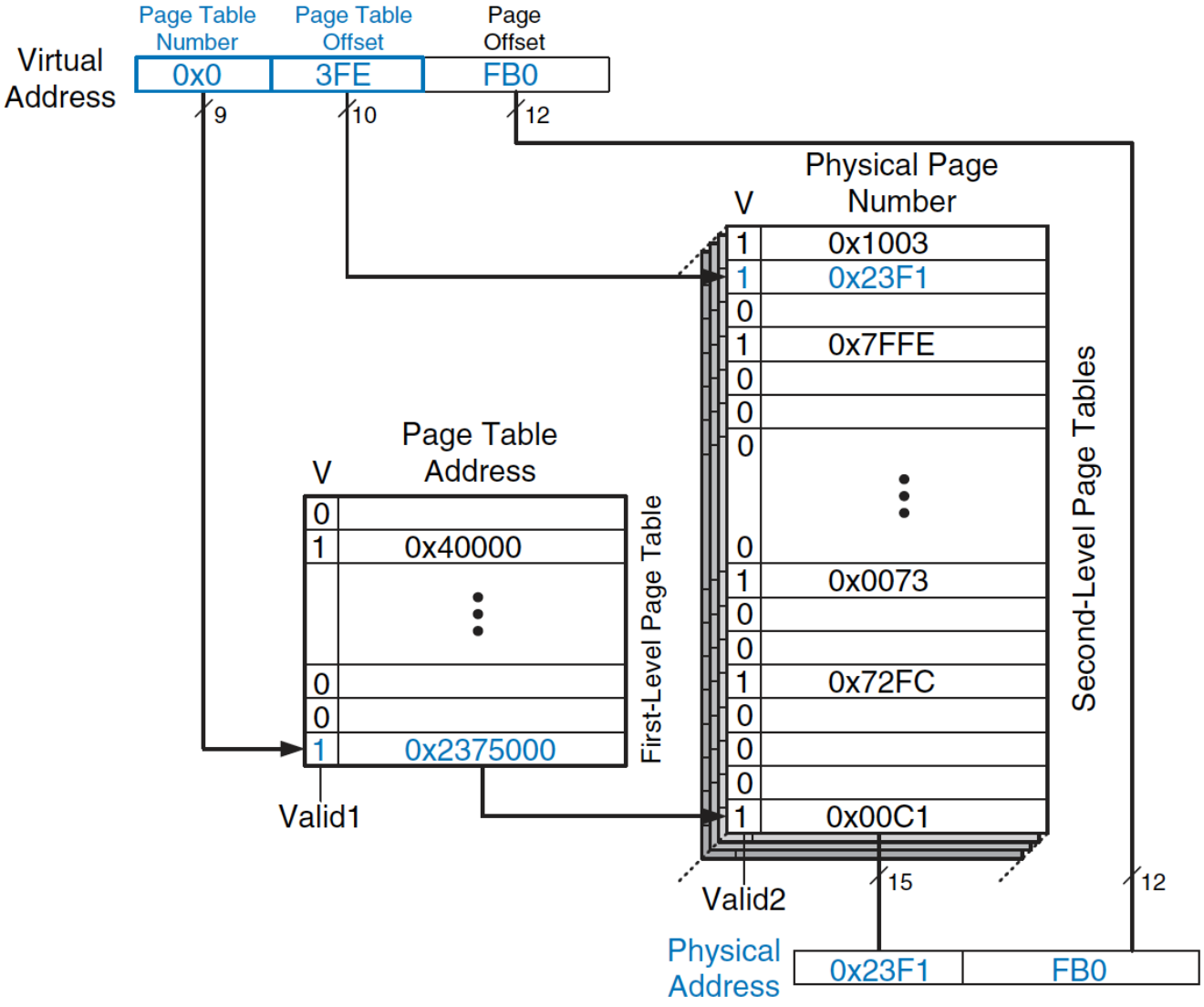
Multi-Level Page Table Example

- First-level page table has to be in physical memory
- Only the needed second-level page tables can be kept in physical memory

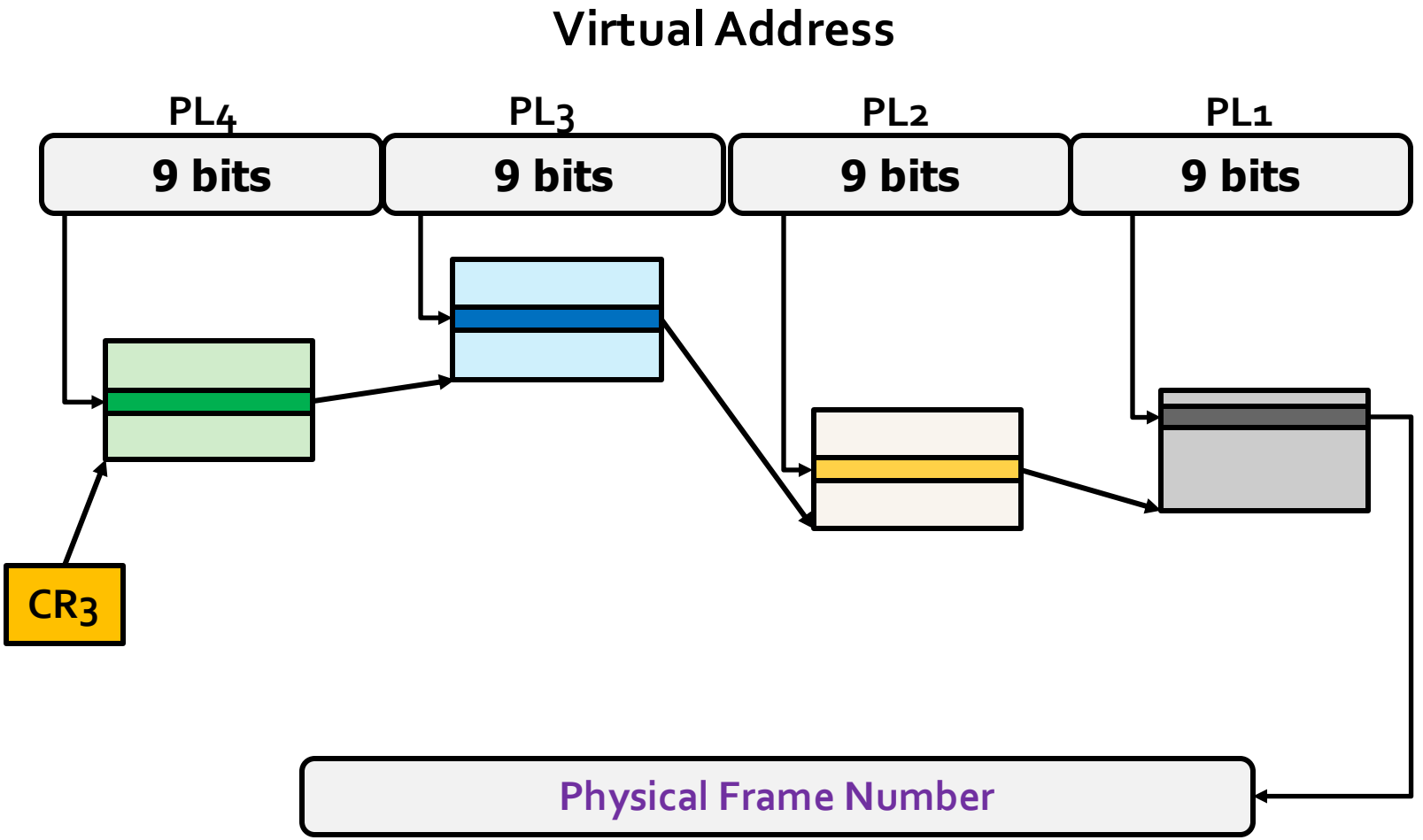


Multi-Level Page Table: Address Translation

- For N-level page table, we need N page table accesses to find the PTE



Page Table Walk in x86-64

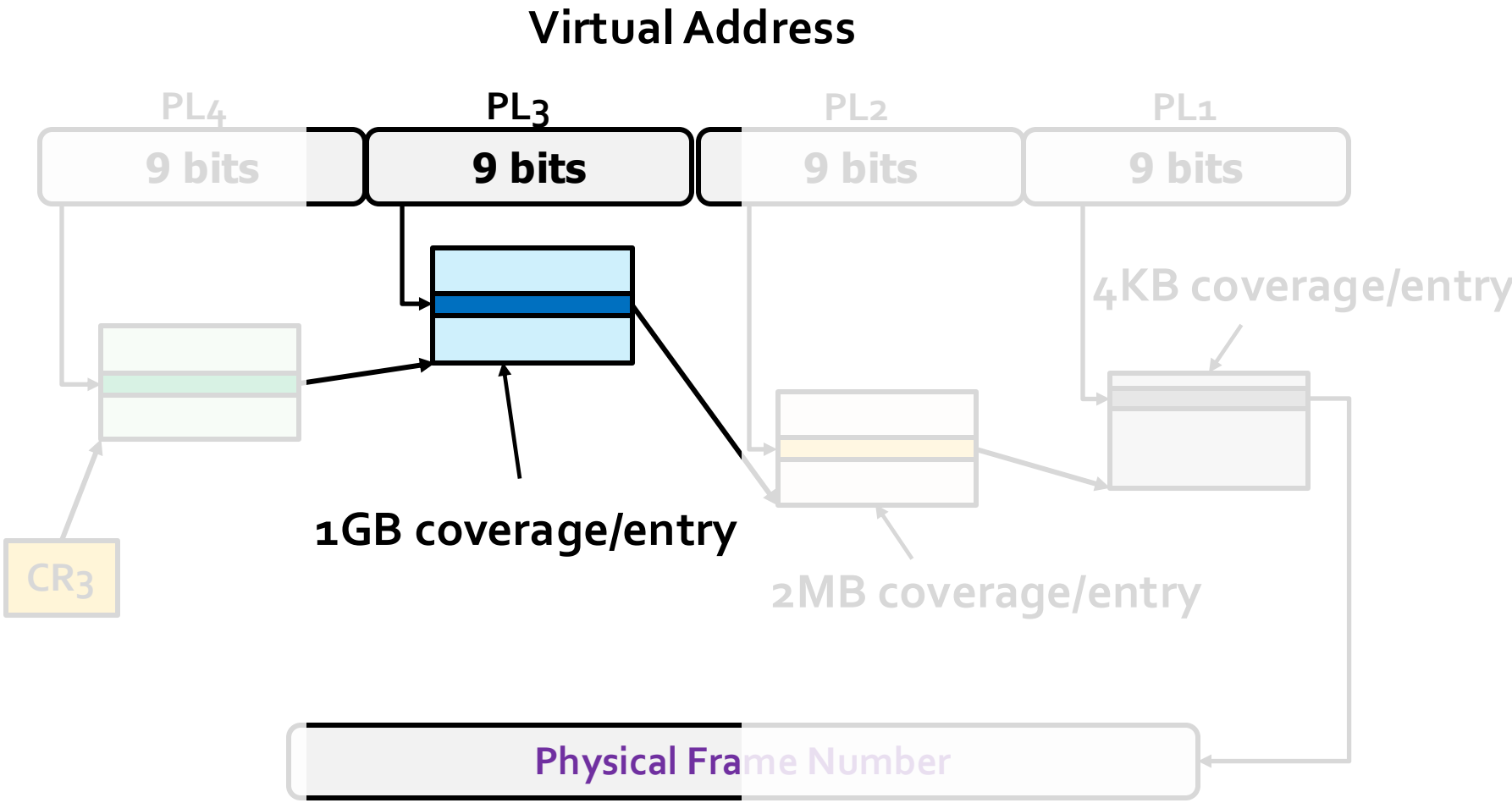


Virtual Pages, Physical Frames (Recall)

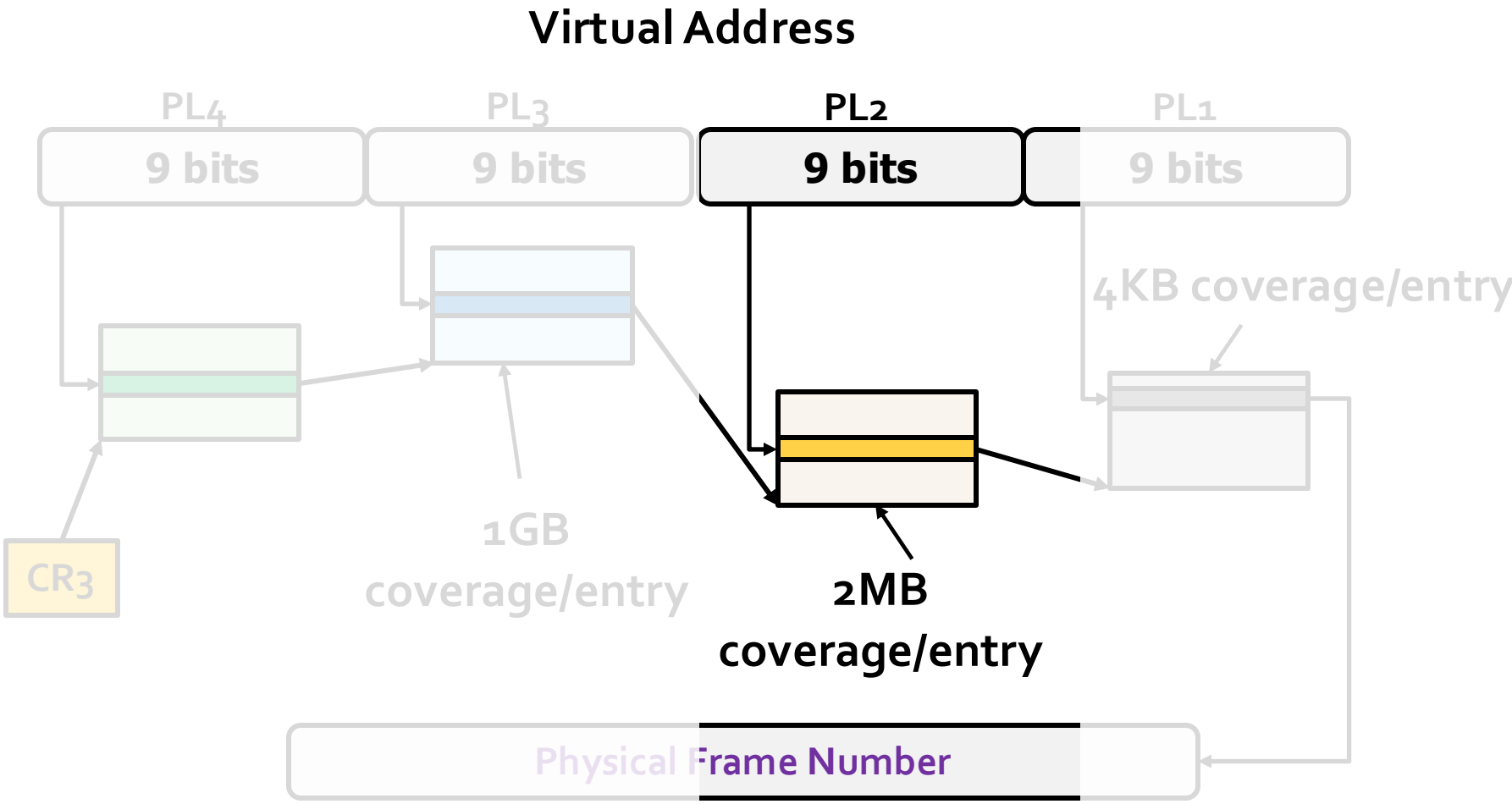
- Virtual and Physical address space divided into pages
  - x86 / x86-64 (Intel & AMD)
    - 4KB – standard, 2MB, 1GB
  - POWER / PowerPC
    - 8 KB — standard, 64 KB, 512 KB, 4 MB, 32 MB
  - macOS/Apple Silicon
    - 16KB — standard

Trade-offs?

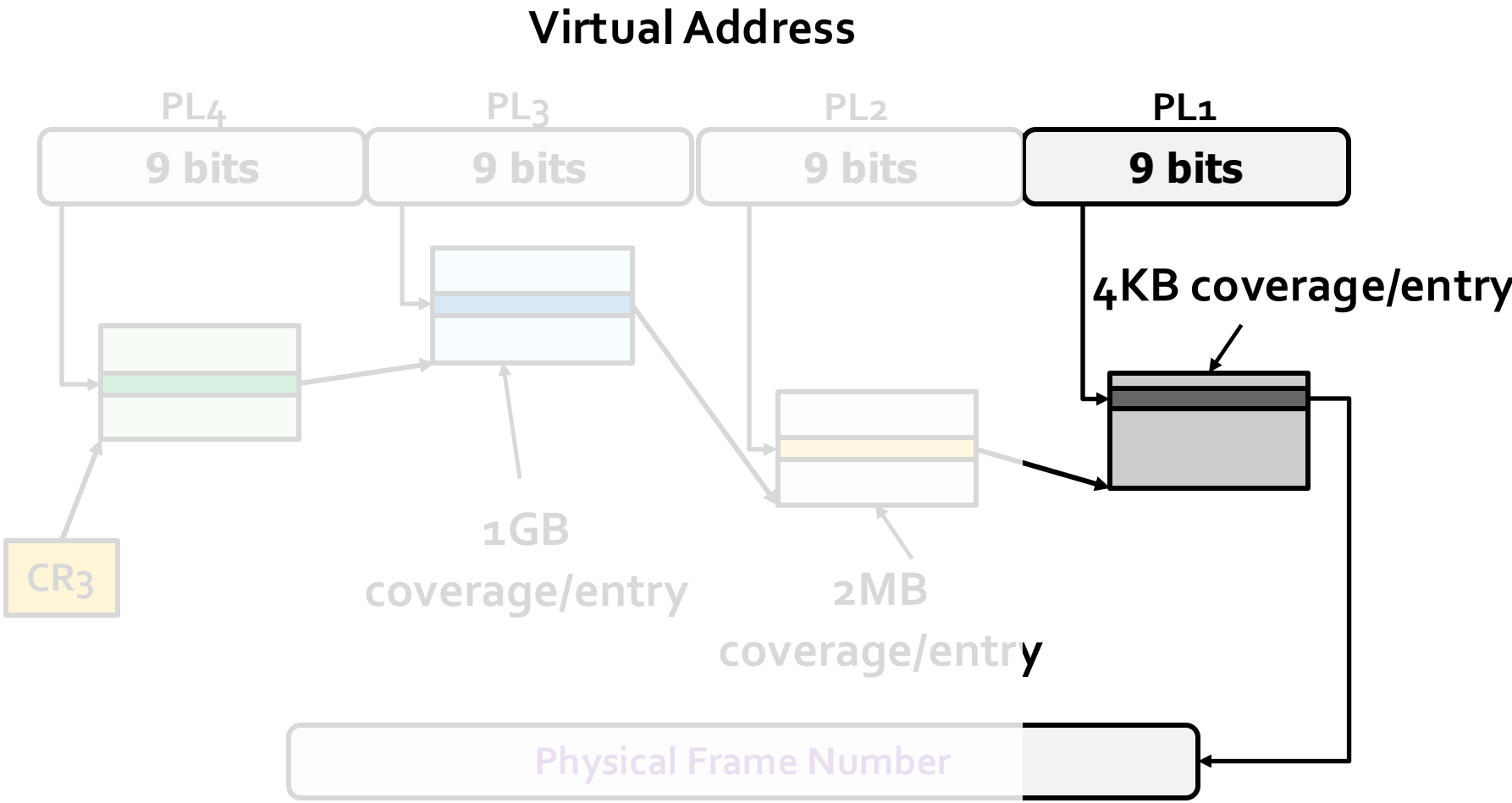
Page Table Walk in x86-64



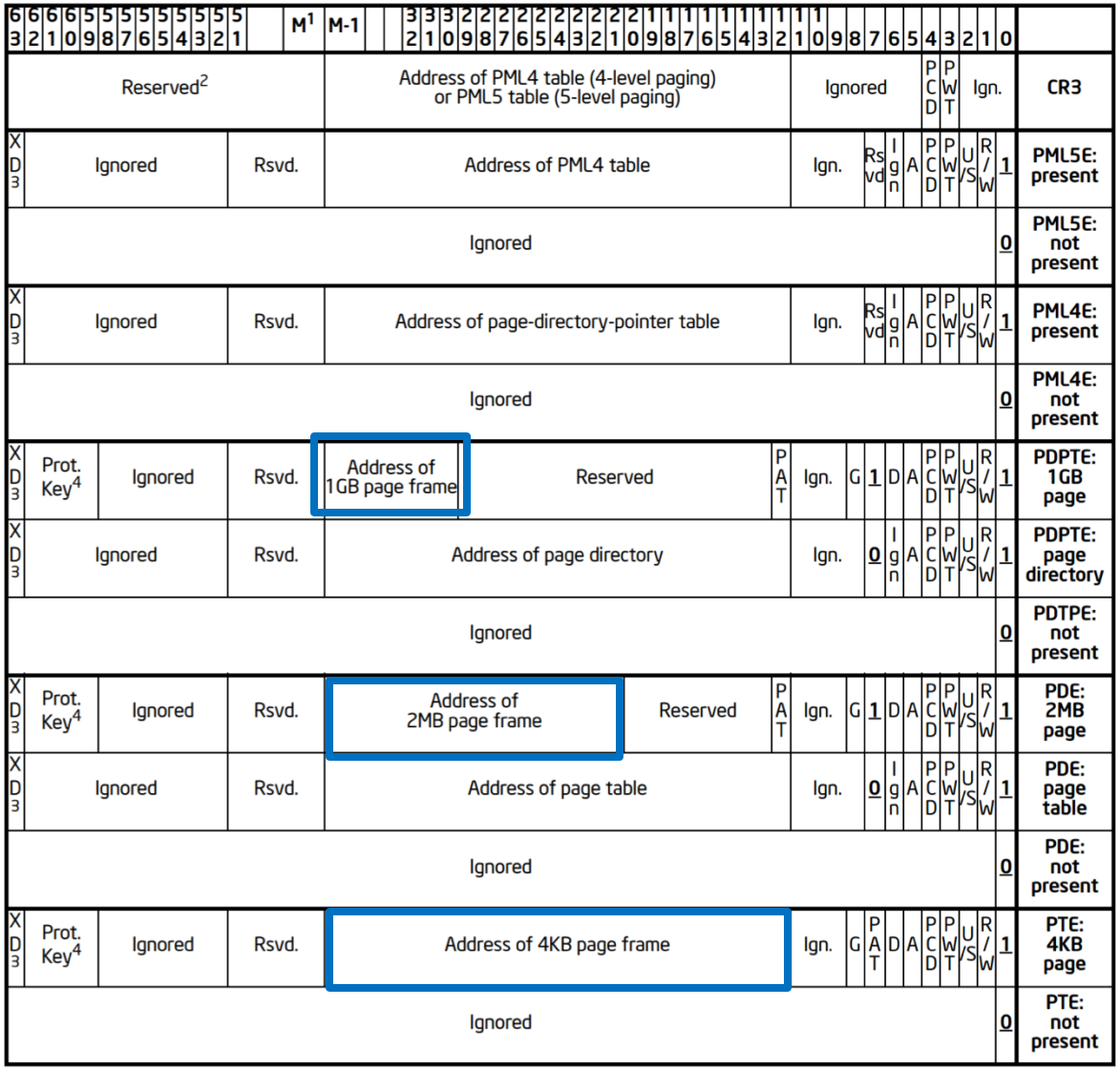
Page Table Walk in x86-64



Page Table Walk in x86-64



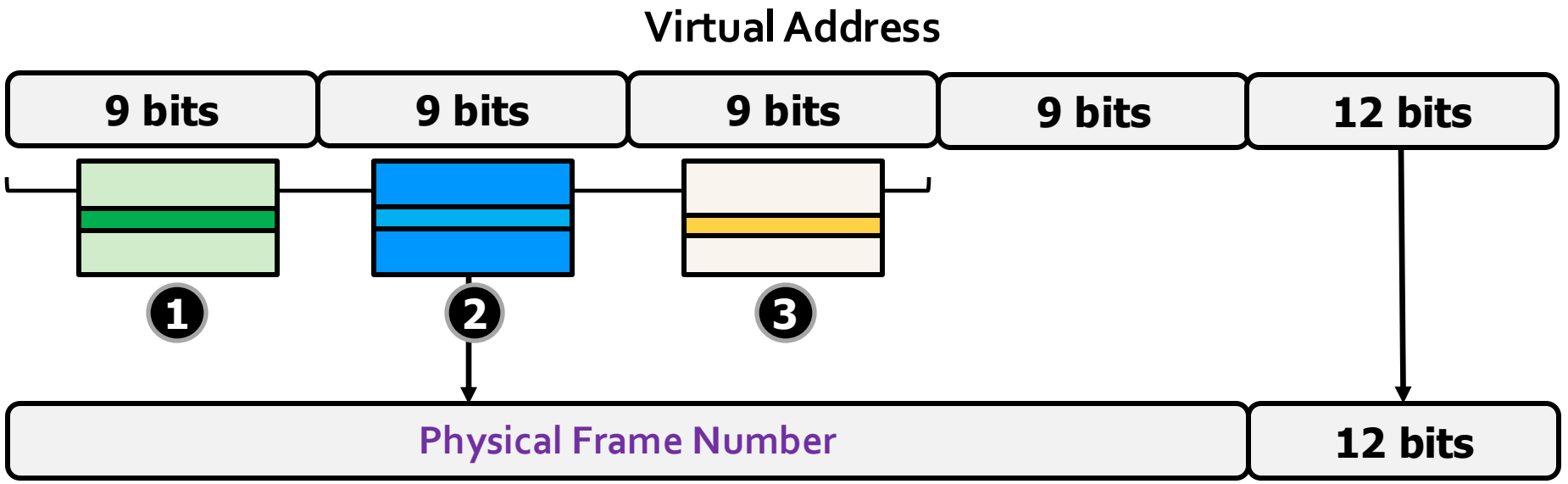
X86-64 Page Table Entry Structure



63	62	61	60	59	58	57	56	55	54	53	52	51	M[1]	M-1	32	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Reserved[2]														Address of PML4 table (4-level paging) or PML5 table (5-level paging)																								Ignored								PCD	PWT	Ign.		CR3
XD	Ignored													Rsvd.	Address of PML4 table																							Ign.				Rsvd	Ign	A	PCD	PWT	U/S	R/W	1	PML5E: present
Ignored																																																0	PML5E: not present
XD	Ignored													Rsvd.	Address of page-directory-pointer table																							Ign.				Rsvd	Ign	A	PCD	PWT	U/S	R/W	1	PML4E: present
Ignored																																																0	PML4E: not present
XD	Prot. Key[4]		Ignored											Rsvd.	Address of 1GB page frame																							Reserved				PAT	Ign.	G	1	D	A	PCD	PWT	U/S	R/W	1	PDPTE: 1GB page
XD	Ignored													Rsvd.	Address of page directory																							Ign.				0	Ign	A	PCD	PWT	U/S	R/W	1	PDPTE: page directory
Ignored																																																0	PDTPE: not present
XD	Prot. Key[4]		Ignored											Rsvd.	Address of 2MB page frame																							Reserved				PAT	Ign.	G	1	D	A	PCD	PWT	U/S	R/W	1	PDE: 2MB page
XD	Ignored													Rsvd.	Address of page table																							Ign.				0	Ign	A	PCD	PWT	U/S	R/W	1	PDE: page table
Ignored																																																0	PDE: not present
XD	Prot. Key[4]		Ignored											Rsvd.	Address of 4KB page frame																							Ign.				G	PAT	D	A	PCD	PWT	U/S	R/W	1	PTE: 4KB page
Ignored																																																0	PTE: not present

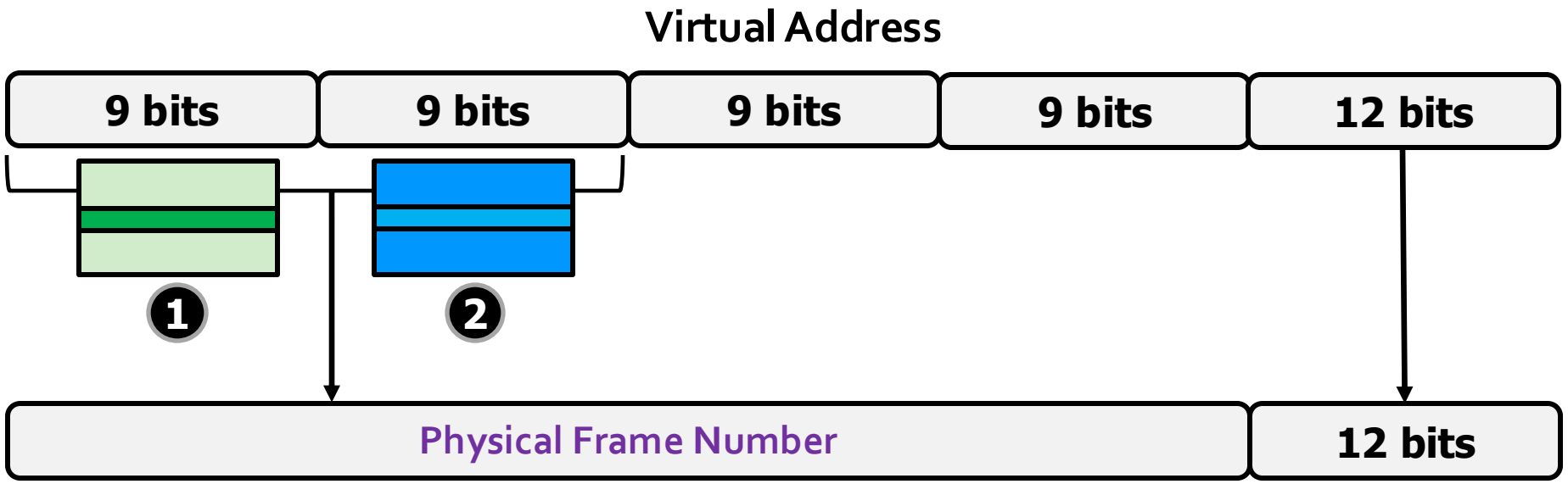
Figure 4-11. Formats of CR3 and Paging-Structure Entries with 4-Level Paging and 5-Level Paging

Page Table Walk in x86-64 (2MB Page)



Three sequential memory accesses in case of a 2MB page

Page Table Walk in x86-64 (1GB Page)



***Two* sequential memory accesses in case of a 1GB page**

Trade-Offs in Page Size

- **Large page size (e.g., 2MB, 1GB)**
  - Pro: Fewer PTEs required ➔ Saves memory space
  - Pro: Faster Page Table Walk
  - Con: Large transfers to/from disk
    - Even when only 1KB is needed, 1GB must be transferred
    - Waste of bandwidth/energy
    - Reduces performance
  - Con: **Internal fragmentation**
    - Even when only 1KB is needed, 1GB must be allocated
    - Waste of space
    - **Q**: What is **external fragmentation**?

Page Table Challenges (II)

- Challenge 1: Page table is large
  - at least part of it needs to be located in physical memory
  - solution: multi-level (hierarchical) page tables

- Challenge 2: Each instruction fetch or load/store requires at least two memory accesses:

  1. one for address translation (page table read)
  2. one to access data with the physical address (after translation)

- Two memory accesses to service an instruction fetch or load/store greatly degrades execution time
  - Num. of memory accesses increases with multi-level page tables
  - Unless we are clever… → speed up the translation…

Real Life Example: Warehouse



Page Table = The Warehouse's Inventory Database

The warehouse management system stores mappings like:

```
Barcode 91273 → Aisle 4, Rack C, Shelf 2, Bin 6
Barcode 44710 → Aisle 12, Rack A, Shelf 1, Bin 1
```

This is the role of the **page table**:

- It maps **virtual page numbers → physical page numbers**
- It defines where every piece of data actually lives.
- It provides isolation: two barcodes can't accidentally point to the same item unless explicitly shared.

Programs use barcodes; only the page table knows real locations.

Performing Address Translation

Supporting Virtual Memory

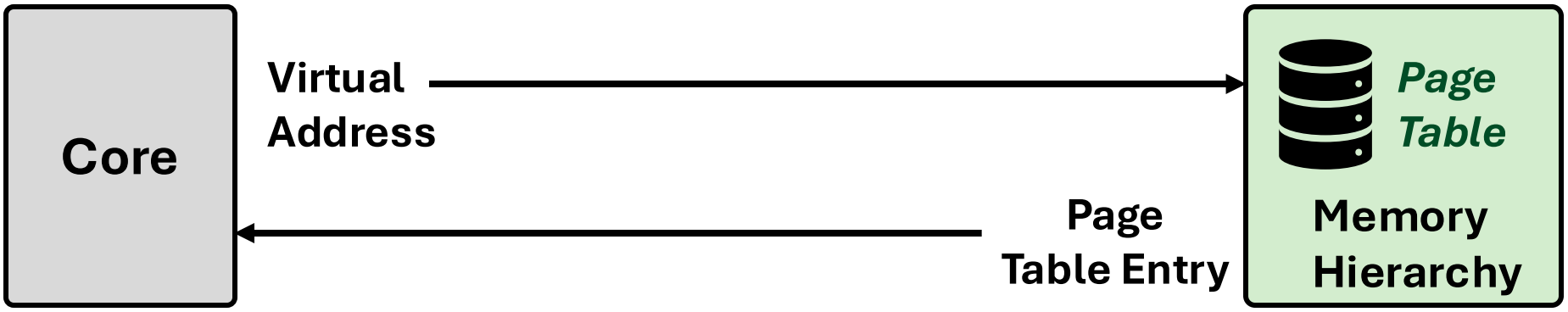
- Virtual memory requires both HW+SW support
  - Page Table is in memory

- It is the job of the software (e.g., the Operating System) to
  - Populate page tables, decide what to replace in physical memory
  - Change the Page Table Base Register on context switch (to use the running thread's page table)
  - Ensure correct mappings

How does the CPU discover the virtual-to-physical mapping?

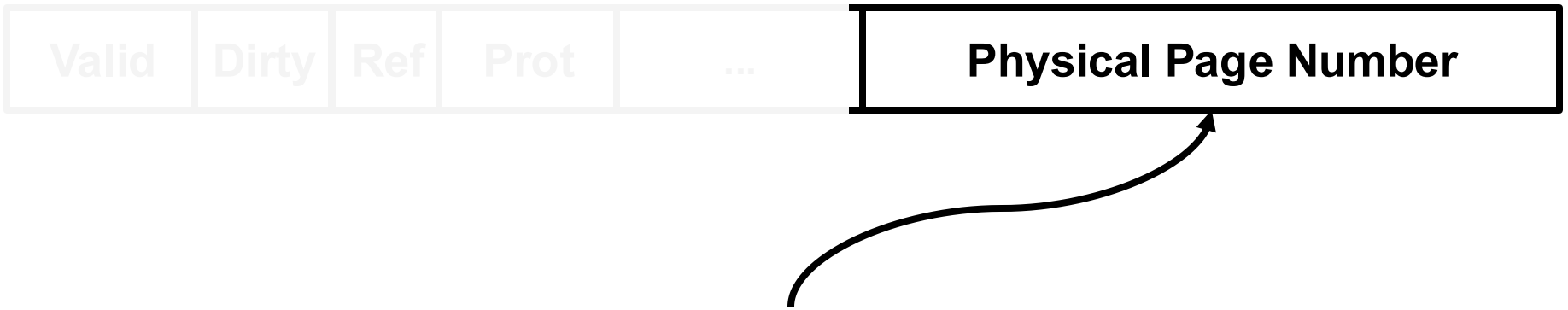
Address Translation

- How to obtain the physical address from a virtual address?



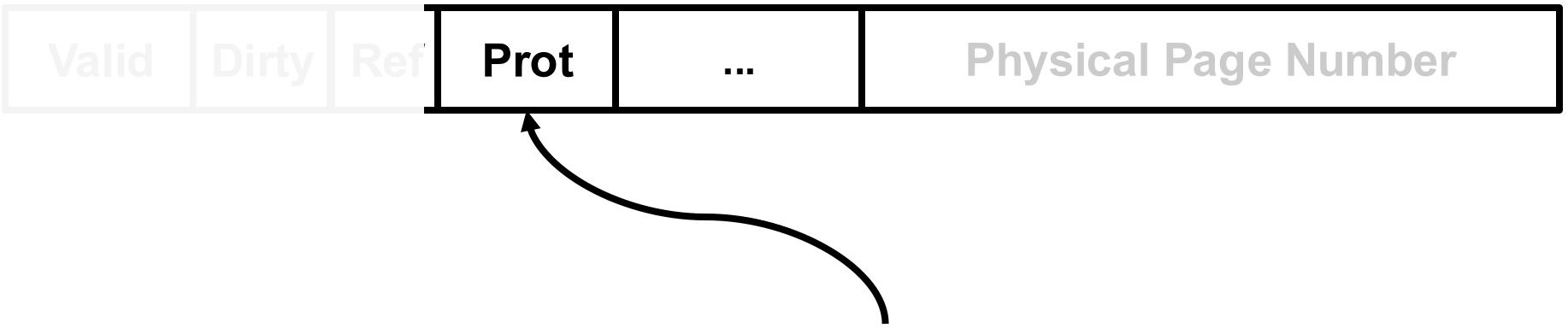
- **Page Table** contains an entry for each virtual page
  - Called **Page Table Entry (PTE)**
  - What is in a PTE?

What Is in a Page Table Entry (PTE)?



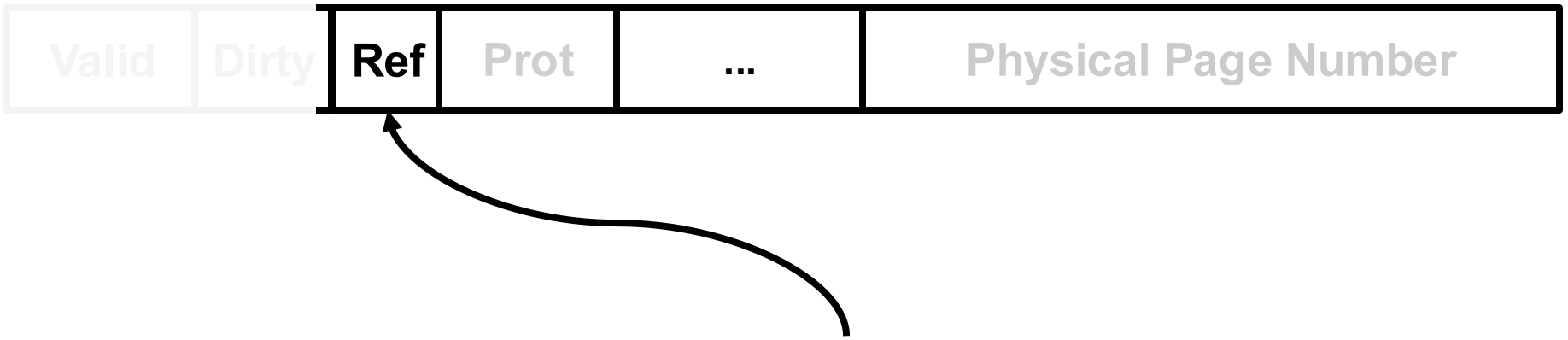
Physical page number the data is stored in (if valid)

What Is in a Page Table Entry (PTE)?



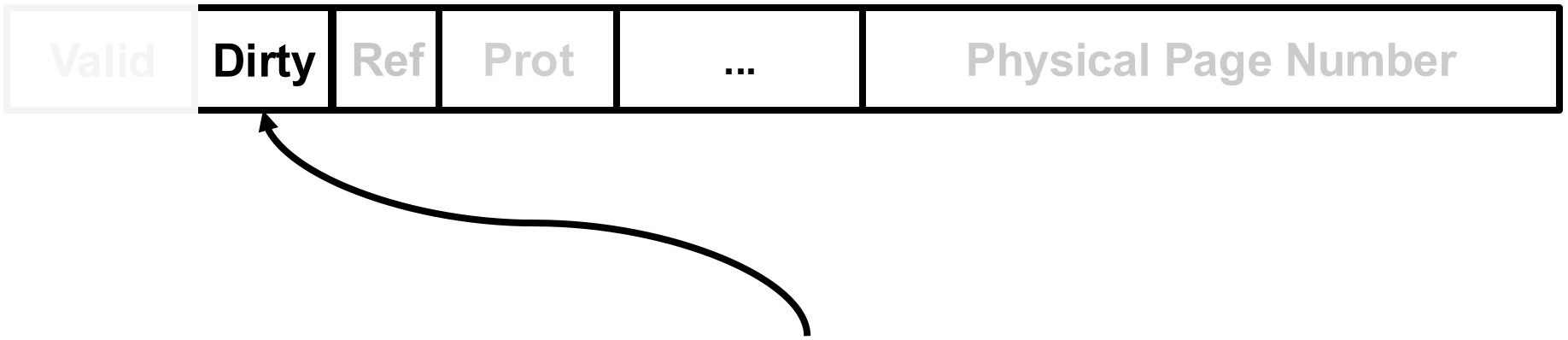
***Prot bits*: Protection or access control bits**

What Is in a Page Table Entry (PTE)?



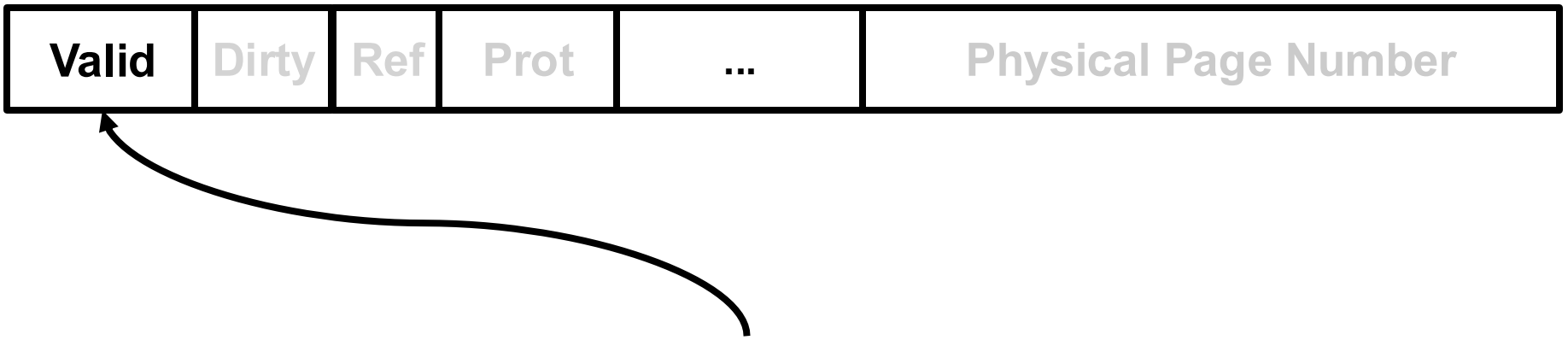
***Ref bits*: Reference or access frequency bits (e.g., did we recently access this page?)**

What Is in a Page Table Entry (PTE)?



Dirty bit: **Do we need to write the page to the disk if evicted from memory?**

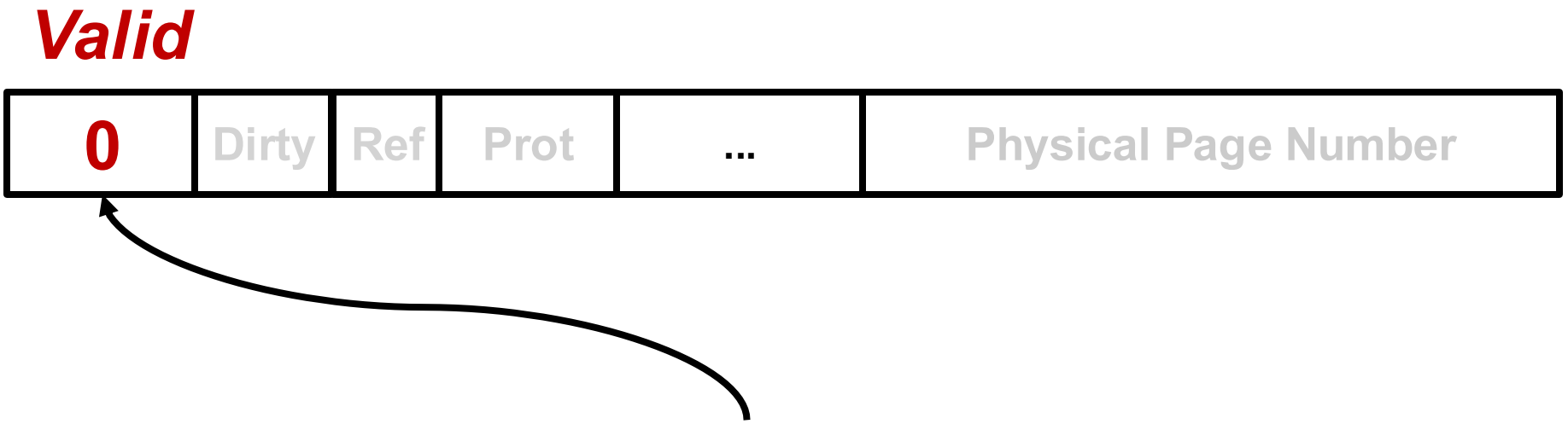
What Is in a Page Table Entry (PTE)?



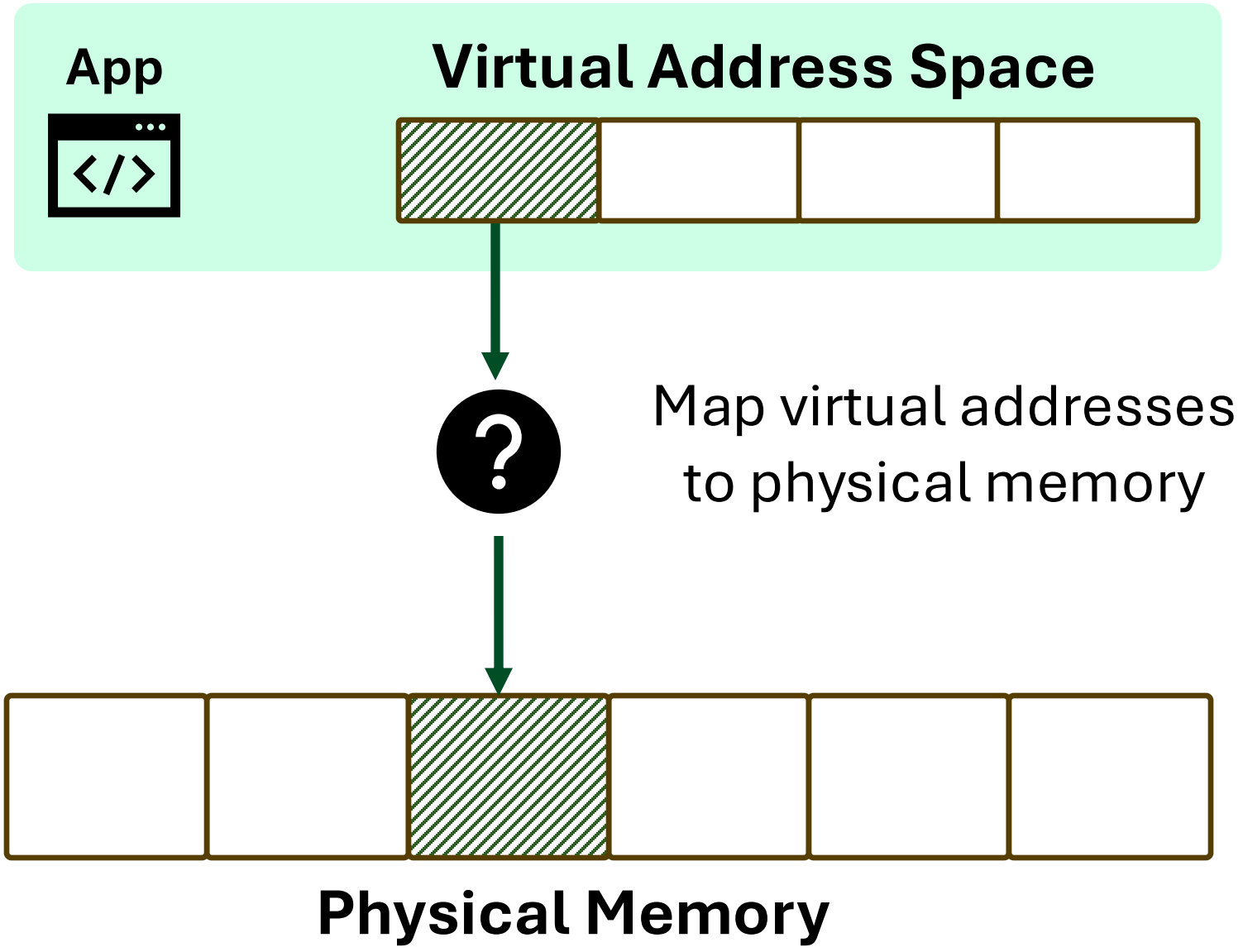
***Valid:* Is the page actually stored in the main memory?**

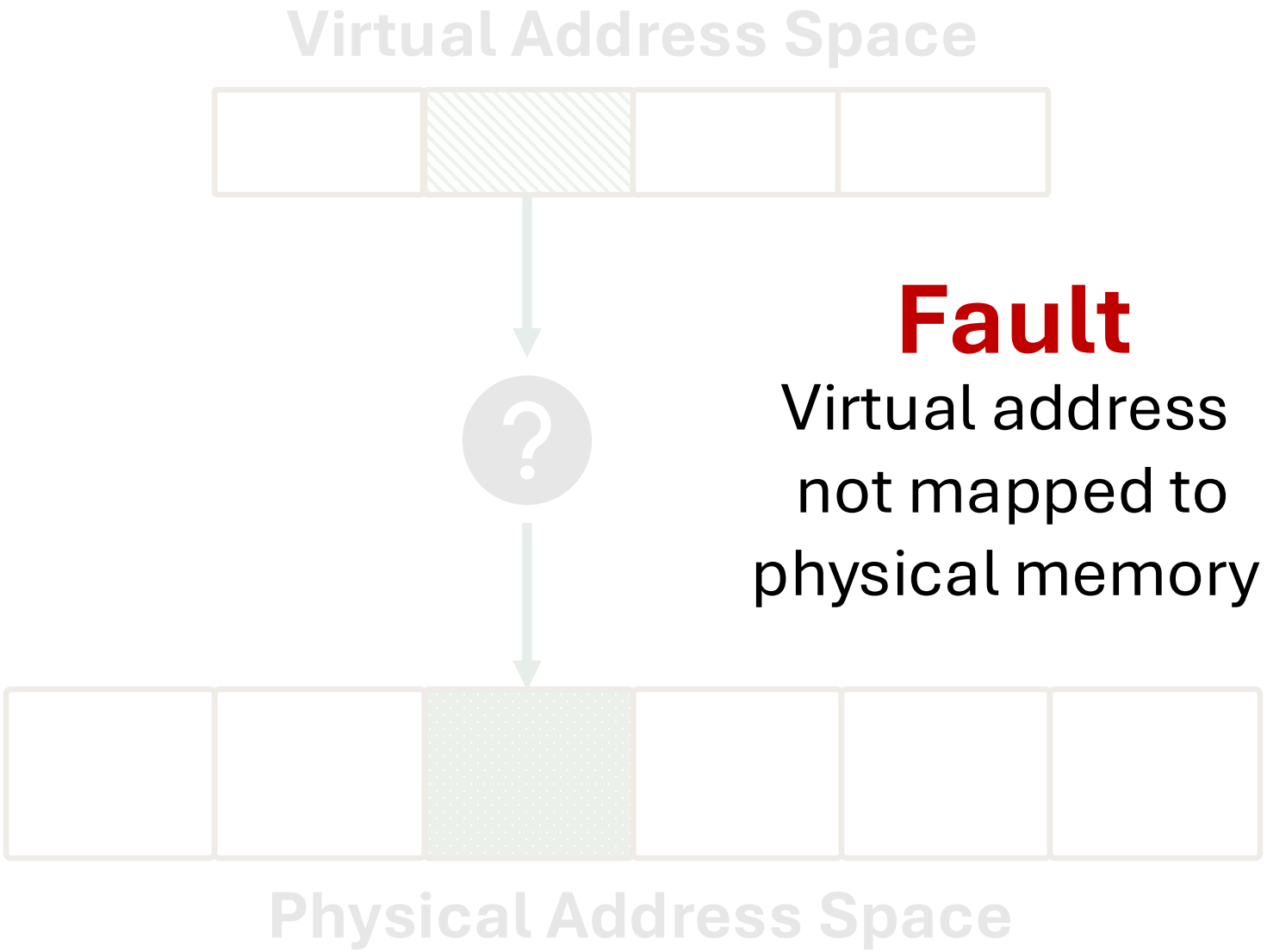
Who establishes virtual-to-physical address mappings?

What Is in a Page Table Entry (PTE)?



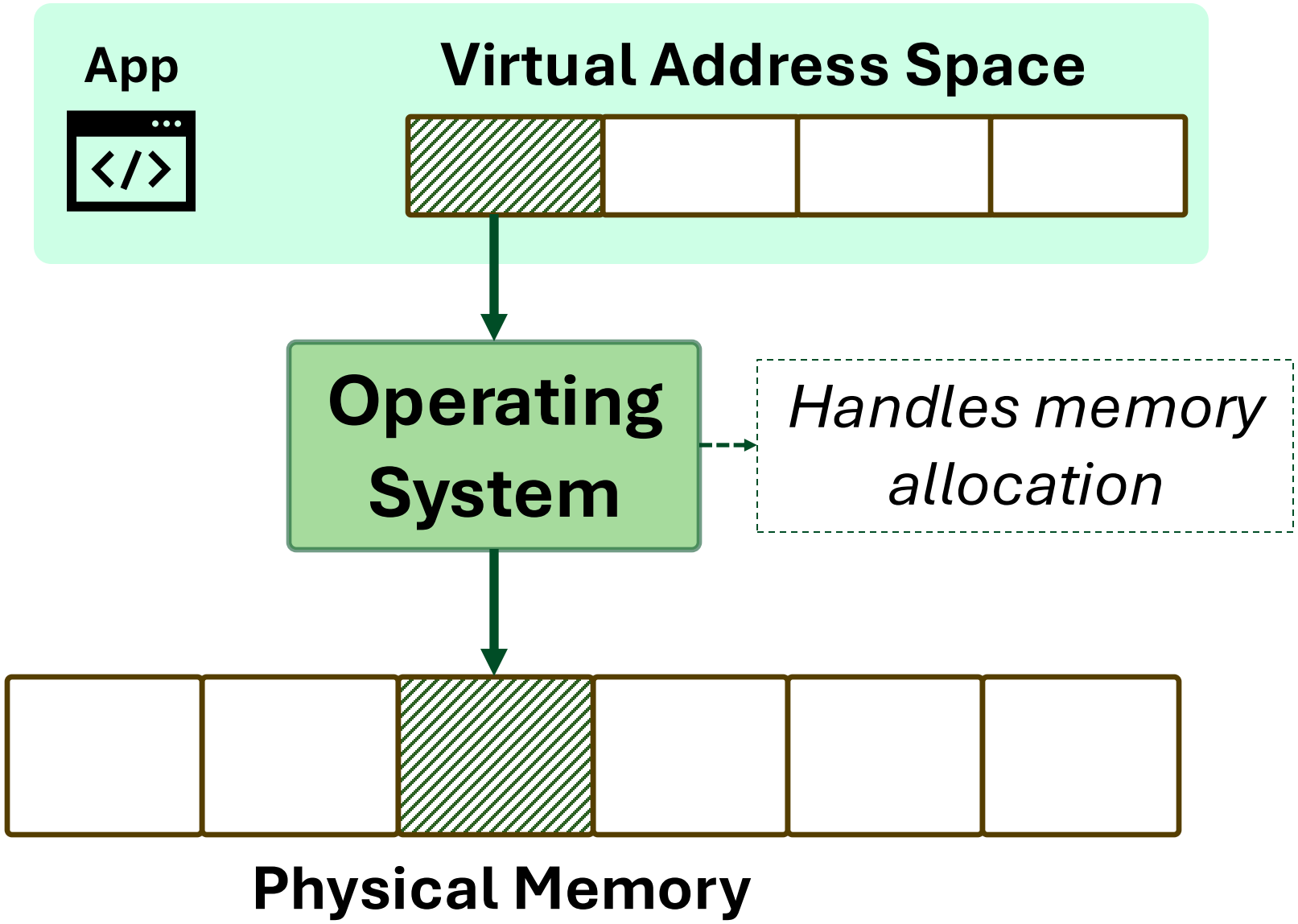
Valid bit is 0:
The virtual-to-physical mapping is not available

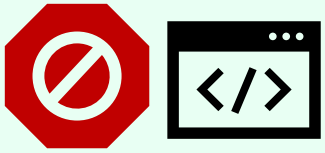

App
Virtual Address Space
?
Map virtual addresses to physical memory
Physical Memory



Fault

Virtual address not mapped to physical memory


App
Virtual Address Space
Operating System
Handles memory allocation
Physical Memory



Application is stalled

Operating System

***Minor** Fault*

Finds free space in physical memory

Order of ~µs

Minor Fault: No Need to Fetch Data from The Disk

- If the data has not been assigned a location in physical memory yet but does not need to be fetched from the disk
  - Page table entry indicates virtual page is not valid
  - Access to such a page triggers a page fault exception
  - OS exception handler invoked to handle the data mapping
    - Other processes can continue executing
    - OS has full control over page placement

If data does not need to fetched from disk, what purpose do minor faults serve?

Common Types of Minor Page Faults (I)

- **Anonymous pages:** Pages not initialized yet but also not backed by a file
  - e.g., Ephemeral data used by an application

Example

```
Data = malloc(size); // Anon
Data[0]= 0; // PAGE FAULT
```

Common Types of Minor Page Faults (II)

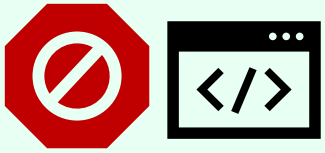
- **Copy-on-Write (CoW):** Multiple processes initially share the same memory pages, but when a process attempts to modify a shared page, a page fault occurs.

- Triggers the creation of a separate copy of the page for the modifying process.

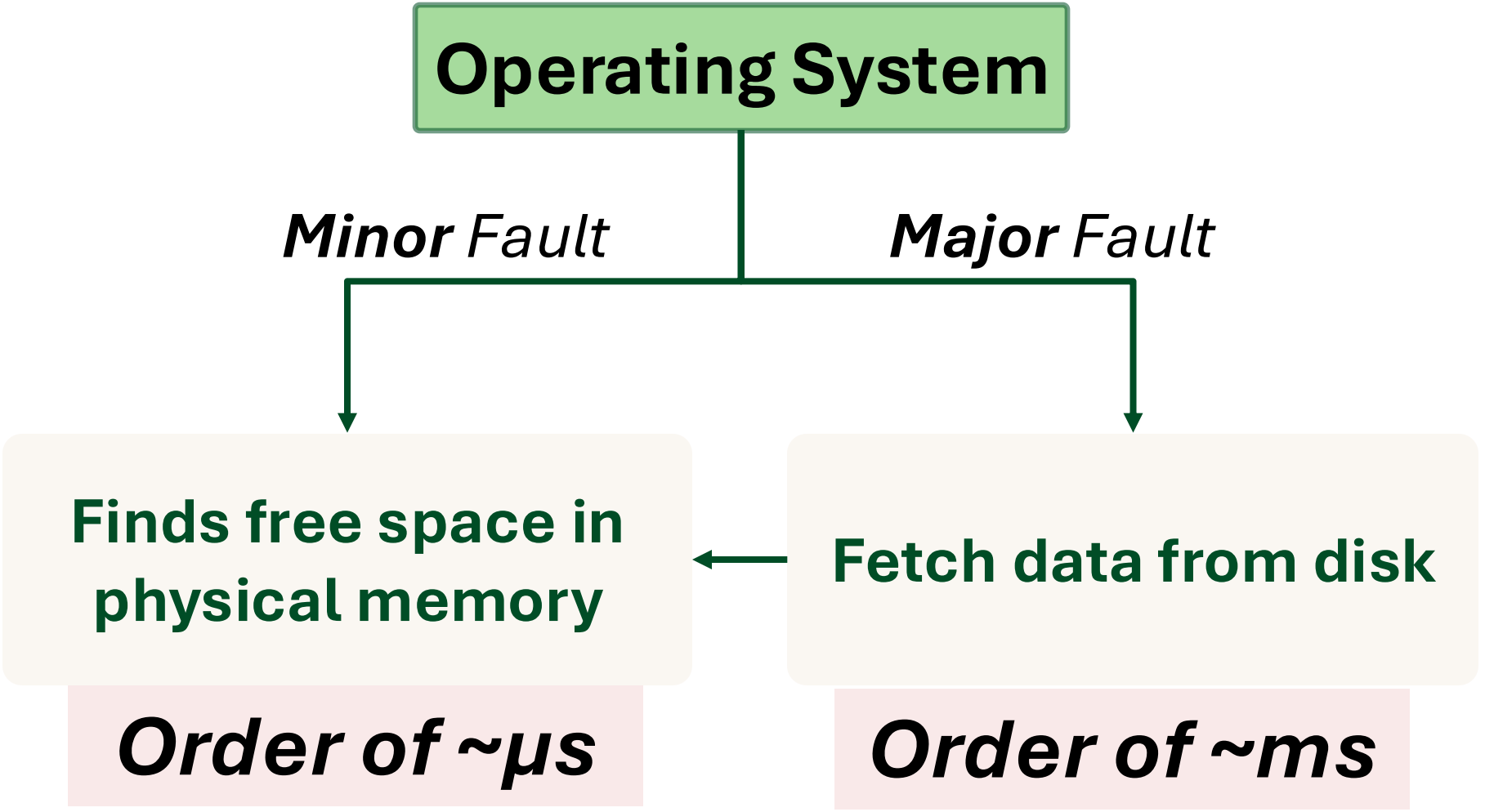
Example

```
x = 10;
pid_t pid = fork(); // Parent/child share page

if (pid == 0)
  x = 20; // Child writes → triggers CoW
```

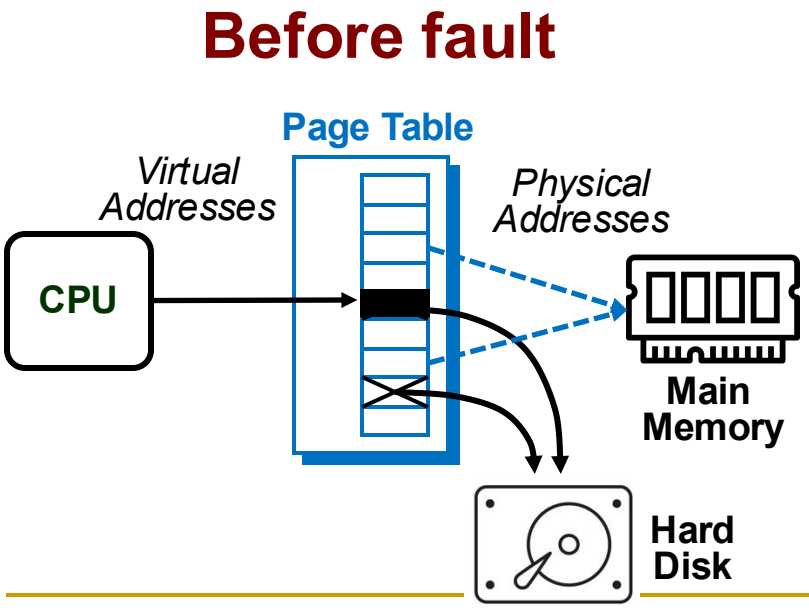


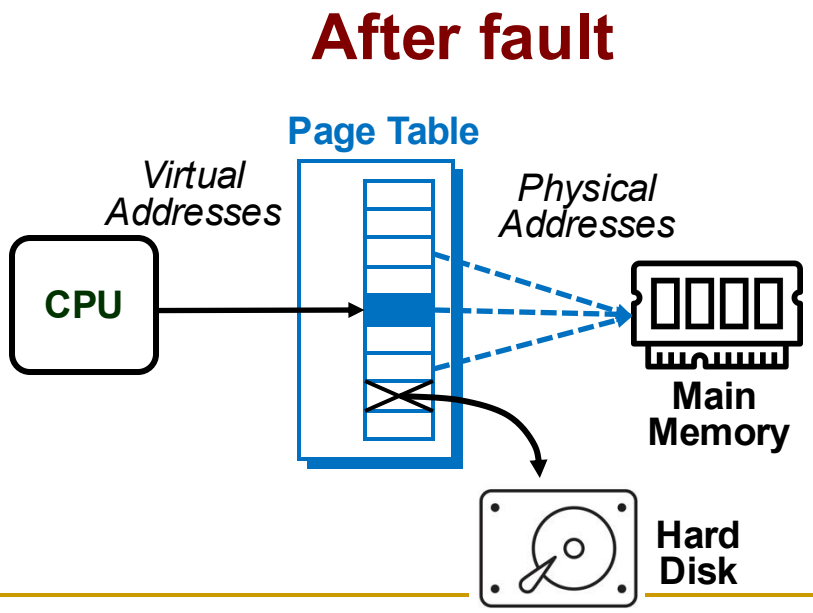
Application is stalled



Major Page Fault ("A Miss in Physical Memory")

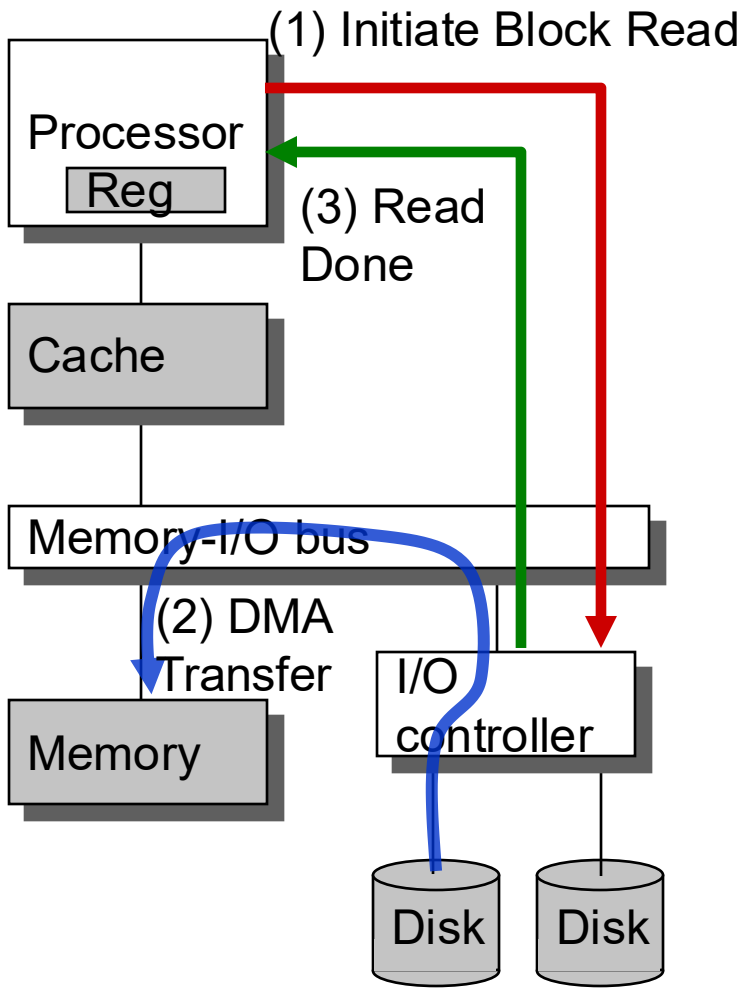
- If a page is not in physical memory but disk
  - Page table entry indicates virtual page not in memory
  - Access to such a page triggers a page fault exception
  - OS exception handler invoked to move data from disk into memory
    - Other processes can continue executing
    - OS has full control over page placement





Servicing a Major Page Fault

1. Processor signals I/O controller
   - Read block of length P starting at disk address X and store starting at memory address Y
2. Disk-to-mem read occurs
   - Direct Memory Access (DMA)
   - Under control of I/O controller
3. Controller signals completion
   - Interrupts processor
   - OS resumes suspended process



Real Life Example: Warehouse



Page Fault = Item Not Currently in Warehouse

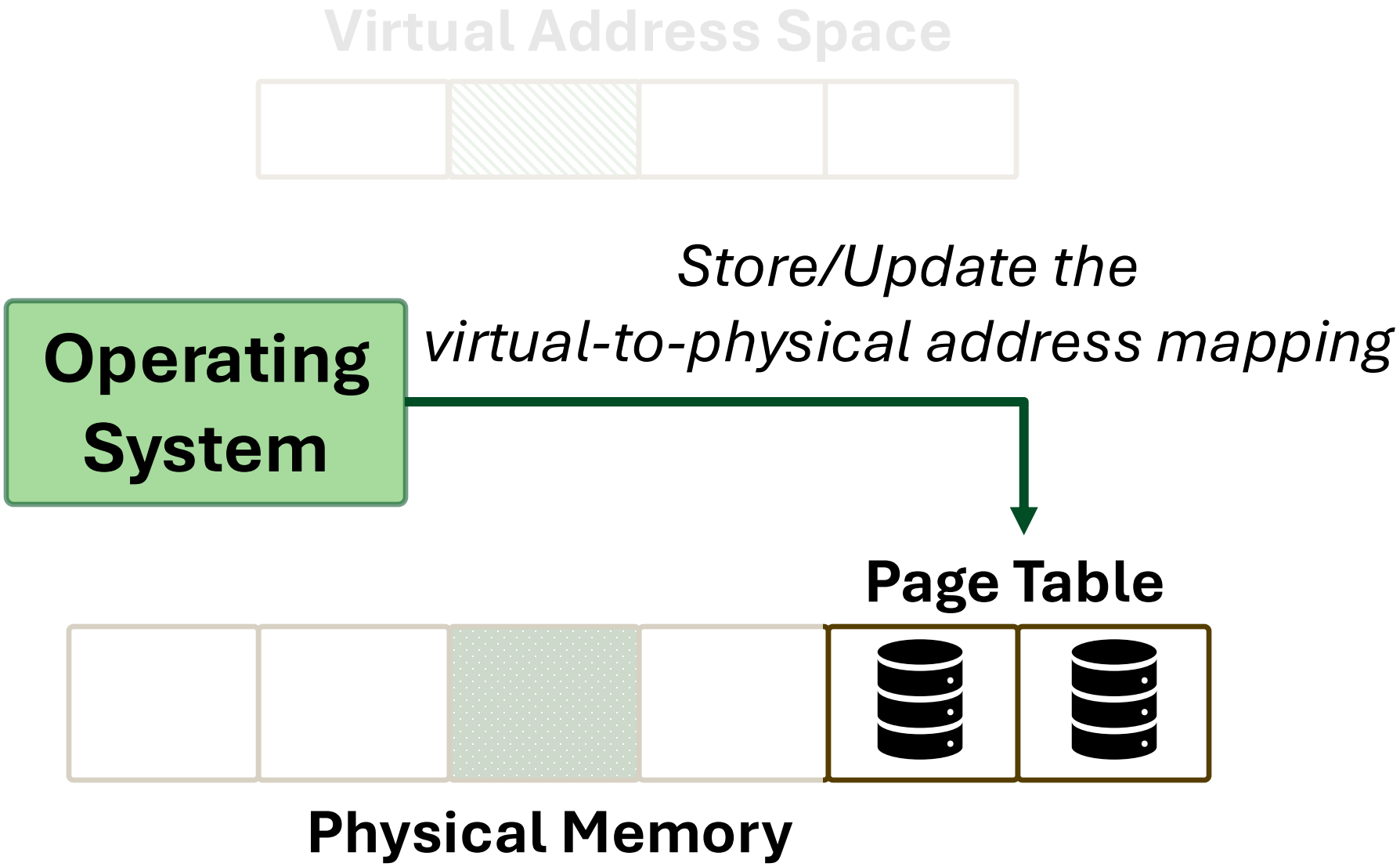
Sometimes the barcode is valid, but the item isn't in the main warehouse:

- It might be in deep storage.
- The warehouse robot must fetch it or make space.
- Workers must wait before accessing it.

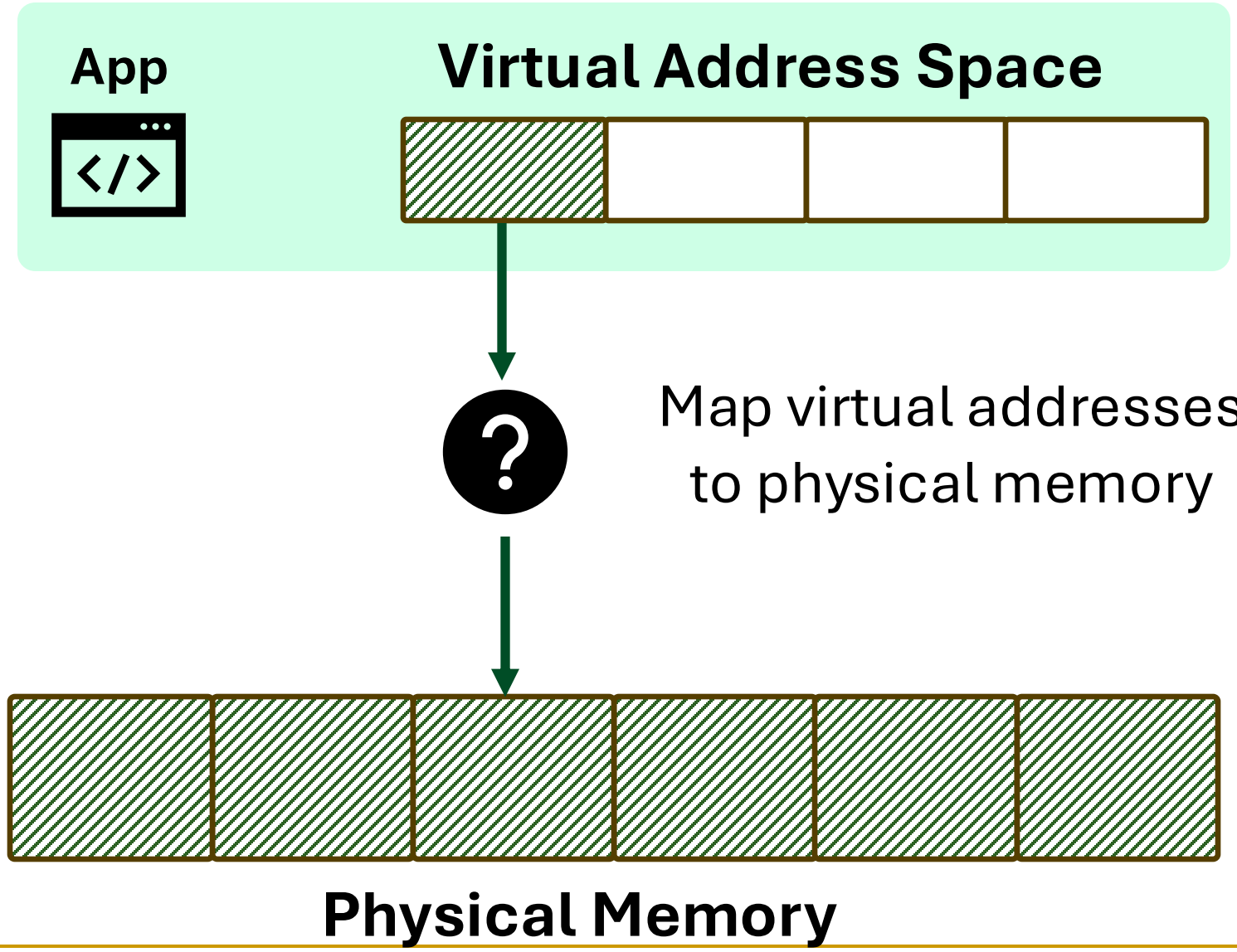
This is a **page fault**:

- The virtual address is valid but not currently mapped to main memory.
- OS fetches the page from disk in some cases.
- The page table is updated, and work continues.

Accessing swapped-out pages is slow.


Virtual Address Space
Store/Update the
virtual-to-physical address mapping
Operating System
Page Table
Physical Memory

Memory is Full



Page Replacement Algorithms

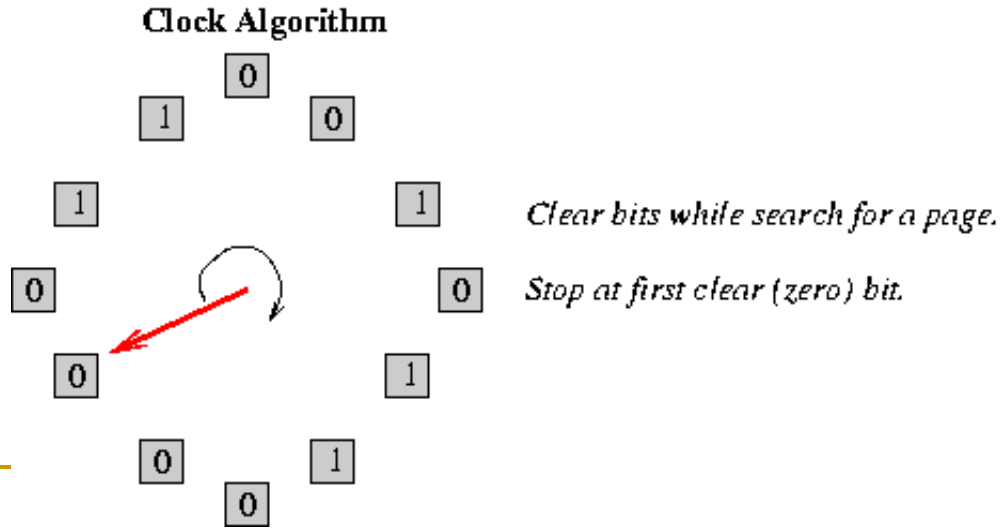
- If physical memory is full (i.e., list of free physical pages is empty), which physical frame to replace on a page fault?

- Is True LRU feasible?
  - 4GB memory, 4KB pages, how many possibilities of ordering?

- Modern systems use approximations of LRU
  - E.g., the CLOCK algorithm
- And, more sophisticated algorithms to take into account "frequency" of use
  - E.g., the ARC algorithm
  - Megiddo and Modha, "ARC: A Self-Tuning, Low Overhead Replacement Cache," FAST 2003.

CLOCK Page Replacement Algorithm

- Keep a circular list of physical frames in memory (OS does)
- Keep a pointer (hand) to the last-examined frame in the list
- When a page is accessed, set the R bit in the PTE
- When a frame needs to be replaced, replace the first frame that has the reference (R) bit not set, traversing the circular list starting from the pointer (hand) clockwise
  - During traversal, clear the R bits of examined frames
  - Set the hand pointer to the next frame in the list



Cache versus Page Replacement

- Physical memory (DRAM) is a cache for disk
  - Managed by system software via the virtual memory subsystem

- Page replacement is similar to cache replacement
- Page table is the "tag store" for physical memory data store

- What is the difference?
  - Required speed of access to cache vs. physical memory
  - Number of blocks in a cache vs. physical memory
  - "Tolerable" amount of time to find a replacement candidate (disk versus memory access latency)
  - Role of hardware versus software

Cache/Virtual Memory Analogues

Cache	Virtual Memory
Block	Page
Block Size	Page Size
Block Offset	Page Offset
Miss	Page Fault
Index	Virtual Page Number
Metadata (Tag) Store	Page Table
Data Store	Physical Memory

Memory Protection

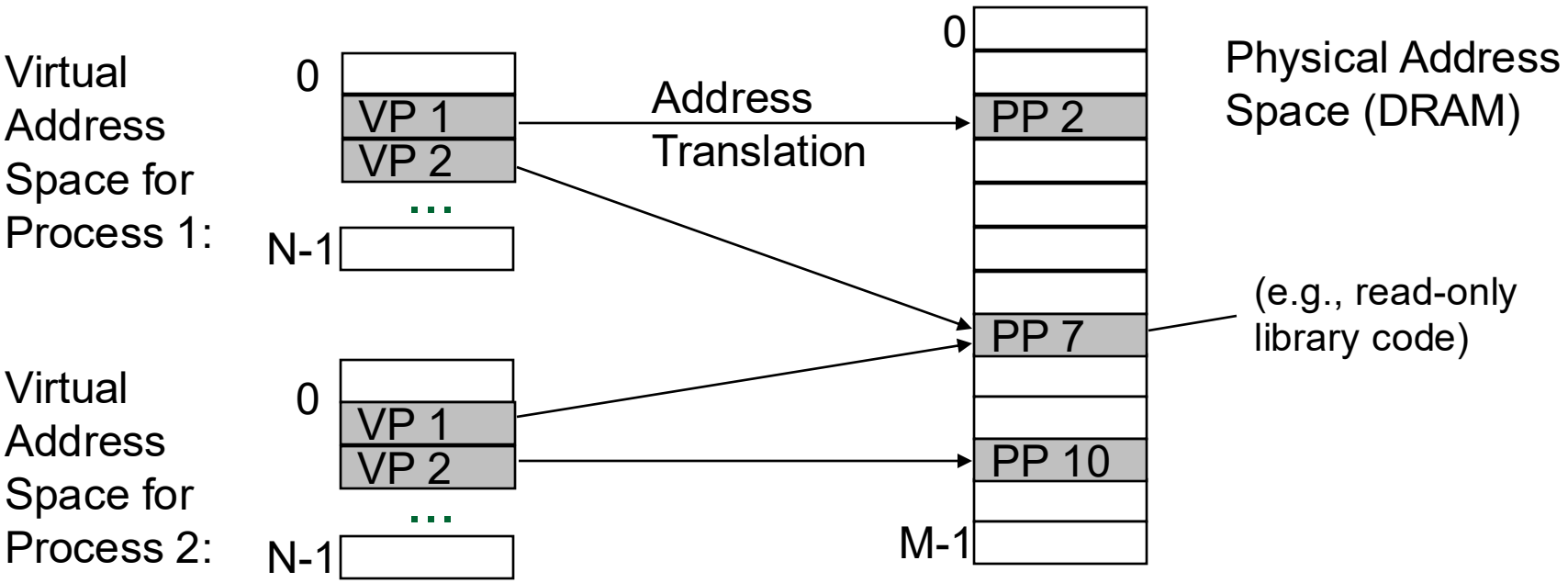
Memory Protection

- Multiple programs (i.e., processes) run concurrently
  - Each process has its own page table
  - Each process can use its entire virtual address space without worrying about where other programs are

- A process can only access physical pages mapped in its page table – cannot overwrite memory of another process
  - Provides protection and isolation between processes
  - Enables access control mechanisms per page

Page Table is Per Process

- Each process has its own virtual address space
  - Full address space for each program
  - Simplifies memory allocation, sharing, linking and loading



Access Protection/Control via Virtual Memory

Real Life Example: Warehouse



Memory Isolation = Workers Can't Access Wrong Items

Workers are issued permissions:

- Some can only pick from Section A.
- Some can't open locked bins.
- Some can access restricted aisles.

This is **memory protection**:

- Processes cannot access memory they do not own.
- Even if two programs use the same virtual address, they map to different physical pages.

Page-Level Access Control (Protection)

- Not every process is allowed to access every page
  - E.g., need supervisor (i.e., kernel) level privilege to access system pages
  - E.g., may not be able to execute "instructions" in some pages

- Idea: Store access control information on a page basis in the process's page table

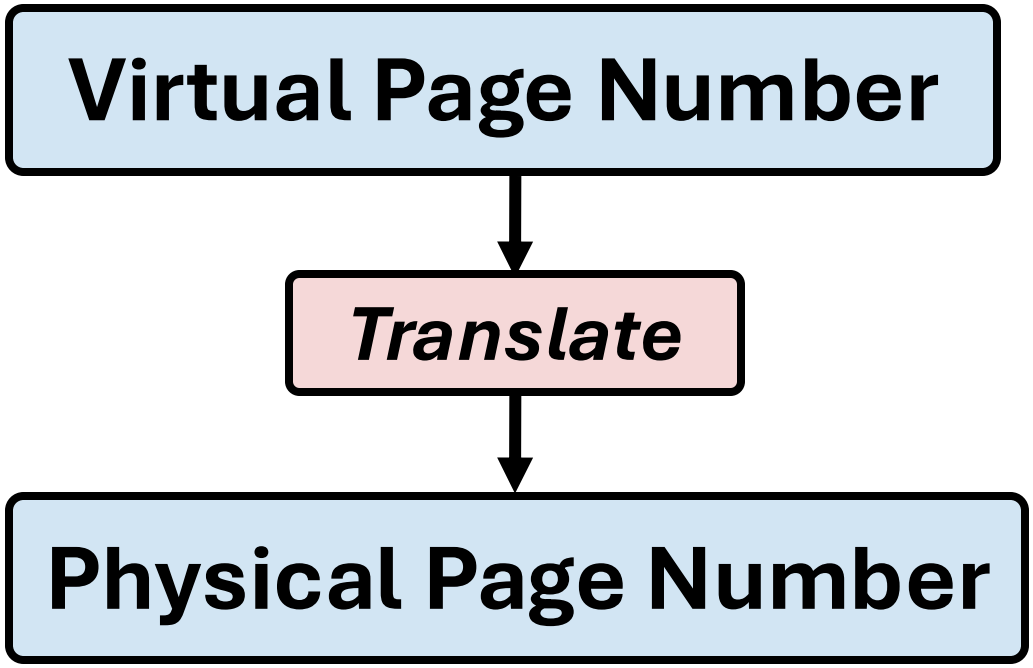
- Enforce access control at the same time as translation

→ Virtual memory system serves two functions today

Address translation (for illusion of large physical memory)

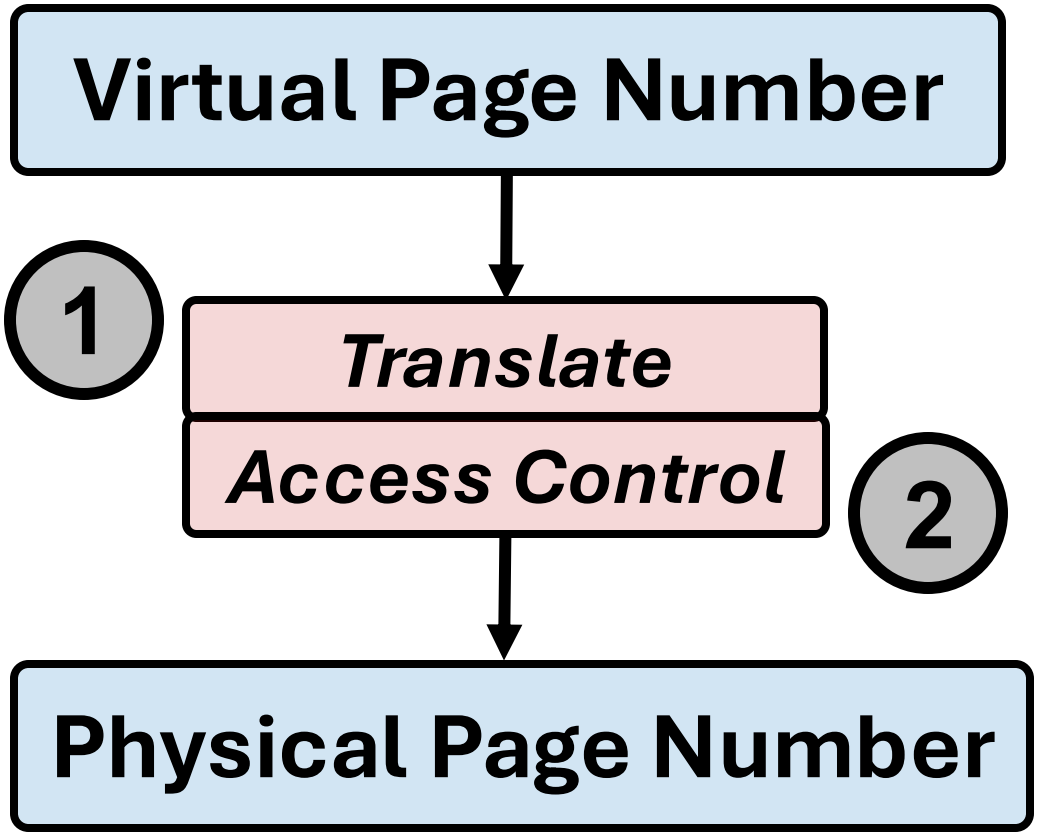
Access control (protection)

Two Functions of Virtual Memory



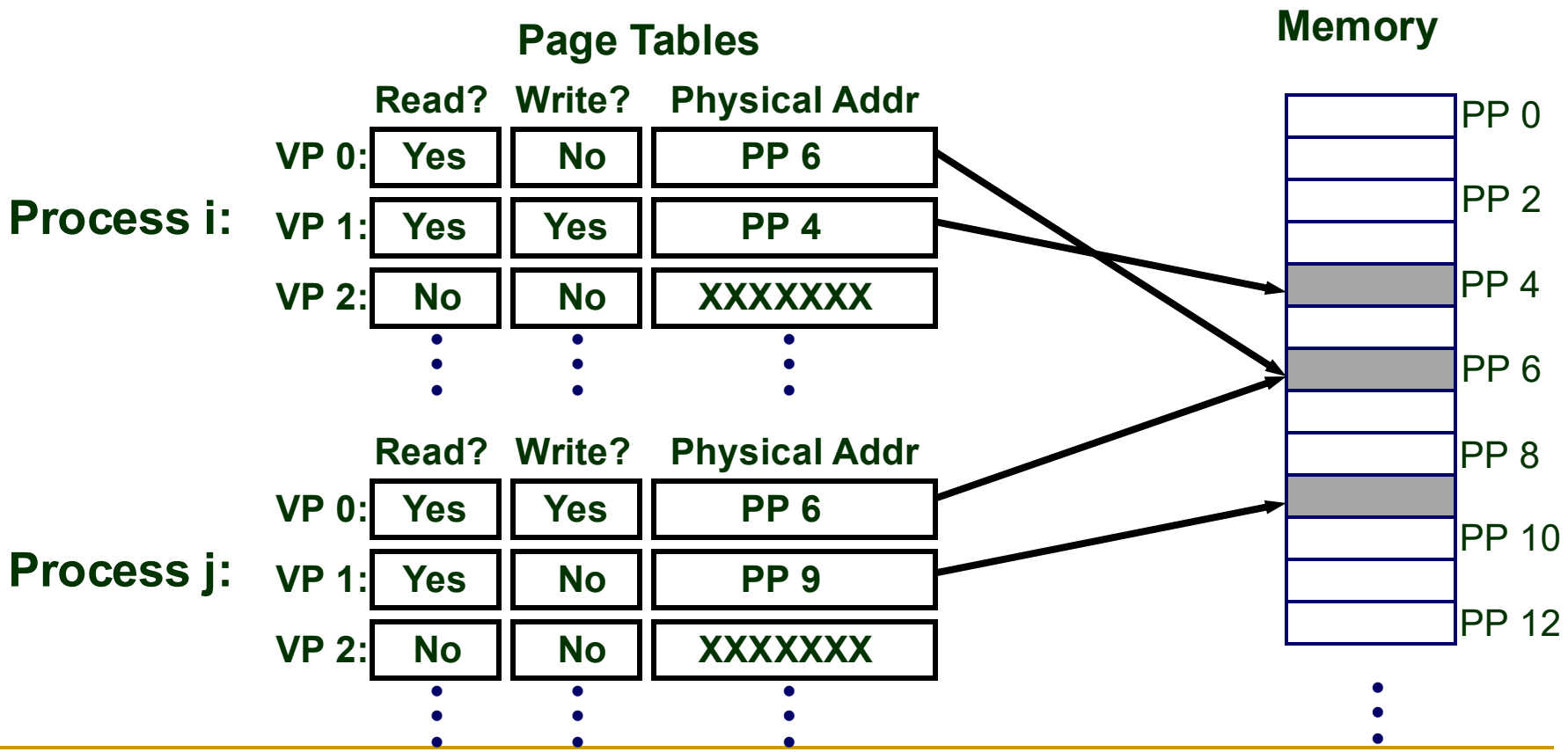
Modern systems rely on VM for one more functionality: **Memory Protection**

Two Functions of Virtual Memory



VM as a Tool for Memory Access Protection

- Extend Page Table Entries (PTEs) with permission bits
- Check bits on each access and during a page fault
  - If violated, generate exception (Access Protection exception)



Privilege Levels in x86

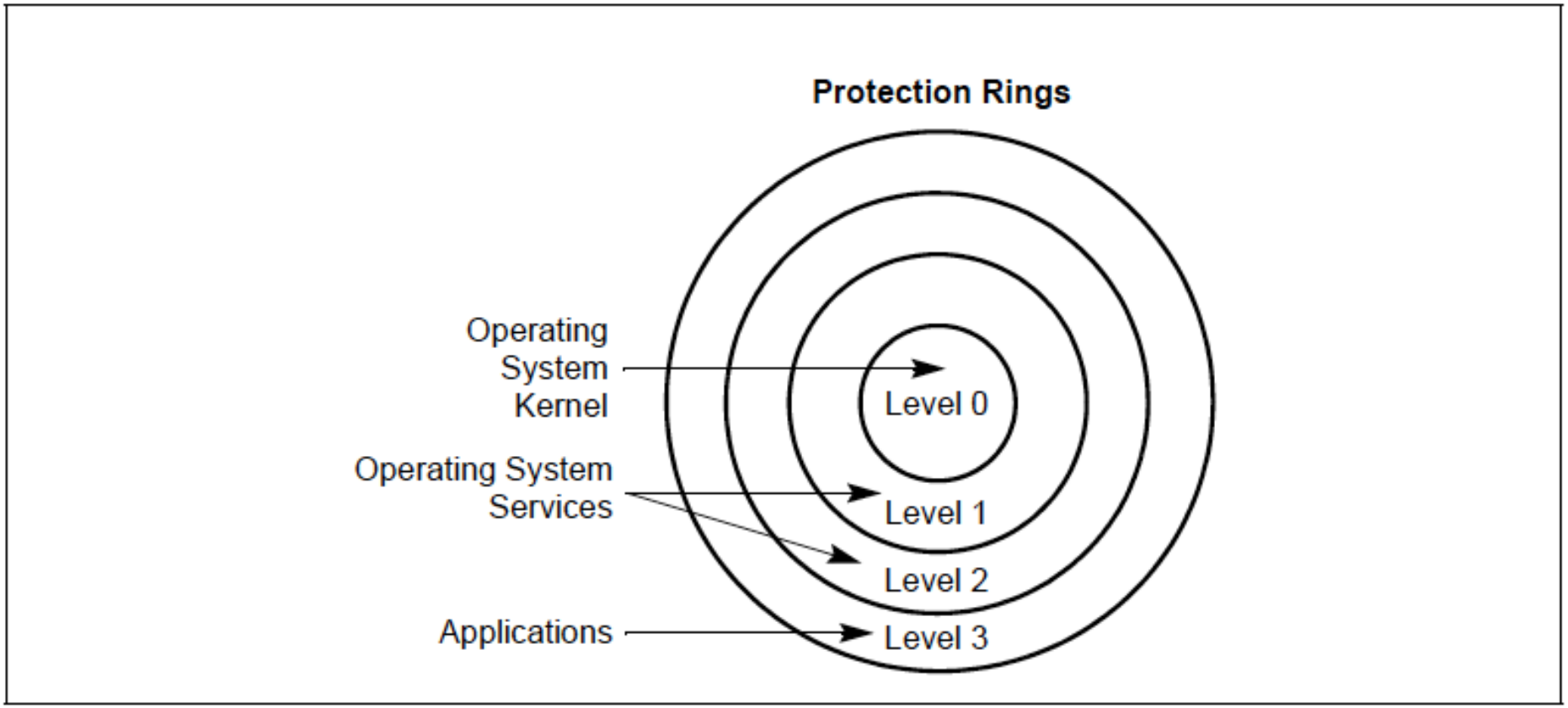


Figure 5-3. Protection Rings

Privilege Levels in x86

- Four **privilege levels** in x86 (referred to as **rings**)
  - Ring 0: Highest privilege (operating system)
  - Ring 1: Not widely used
  - Ring 2: Not widely used
  - Ring 3: Lowest privilege (user applications)

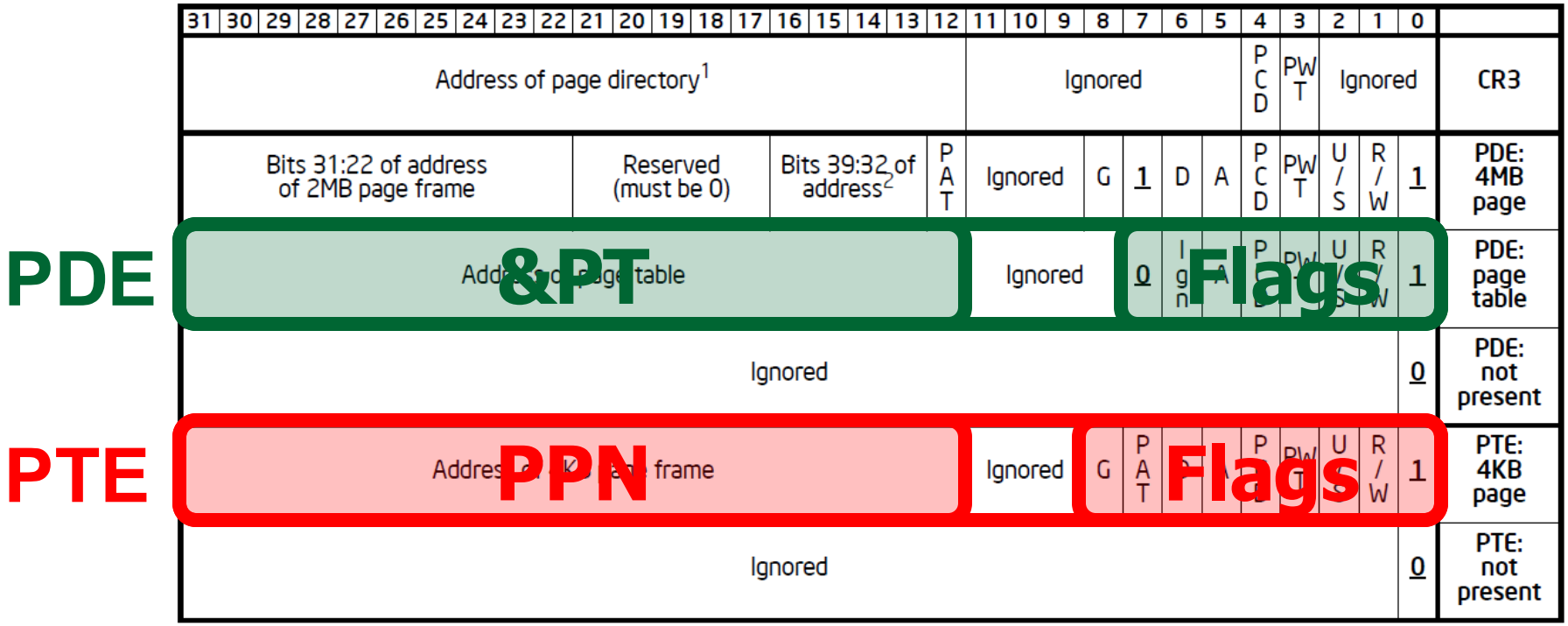
"Supervisor"

"User"

- Supervisor = Kernel (in modern terminology)

x86: A Closer Look at the PDE/PTE

- **PDE:** Page Directory Entry (32 bits)
- **PTE:** Page Table Entry (32 bits)



31–12	11–9	8	7	6	5	4	3	2	1	0	
Address of page directory[1]	Ignored					PCD	PWT	Ignored			CR3
Bits 31:22 of address of 2MB page frame	Reserved (must be 0)	Bits 39:32 of address[2]	PAT	Ignored	G	1	D	A	PCD	PWT	U/S	R/W	1	PDE: 4MB page
Address of page table	Ignored												1	PDE: page table
Ignored													0	PDE: not present
Address of 4KB page frame	Ignored		G	PAT							U/S	R/W	1	PTE: 4KB page
Ignored													0	PTE: not present

Figure 4-4. Formats of CR3 and Paging-Structure Entries with 32-Bit Paging

Protection: PDE's Flags

- Protects all 1024 pages in a page table

Table 4-5. Format of a 32-Bit Page-Directory Entry that References a Page Table

Bit Position(s)	Contents
0 (P)	Present; must be 1 to reference a page table
1 (R/W)	Read/write; if 0, writes may not be allowed to the 4-MByte region controlled by this entry (see Section 4.6)
2 (U/S)	User/supervisor; if 0, user-mode accesses are not allowed to the 4-MByte region controlled by this entry (see Section 4.6)
3 (PWT)	Page-level write-through; indirectly determines the memory type used to access the page table referenced by this entry (see Section 4.9)
4 (PCD)	Page-level cache disable; indirectly determines the memory type used to access the page table referenced by this entry (see Section 4.9)
5 (A)	Accessed; indicates whether this entry has been used for linear-address translation (see Section 4.8)
6	Ignored
7 (PS)	If CR4.PSE = 1, must be 0 (otherwise, this entry maps a 4-MByte page; see Table 4-4); otherwise, ignored

Protection: PTE's Flags

- Protects one page at a time

Table 4-6. Format of a 32-Bit Page-Table Entry that Maps a 4-KByte Page

Bit Position(s)	Contents
0 (P)	Present; must be 1 to map a 4-KByte page
1 (R/W)	Read/write; if 0, writes may not be allowed to the 4-KByte page referenced by this entry (see Section 4.6)
2 (U/S)	User/supervisor; if 0, user-mode accesses are not allowed to the 4-KByte page referenced by this entry (see Section 4.6)
3 (PWT)	Page-level write-through; indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9)
4 (PCD)	Page-level cache disable; indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9)
5 (A)	Accessed; indicates whether software has accessed the 4-KByte page referenced by this entry (see Section 4.8)
6 (D)	Dirty; indicates whether software has written to the 4-KByte page referenced by this entry (see Section 4.8)
7 (PAT)	If the PAT is supported, indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9.2); otherwise, reserved (must be 0)[1]
8 (G)	Global; if CR4.PGE = 1, determines whether the translation is global (see Section 4.10); ignored otherwise

Page Level Protection in x86

Table 5-3. Combined Page-Directory and Page-Table Protection

Page-Directory Entry		Page-Table Entry		Combined Effect	
Privilege	Access Type	Privilege	Access Type	Privilege	Access Type
User	Read-Only	User	Read-Only	User	Read-Only
User	Read-Only	User	Read-Write	User	Read-Only
User	Read-Write	User	Read-Only	User	Read-Only
User	Read-Write	User	Read-Write	User	Read/Write
User	Read-Only	Supervisor	Read-Only	Supervisor	Read/Write*
User	Read-Only	Supervisor	Read-Write	Supervisor	Read/Write*
User	Read-Write	Supervisor	Read-Only	Supervisor	Read/Write*
User	Read-Write	Supervisor	Read-Write	Supervisor	Read/Write
Supervisor	Read-Only	User	Read-Only	Supervisor	Read/Write*
Supervisor	Read-Only	User	Read-Write	Supervisor	Read/Write*
Supervisor	Read-Write	User	Read-Only	Supervisor	Read/Write*
Supervisor	Read-Write	User	Read-Write	Supervisor	Read/Write
Supervisor	Read-Only	Supervisor	Read-Only	Supervisor	Read/Write*
Supervisor	Read-Only	Supervisor	Read-Write	Supervisor	Read/Write*
Supervisor	Read-Write	Supervisor	Read-Only	Supervisor	Read/Write*
Supervisor	Read-Write	Supervisor	Read-Write	Supervisor	Read/Write

Protection: PDE + PTE = ???

Table 5-3. Combined Page-Directory and Page-Table Protection

Page-Directory Entry		Page-Table Entry		Combined Effect	
Privilege	Access Type	Privilege	Access Type	Privilege	Access Type
User	Read-Only	User	Read-Only	User	Read-Only
User	Read-Only	User	Read-Write	User	Read-Only
User	Read-Write	User	Read-Only	User	Read-Only
User	Read-Write	User	Read-Write	User	Read/Write
User	Read-Only	Supervisor	Read-Only	Supervisor	Read/Write*
User	Read-Only	Supervisor	Read-Write	Supervisor	Read/Write*
User	Read-Write	Supervisor	Read-Only	Supervisor	Read/Write*
User	Read-Write	Supervisor	Read-Write	Supervisor	Read/Write
Supervisor	Read-Only	User	Read-Only	Supervisor	Read/Write*
Supervisor	Read-Only	User	Read-Write	Supervisor	Read/Write*
Supervisor	Read-Write	User	Read-Only	Supervisor	Read/Write*
Supervisor	Read-Write	User	Read-Write	Supervisor	Read/Write
Supervisor	Read-Only	Supervisor	Read-Only	Supervisor	Read/Write*
Supervisor	Read-Only	Supervisor	Read-Write	Supervisor	Read/Write*
Supervisor	Read-Write	Supervisor	Read-Only	Supervisor	Read/Write*
Supervisor	Read-Write	Supervisor	Read-Write	Supervisor	Read/Write

NOTE:

* If CR0.WP = 1, access type is determined by the R/W flags of the page-directory and page-table entries. IF CR0.WP = 0, supervisor privilege permits read-write access.

Food for Thought: What If?

- Your hardware is unreliable and someone can flip the access protection bits
  - such that a user-level program can gain supervisor-level access (i.e., access to all data on the system)
  - by flipping the access control bit from user to supervisor!

- Can this happen?

Remember RowHammer?

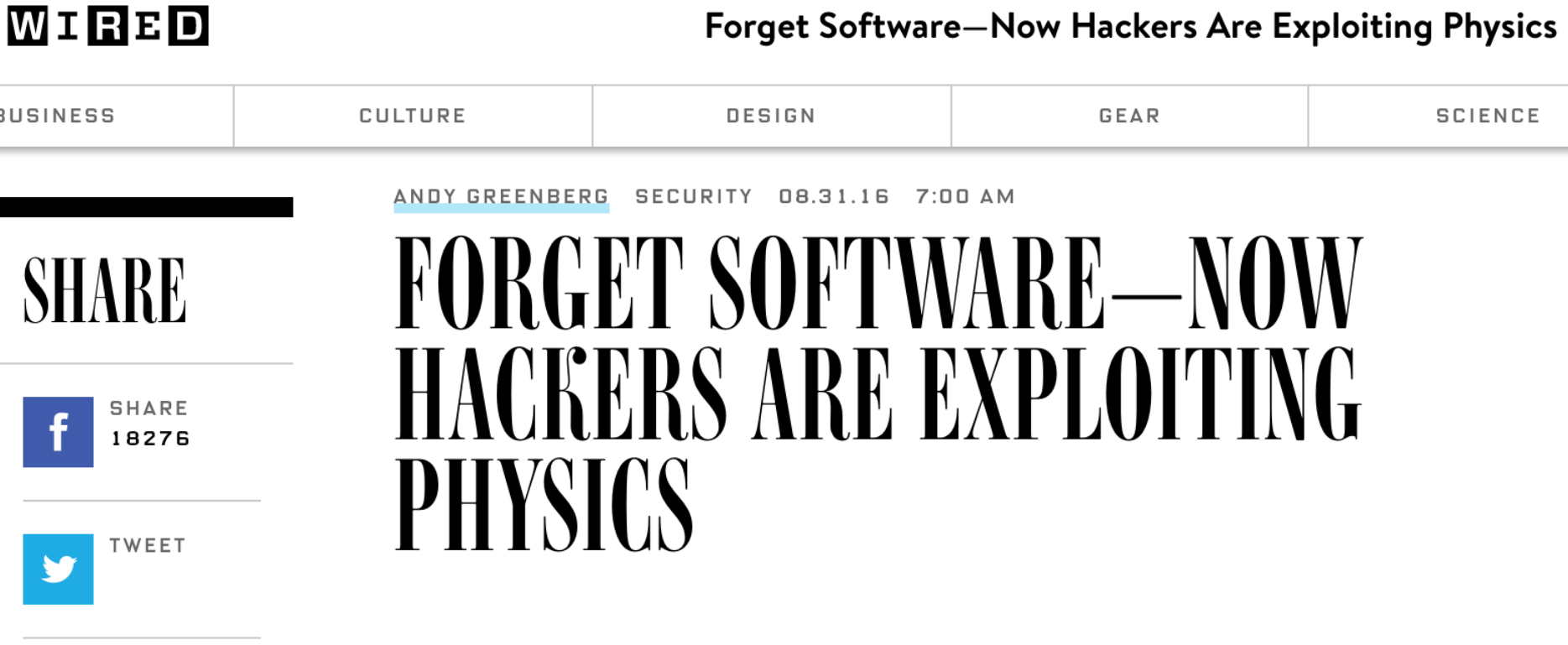
One can

predictably induce errors

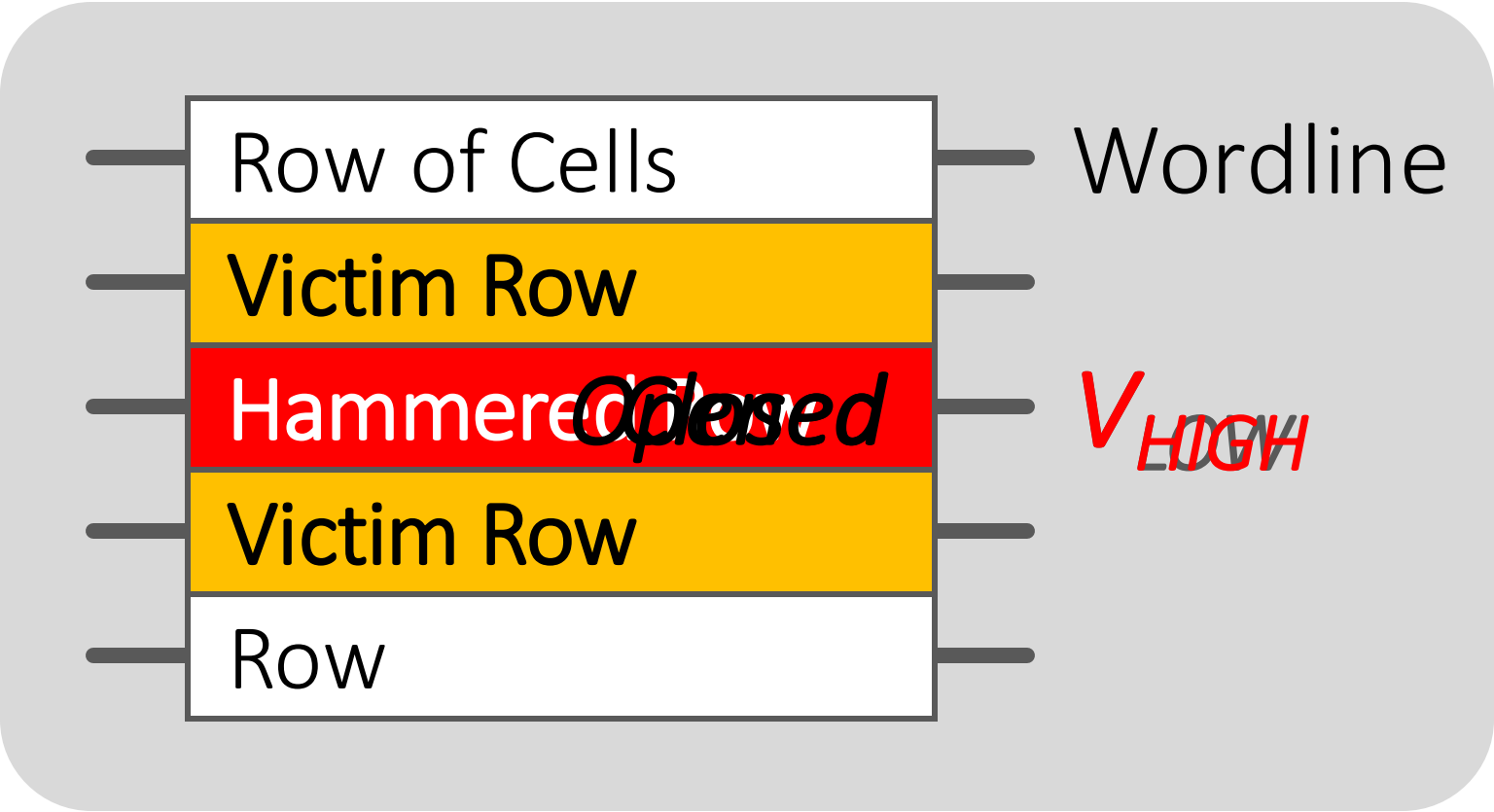
in most DRAM memory chips

Remember RowHammer?

- One can predictably induce bit flips in commodity DRAM chips
  - >80% of the tested DRAM chips are vulnerable
- First example of how a simple hardware failure mechanism can create a widespread system security vulnerability



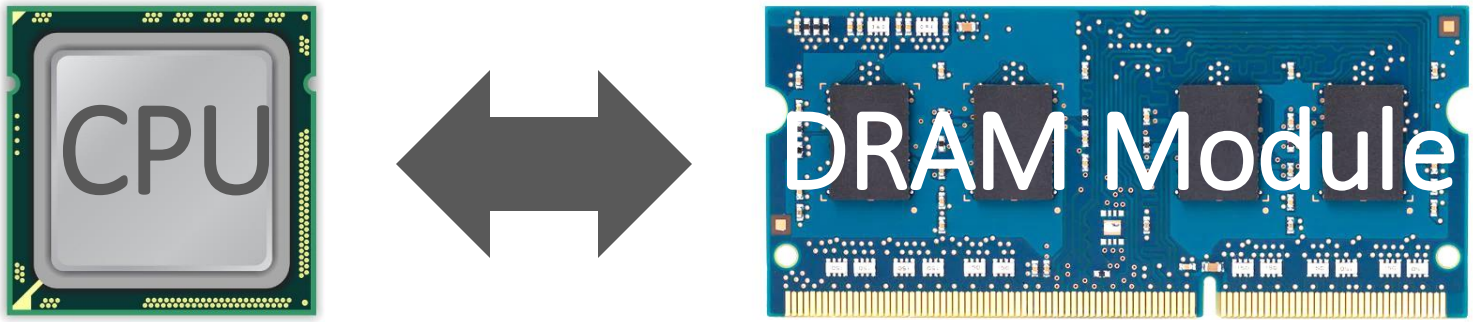
Modern DRAM is Prone to Disturbance Errors



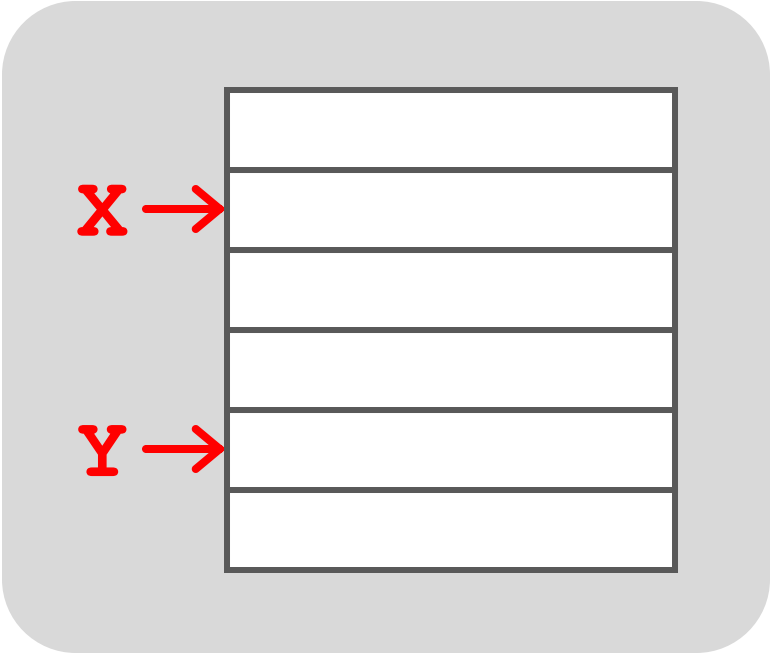
Repeatedly reading a row enough times (before memory gets refreshed) induces disturbance errors in **adjacent rows** in most real DRAM chips you can buy today

Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors, (Kim et al., ISCA 2014)

A Simple Program Can Induce Many Errors

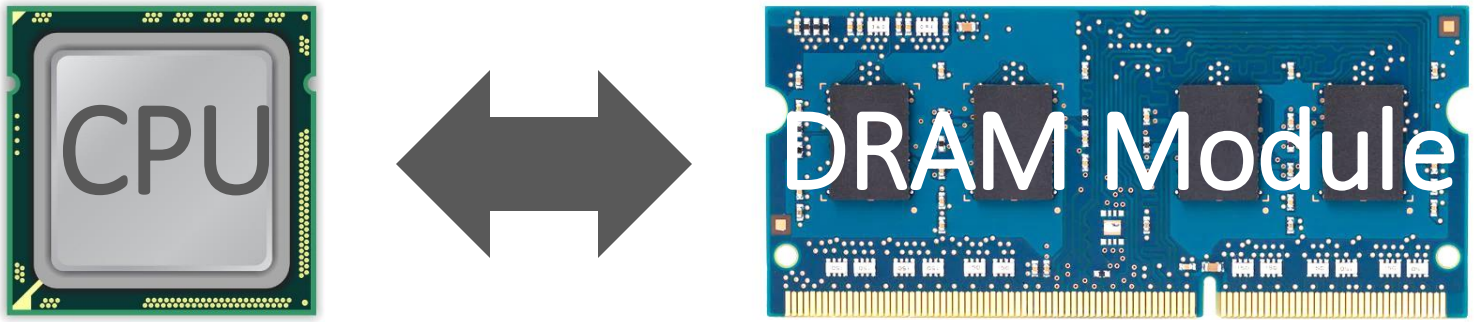



```
loop:
   mov (X), %eax
   mov (Y), %ebx
   clflush (X)
   clflush (Y)
   mfence
   jmp loop
```



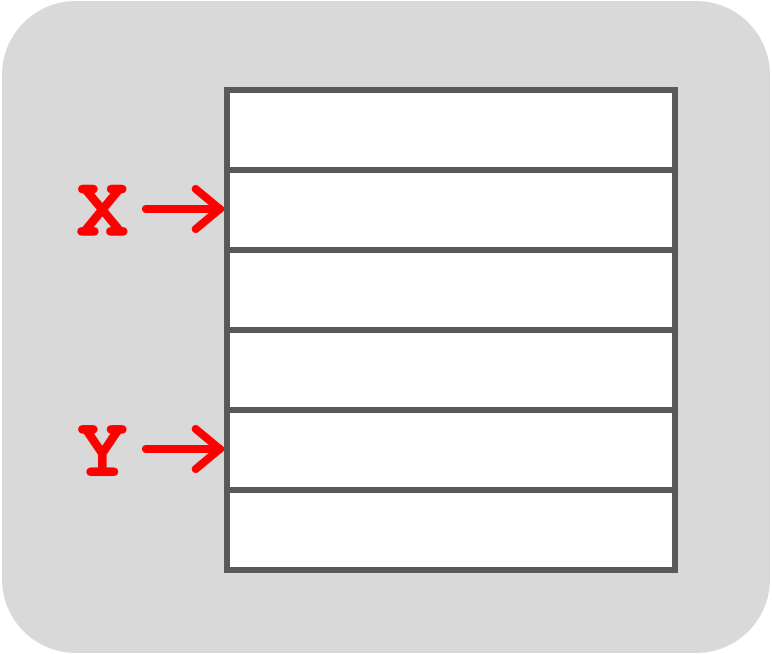
Download from: https://github.com/CMU-SAFARI/rowhammer

A Simple Program Can Induce Many Errors



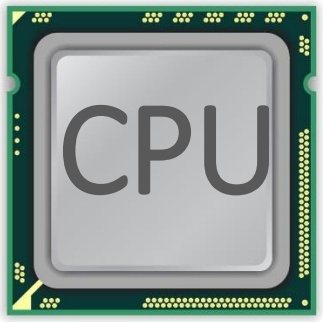
1. Avoid ***cache hits***
   – Flush **X** from cache

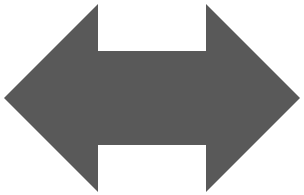
2. Avoid ***row hits*** to **X**
   – Read **Y** in another row

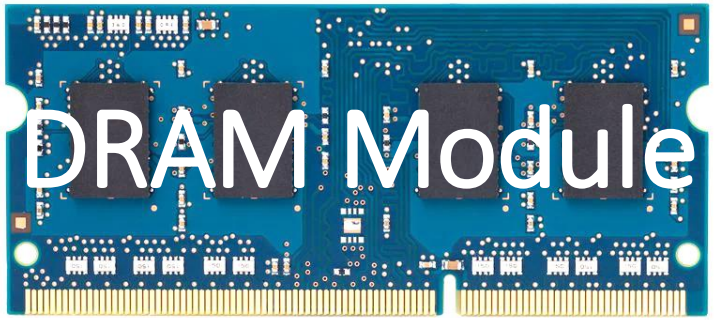


Download from: https://github.com/CMU-SAFARI/rowhammer

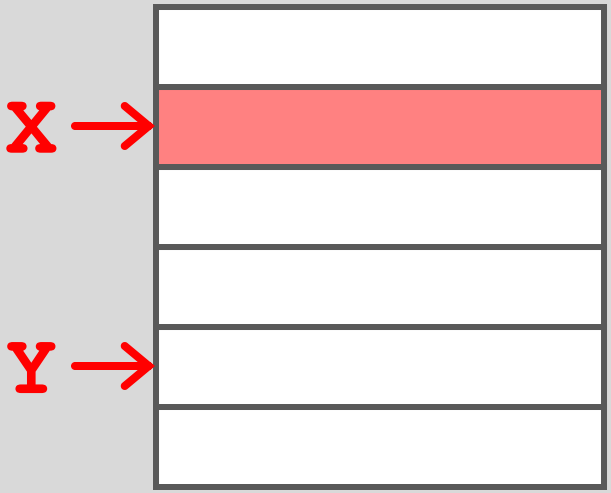
A Simple Program Can Induce Many Errors





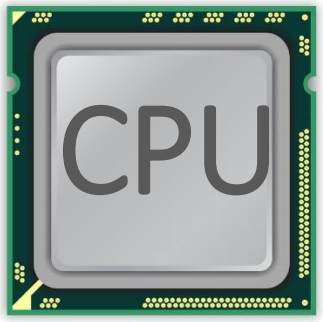


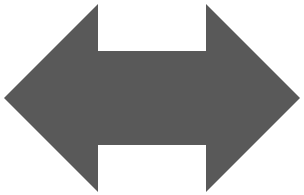

```
loop:
  mov (X), %eax
  mov (Y), %ebx
  clflush (X)
  clflush (Y)
  mfence
  jmp loop
```

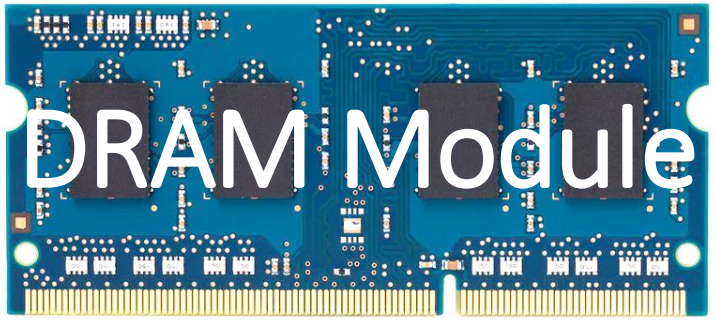


Download from: https://github.com/CMU-SAFARI/rowhammer

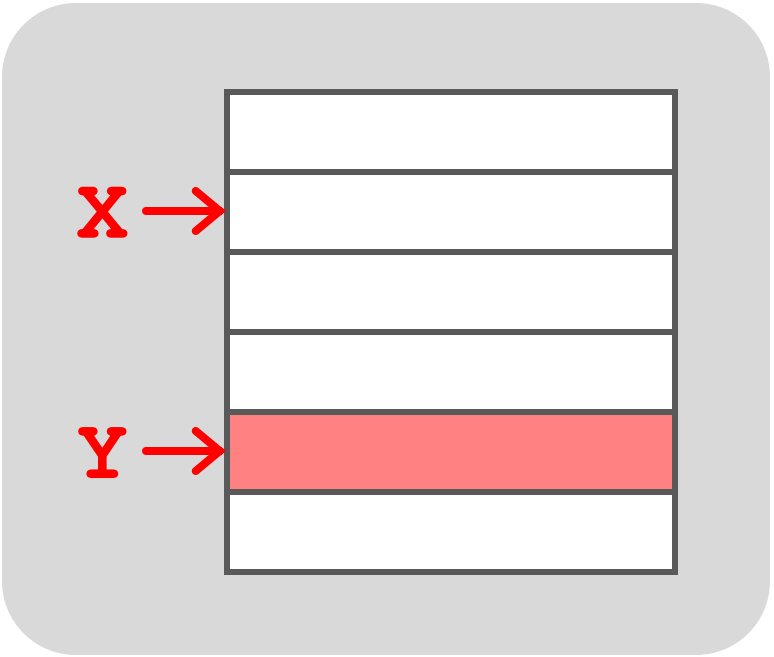
A Simple Program Can Induce Many Errors





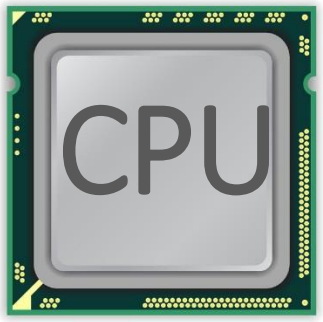


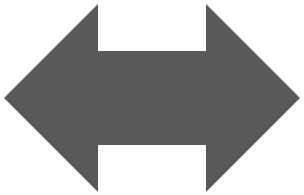

```
loop:
   mov (X), %eax
   mov (Y), %ebx
   clflush (X)
   clflush (Y)
   mfence
   jmp loop
```

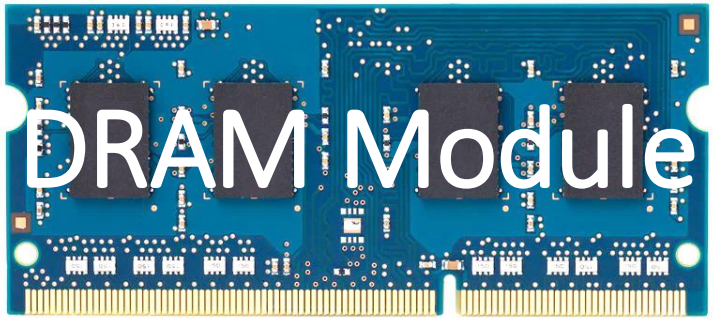


Download from: https://github.com/CMU-SAFARI/rowhammer

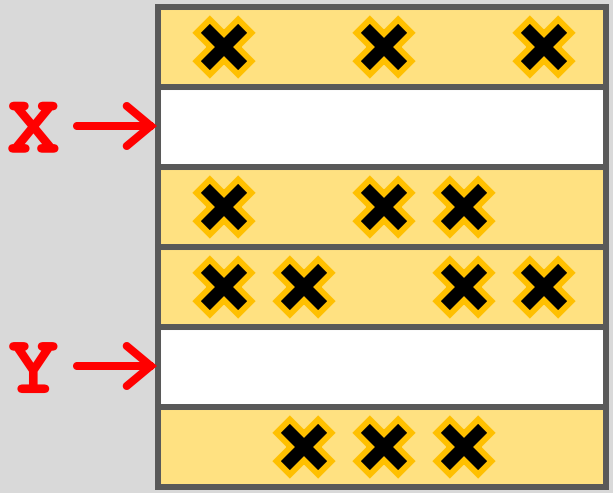
A Simple Program Can Induce Many Errors








```
loop:
   mov (X), %eax
   mov (Y), %ebx
   clflush (X)
   clflush (Y)
   mfence
   jmp loop
```



Download from: https://github.com/CMU-SAFARI/rowhammer

Observed Errors in Real Systems

CPU Architecture	Errors	Access-Rate
Intel Haswell (2013)	22.9K	12.3M/sec
Intel Ivy Bridge (2012)	20.7K	11.7M/sec
Intel Sandy Bridge (2011)	16.1K	11.6M/sec
AMD Piledriver (2012)	59	6.1M/sec

A real reliability & security issue

Kim+, "Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors," ISCA 2014.

One Can Take Over an Otherwise-Secure System

Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors

Abstract. Memory isolation is a key property of a reliable and secure computing system — an access to one memory address should not have unintended side effects on data stored in other addresses. However, as DRAM process technology

Project Zero

[Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors] (Kim et al., ISCA 2014)

News and updates from the Project Zero team at Google

[Exploiting the DRAM rowhammer bug to gain kernel privileges] (Seaborn, 2015)

Monday, March 9, 2015

Exploiting the DRAM rowhammer bug to gain kernel privileges

Google's Original RowHammer Attack

The following slides are from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf

Kernel exploit

- x86 page tables entries (PTEs) are **dense and trusted**
  - They control access to physical memory
  - A bit flip in a PTE's physical page number can give a process access to a different physical page
- Aim of exploit: Get access to a page table
  - Gives access to all of physical memory
- Maximise chances that a bit flip is useful:
  - Spray physical memory with page tables
  - Check for useful, repeatable bit flip first

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf

x86-64 Page Table Entries (PTEs)

- Page table is a 4k page containing array of 512 PTEs
- Each PTE is 64 bits, containing:

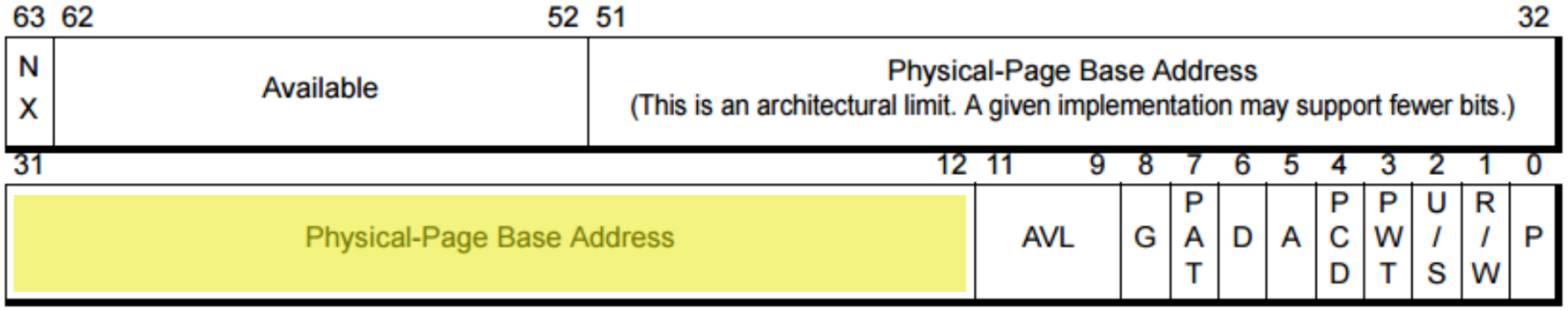
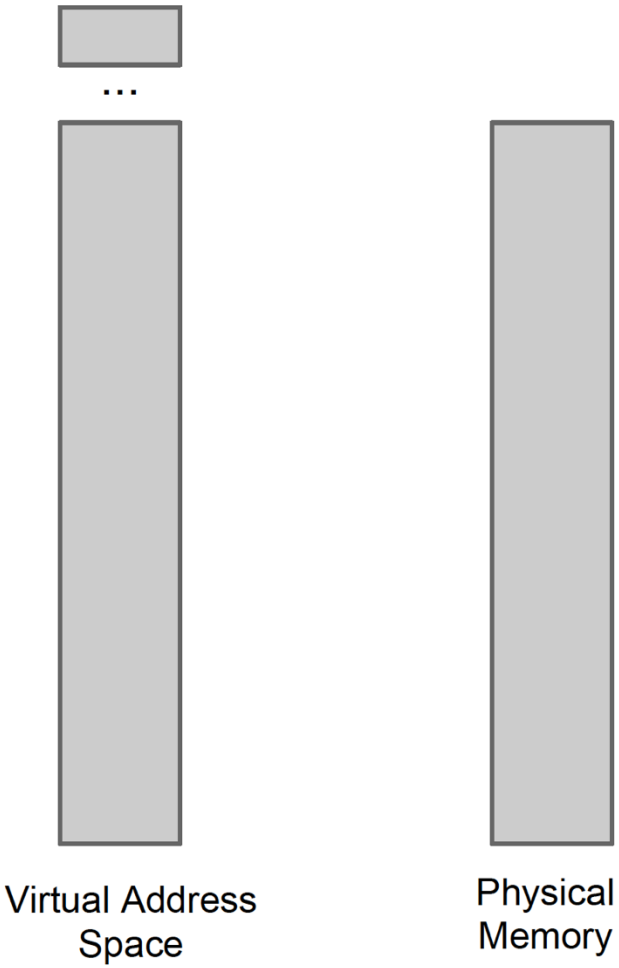


Figure 5-21. 4-Kbyte PTE—Long Mode

- Could flip:
  - "Writable" permission bit (RW): 1 bit → 2% chance
  - Physical page number: 20 bits on 4GB system → 31% chance

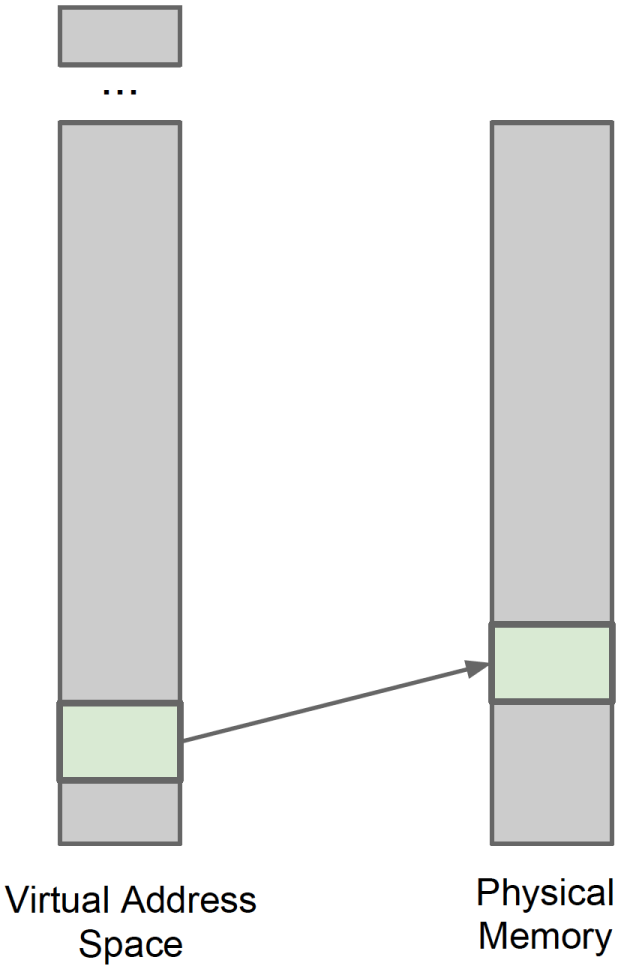
This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf



This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

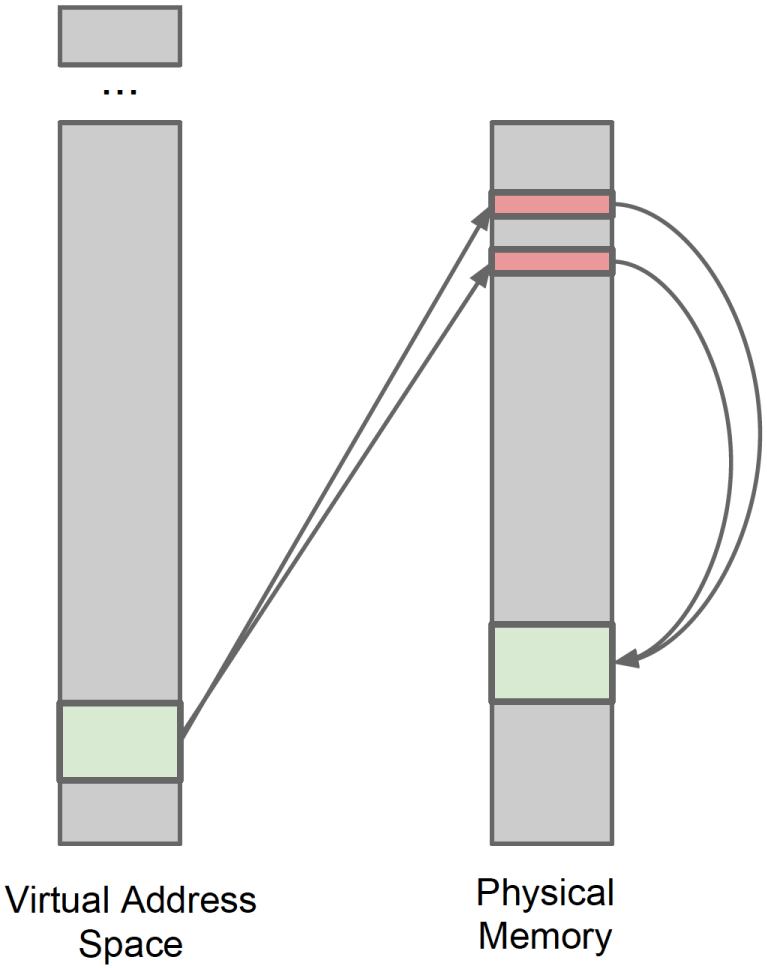
https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf



What happens when we map a file with read-write permissions?

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

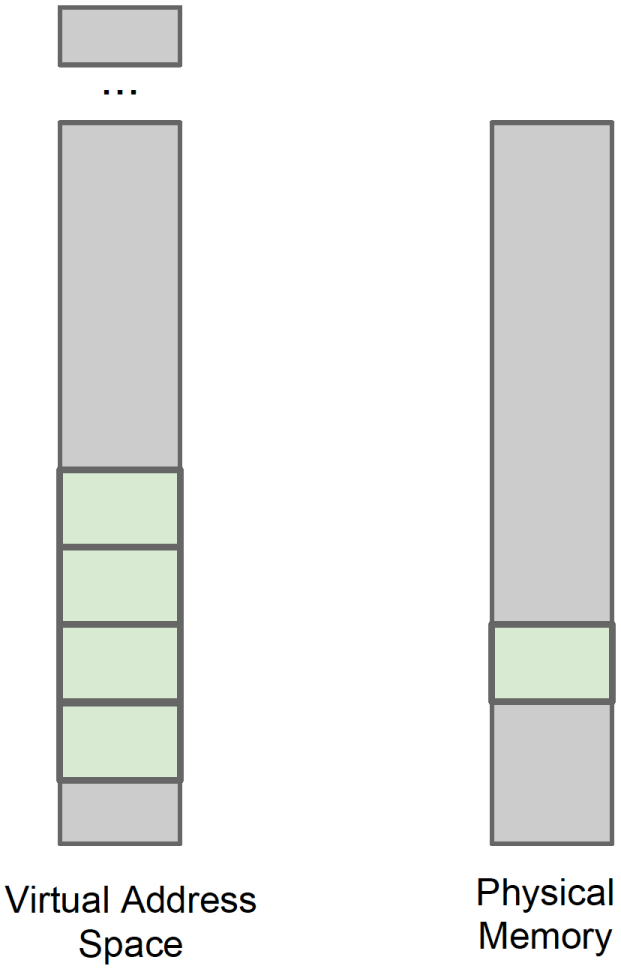
https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf



What happens when we map a file with read-write permissions? Indirection via page tables.

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

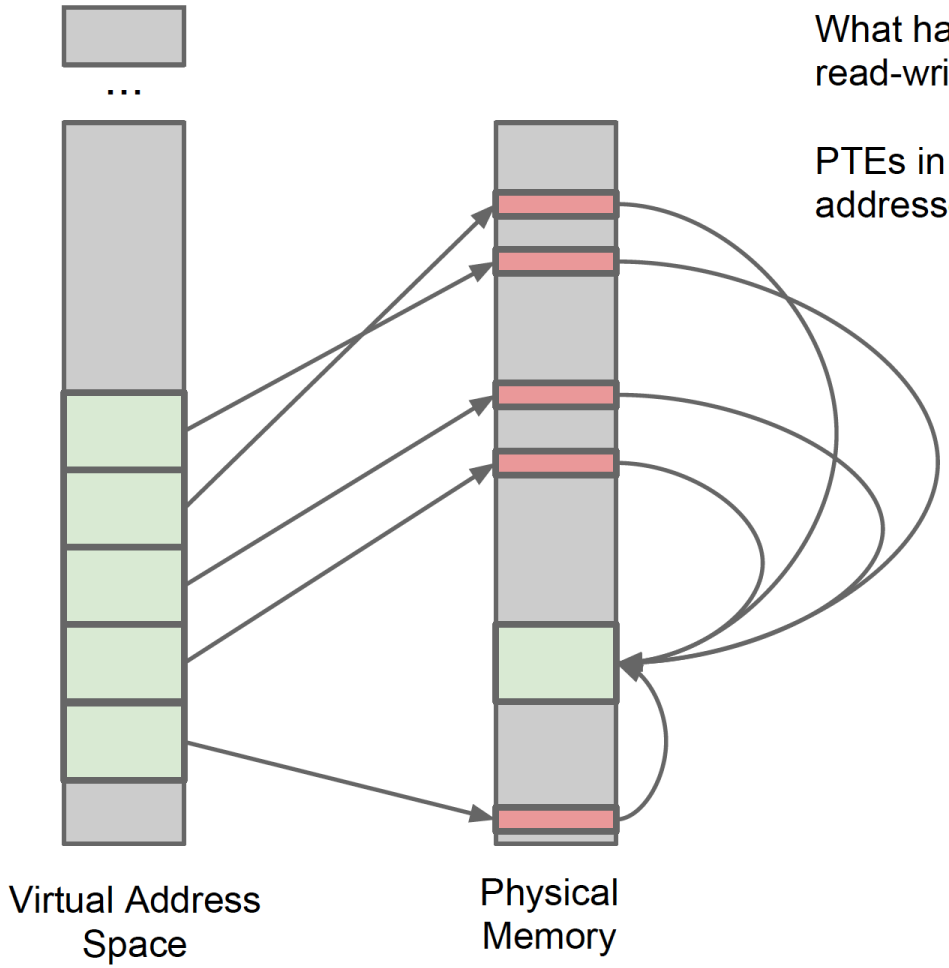
https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf



What happens when we repeatedly map a file with read-write permissions?

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf

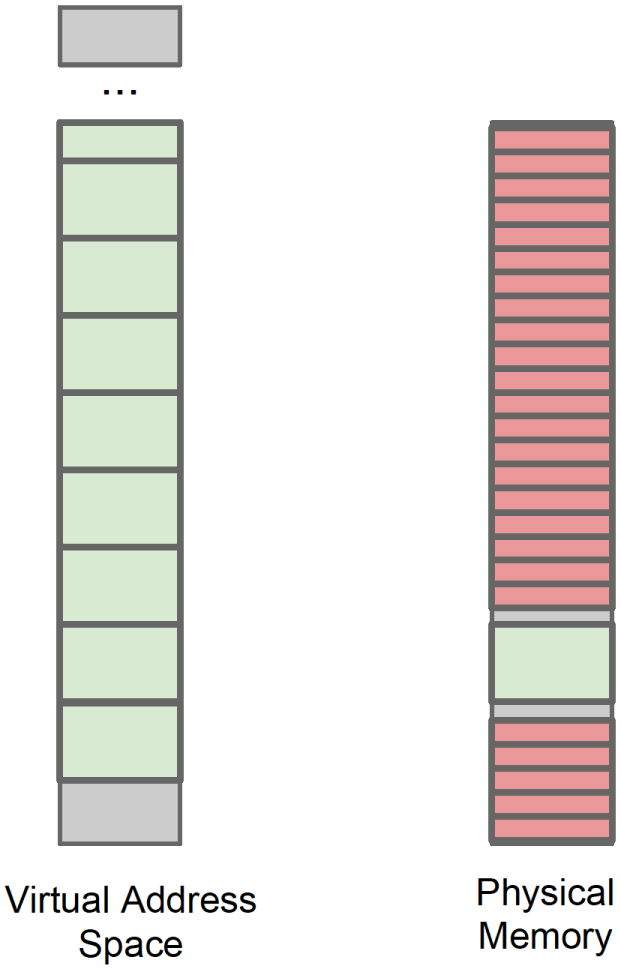


What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf



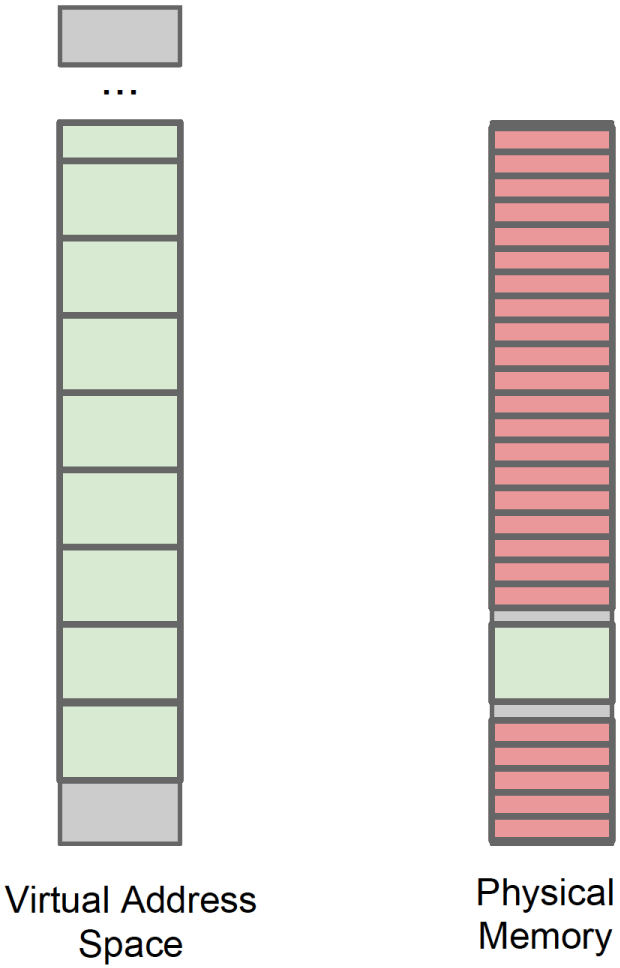
What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf



What happens when we repeatedly map a file with read-write permissions?

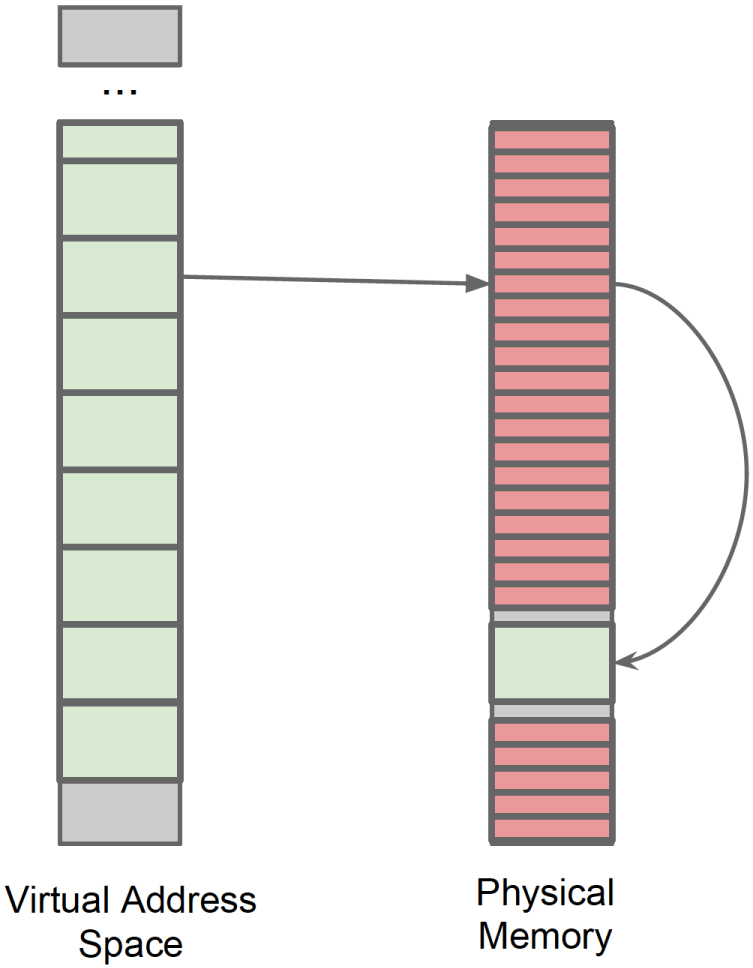
PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

Each of them points to pages in the same physical file mapping.

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf



What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

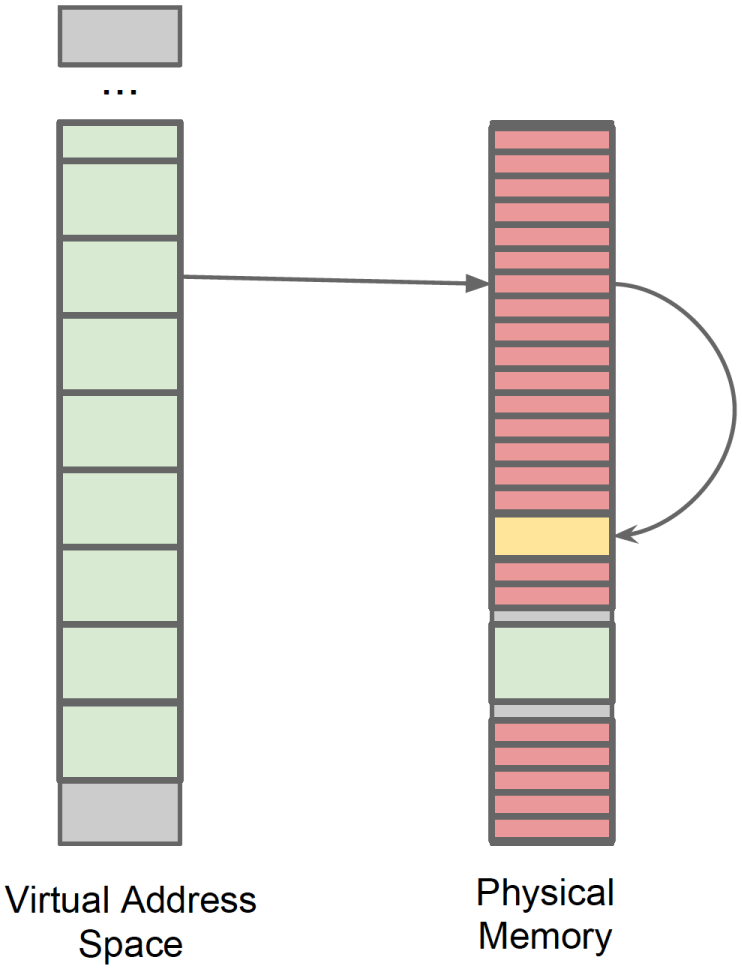
We can fill physical memory with PTEs.

Each of them points to pages in the same physical file mapping.

If a bit in the right place in the PTE flips ...

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf



What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

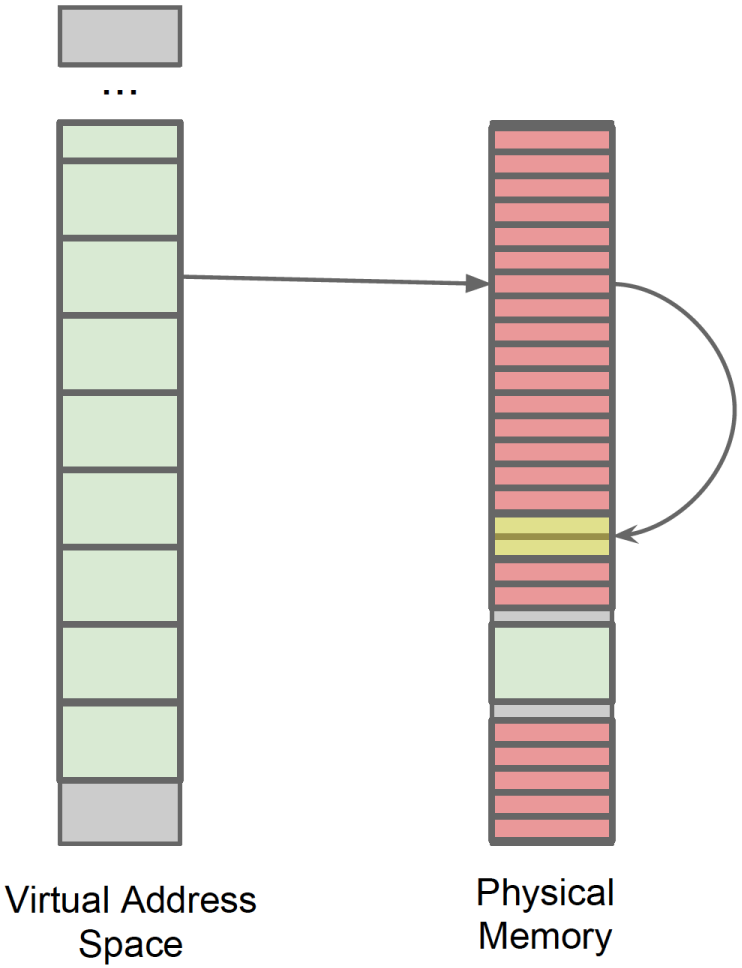
Each of them points to pages in the same physical file mapping.

If a bit in the right place in the PTE flips …

… the corresponding virtual address now points to a wrong physical page - with RW access.

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf



What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

Each of them points to pages in the same physical file mapping.

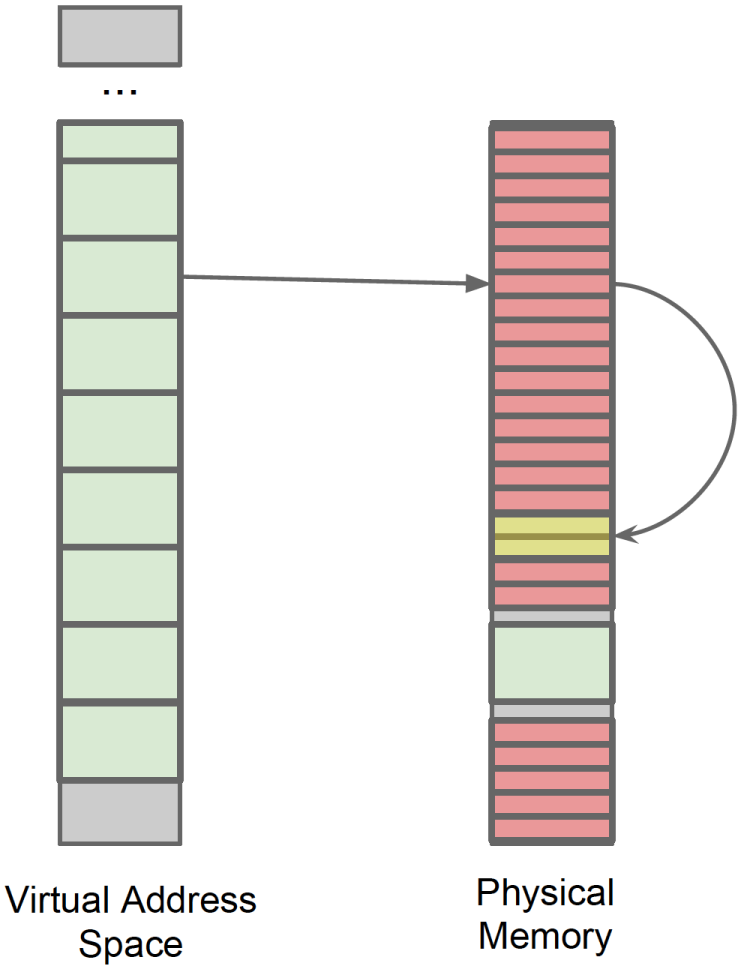
If a bit in the right place in the PTE flips …

… the corresponding virtual address now points to a wrong physical page - with RW access.

Chances are this wrong page contains a page table itself.

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf



What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

Each of them points to pages in the same physical file mapping.

If a bit in the right place in the PTE flips …

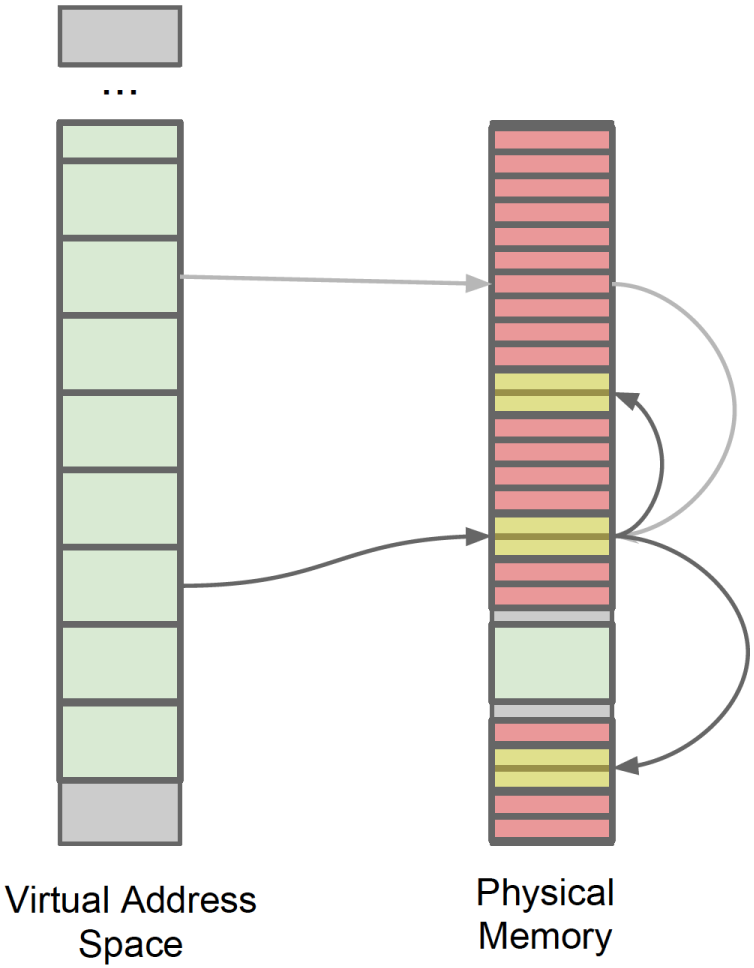
… the corresponding virtual address now points to a wrong physical page - with RW access.

Chances are this wrong page contains a page table itself.

An attacker that can read / write page tables …

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf



What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

Each of them points to pages in the same physical file mapping.

If a bit in the right place in the PTE flips …

… the corresponding virtual address now points to a wrong physical page - with RW access.

Chances are this wrong page contains a page table itself.

An attacker that can read / write page tables can use that to map **any** memory read-write.

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf

Exploit strategy

Privilege escalation in 7 easy steps …

1. Allocate a large chunk of memory
2. Search for locations prone to flipping
3. Check if they fall into the "right spot" in a PTE for allowing the exploit
4. Return that particular area of memory to the operating system
5. Force OS to re-use the memory for PTEs by allocating massive quantities of address space
6. Cause the bitflip - shift PTE to point into page table
7. Abuse R/W access to all of physical memory

In practice, there are many complications.

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf

Security Implications



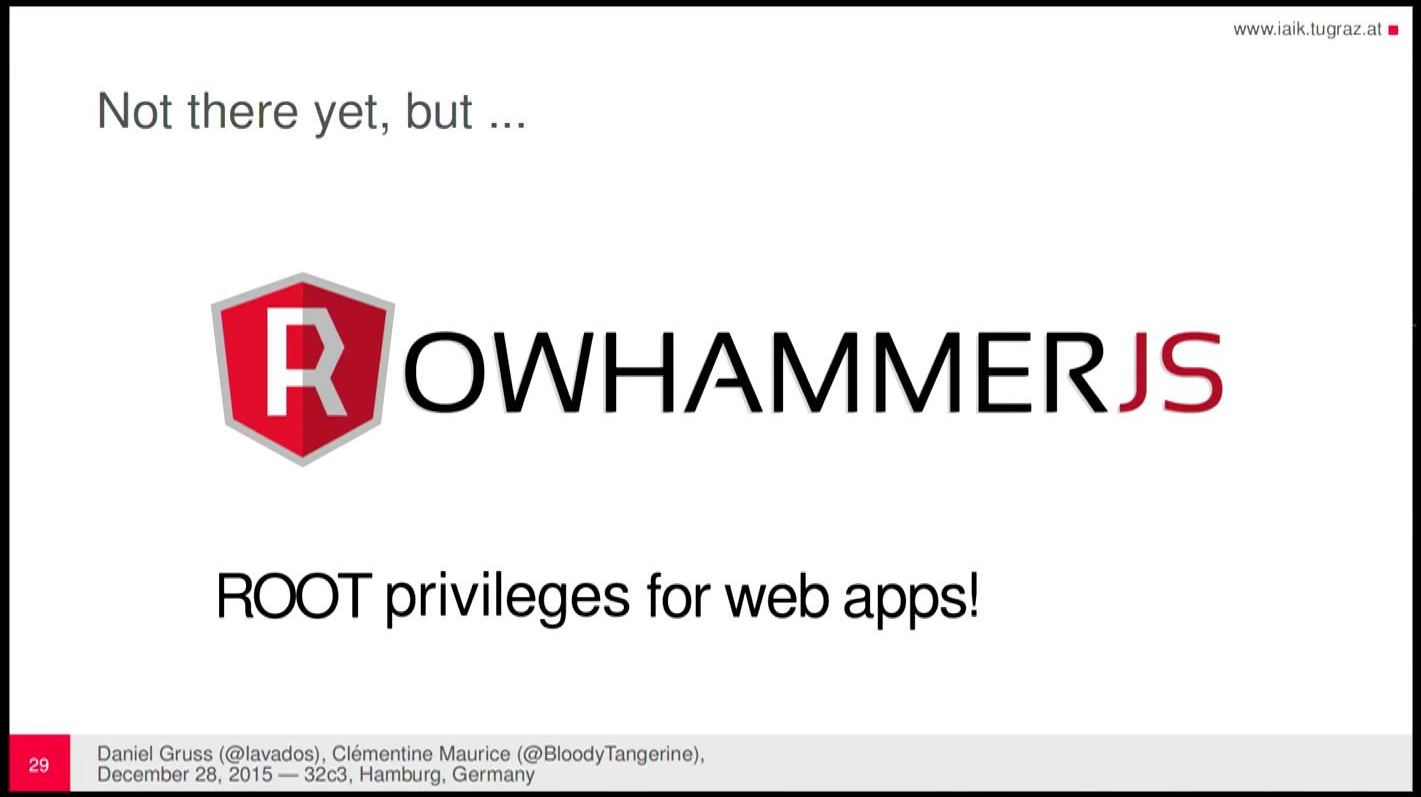
Security Implications



It's like breaking into an apartment by repeatedly slamming a neighbor's door until the vibrations open the door you were after

More Security Implications (I)

"We can gain unrestricted access to systems of website visitors."







Rowhammer.js: A Remote Software-Induced Fault Attack in JavaScript (DIMVA'16)

Source: https://lab.dsst.io/32c3-slides/7197.html

Takeaway and Food for Thought

- If hardware is unreliable, higher-level security and protection mechanisms (as in virtual memory) may be compromised

- The root of security and trust is at the very low levels…
  - in the hardware itself
  - RowHammer, Spectre, Meltdown are recent key examples…

- What should we assume the hardware provides?
- How do we keep hardware reliable?
- How do we design secure hardware?
- How do we design secure hardware with high performance, high energy efficiency, low cost, convenient programming?

Plenty of exciting and highly-relevant research questions

Some Issues in Virtual Memory

Three Major Issues in Virtual Memory

1. How large is the page table and how do we store and access it?

2. How can we speed up translation & access control check?

3. When do we do the translation in relation to cache access?

- There are many other issues we will not cover in detail
  - What happens on a context switch?
  - How can you handle multiple page sizes?
  - …

Three Major Issues in Virtual Memory

1. How large is the page table and how do we store and access it?

2. How can we speed up translation & access control check?

3. When do we do the translation in relation to cache access?

- There are many other issues we will not cover in detail
  - What happens on a context switch?
  - How can you handle multiple page sizes?
  - …

Virtual Memory Issue II

- How fast is the address translation?
  - How can we make it fast?

- Idea: Use a hardware structure that caches PTEs and reduces the number of memory accesses to lookup the page table

Address Translation



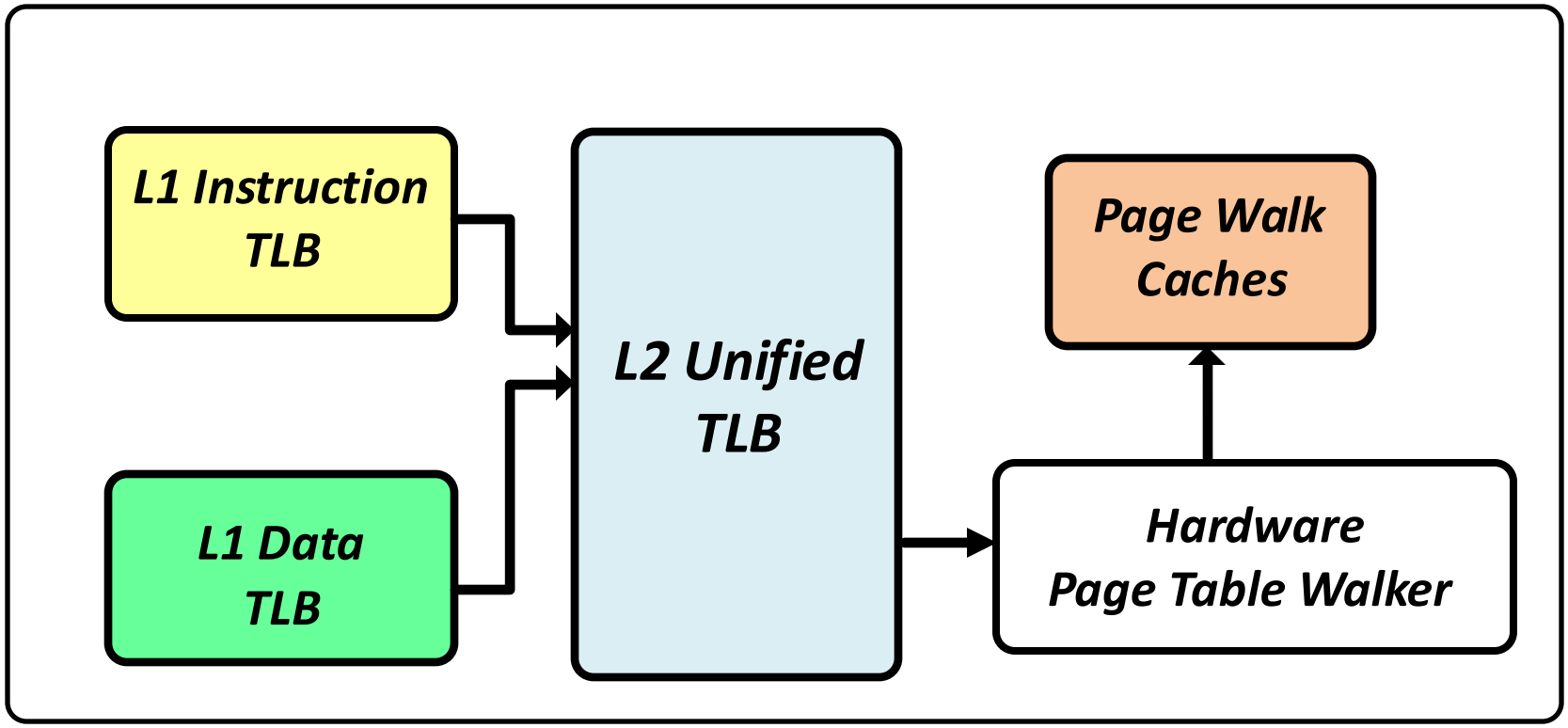
Hardware unit responsible for address translation

Memory Management Unit

- The **Memory Management Unit (MMU) is** responsible for resolving address translation requests
  - One MMU per core (usually)

- MMU typically has three key components:
  - **Translation Lookaside Buffers** that cache recently-used virtual-to-physical translations (PTEs)
  - **Page Table Walk Caches** that offer fast access to the intermediate levels of a multi-level page table
  - **Hardware Page Table Walker** that sequentially accesses the different levels of the Page Table to fetch the required PTE

Intel Skylake: MMU



https://www.7-cpu.com/cpu/Skylake.html

Translation Lookaside Buffer (TLB)

- Idea: Cache the Page Table Entries (PTEs) in a hardware structure in the processor to speed up address translation

- Translation lookaside buffer (TLB)
  - Small cache of most recently used Page Table Entries, i.e., recently used Virtual-to-Physical translations
  - Reduces the number of memory accesses required for *most* instruction fetches and loads/stores to only one TLB access

Real Life Example: Warehouse



TLB = A Worker's Pocket Notepad

Workers can't check the database for every single barcode — too slow.

Instead, they keep a **tiny notebook** of the most recently used barcode→location mappings.

This notebook is the **TLB (Translation Lookaside Buffer)**:

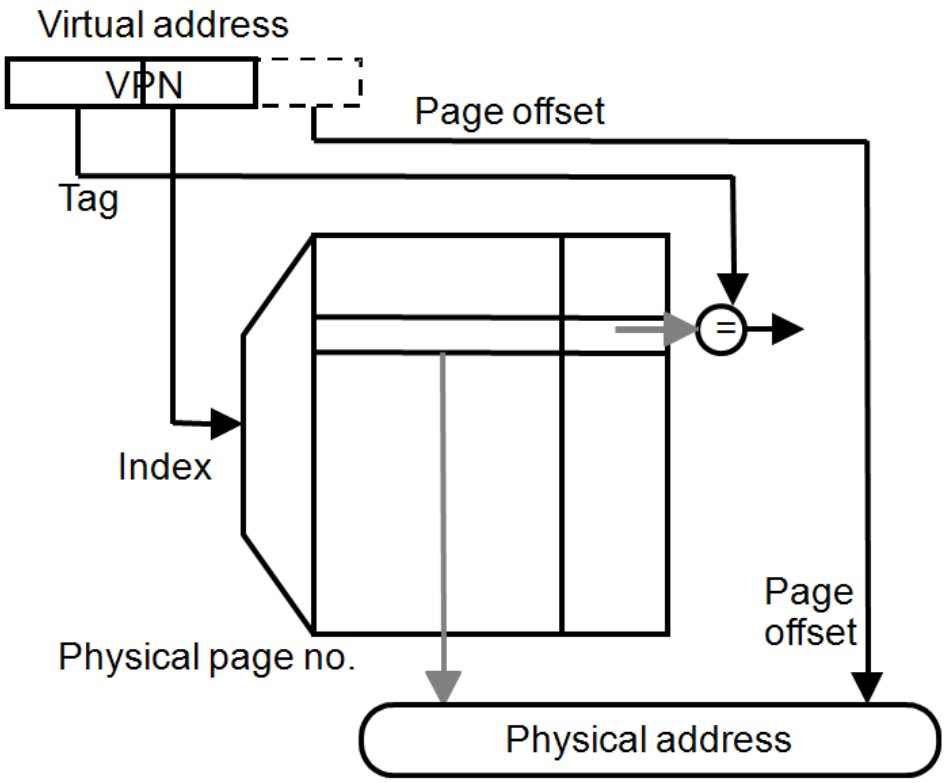
- A small, fast cache of translations.
- If a translation is found in the TLB → extremely fast access.
- If not → must check the slower warehouse database (page table).

Speeding up Translation with a TLB

- A cache of address translations
  - Avoids accessing the page table on every memory access

- **Index** = lower bits of VPN (virtual page #)
- **Tag** = unused bits of VPN + process ID
- **Data** = a page-table entry
- **Status** = valid, dirty

The usual cache design choices (placement, replacement policy, multi-level, etc.) apply here too.



Handling TLB Misses

- The TLB is small; it cannot hold **all** PTEs
  - Some translation requests will inevitably miss in the TLB
  - Must access memory to find the required PTE
    - Called **walking** the page table
    - Large performance penalty

- Better TLB management & prefetching can reduce TLB misses

- Who handles TLB misses?
  - Hardware or software?

Handling TLB Misses (II)

- Approach #1. **Hardware-Managed** (e.g., x86)
  - The hardware does the **page walk**
  - The hardware fetches the PTE and inserts it into the TLB
    - If the TLB is full, the entry **replaces** another entry
  - Done transparently to system software
  - Can employ specialized structures and caches
    - E.g., page walkers and page walk caches

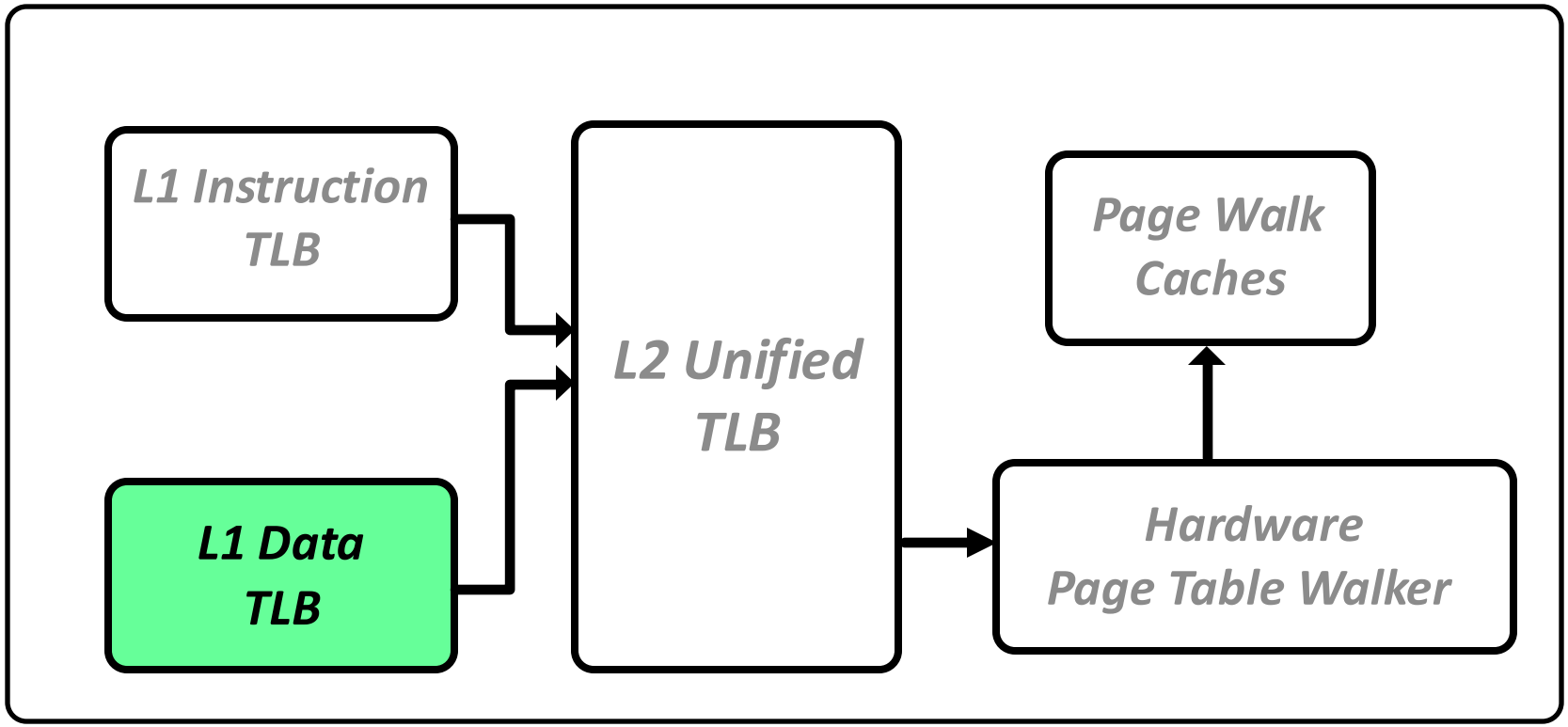
- Approach #2. **Software-Managed** (e.g., MIPS)
  - The hardware raises an exception
  - The operating system does the **page walk**
  - The operating system fetches the PTE
  - The operating system inserts/evicts entries in the TLB

Handling TLB Misses (III)

- Hardware-Managed TLB
  - + No exception on TLB miss. Instruction just stalls
  - + Independent instructions may execute and help tolerate latency
  - + No extra instructions/data brought into caches
  - -- Page directory/table organization is fetched into the system: OS has little flexibility in deciding these

- Software-Managed TLB
  - + The OS can define the page table organization
  - + More sophisticated TLB replacement policies are possible
  - -- Need to generate an exception → performance overhead due to pipeline flush, exception handler execution, extra instructions brought to caches

Intel Skylake: L1 Data TLB



Intel Skylake: L1 Data TLB

- Separate L1 Data TLB structures for 4KB, 2MB, and 1GB pages

- L1 DTLB
  - **4KB**: 64-entry, 4-way, 1 cycle access, 9 cycle miss
  - **2MB**: 32-entry, 4-way, 1 cycle access, 9 cycle miss
  - **1GB**: 4 entry, fully-associative

- Virtual-to-physical mappings are inserted in the corresponding TLB after a TLB miss

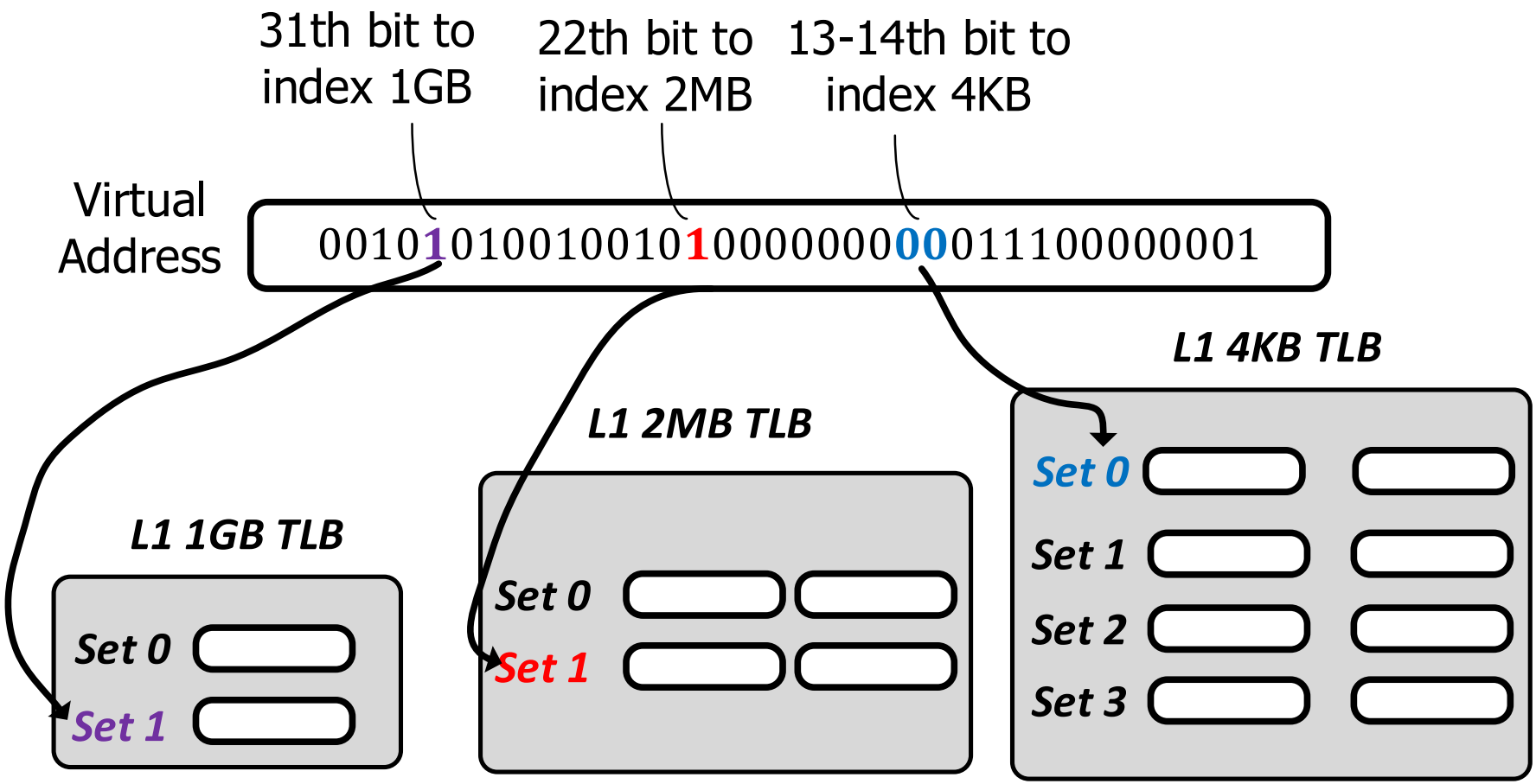
- During a translation request, all three L1 TLBs are looked up in parallel

https://www.7-cpu.com/cpu/Skylake.html

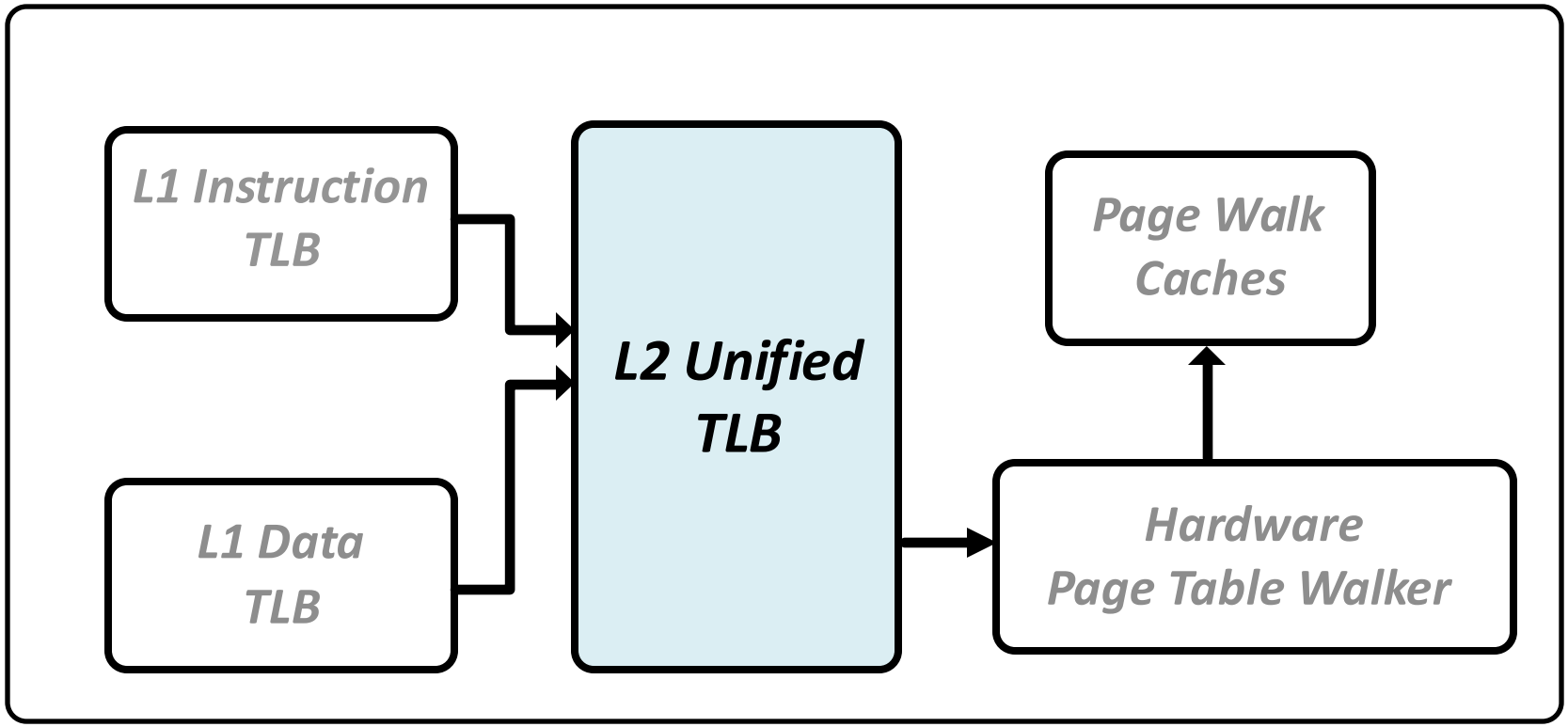
Recall: Trade-Offs in Page Size

- **Large page size (e.g., 1GB)**
  - Pro: Fewer PTEs required ➔ Saves memory space
  - Pro: Fewer TLB misses ➔ Improves performance
  - Con: Cannot have fine-grained permissions
  - Con: Large transfers to/from disk
    - Even when only 1KB is needed, 1GB must be transferred
    - Waste of bandwidth/energy
    - Reduces performance
  - Con: **Internal fragmentation**
    - Even when only 1KB is needed, 1GB must be allocated
    - Waste of space
    - **Q**: What is **external fragmentation**?

L1 Data TLB: Parallel Lookup Example



Intel Skylake: L2 Unified I/D TLB



Intel Skylake: L2 Unified TLB

- L2 Unified TLB caches translations for both instr. and data
  - private per individual core
- 2 separate L2 TLB structures for 4KB/2MB and 1GB pages
- L2 TLB
  - **4KB/2MB**: 1536-entry, 12-way, 14 cycle access, 9 cycle miss
  - **1GB**: 16-entry, 4-way, 1 cycle access, 9 cycle miss penalty
- Challenge: How can the L2 TLB support both 4KB and 2MB pages using a single structure?

  (Not enough publicly available information for Intel Skylake)

https://www.7-cpu.com/cpu/Skylake.html

L2 Unified TLB: Accessing the TLB

- The 4KB/2MB structure of the L2 TLB is probed in 2 steps

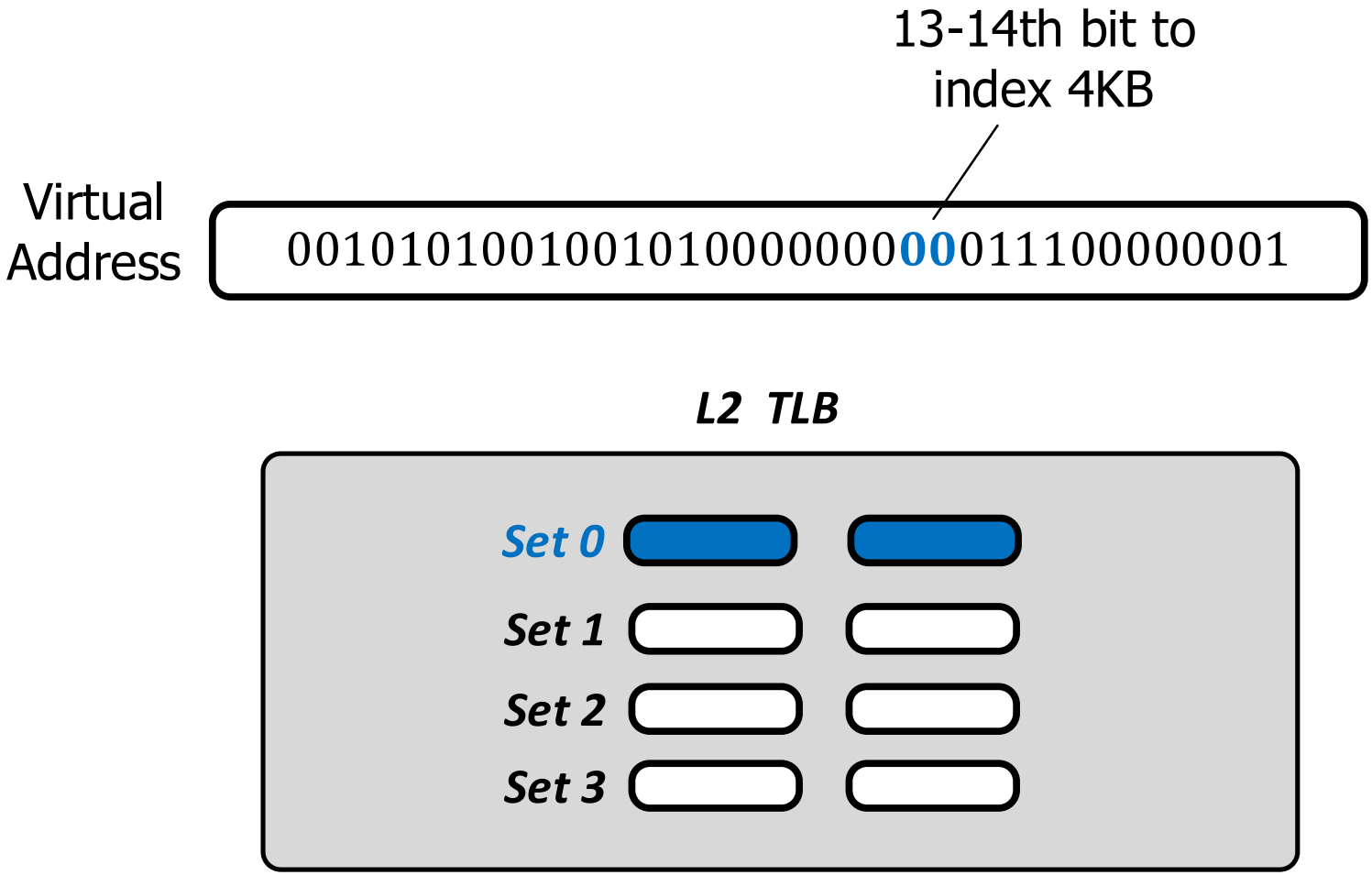
- Step 1: Assume the page size is 4KB, calculate the index bits and access the L2 TLB
  - If the tag matches, it is a hit. If the tag does not match, go to Step 2.

- Step 2: Assume the page size is 2MB, **re-calculate** the index and access the L2 TLB.
  - If the tag matches, it is a hit. If the tag does not match, it is an L2 TLB miss.

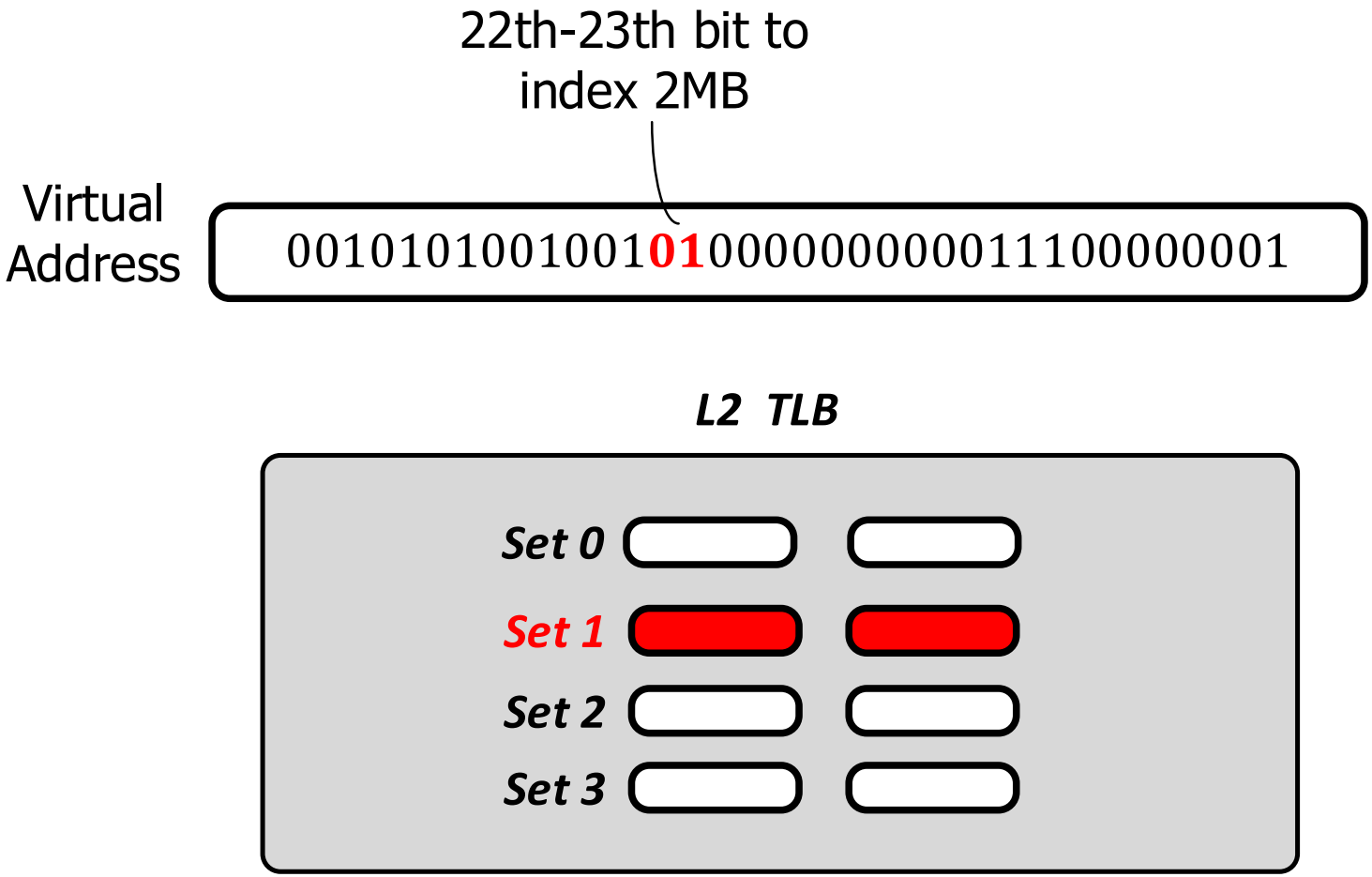
- **General algorithm**:

  Re-calculate index and probe TLB for all remaining page sizes

Step 1: Calculate Index for 4KB



Step 2: Re-calculate Index for 2MB



L2 TLB: N-Step Index Re-Calculation

- Pros:

  + Simple and practical implementation

- Cons:

  - Varying L2 TLB hit latency (faster for 4KB, slower for 2MB)

  - Slower identification of L2 TLB Miss as all page sizes need to be tested

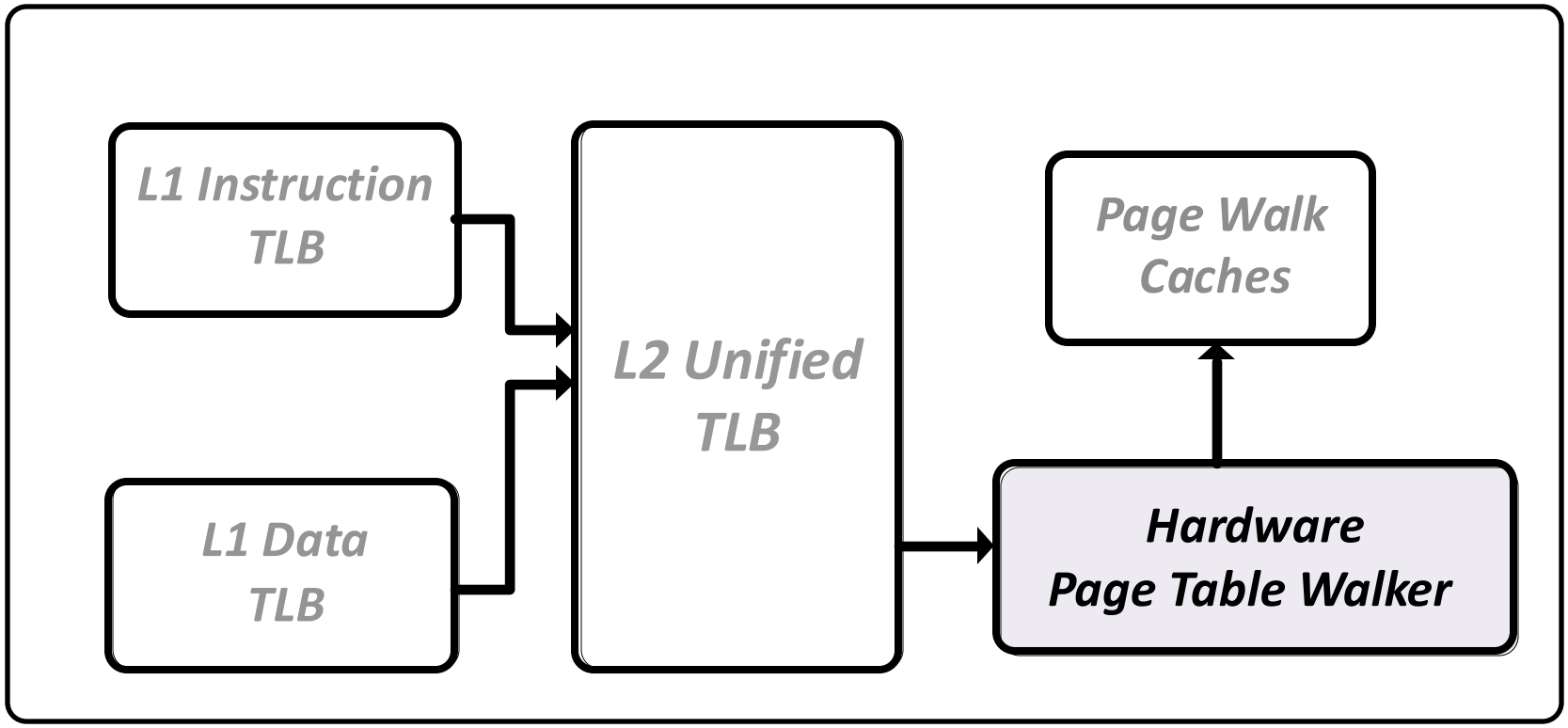
- Potential Optimizations:

  1. Parallel Lookup: Look up for 4KB and 2MB pages in parallel

  2. Page Size Prediction: Predict the probing order

Tradeoffs are similar to "associativity in time" (also called pseudo-associativity)

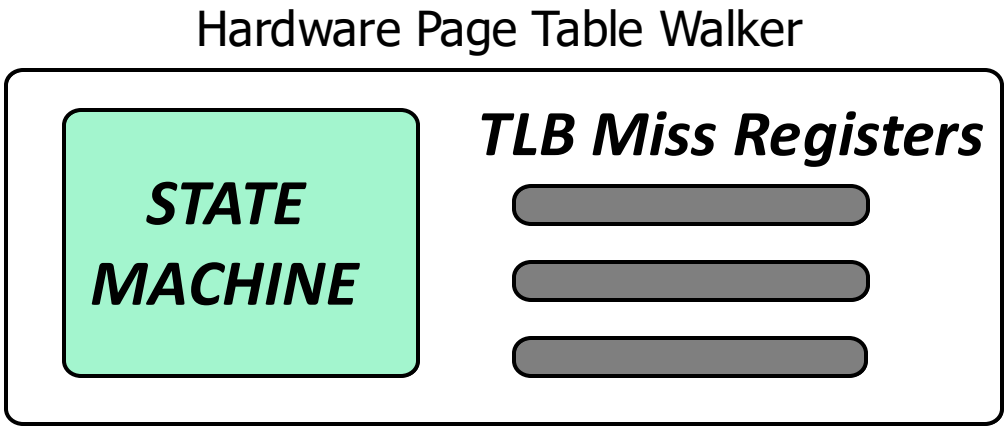
Hardware Page Table Walker



Hardware Page Table Walker (I)

- A per-core hardware component that walks the multi-level page table to avoid expensive context switches & SW handling

- HW PTW consists of 2 components:
  - A state machine that is designed to be aware of the architecture's page table structure
  - Registers that keep track of outstanding TLB misses



Hardware Page Table Walker (II)

- Pros:
  + Avoids the need for context switch on TLB miss
  + Overlaps TLB misses with useful computation
  + Supports concurrent TLB misses

- Cons:
  - Hardware area and power overheads
  - Limited flexibility compared to software page table walk

Hardware Page Table Walker (III)

- PTW accesses the CR3 register that maintains information about the physical address of the root of the page table (PML4)
- PTW concatenates the content of CR3 with the first 9 bits of the virtual address

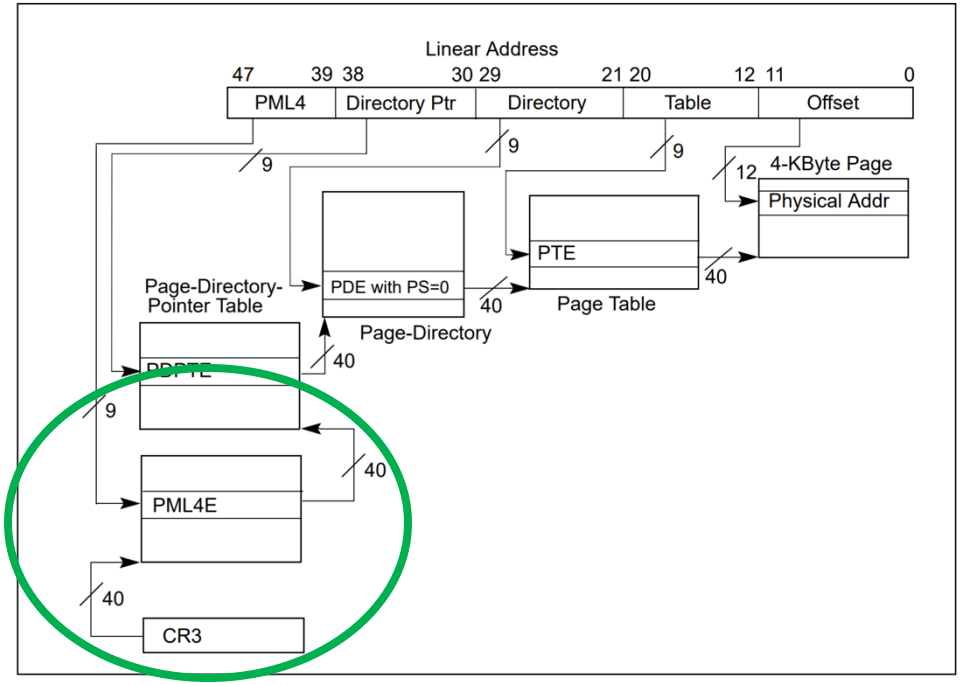
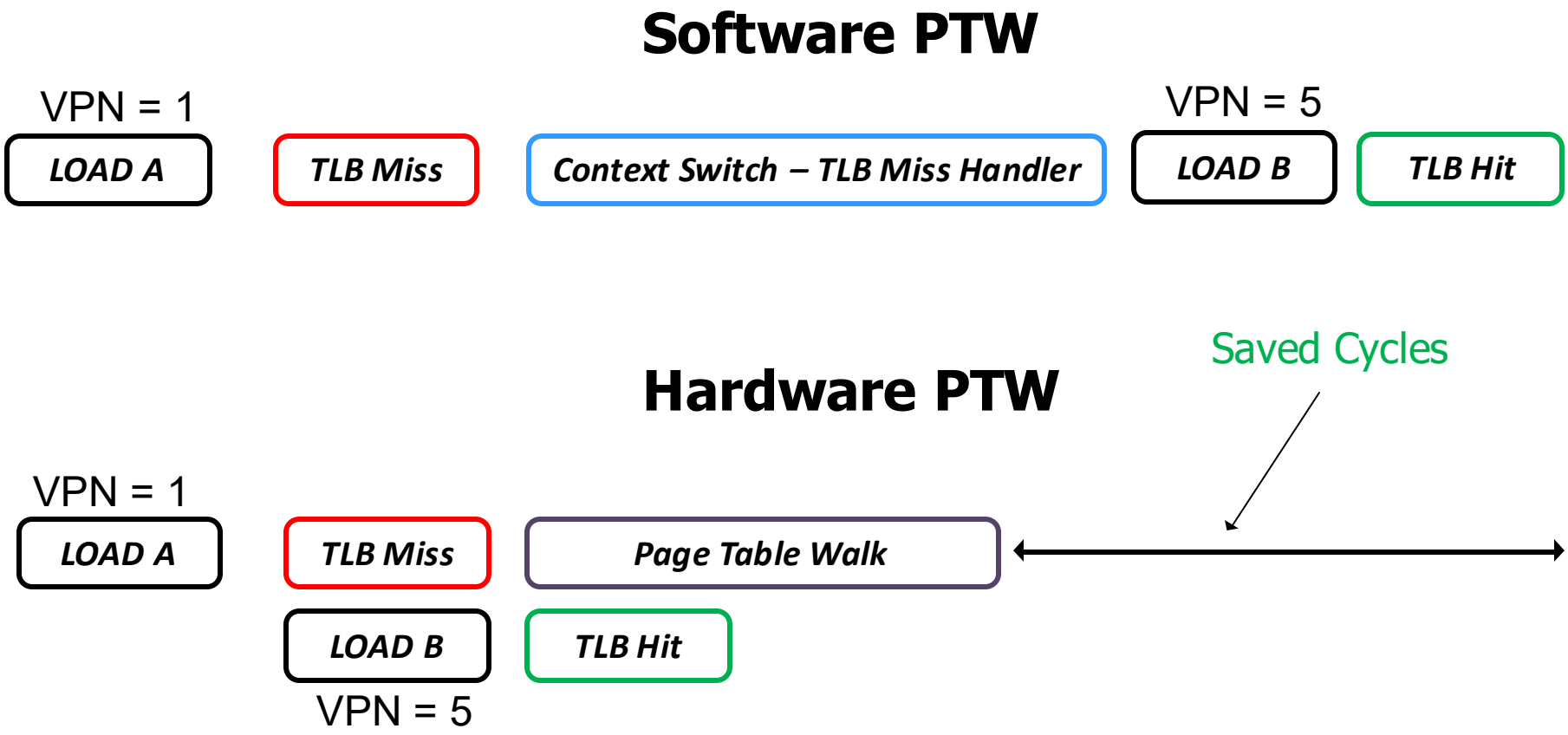


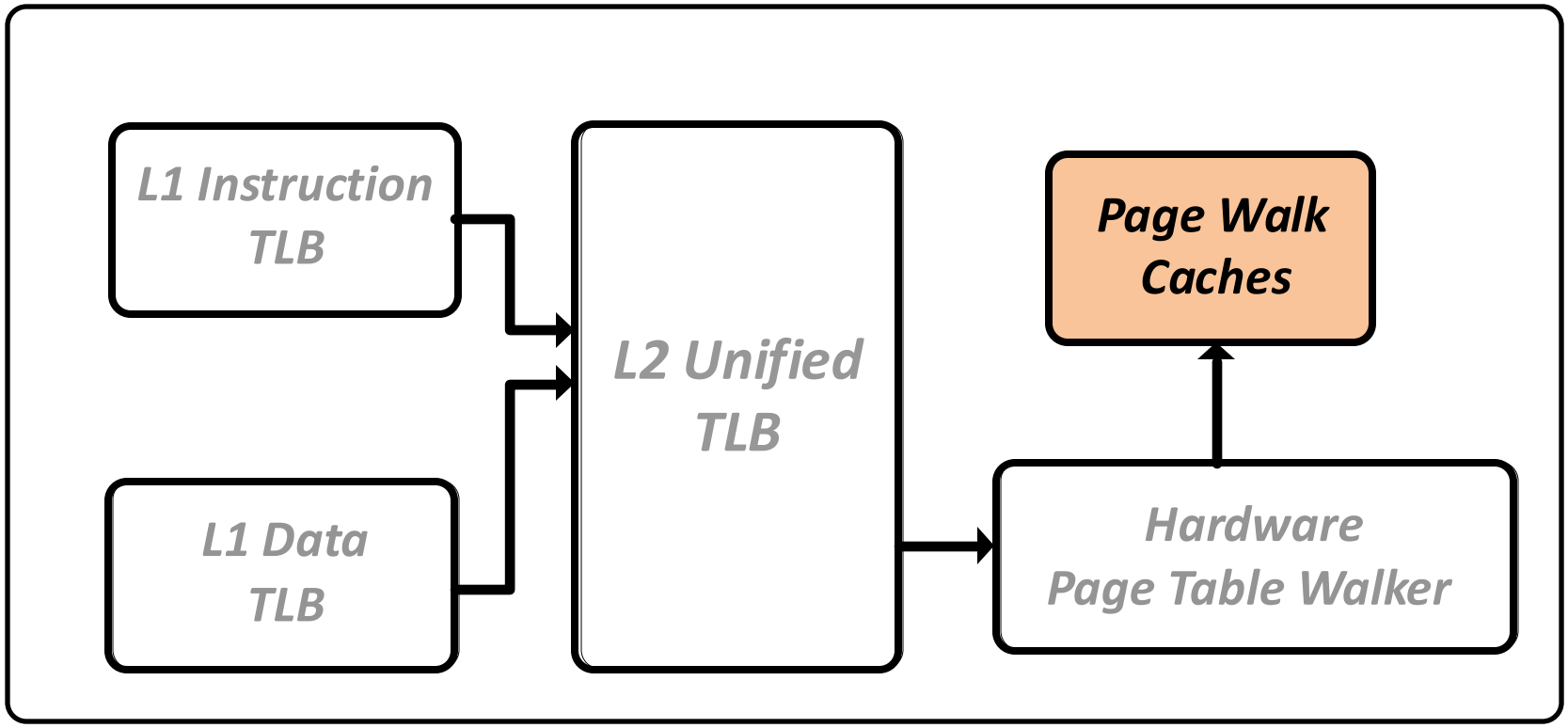
Figure 4-8. Linear-Address Translation to a 4-KByte Page using 4-Level Paging

Hardware Page Table Walker (IV)

- Hardware PTWs allow overlapping TLB misses with useful computation



Page Walk Caches



Page Walk Caches

- Page Walk Caches cache translations from non-leaf levels of a multi-level page table to accelerate page table walks

- Page Walk Caches are low-latency caches that provide faster access to the page table levels
  - compared to accessing the regular cache/memory hierarchy for every page table walk

Three Major Issues in Virtual Memory

1. How large is the page table and how do we store and access it?

2. How can we speed up translation & access control check?

3. When do we do the translation in relation to cache access?

- There are many other issues we will not cover in detail
  - What happens on a context switch?
  - How can you handle multiple page sizes?
  - …

Teaser: Virtual Memory Issue III

- When do we do the address translation?
  - Before or after accessing the L1 cache?

Address Translation and Caching

- When do we do the address translation?
  - Before or after accessing the L1 cache?

- In other words, is the cache virtually addressed or physically addressed?
  - Virtual versus physical cache

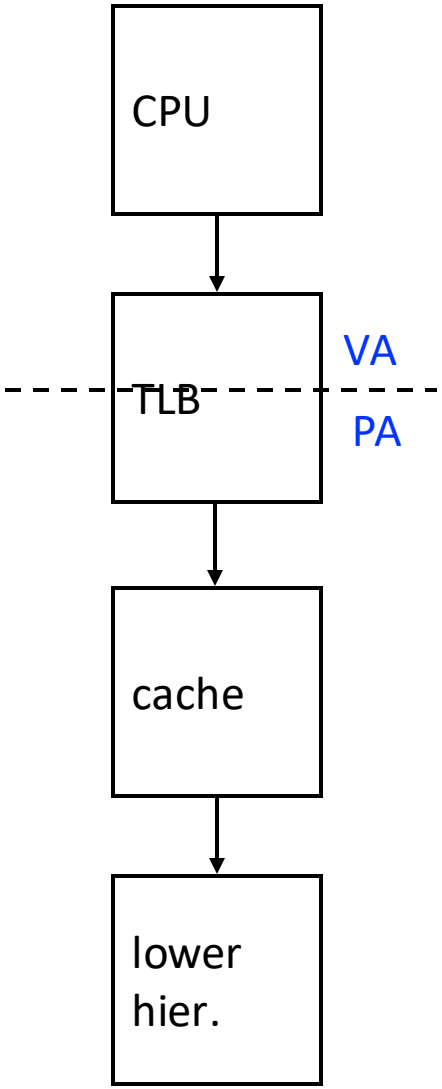
- What are the issues with a virtually addressed cache?

- Synonym problem:
  - Two different virtual addresses can map to the same physical address → same physical address can be present in multiple locations in the cache → can lead to inconsistency in data

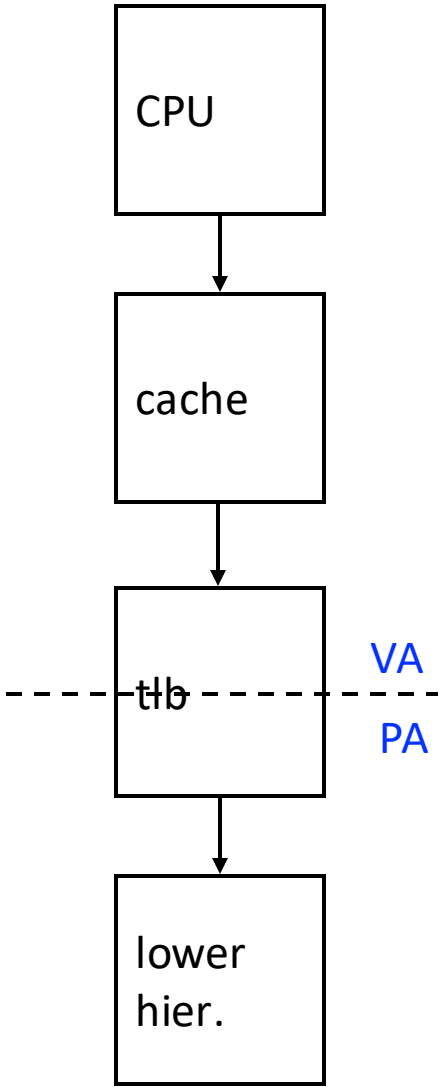
Homonyms and Synonyms

- Homonym: Same VA can map to two different PAs
  - Why?
    - VA is in different processes
- Synonym: Different VAs can map to the same PA
  - Why?
    - Different pages can share the same physical frame within or across processes
    - Reasons: shared libraries, shared data, copy-on-write pages within the same process, …
- Do homonyms and synonyms create problems when we have a cache?
  - Is the cache virtually or physically addressed?

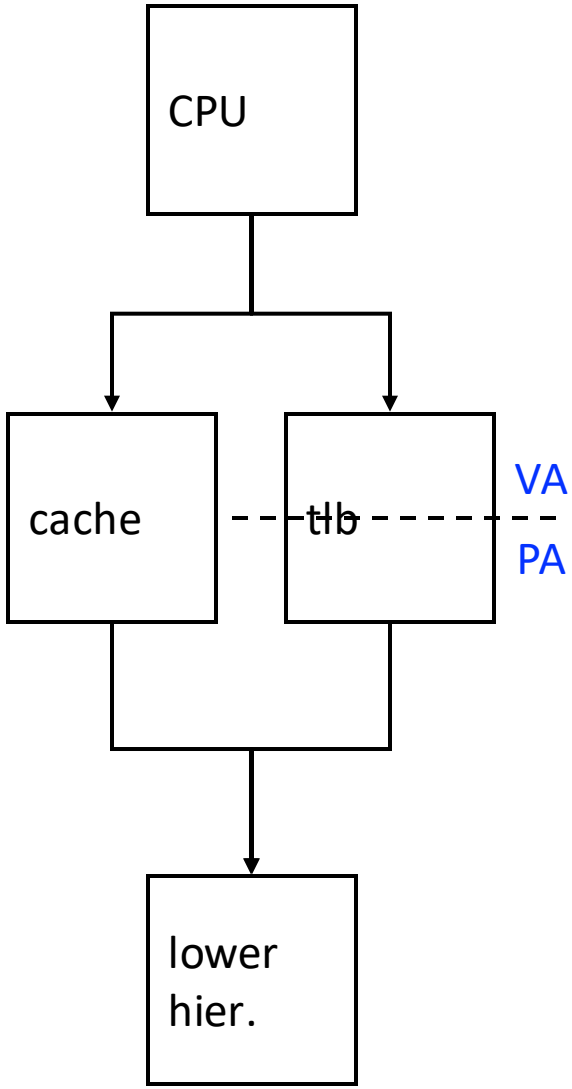
Cache-VM Interaction



physical cache



virtual (L1) cache

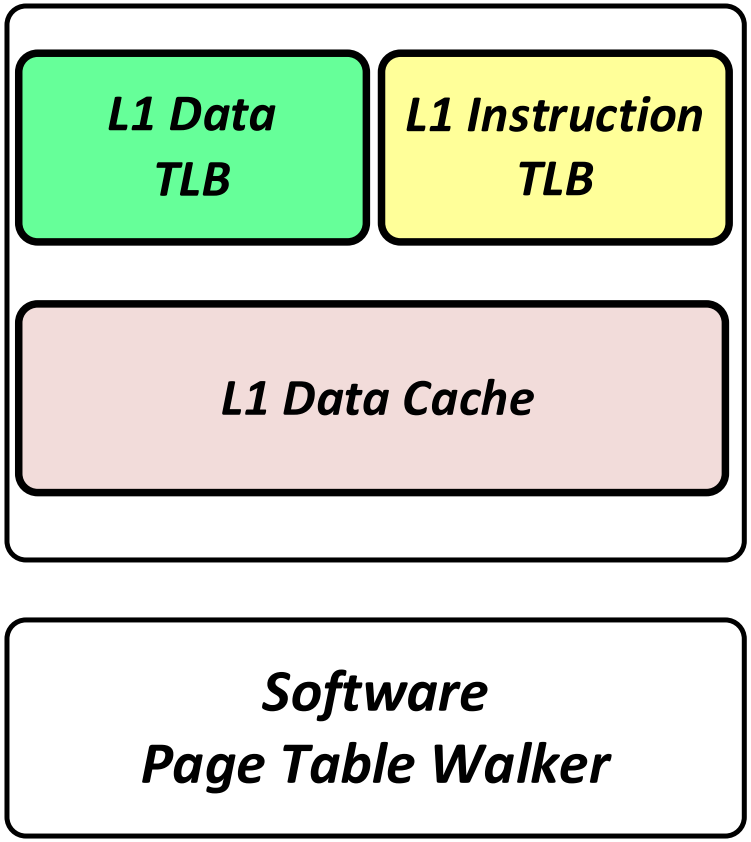


virtual-physical cache

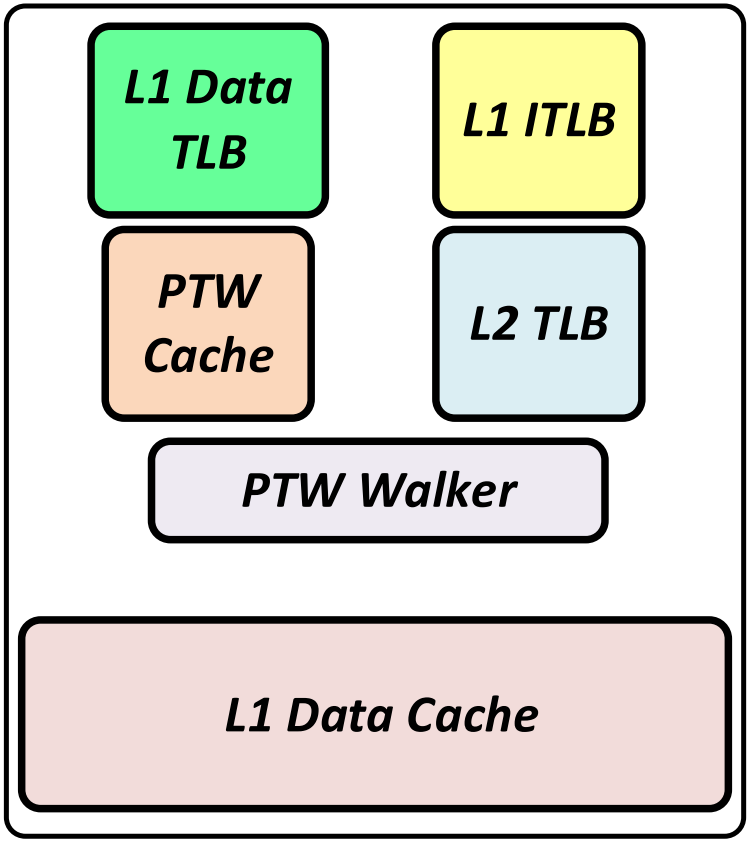
Evolution of the Virtual Memory System

Evolution of Address Translation

Simple Address Translation



Modern Address Translation

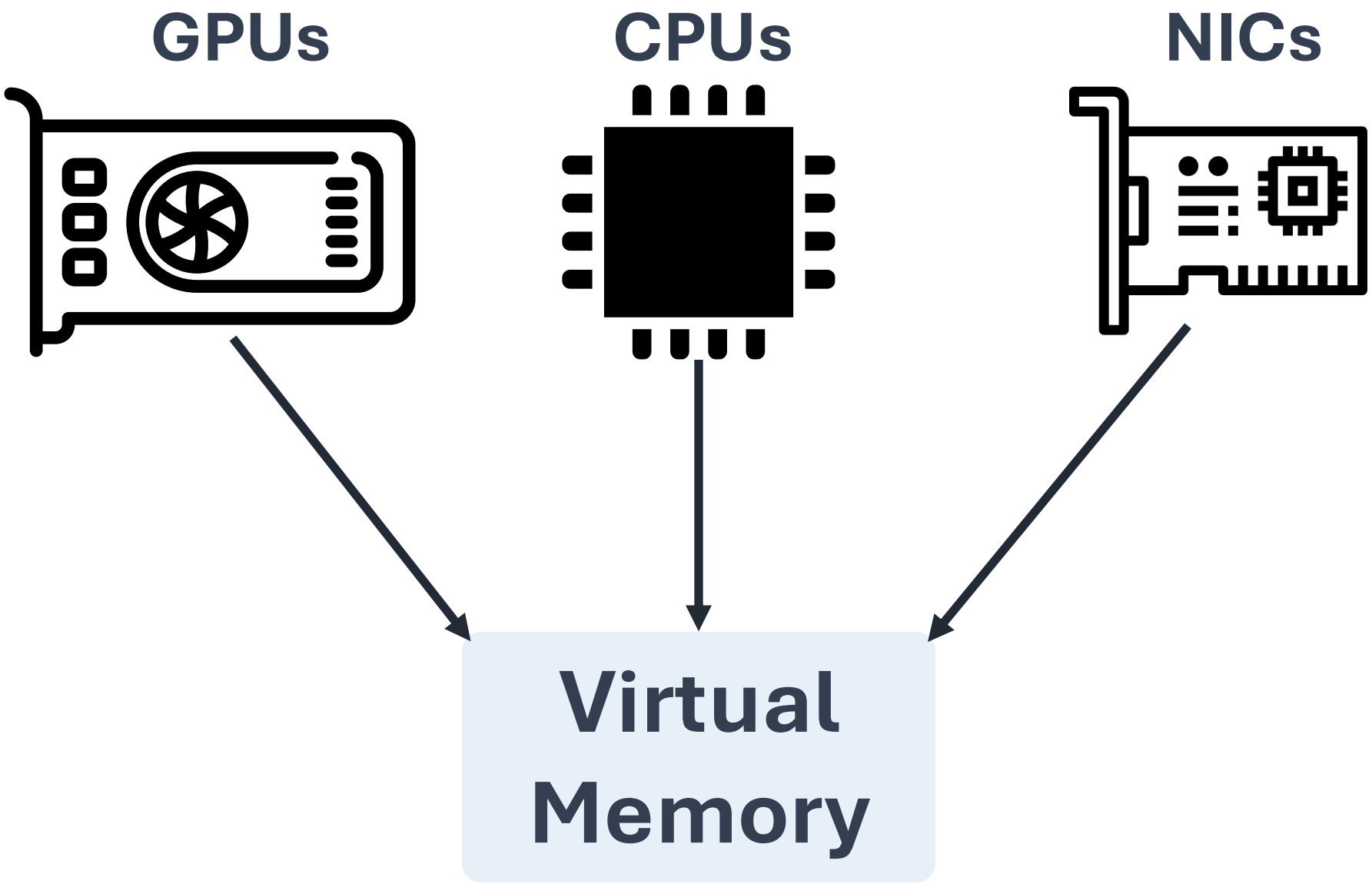


Modern Virtual Memory Designs

	A14 "Firestorm" (iPhone 12 Pro)	Intel/AMD/ARM
Decode width	8	4, 5 (Samsung M3), 5 (Cortex-X1)
ROB size	630	352 (Intel Willow Cove)
Load/store queue size	~148 outstanding loads ~106 outstanding stores	Intel Sunny Cove (128-LQ, 72-SQ) AMD Zen3 (64-LQ, 44-SQ)
L1-TLB	256 entries	64 entries
L2-TLB	3072 entries	1536 entries
Page size	16KB	4KB
L1-I cache	192KB	48KB (Intel Ice Lake)
L1-D cache	128KB, 3-cycles	32KB (Intel/AMD), 4-cycles
L2 cache	8MB shared across two big-cores, ~16-cycles	1MB (Intel Cascade Lake)
L3 cache	16MB shared across all CPU cores and integrated GPU	1.375 MB/core

https://www.anandtech.com/show/16226/apple-silicon-m1-a14-deep-dive/2
https://news.ycombinator.com/item?id=25257932

Virtual Memory Research


GPUs
CPUs
NICs
Virtual Memory





Virtual Memory is a cornerstone of modern computing systems

Does Virtual Memory come for free?

Virtual memory causes high **performance overheads**

Memory Allocation

1

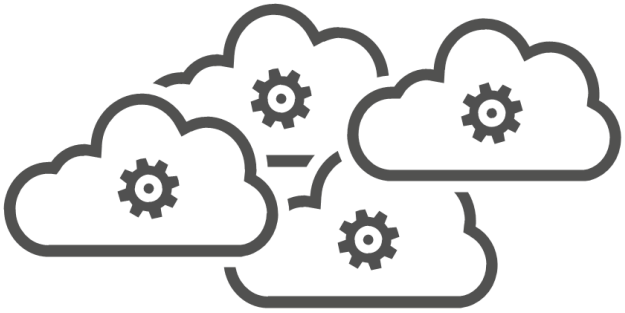
Address Translation

2

Emerging Workloads



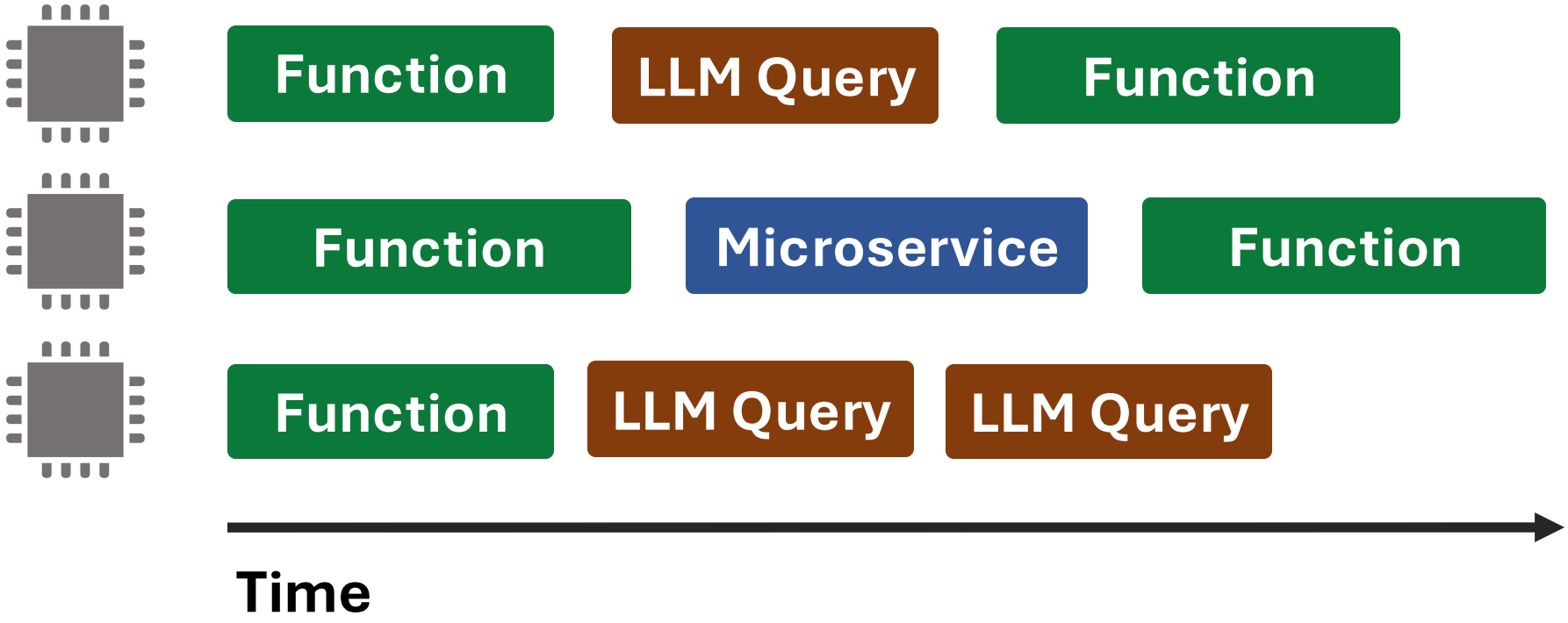
Serverless Functions



Microservices

Short-input Short-output LLM Inference

Emerging Workloads



1. Stateless execution
2. Bursty Patterns
3. Short Execution Time

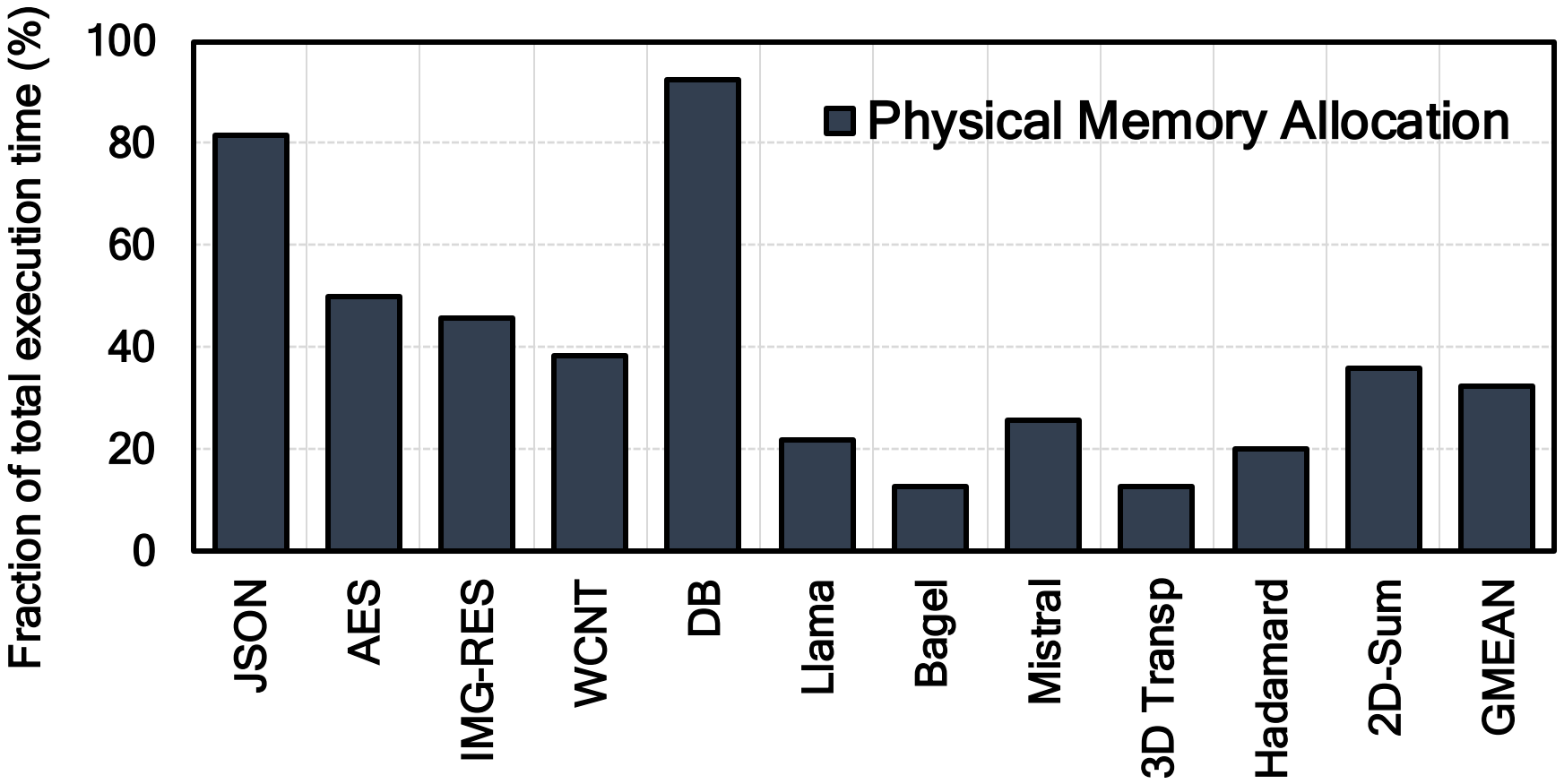
Memory Allocation Overheads

1



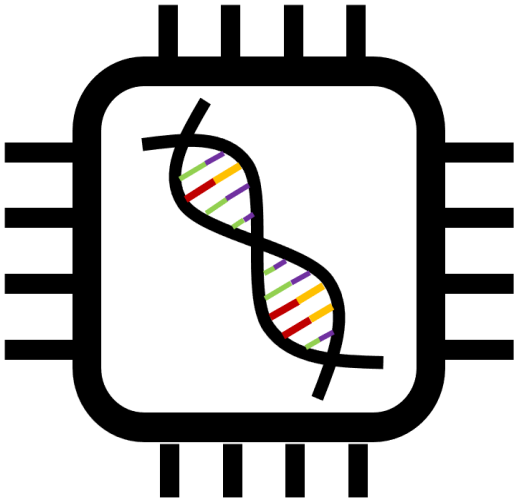
Time spent in OS cannot be amortized due to short execution

Physical Memory Allocation Overheads

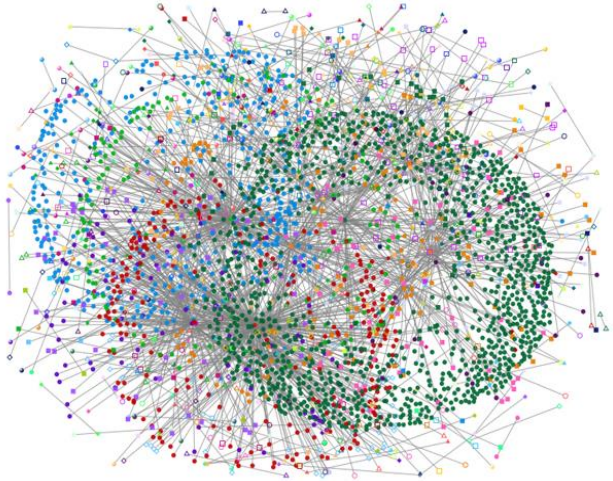


On average 32% of execution time (measured in a real system) spent on physical memory allocation

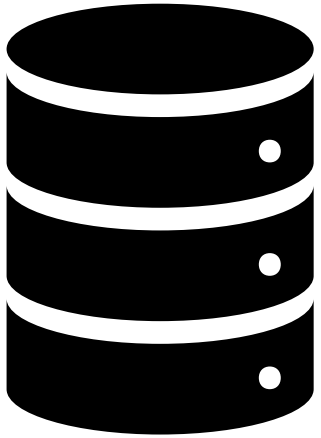
Traditional Workloads



Bioinformatics



Graph Analytics

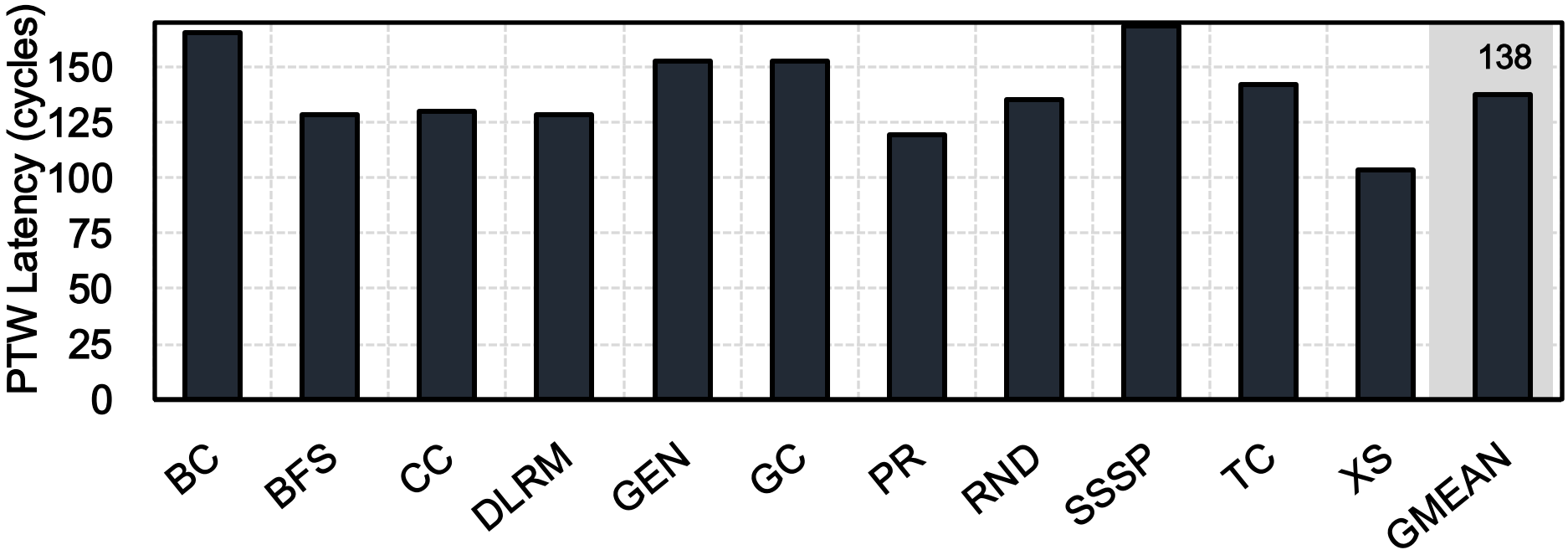


Databases

Traditional Workloads

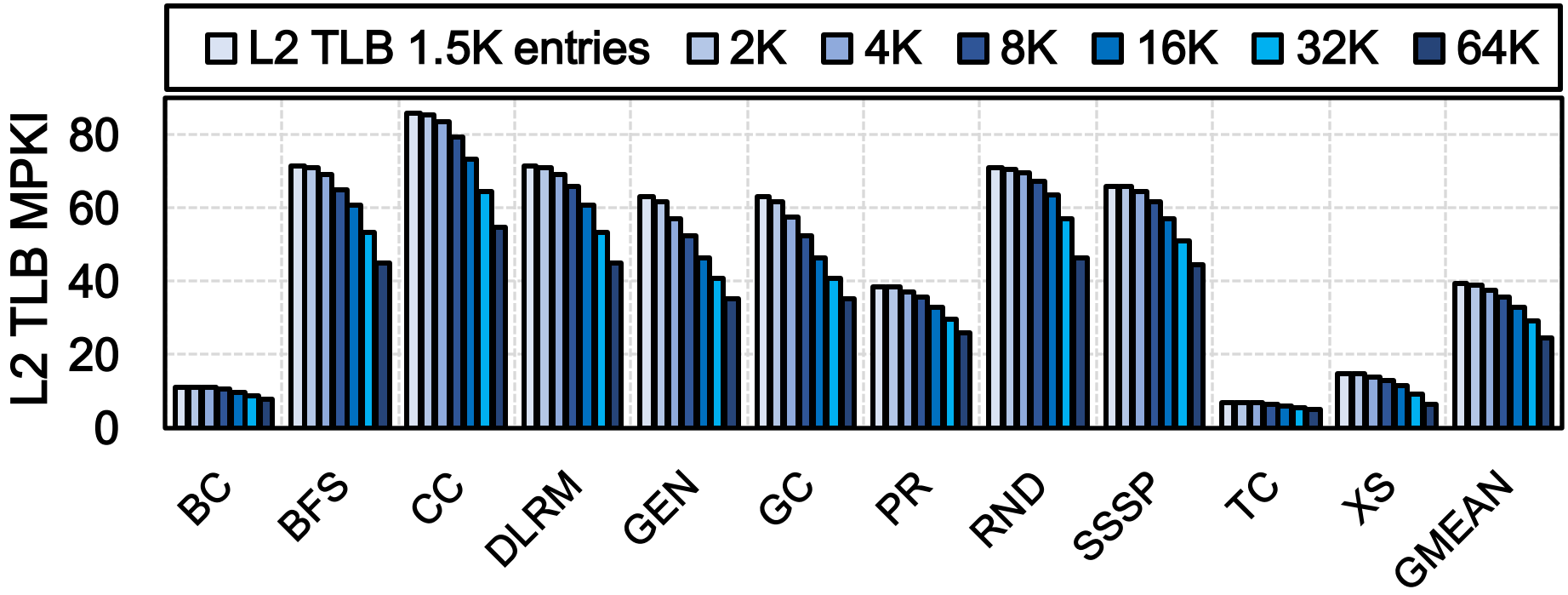
1. Long Running
2. Irregular Memory Accesses
3. Large Data Footprints

High Latency PTWs



138 cycles on average to perform a page table walk

Frequent PTWs



Even the largest 64K-entry L2 TLB experiences 24 MPKI on average

Traditional Workloads

2

High Latency Address Translation



Address translation is a first-order bottleneck

Address Translation Overheads



On average 26% of execution time (measured in a real system) is spent on address translation

Address Translation Overheads

Industrial[1,2] studies also show that the modern web, machine learning and database workloads spend up to ~30% of execution time on address translation

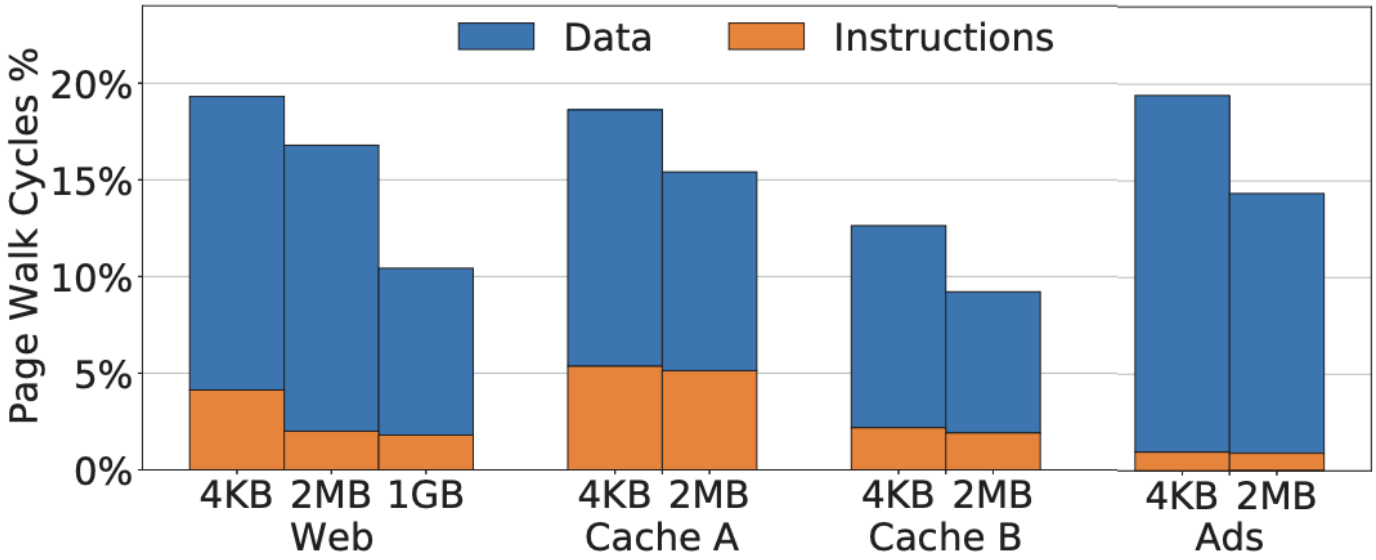
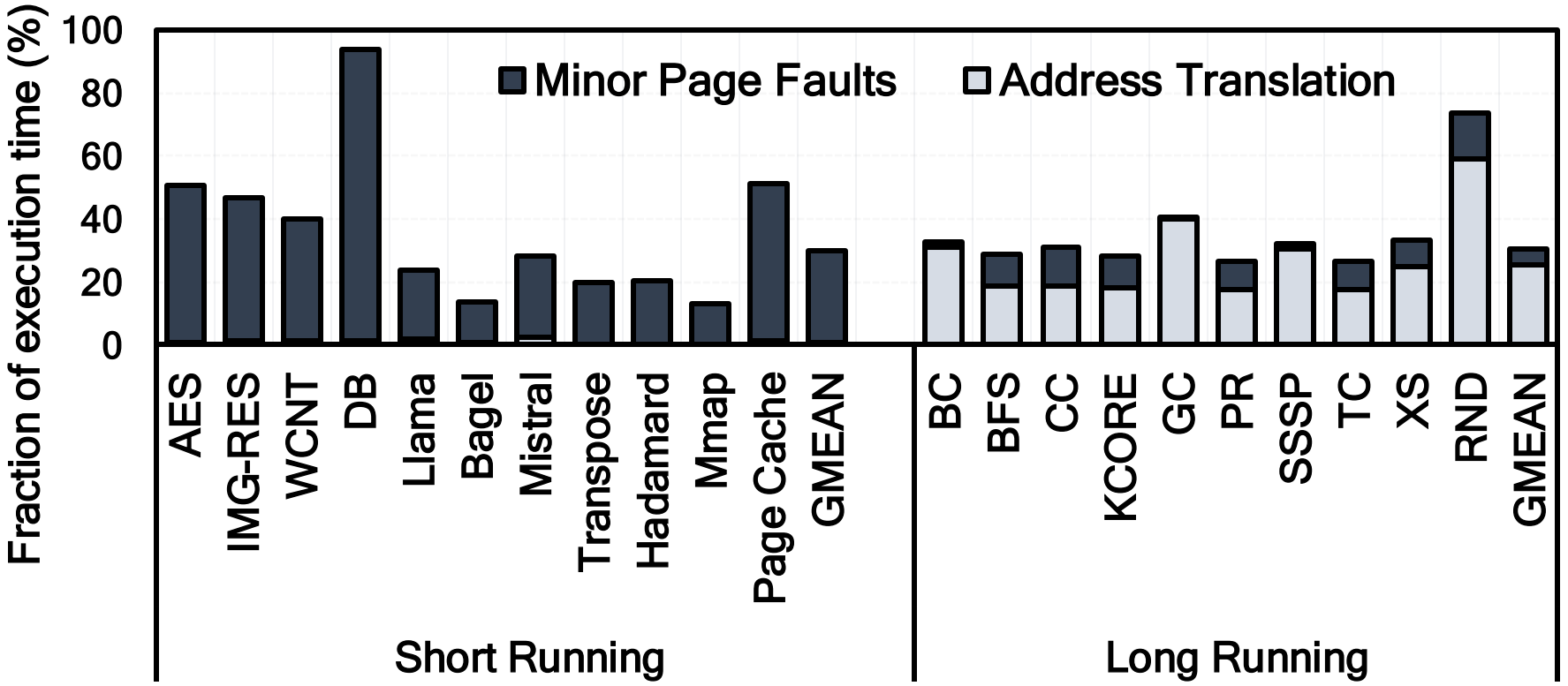


Figure 3: Percentage of cycles lost due to page walks.

[1] Contiguitas: The Pursuit of Physical Memory Contiguity in Datacenters ISCA '23

[2] Radiant: Efficient Page Table Management for Tiered Memory Systems ISMM '22

High Virtual Memory Overheads

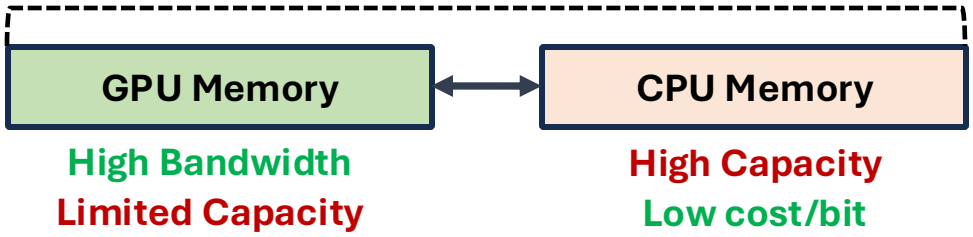


VM causes high overheads in diverse emerging workloads

Virtual Memory overheads are **expected to increase** as we transition to larger physical address spaces

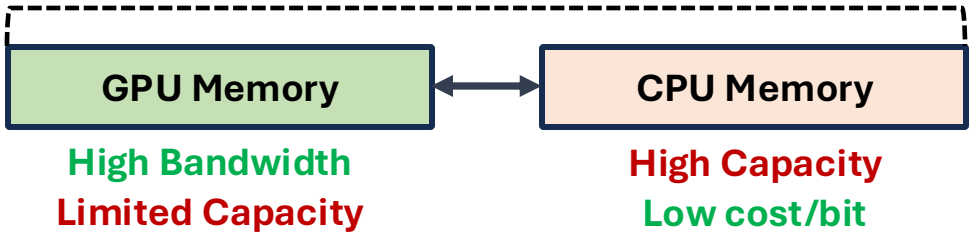
Going into the Future

1 Unified Virtual Memory



Going into the Future

1 Unified Virtual Memory



2 Direct Storage Access

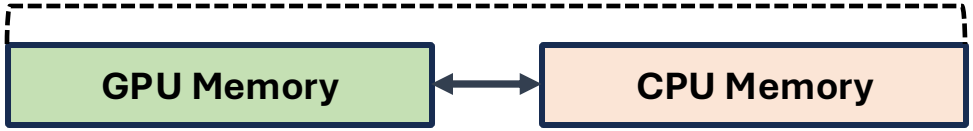


High-end SSD

Byte-addressable interface

Going into the Future

1 Unified Virtual Memory



High Bandwidth
Limited Capacity

High Capacity
Low cost/bit

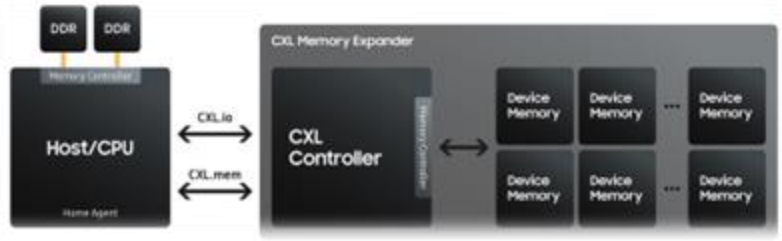
2 Direct Storage Access



High-end SSD

Byte-addressable interface

Memory Disaggregation 3



Access to both local and remote memory modules



Researchers try to save the day

Co-design the hardware and the OS to reduce VM overheads

Software-managed TLB Subsystem

Nagle+ ISCA 1993

Design Tradeoffs for Software-Managed TLBs

Ryoo+ ISCA 2017

Rethinking TLB Designs in Virtualized Environments: A Very Large Part-of-Memory TLB

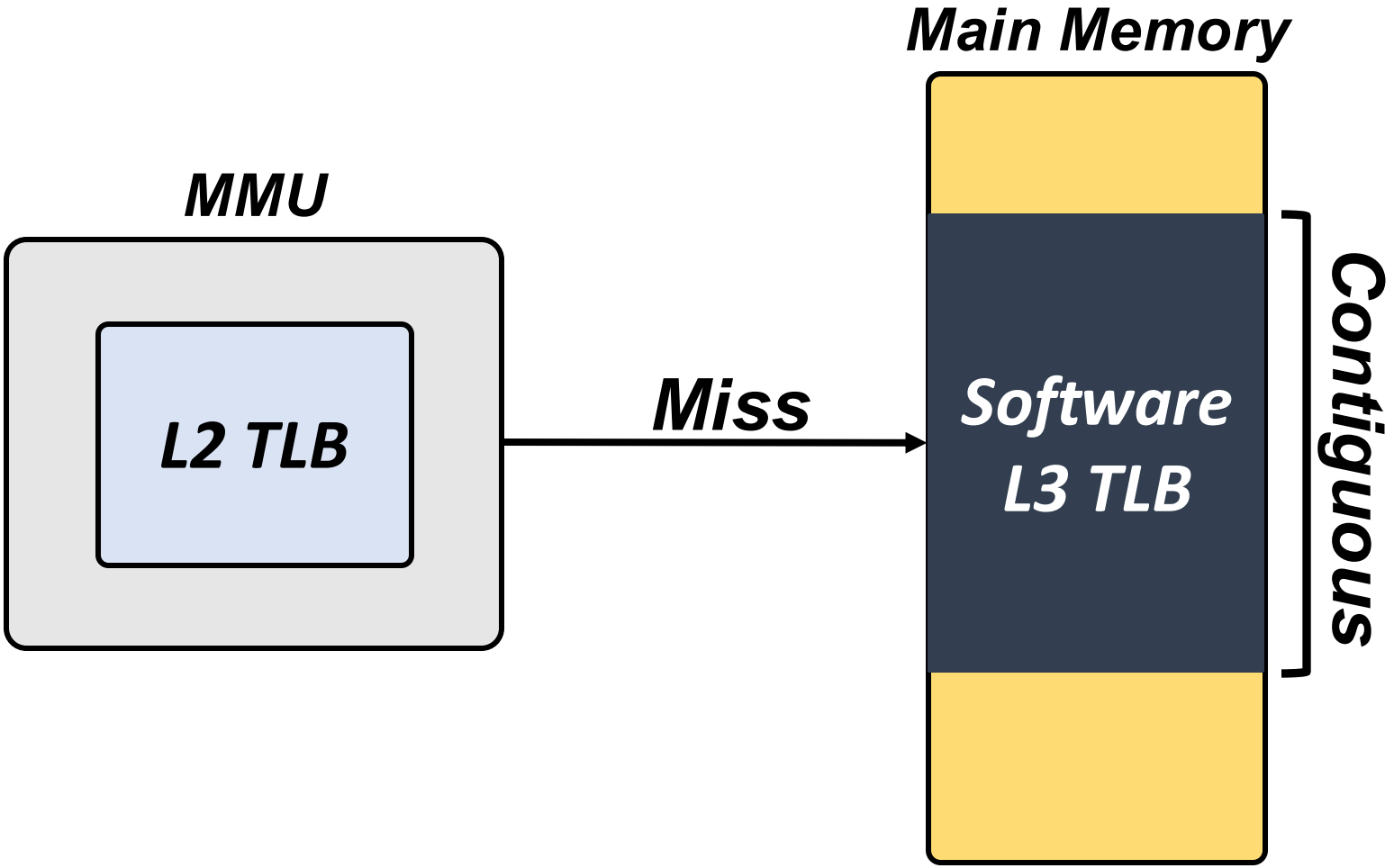
Marathe+ MICRO 2017

CSALT: Context Switch Aware Large TLB

Jaleel+ TACO 2019

DUCATI: High-performance Address Translation by Extending TLB Reach of GPU-accelerated Systems

Large Software-Managed L3 TLB



Ryoo+ ISCA 2017

Rethinking TLB Designs in Virtualized Environments:
A Very Large Part-of-Memory TLB

Co-design the hardware and the OS to reduce VM overheads

Leveraging VA-to-PA Contiguity

Pham+ MICRO 2012

CoLT: Coalesced Large-Reach TLBs

Karakostas*,Ghandhi*+ ISCA 2015

Redundant Memory Mappings for Fast Access to Large Memories

Ausavarungnirun+ MICRO 2017

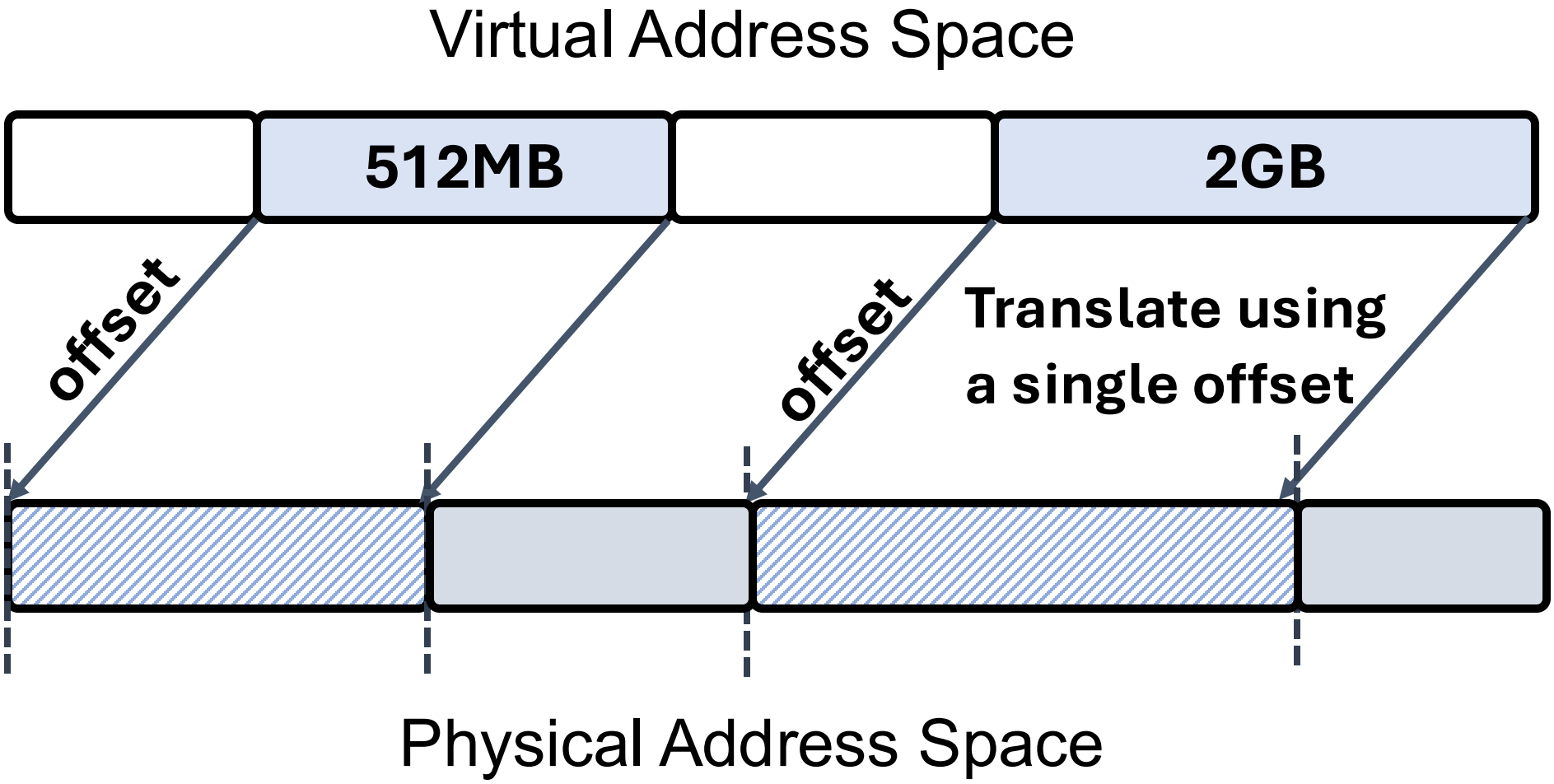
Mosaic: An Application-Transparent Hardware–Software Cooperative Memory Manager for GPUs

Zhao+ ISCA 2022

Contiguitas: The Pursuit of Physical Memory Contiguity in Datacenters

Software-managed TLB Subsystem

Range-based Mappings



Karakostas*,Ghandhi*+ ISCA 2015

Redundant Memory Mappings for Fast Access to Large Memories

Co-design the hardware and the OS to reduce VM overheads

Accelerating Memory Allocation

Lee+ ISCA 2020

A Case for Hardware-Based Demand Paging

Tirumalasetty+ TACO 2022

Reducing Minor Page Fault Overheads through Enhanced Page Walker

Wang+ MICRO 2023

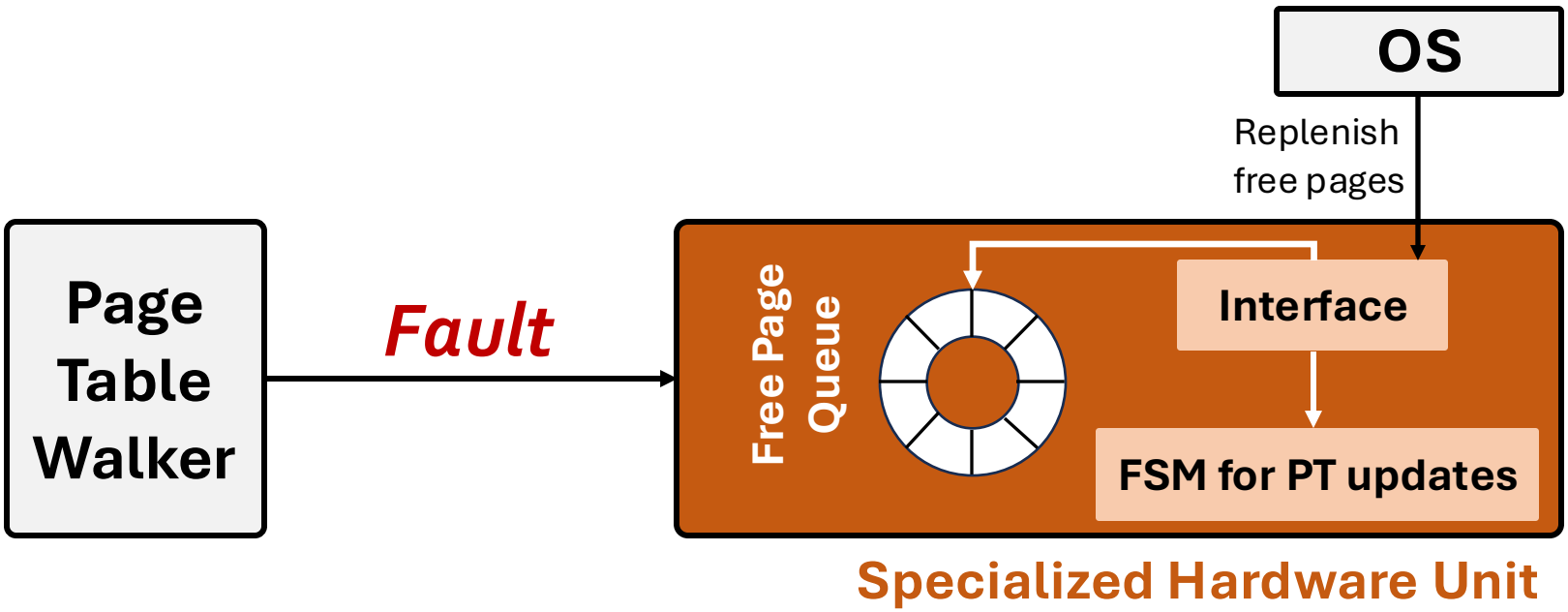
Memento: Architectural Support for Ephemeral Memory Management in Serverless Environments

Software-managed TLB Subsystem

Leveraging VA-to-PA Contiguity

Prior Works: Delegate to HW

Idea: Delegate parts of memory allocation at the hardware level



[1] Reducing Minor Page Fault Overheads through Enhanced Page Walker, TACO 2022
[2] Memento: Architectural Support for Ephemeral Memory Management in Serverless Environments, MICRO 2023
[3] A Case for Hardware-Based Demand Paging, ISCA 2020

Co-design the hardware and the OS to reduce VM overheads

Alternative Address Mappings

Picorel+ PACT 2016

Near-Memory Address Translation

Gosakan+ ASPLOS 2023

Mosaic Pages: Big TLB Reach with Small Pages

Kanellopoulos+ MICRO 2023

Utopia: Fast and Efficient Address Translation via Hybrid Restrictive & Flexible Virtual-to-Physical Address Mappings

Software-managed TLB Subsystem

Leverage VA-to-PA Contiguity

Accelerating Memory Allocation

Rethinking Virtual Memory

Konstantinos Kanellopoulos, Rahul Bera, Kosta Stojiljkovic, Nisa Bostanci, Can Firtina, Rachata Ausavarungnirun, Rakesh Kumar, Nastaran Hajinazar, Mohammad Sadrosadati, Nandita Vijaykumar, and Onur Mutlu,
"Utopia: Fast and Efficient Address Translation via Hybrid Restrictive & Flexible Virtual-to-Physical Address Mappings"
*Proceedings of the 56th International Symposium on Microarchitecture (**MICRO**)*, Toronto, ON, Canada, November 2023.
[Slides (pptx) (pdf)]
[arXiv version]
[Utopia Source Code]

Utopia: Fast and Efficient Address Translation via Hybrid Restrictive & Flexible Virtual-to-Physical Address Mappings

Konstantinos Kanellopoulos[1] Rahul Bera[1] Kosta Stojiljkovic[1] Nisa Bostanci[1] Can Firtina[1]
Rachata Ausavarungnirun[2] Rakesh Kumar[3] Nastaran Hajinazar[4] Mohammad Sadrosadati[1]
Nandita Vijaykumar[5] Onur Mutlu[1]

[1]ETH Zürich [2]King Mongkut's University of Technology North Bangkok
[3]Norwegian University of Science and Technology [4]Intel Labs [5]University of Toronto

Utopia: Key Idea

We propose **Utopia**, a new virtual-to-physical mapping scheme that enables both:

Restrictive Mapping **Flexible Mapping**

Harmoniously co-exist in the system

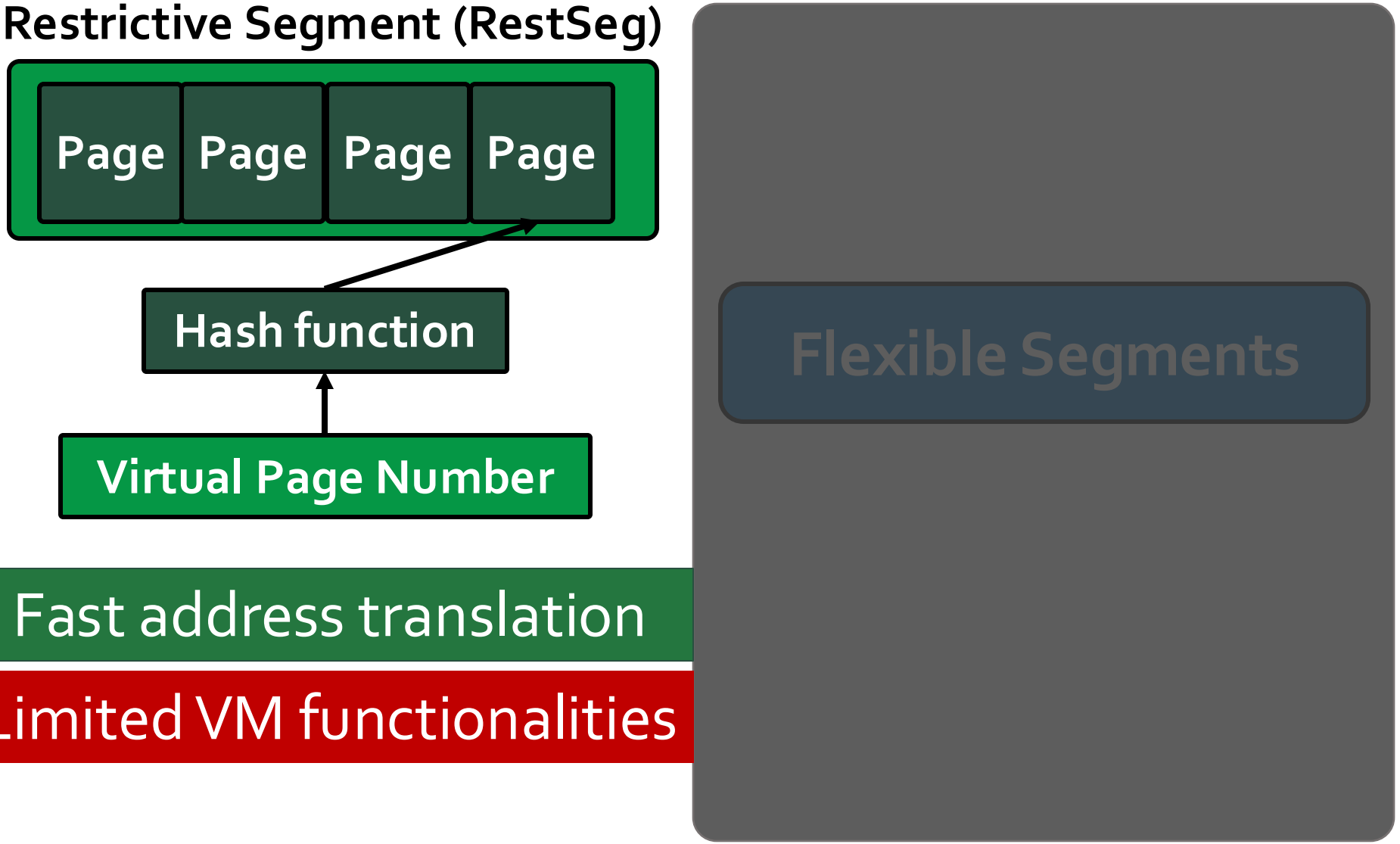
Utopia: Key Idea

Manage physical memory using two types of physical memory segments:

Restrictive Segments

Flexible Segments

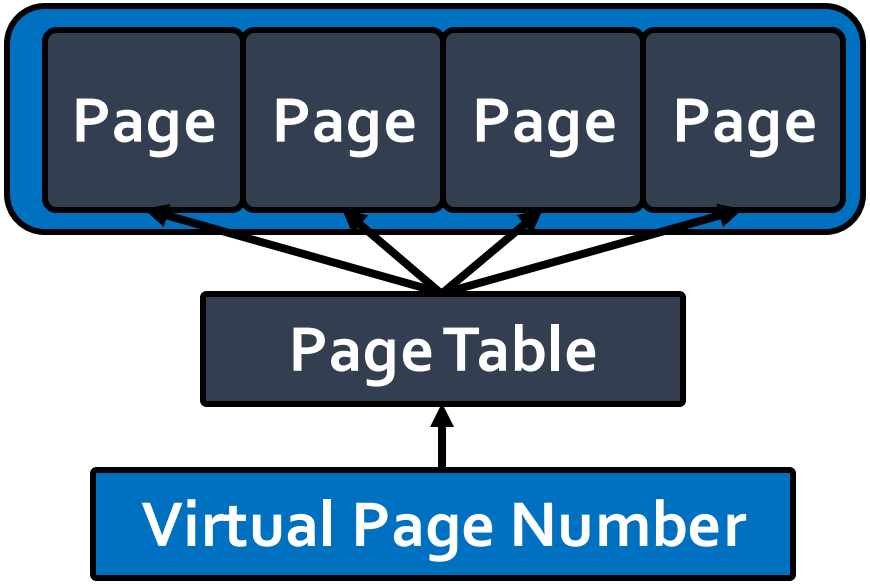
Utopia: Key Idea (I)



Utopia: Key Idea (II)

Restrictive Segments

Flexible Segment (FlexSeg)



Supports all conventional VM features

High-latency address translation

Co-design the hardware and the OS to reduce VM overheads

Rethinking the Page Table Design

Software-managed TLB Subsystem

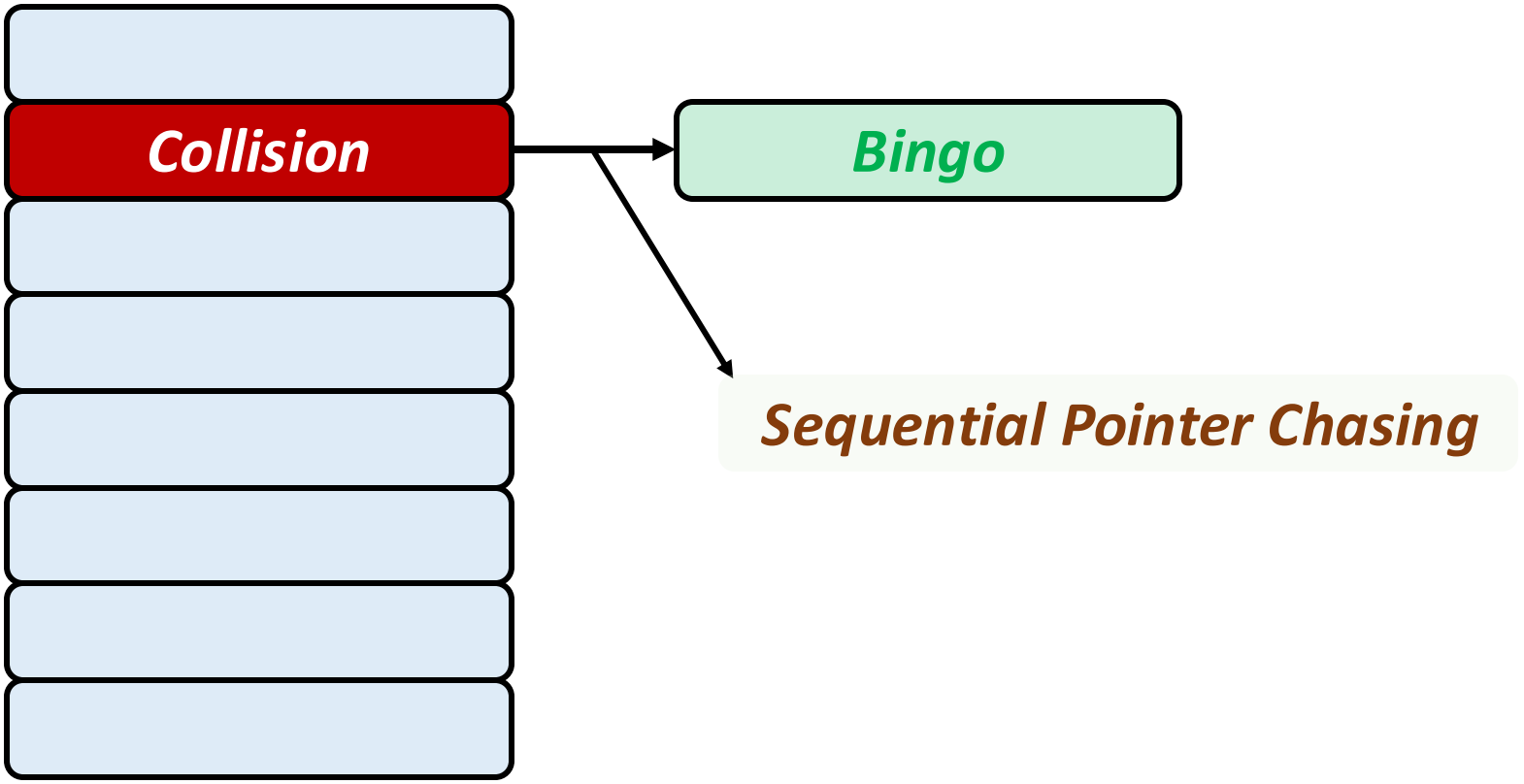
Leverage VA-to-PA Contiguity

Accelerating Memory Allocation

Alternative Address Mappings

Hash-based Page Table: Chaining

Chain upon Collision



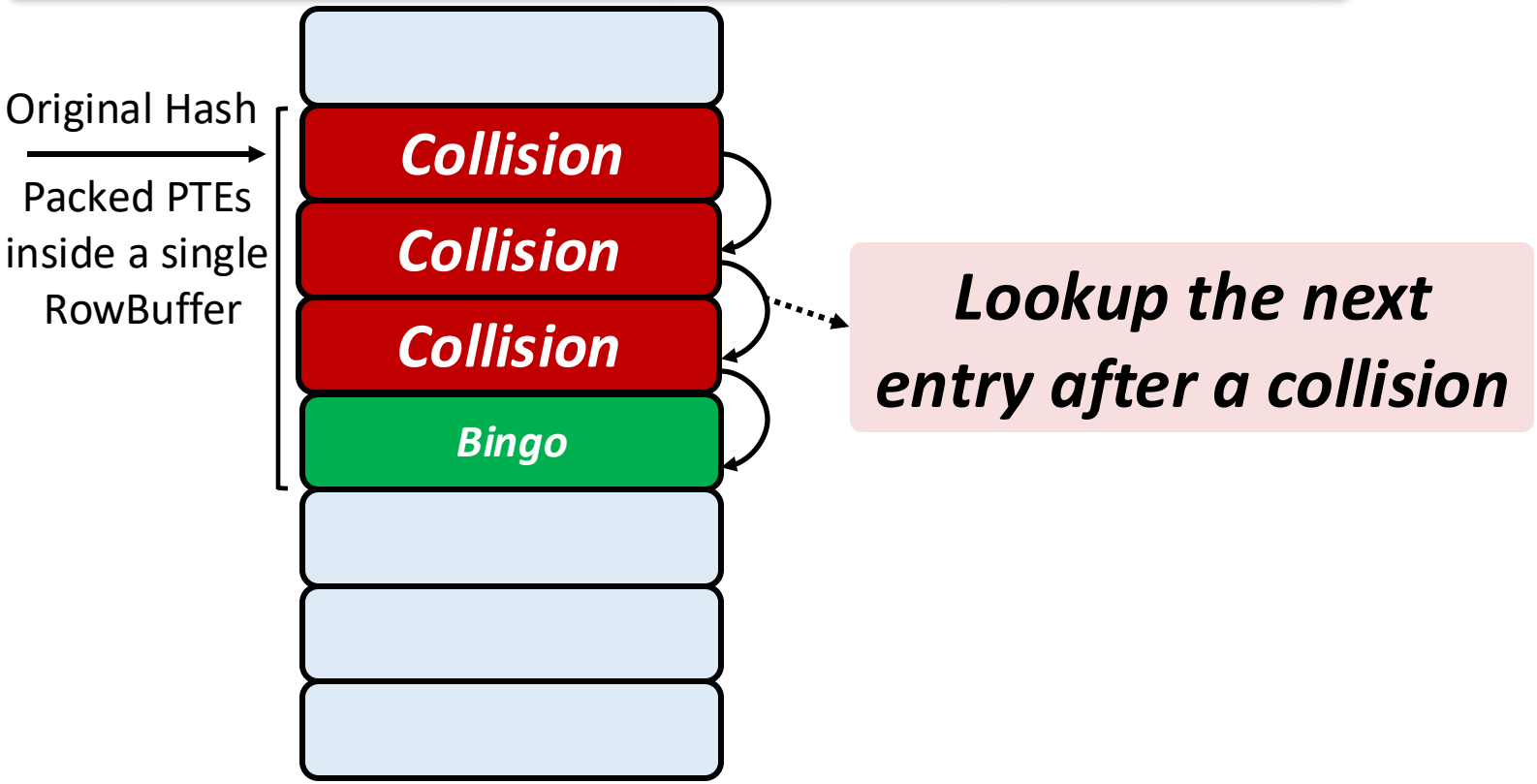
Hash-based Page Table: Open Addressing

Hash, Don't Cache (the Page Table)

Idan Yaniv
Technion – Israel Institute of Technology
idanyani@cs.technion.ac.il

Dan Tsafrir
Technion – Israel Institute of Technology
dan@cs.technion.ac.il

SIGMETRICS 2016

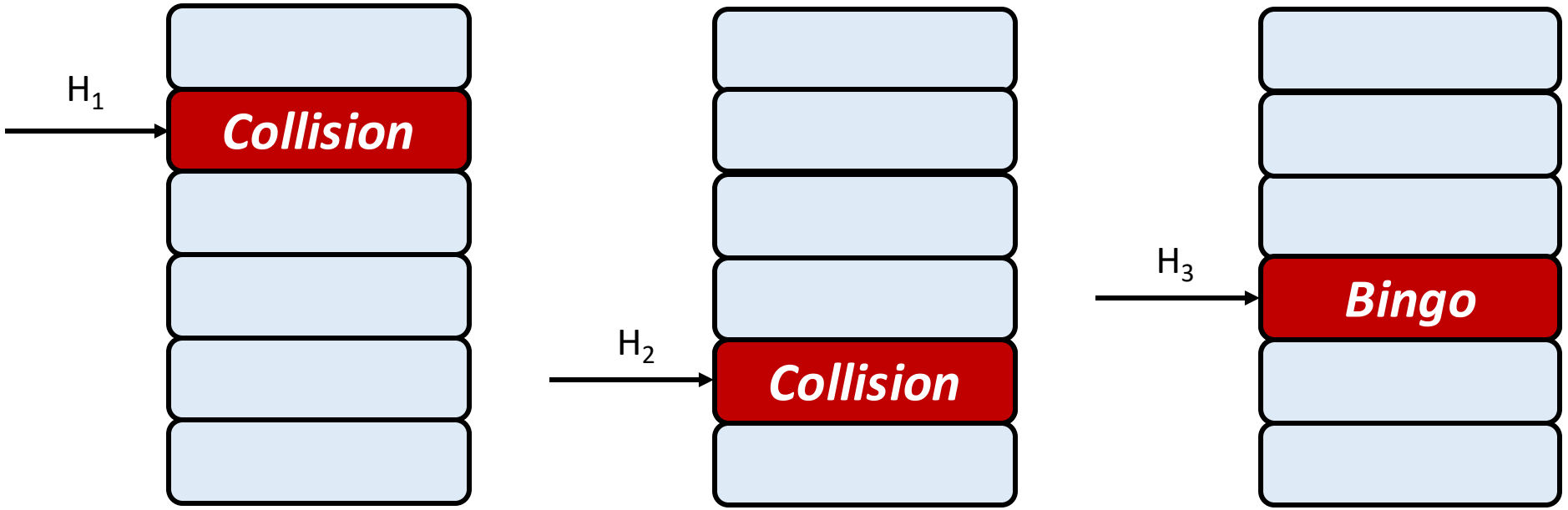


Cuckoo Hash-based Page Tables

Elastic Cuckoo Page Tables: Rethinking Virtual Memory Translation for Parallelism

Dimitrios Skarlatos, Apostolos Kokolis, Tianyin Xu, and Josep Torrellas
University of Illinois at Urbana-Champaign
{skarlat2,kokolis2,tyxu,torrella}@illinois.edu

ASPLOS 2020



Multiple parallel memory accesses to different tables

Co-design the hardware and the OS to reduce VM overheads

Rethinking the Page Table Design

Employing Multiple Page Sizes

Example: Transparent Huge Pages



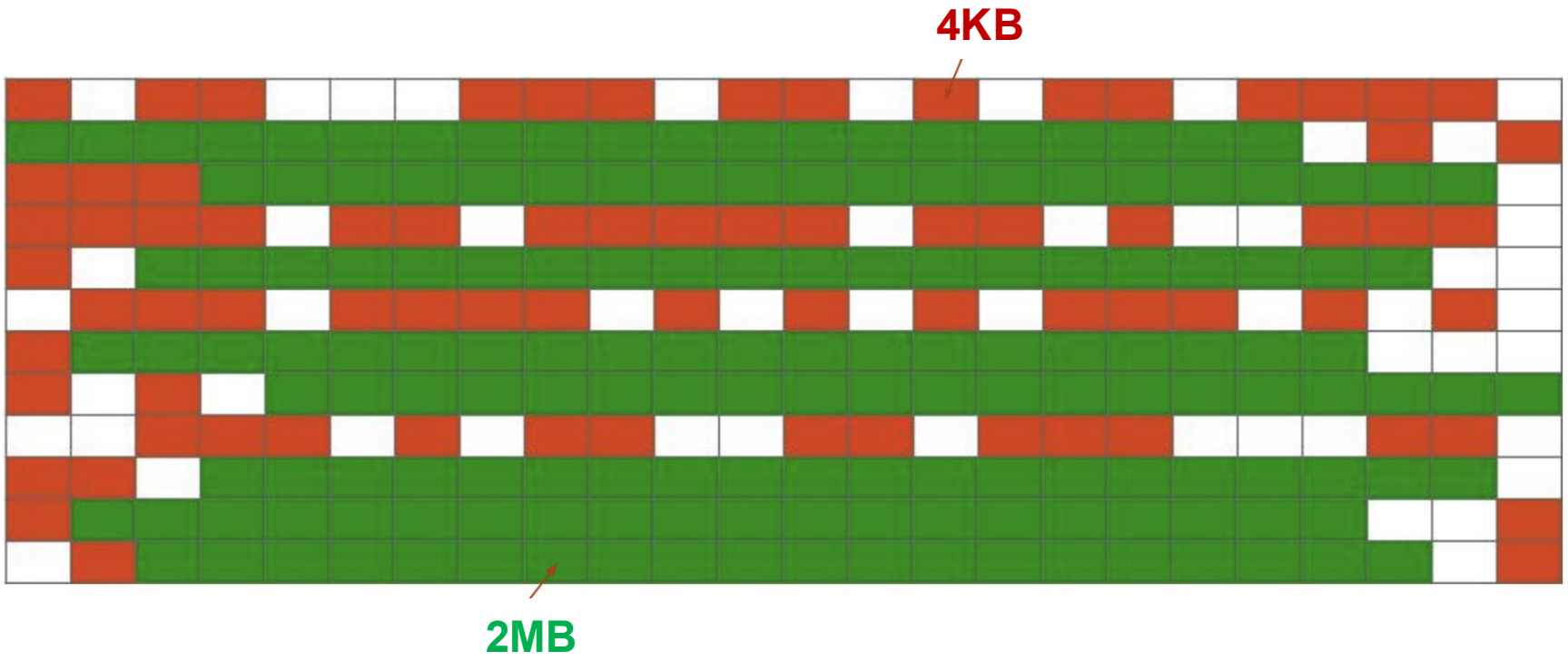
THP works by quietly substituting huge pages into a process's address space when 1) those page are available and
2) it appears that the process would benefit from the change.

The kernel works in the background (`khugepaged`) trying to:

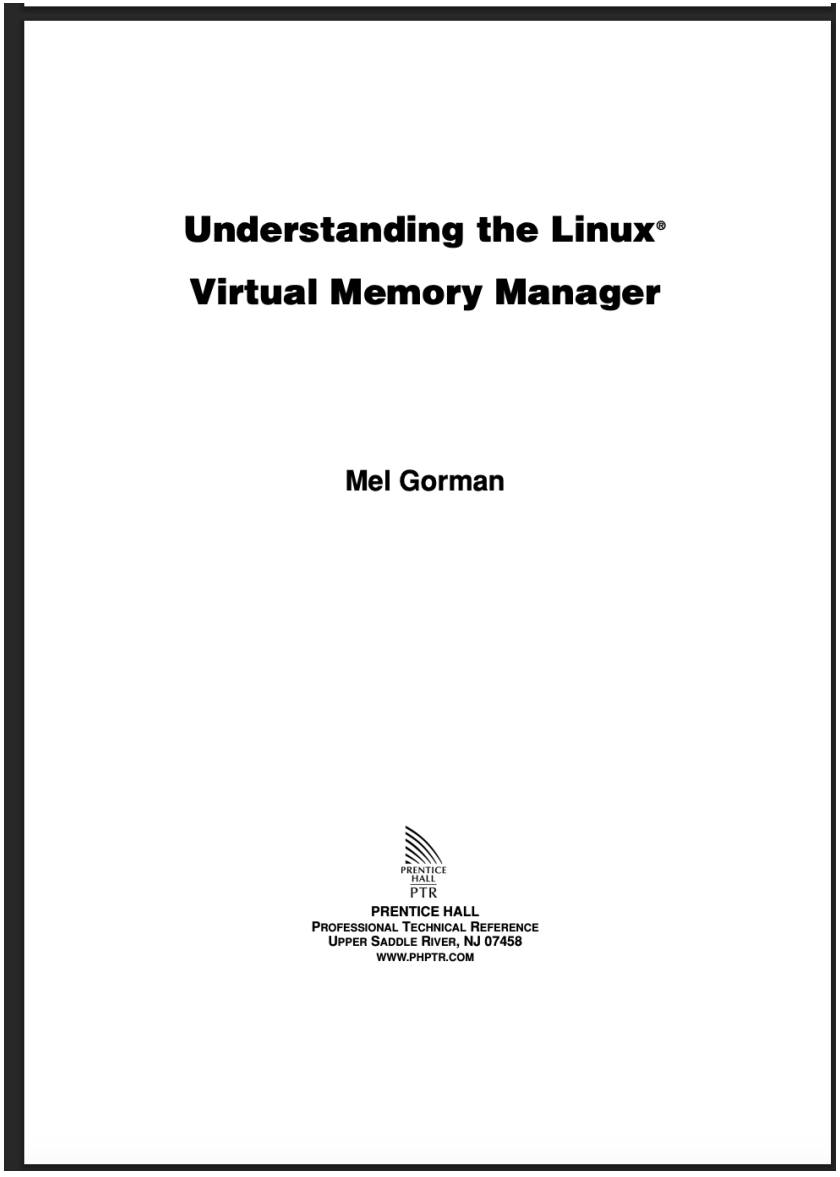
- "create" huge pages
  - find enough contiguous blocks of memory
  - "convert" them into a huge page
- *transparently* allocate them to processes when there is a "fit"
  - shouldn't provide a 2M-page for someone asking 128K

Introduced in Linux in 2.6.38 (2011) by Andrea Arcangeli

Example: Transparent Huge Pages



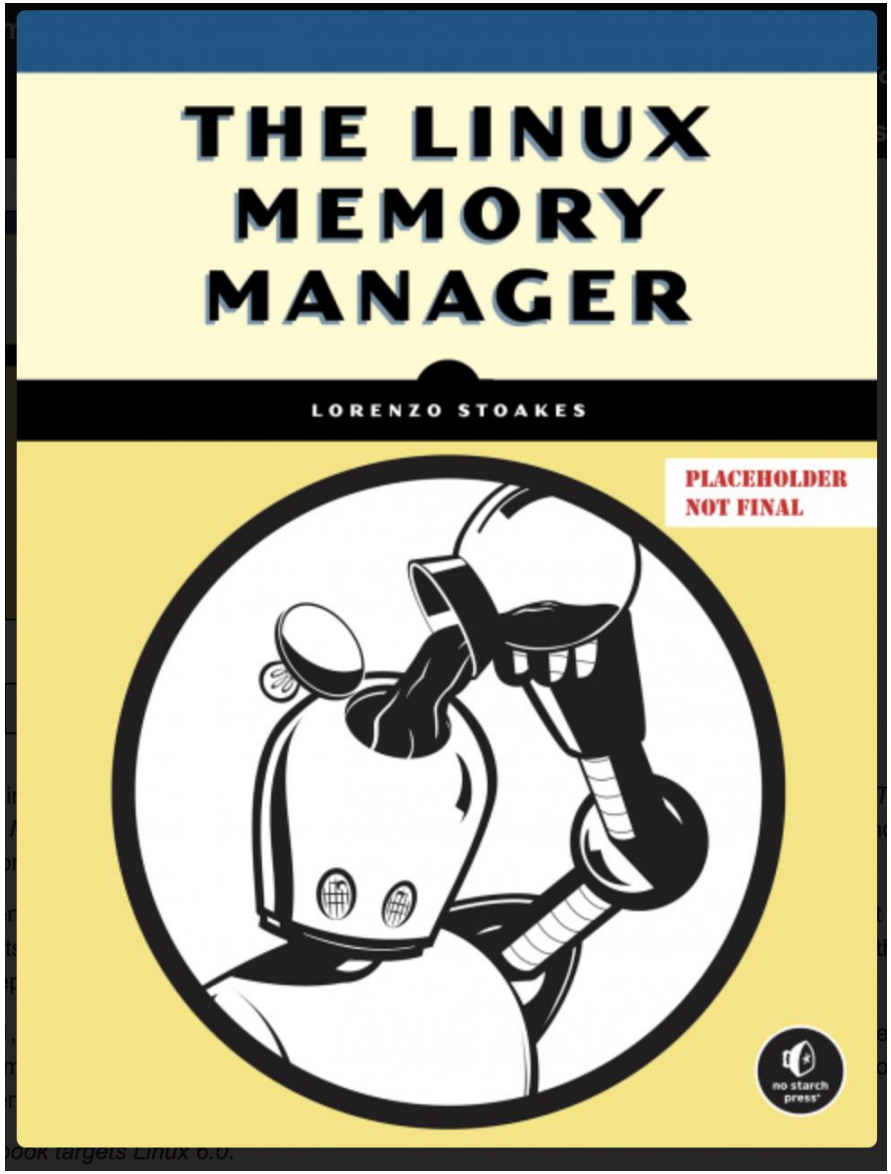
Resources



Understanding the Linux Virtual Memory

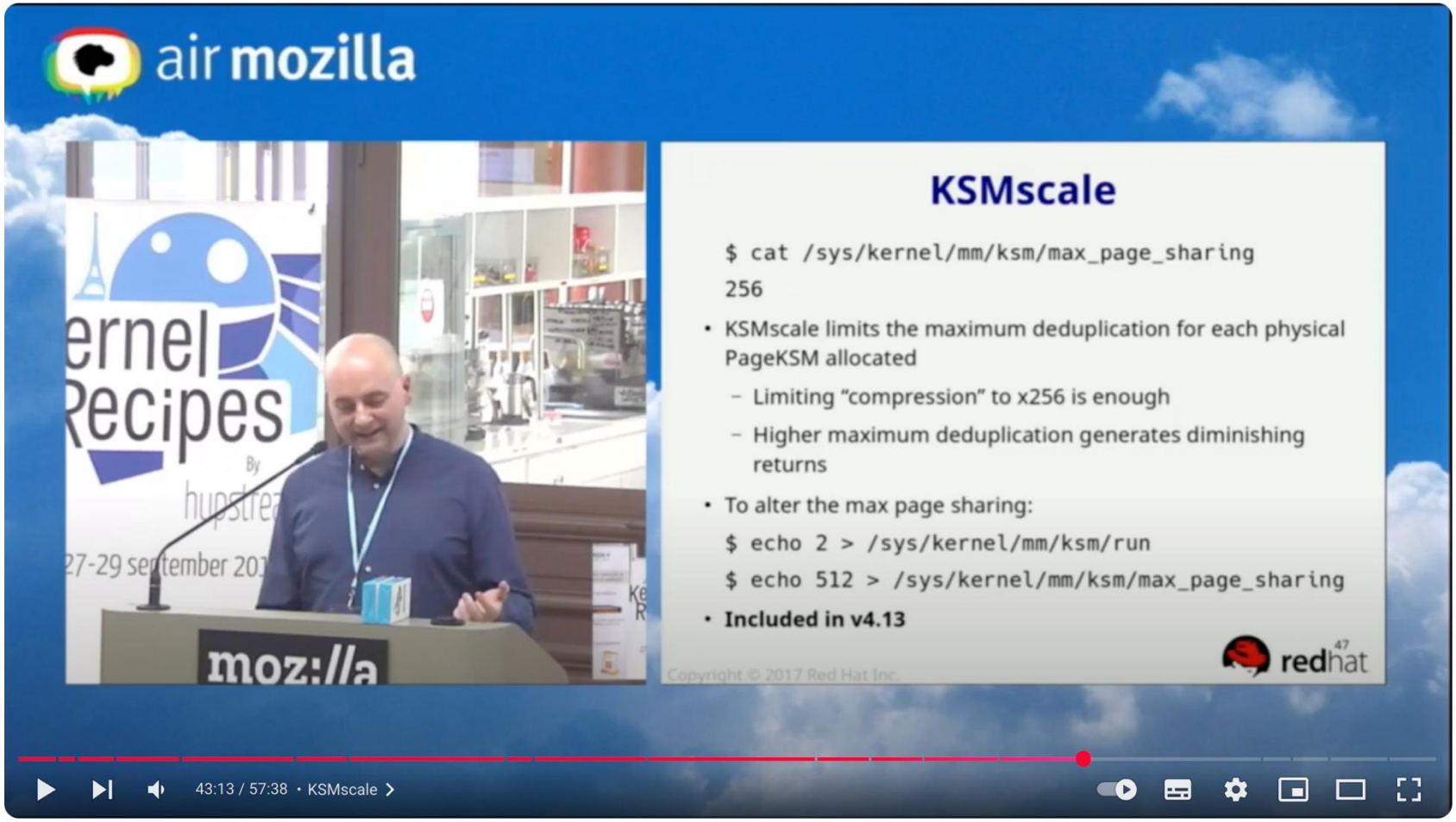
Book by Mel Gorman

Resources



The Linux Memory Manager
by Lorenzo Stoakes

Resources



Kernel Recipes 2017 - 20 years of Linux virtual memory - Andrea Arcangeli

https://www.youtube.com/watch?v=pZghXbeCH5s

Co-design the hardware and the OS to reduce VM overheads



- Rethinking the Page Table Design
- Employing multiple page sizes
- Software-managed TLB Subsystem
- Leverage VA-to-PA Contiguity
- Accelerate Memory Allocation
- Alternative Address Mappings

Wide range of hardware-OS co-design techniques that aim to improve virtual memory

Effectively evaluating VM techniques is crucial for progress in the domain



Enabling Fast and Accurate Virtual Memory Research with an Imitation-based Operating System Simulation Methodology

Presented at ASPLOS 2025

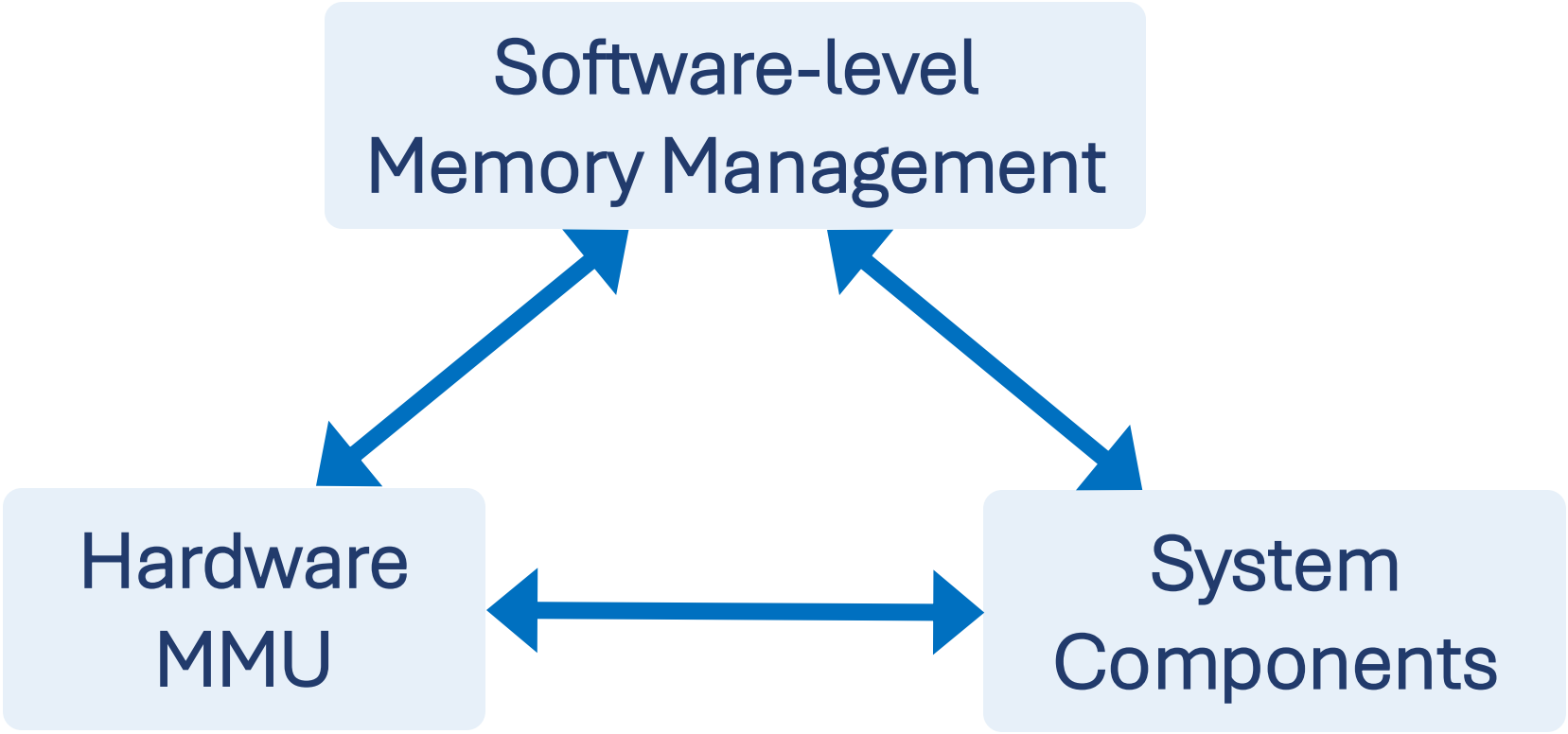
Konstantinos Kanellopoulos Konstantinos Sgouras F. Nisa Bostanci
Andreas Kosmas Kakolyris Berkin Kerim Konar Rahul Bera
Mohammad Sadrosadati Rakesh Kumar Nandita Vijaykumar Onur Mutlu

https://github.com/CMU-SAFARI/Virtuoso

SAFARI
SAFARI Research Group
safari.ethz.ch
ETH zürich
UNIVERSITY OF TORONTO
NTNU

Challenges in Evaluation

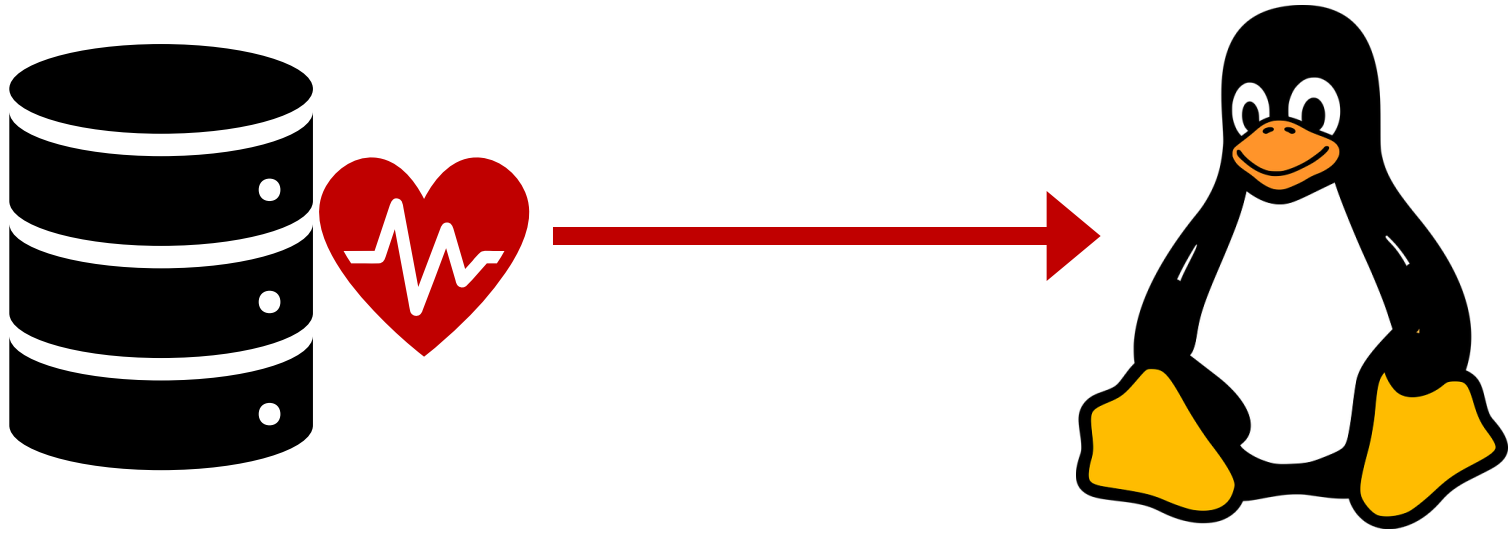
Interplay between system components



Example: Page Table and Large Pages

Page Table Size

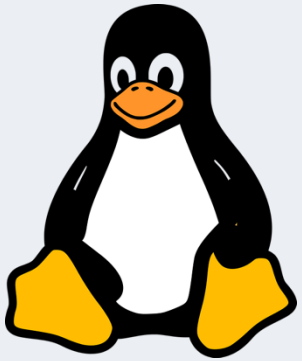
OS Allocation Policy



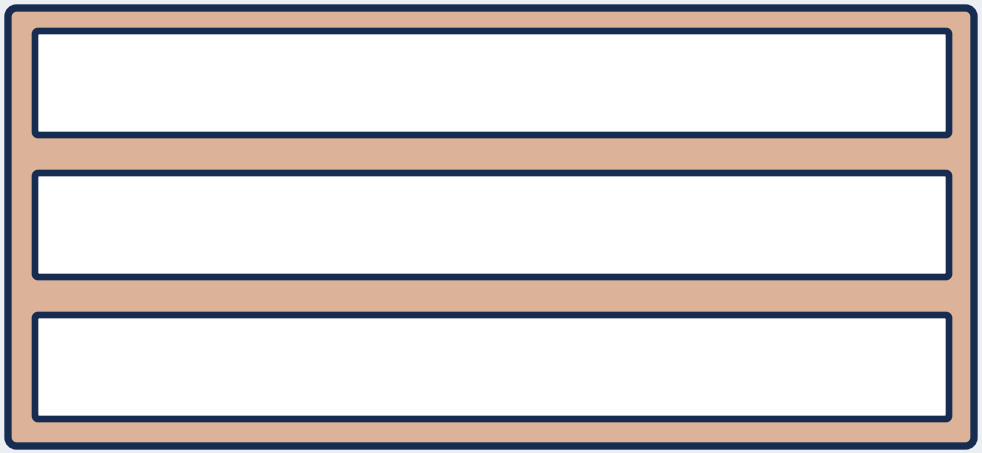
Size of the page table depends on the number of large pages provided by the OS

Example: Page Table and Large Pages

OS Kernel

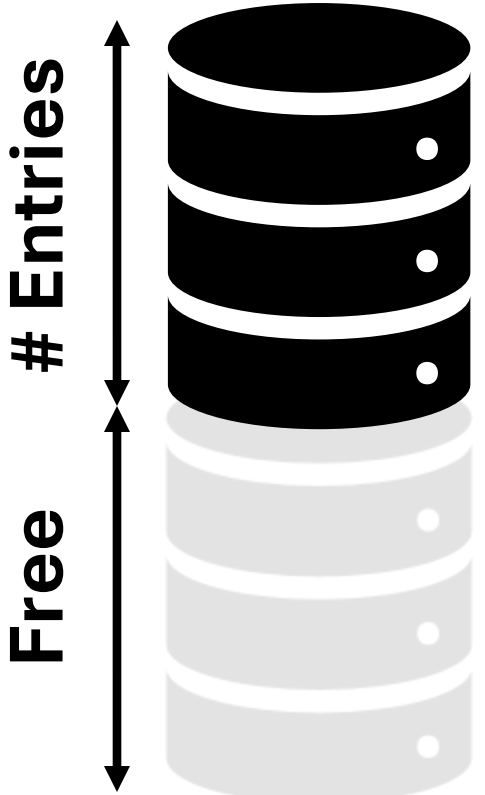


”I can provide plenty of large pages”



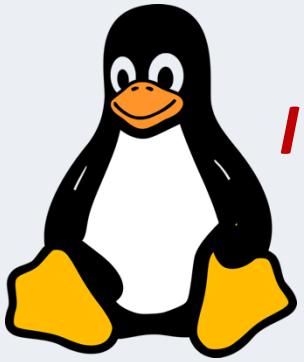
2MB Pages

Page Table

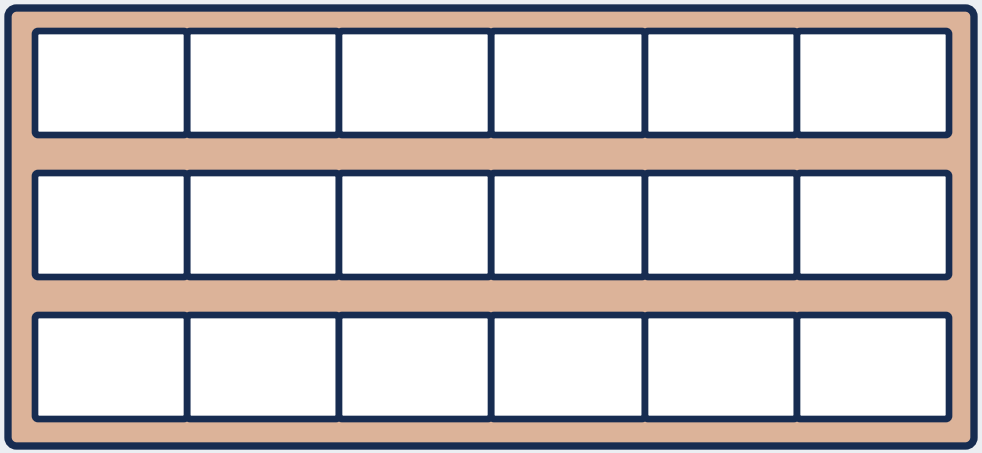


Example: Page Table and Large Pages

OS Kernel

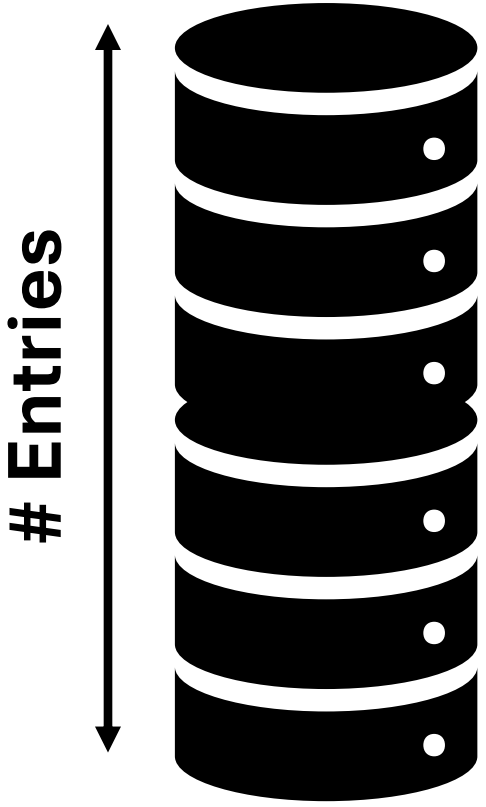


"Fragmentation is rising so I cannot allocate 2MB pages"

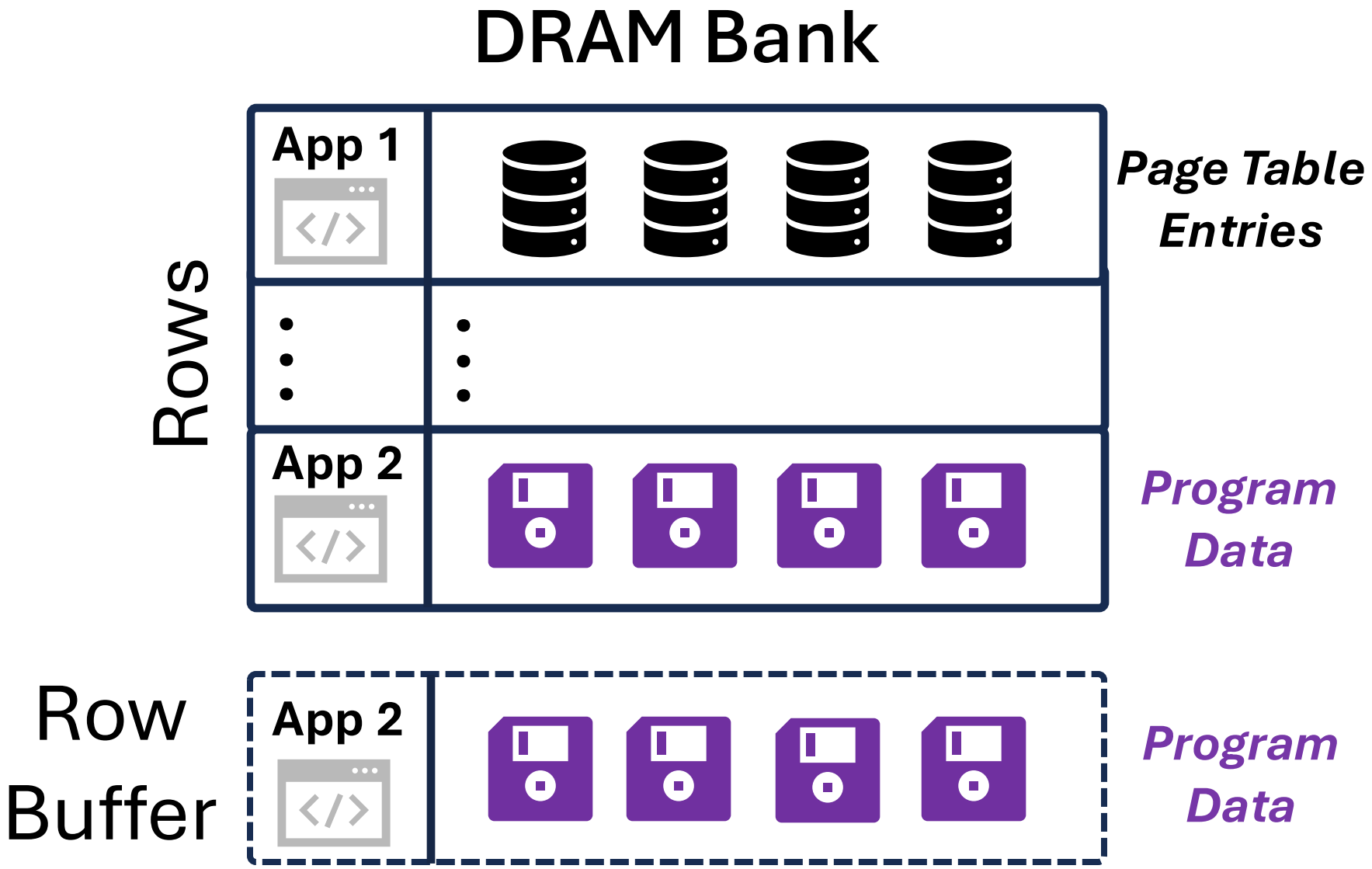


4KB Pages

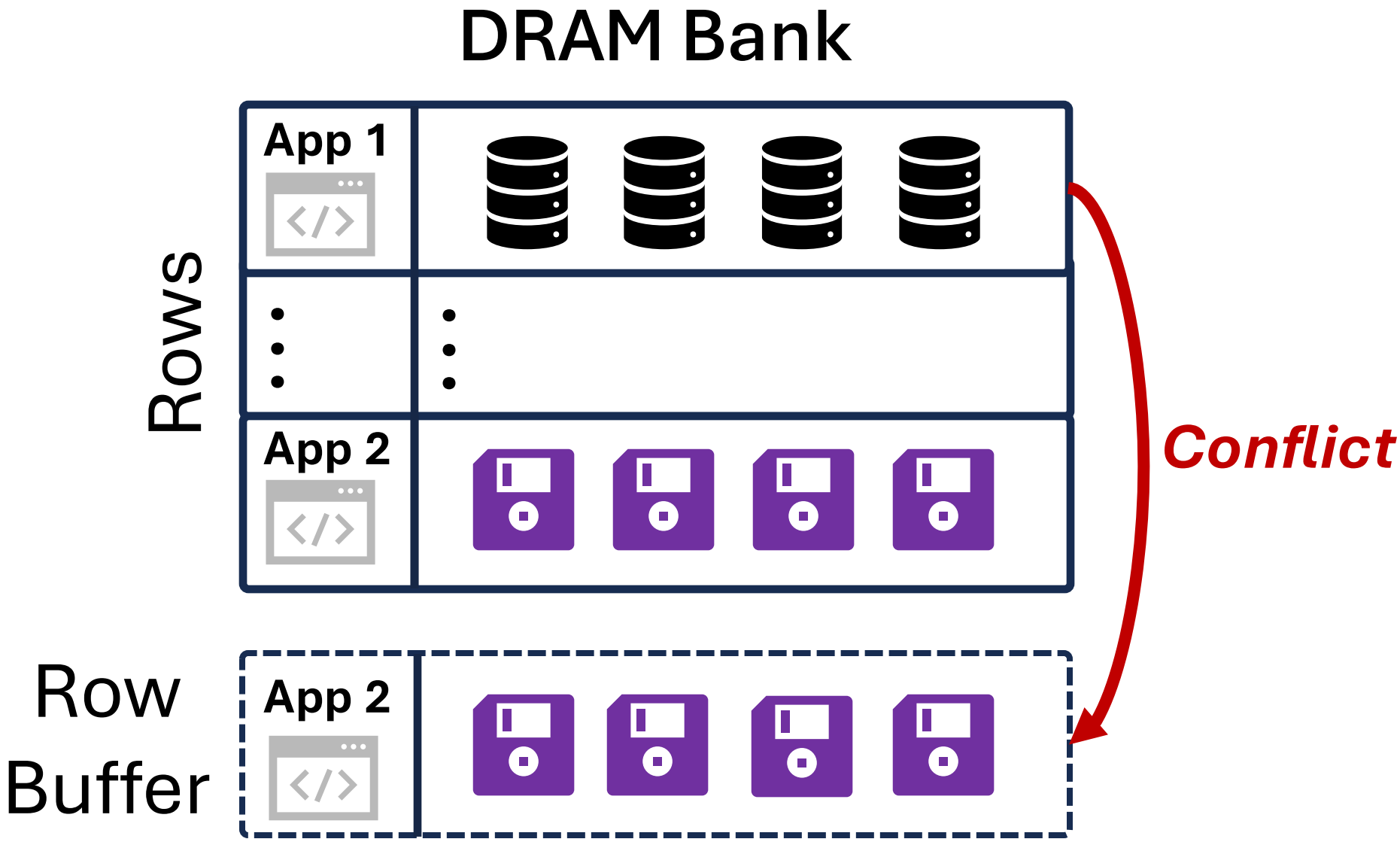
Page Table



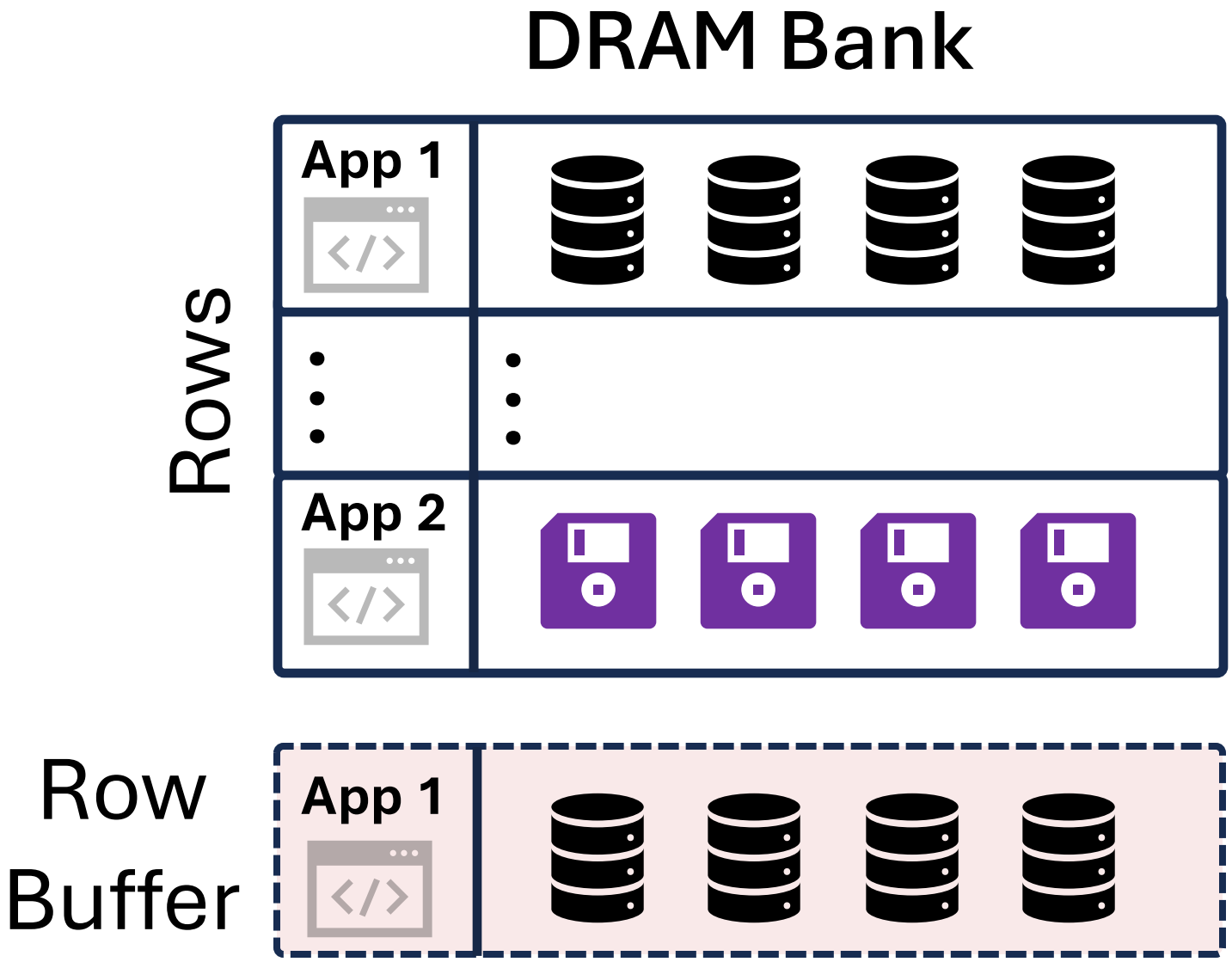
Page Table and Memory Interference



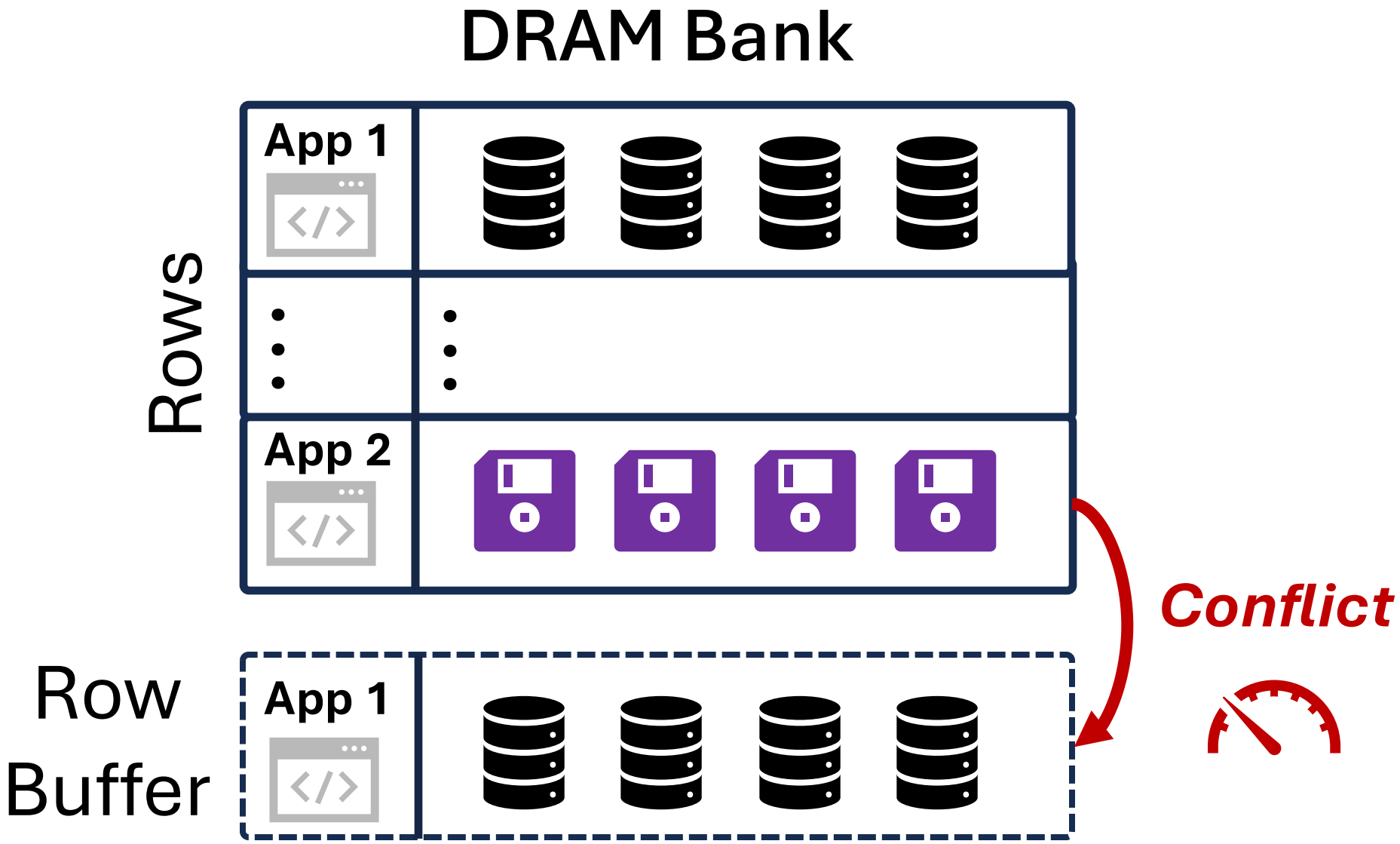
Page Table and Memory Interference



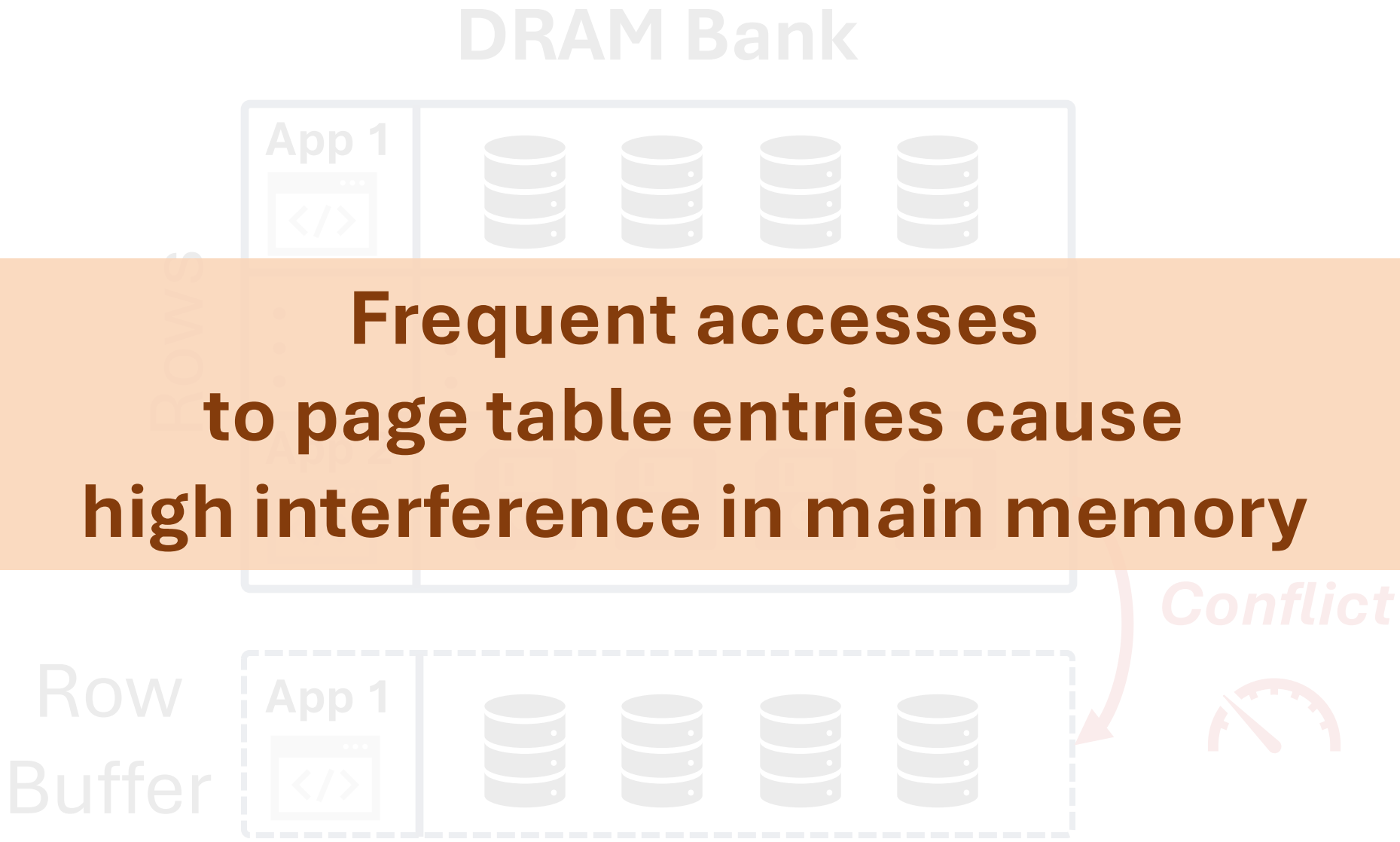
Page Table and Memory Interference



Page Table and Memory Interference

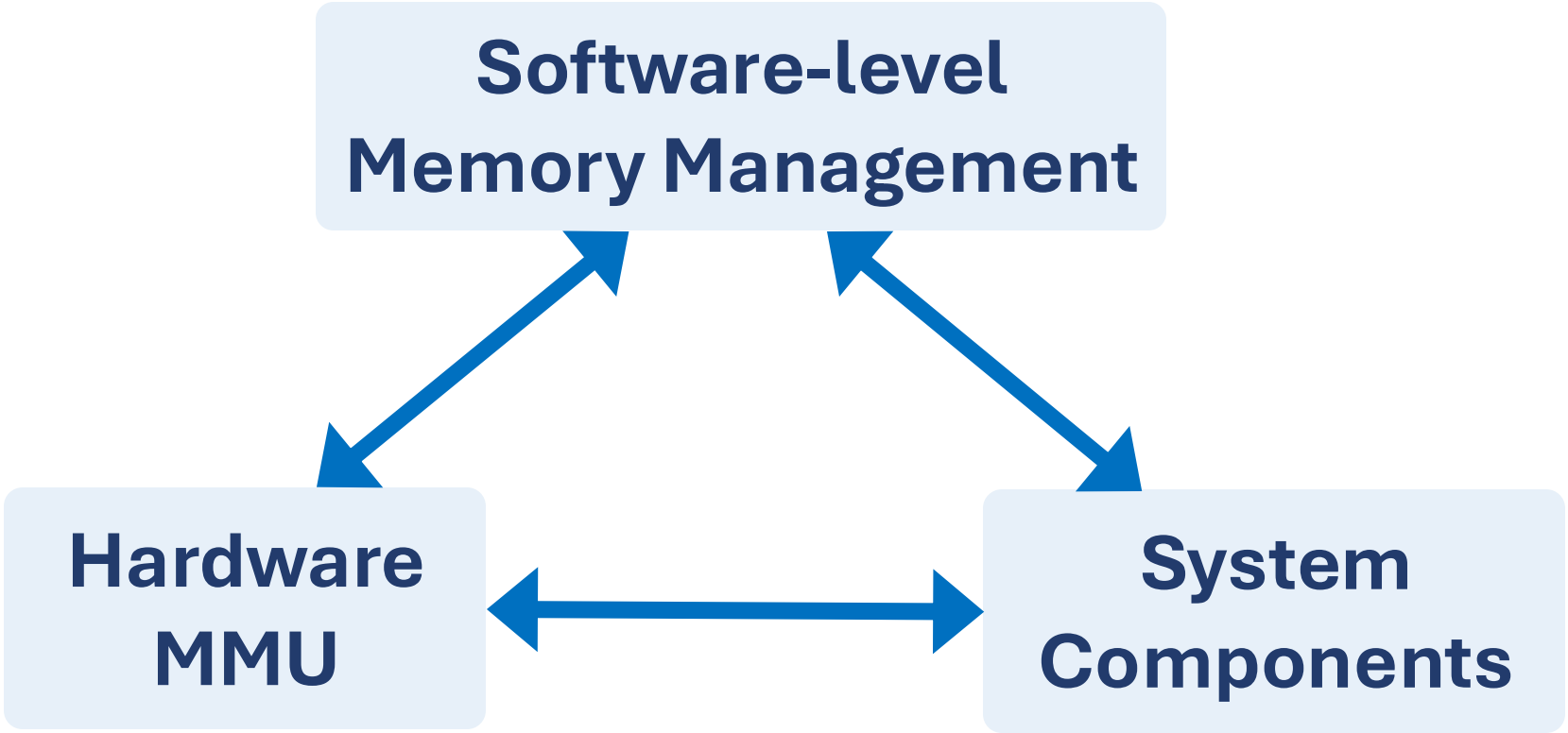


Page Table and Memory Interference



Frequent accesses to page table entries cause high interference in main memory

Challenges in Evaluation



Challenges in Evaluation

Evaluating the interplay between components is critical when assessing current and new VM techniques

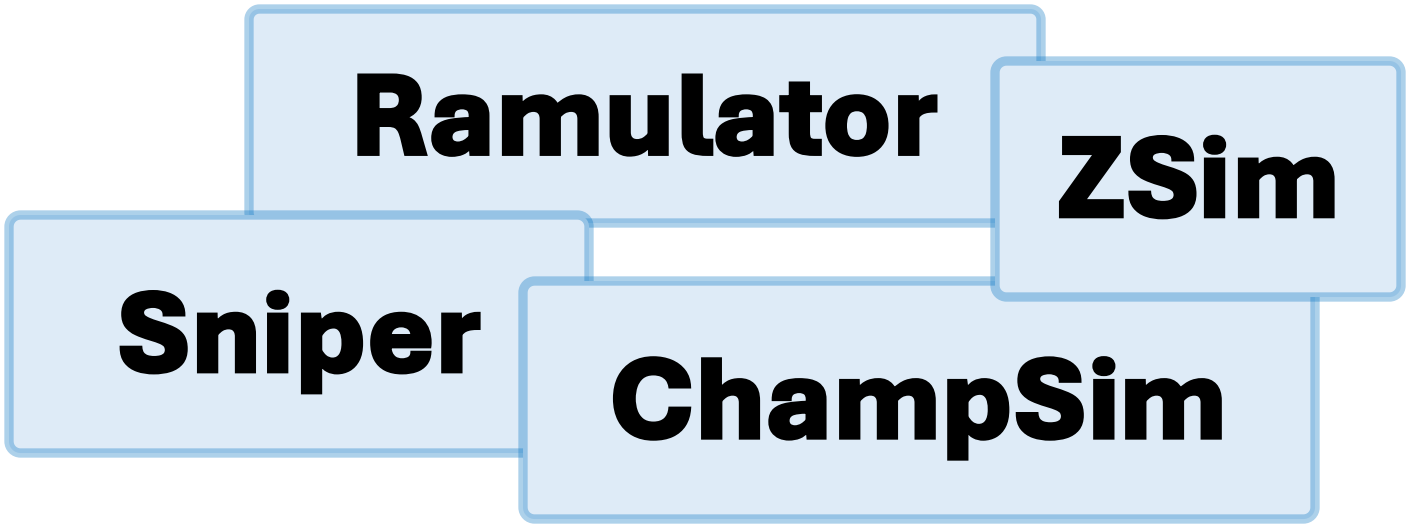
Researchers have architectural simulators at their disposal

Existing Architectural Simulators

Two main classes of simulators

1. **Emulation-based**
2. **Full-system**

Emulation-based Simulators



High simulation speed with a focus on modeling microarchitectural features

Limited support for OS primitives

Analytically estimate VM Overheads

Analytical Estimation of VM Overheads

First-order Models

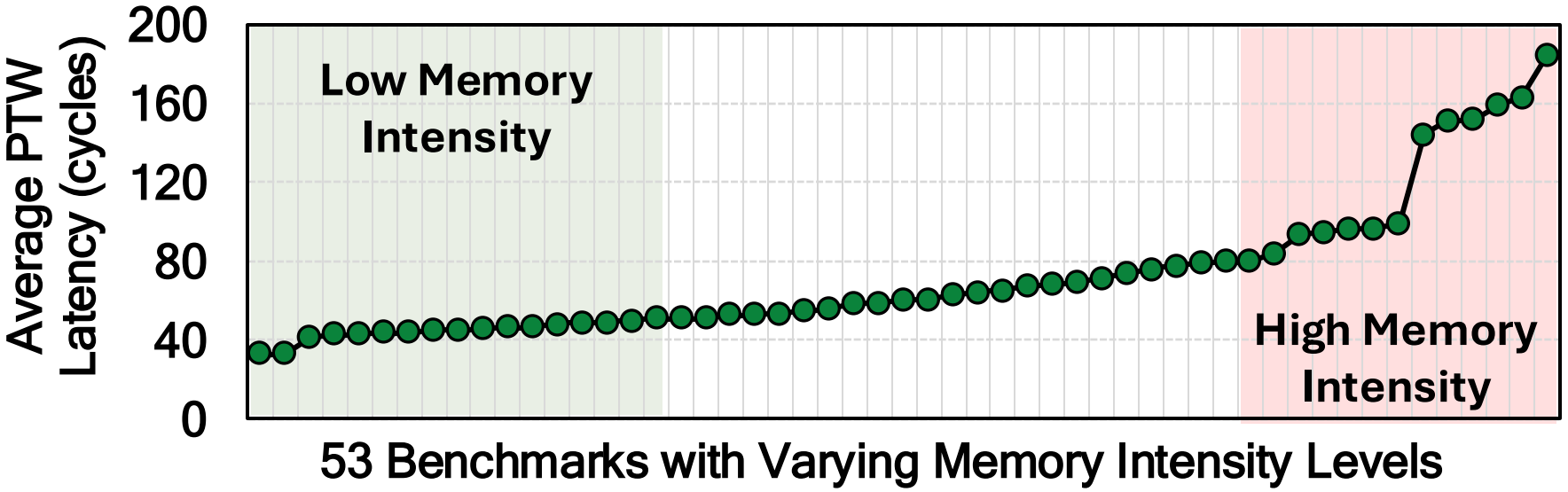
Simple mathematical or algorithmic approximation

Example

Page Table Walk Latency = 100 cycles (fixed)
Minor Page Fault Latency = 2000 cycles (fixed)

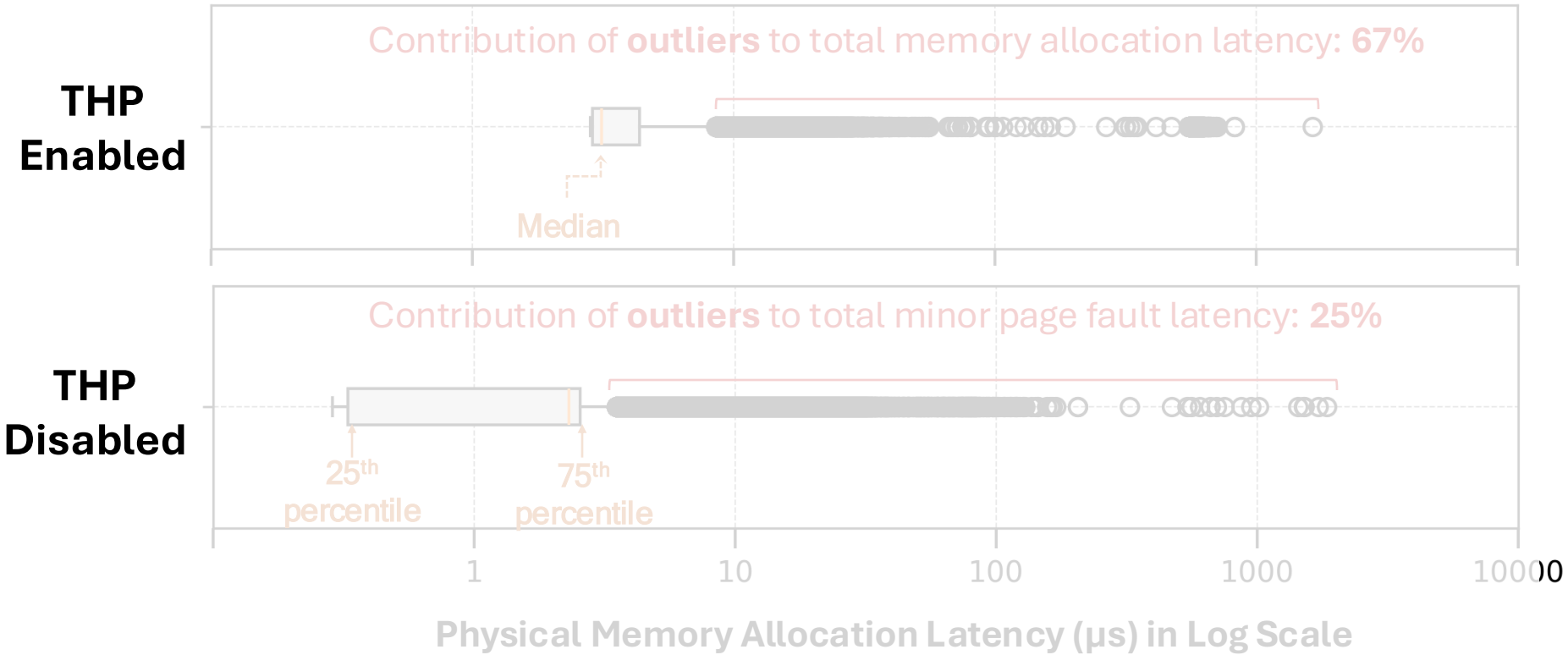
Is this good enough?

Variable PTW Latency

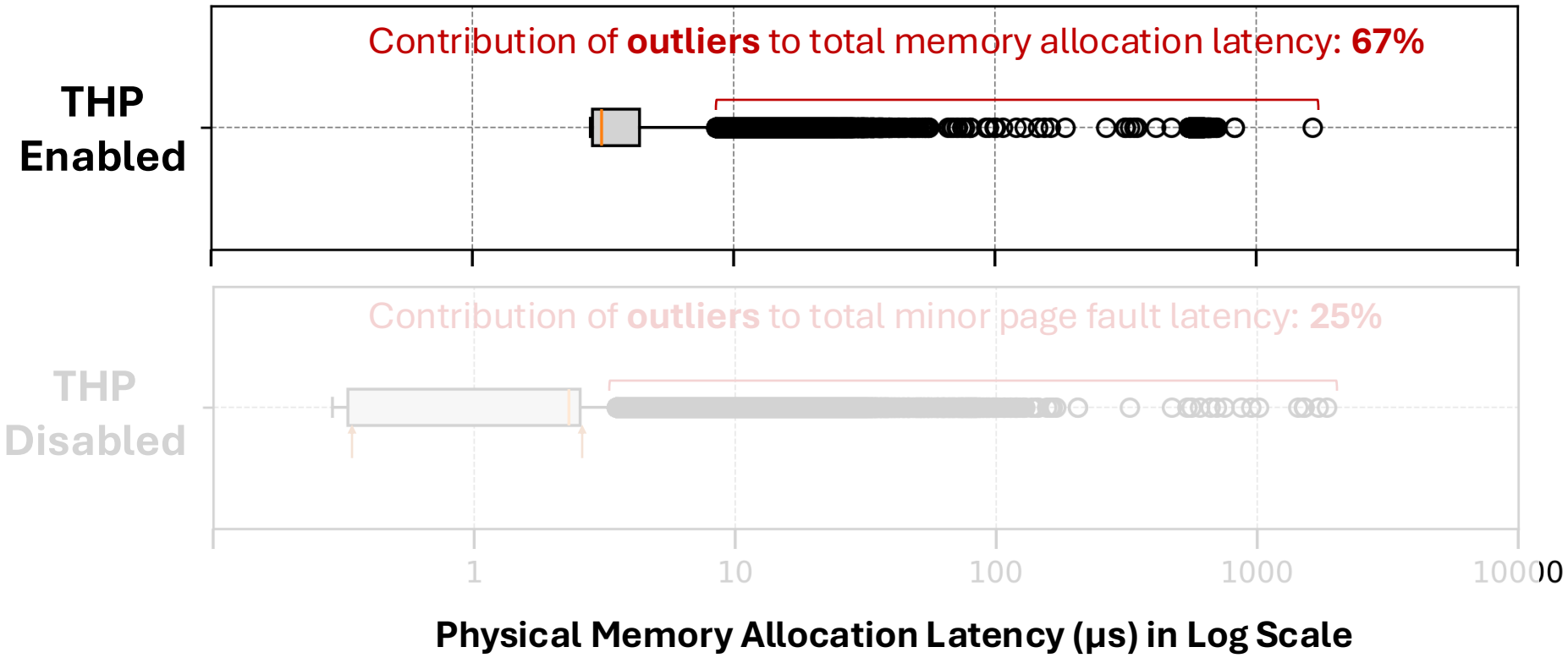


Average PTW Latency (measured in a real system) ranges from 33 up to 184 cycles

Variable Memory Allocation Latency

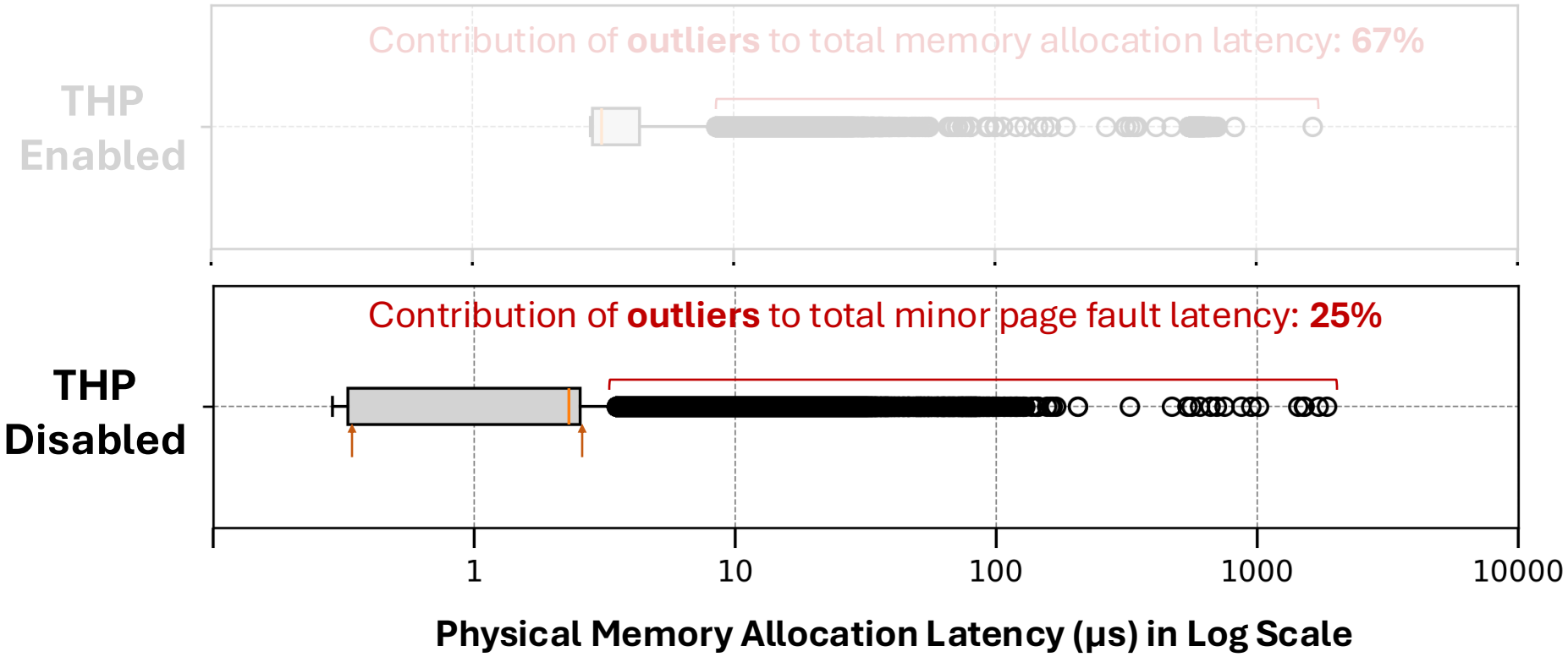


Variable Memory Allocation Latency



The latency of handling memory allocation (measured in a real system) exhibits high variability

Variable Memory Allocation Latency



The latency of handling memory allocation (measured in a real system) exhibits high variability

Analytical Estimation of VM Overheads

Use of First-order Models

First-order models do not accurately capture the dynamic nature of VM overheads

Full-System Simulators

gem5-FS

QFlex

PTLsim

Enable the execution of a full-blown OS on top of a hardware simulator

Low simulation speed

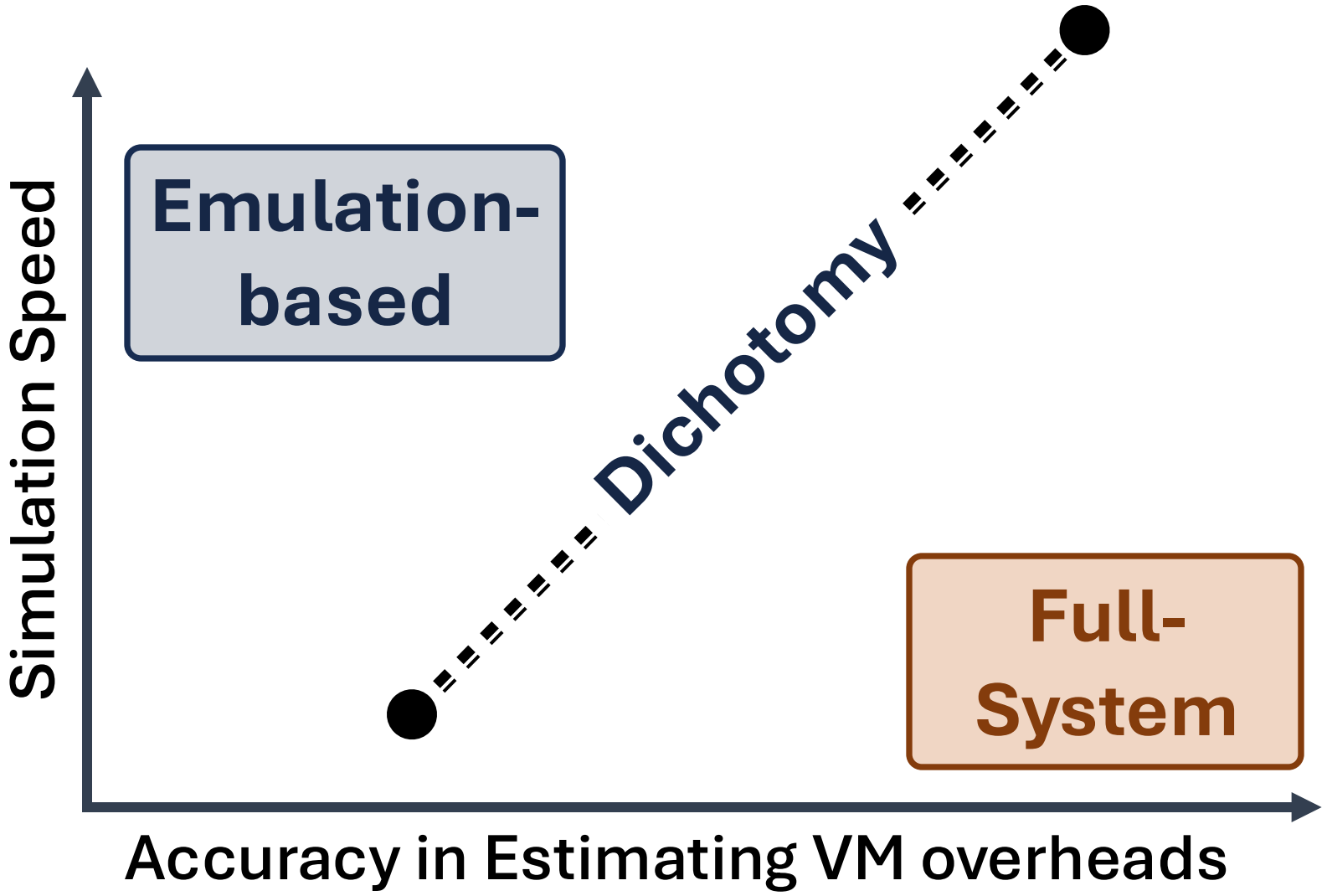
Hard to develop

Existing Architectural Simulators

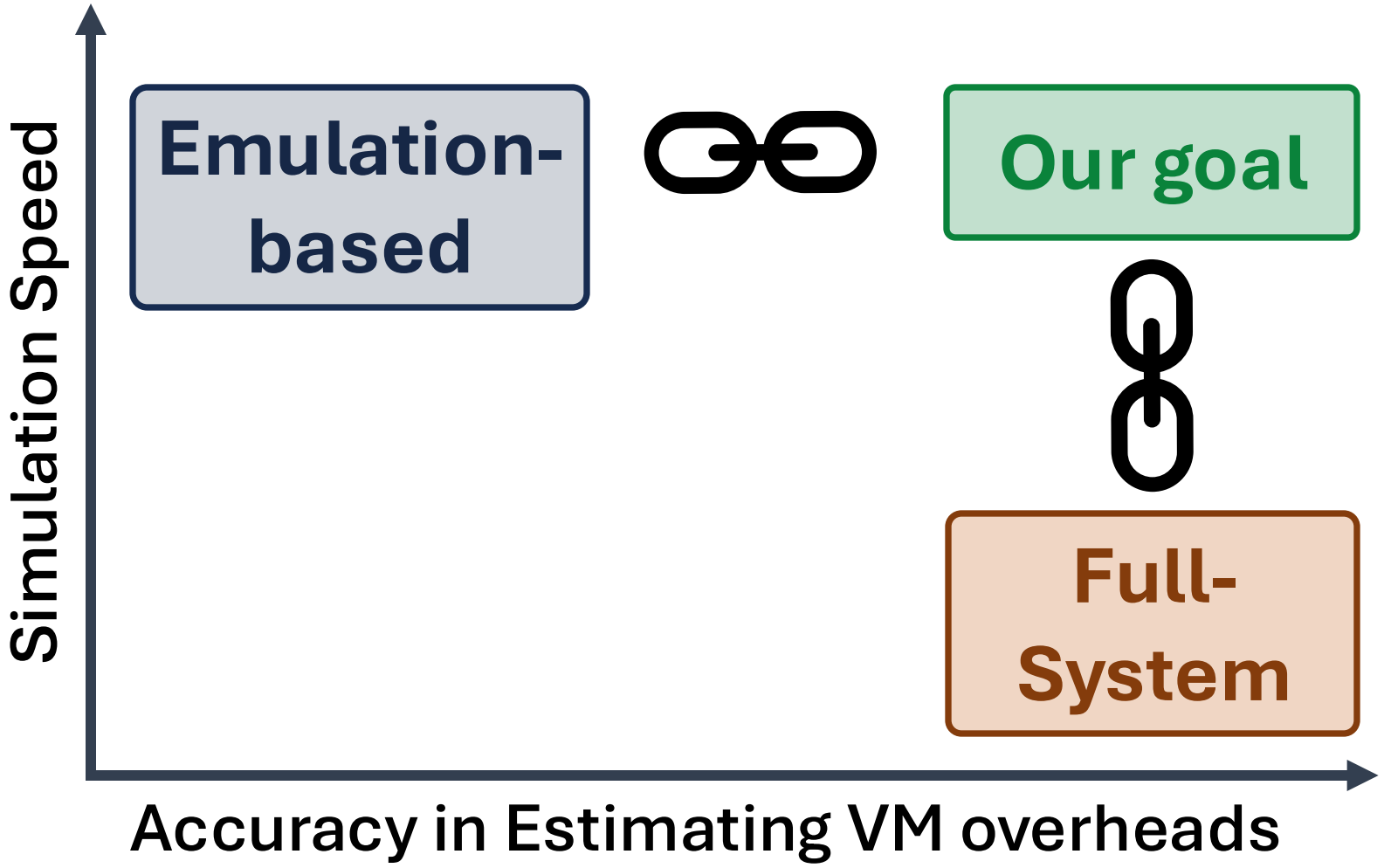
Two main classes of simulators

1. **Emulation-based**
2. **Full-System**

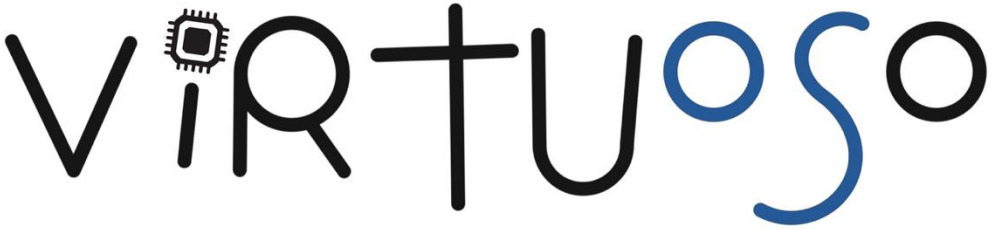
Existing Architectural Simulators



Existing Architectural Simulators



Enabling Fast and Accurate VM Research



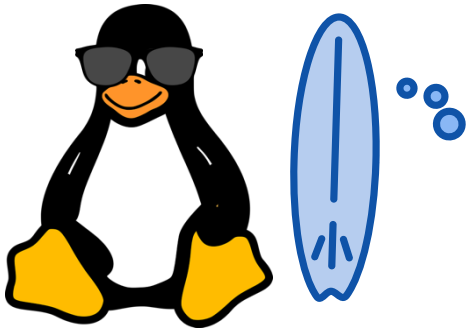
New simulation framework that enables **fast and accurate** prototyping and evaluation of the ***software and hardware*** components of VM

Imitation-based Simulation Methodology

Lightweight Userspace Kernel

written in a high-level programming language

"I will try to imitate the full-blown OS"



Choose OS modules only related to the desired research

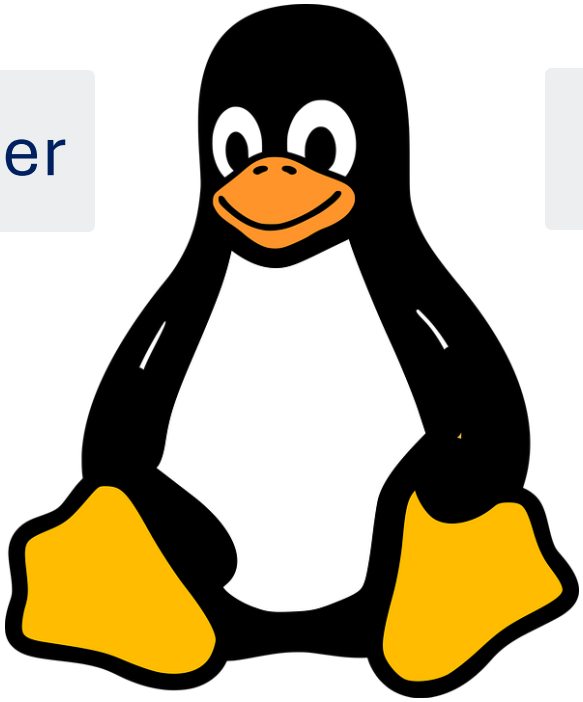
Imitation-based Simulation Methodology

Monitoring

Full-blown Kernel

Interrupts

Thread Scheduler



Managing NUMA

Data Sharing Support

Transparent Huge Pages

Swapping

Virtual Memory Area Handling

Cgroups

Boot process

Protection

Block device driver

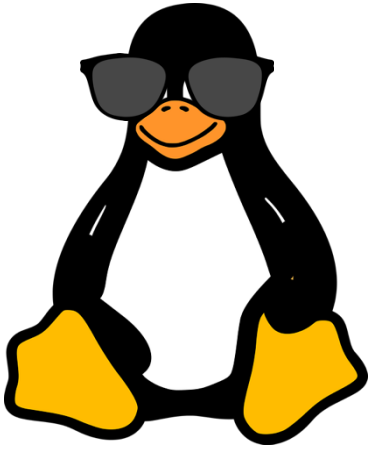
SLAB Allocator

hugetlbfs

Imitation-based Simulation Methodology

Lightweight Userspace Kernel

- Swapping
- Virtual Memory Area Handling
- Transparent Huge Pages



Imitation-based Simulation Methodology

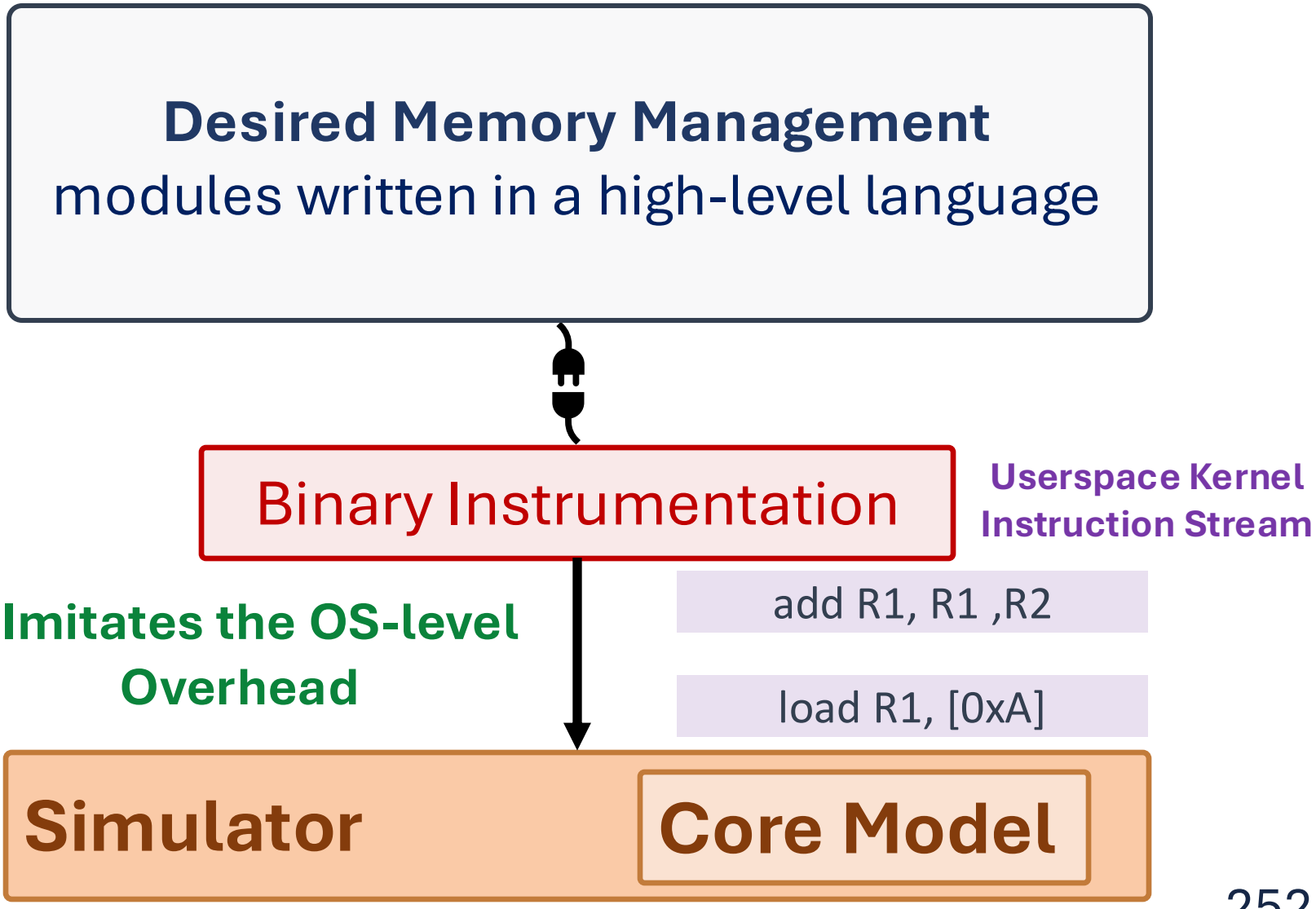
1. **Rapid development and prototyping**

2. **High simulation speed** by executing only the desired OS kernel functionality

3. **Accurate evaluation** by integrating Virtuoso with an architectural simulator

Imitation-based Simulation Methodology

Lightweight Userspace Kernel



Imitation-based Simulation Methodology

Lightweight Userspace Kernel

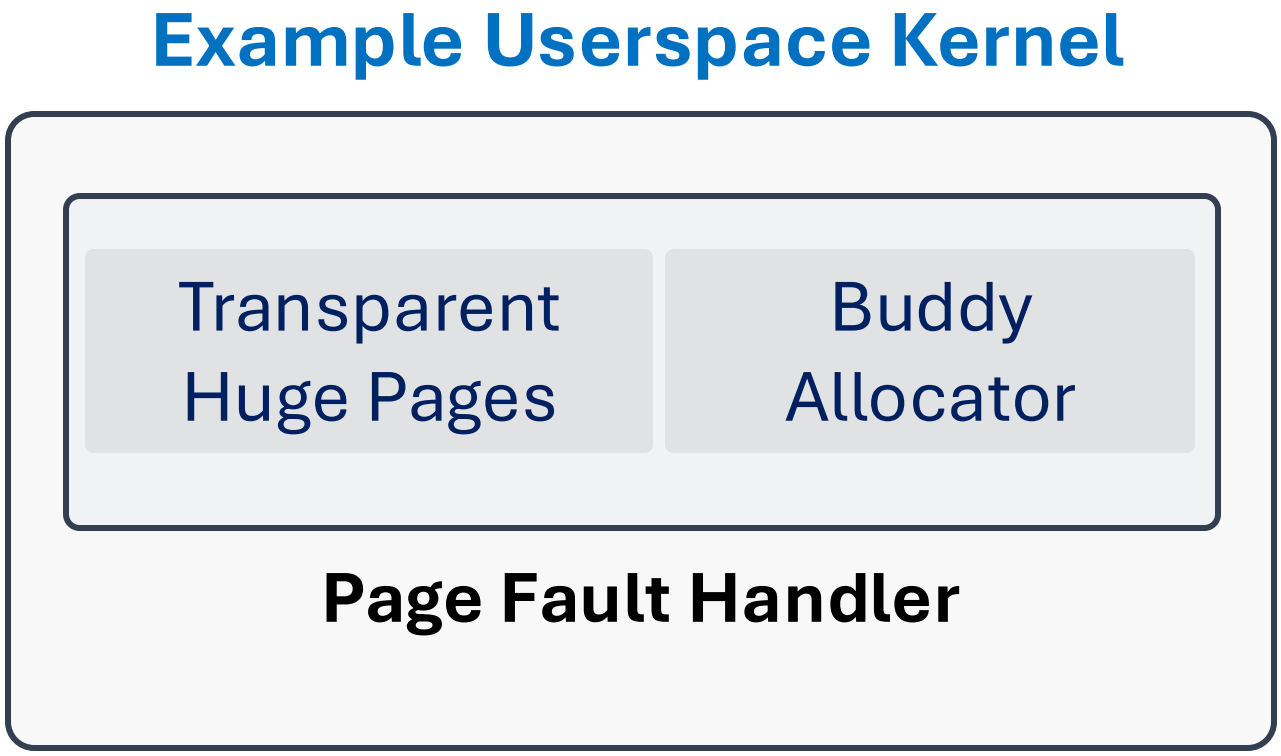
Desired Memory Management
modules written in a high-level language

Functional Outcome

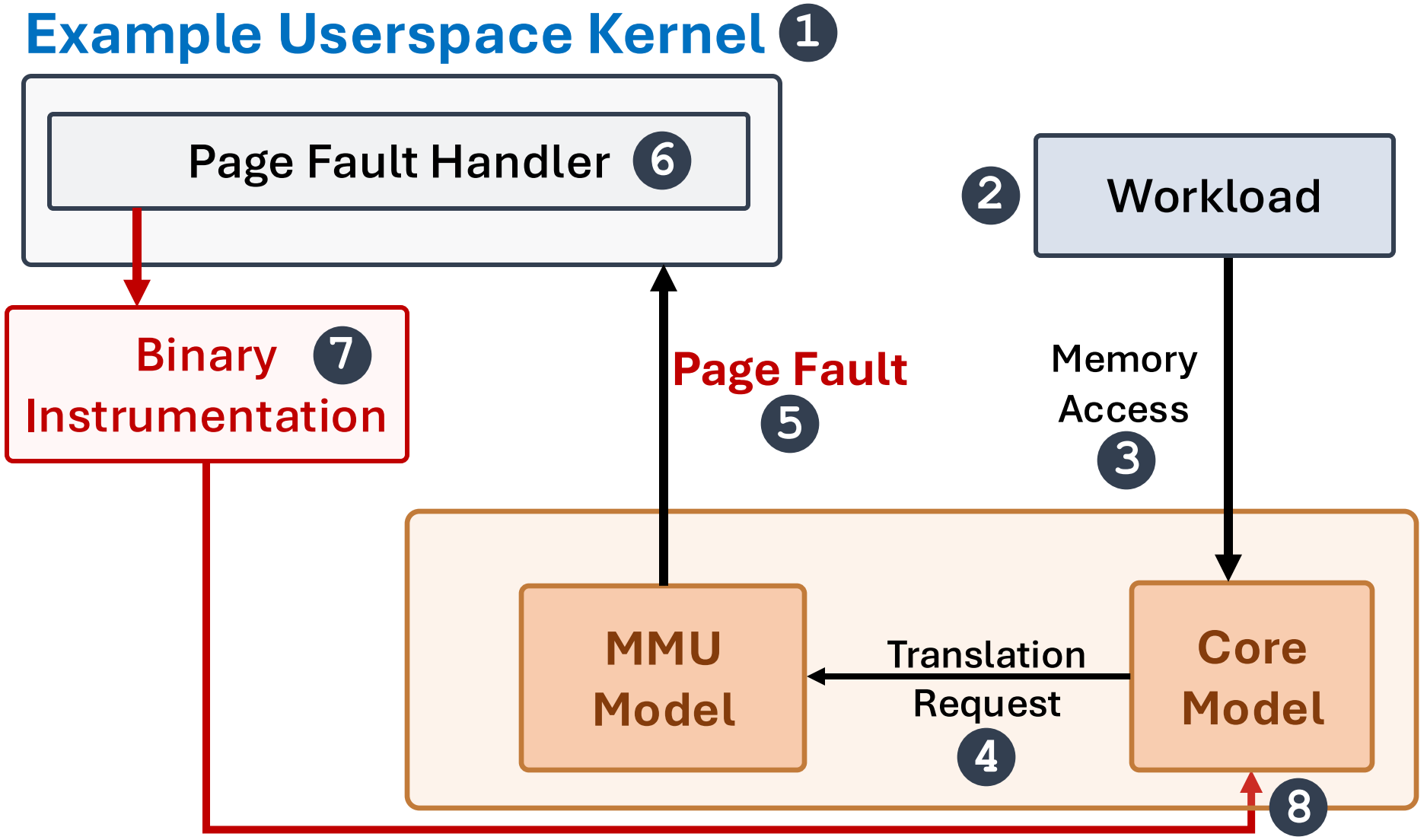
+ Enables emulation of kernel modules

Simulator **Core Model**

Workflow: Page Fault Handling Example

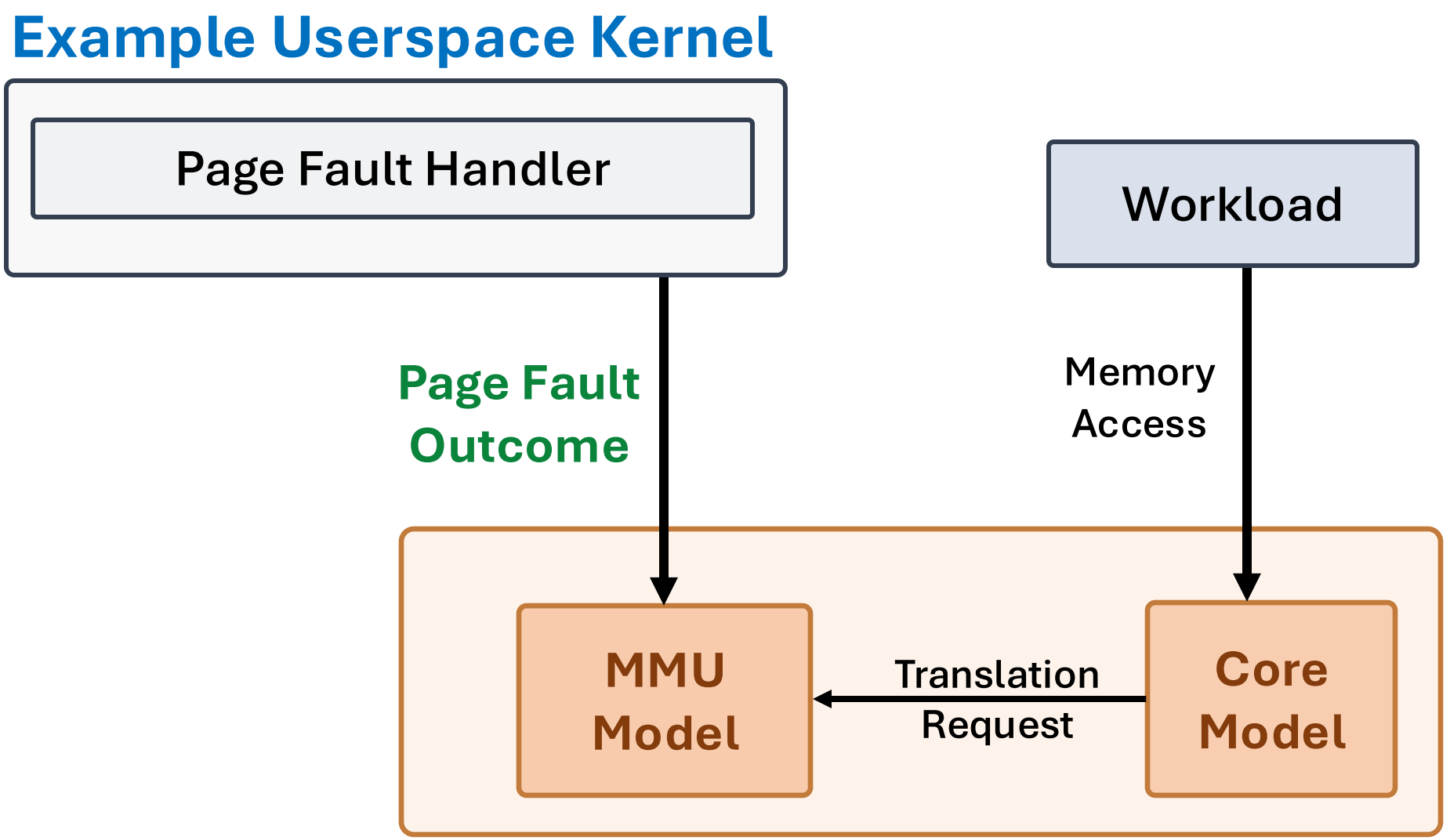


Workflow: Page Fault Handling Example



Example: Sniper Simulator [Carlson+ TACO 12]

Workflow: Page Fault Handling Example



Example: Sniper Simulator [Carlson+ TACO 12]

MimicOS: Imitating the Linux Kernel

MimicOS modules written in C++

Buddy Allocator	Radix-based page table	hugetlbfs
Transparent Huge Pages	Swap Space	Page Cache

More details in the paper

https://arxiv.org/pdf/2403.04635

VirTool: State-of-the-art VM techniques



Toolset encompassing the HW/SW components of multiple state-of-the-art VM techniques

- 4 page table designs
- Software-Managed TLB
- Page-Size Prediction
- Nested MMU Support
- 2 Hash-based address mapping schemes
- 2 Memory Tagging Schemes
- 2 THP Policies
- 2 Contiguity-aware Schemes
- 2 Intermediate Address Space Schemes
- Speculative Translation
- TLB Prefetching

VirTool: State-of-the-art VM techniques

Toolset encompassing multiple state-of-the-art VM techniques

4 page table

Software-

Page-Size Prediction

Enables researchers to evaluate and prototype current and brand new VM techniques

2 THP Policies

2 Intermediate Address Space Schemes

VirTool: State-of-the-art VM techniques

Toolset encompassing multiple state-of-the-art VM techniques

4 page table

Software-

Page-Size Prediction

Virtuoso is a highly versatile simulation framework

2 IHP Policies

2 Intermediate Address Space Schemes

Integrated Virtuoso with 5 Diverse Architectural Simulators

gem5-SE

System-call emulation mode of gem5

[Lowe-Power+ arXiv 2020]

[Binkert+ SIGARCH News 2011]

http://gem5.org/

Integrated Virtuoso with 5 Diverse Architectural Simulators

gem5-SE

Ramulator

Focus on main memory subsystem

[Luo+ CAL 2023]

https://github.com/CMU-SAFARI/ramulator2

Integrated Virtuoso with 5 Diverse Architectural Simulators

gem5-SE

Ramulator

Sniper

Focus on multicore systems

[Carlson+ TACO 2012]

https://github.com/snipersim/snipersim

Integrated Virtuoso with 5 Diverse Architectural Simulators

gem5-SE

Ramulator

Sniper

ChampSim

Focus on microarchitecture

[Gober+ arXiv 2022]

https://github.com/ChampSim/ChampSim

Integrated Virtuoso with 5 Diverse Architectural Simulators

gem5-SE

Ramulator

Sniper

ChampSim

Focus on storage devices

[Tavakkol+ FAST 2018]

MQSim

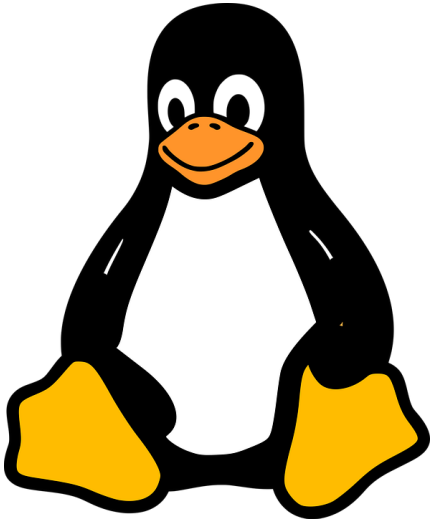
https://github.com/CMU-SAFARI/MQSim

Validation against a Real System

VIRTUOSO ∞ Sniper

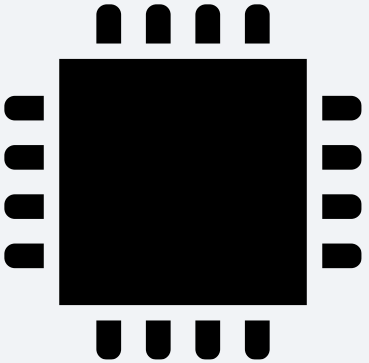
Against

Linux Kernel

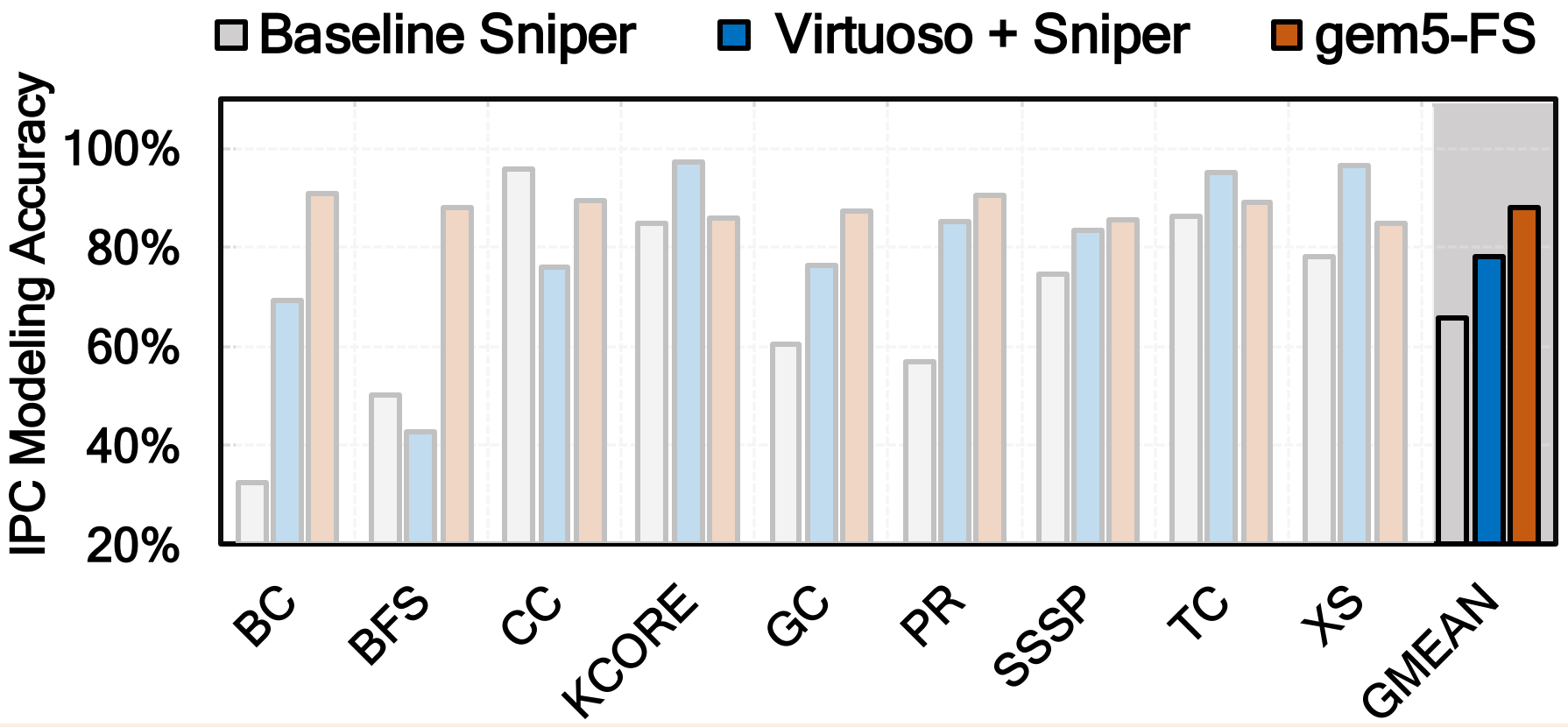


∞

High-end Server-grade CPU

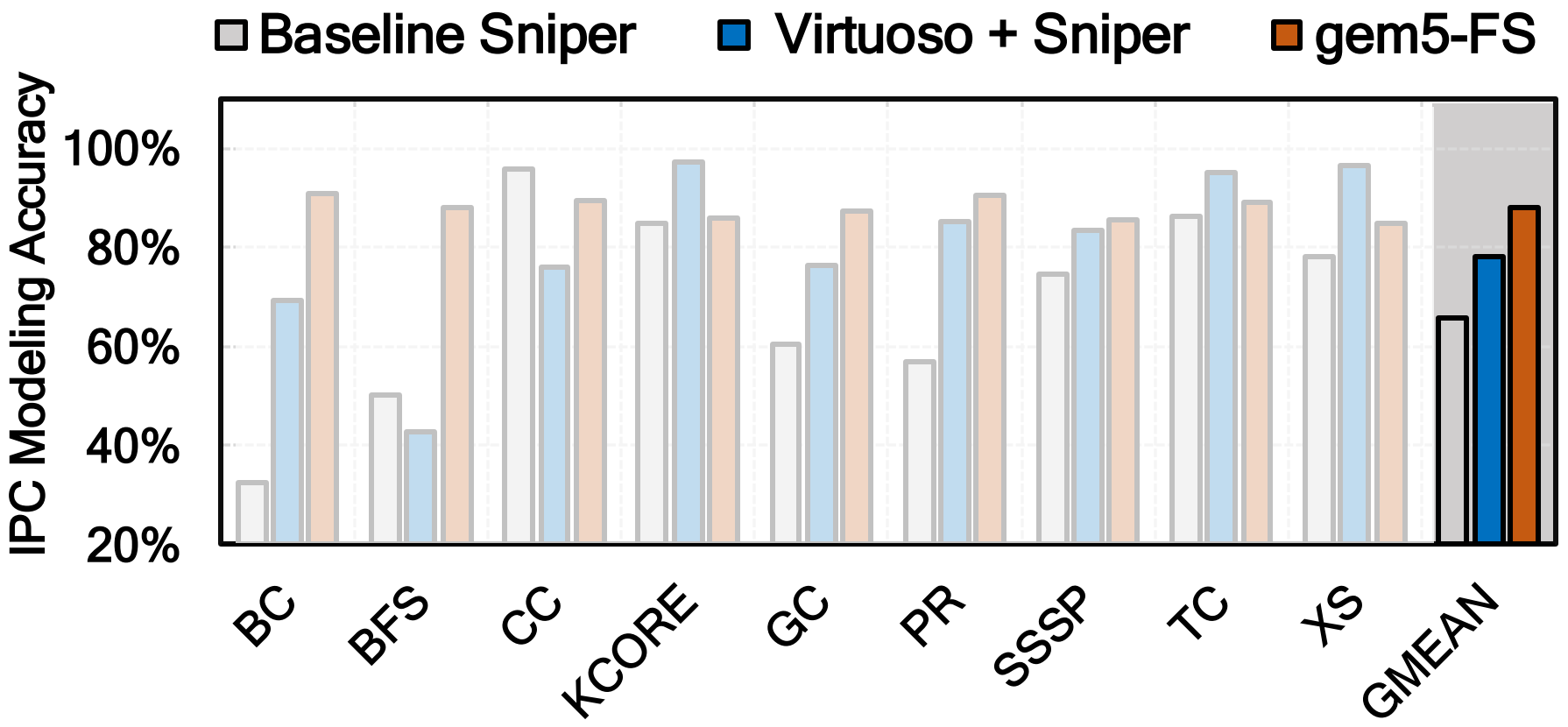


Validation: Instructions Per Cycle (IPC)



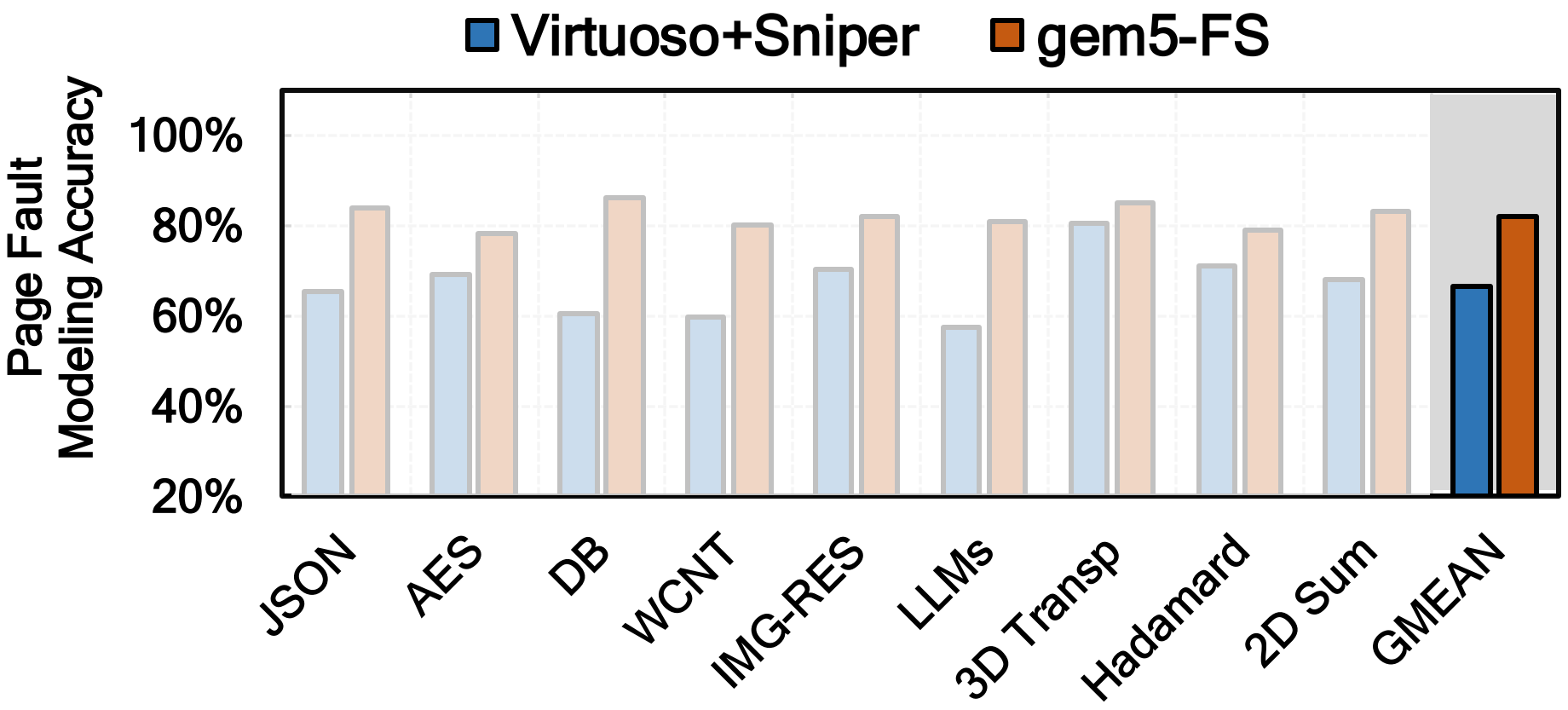
Virtuoso integrated with Sniper improves IPC modeling accuracy by 21% compared to baseline Sniper

Validation: Instructions Per Cycle (IPC)



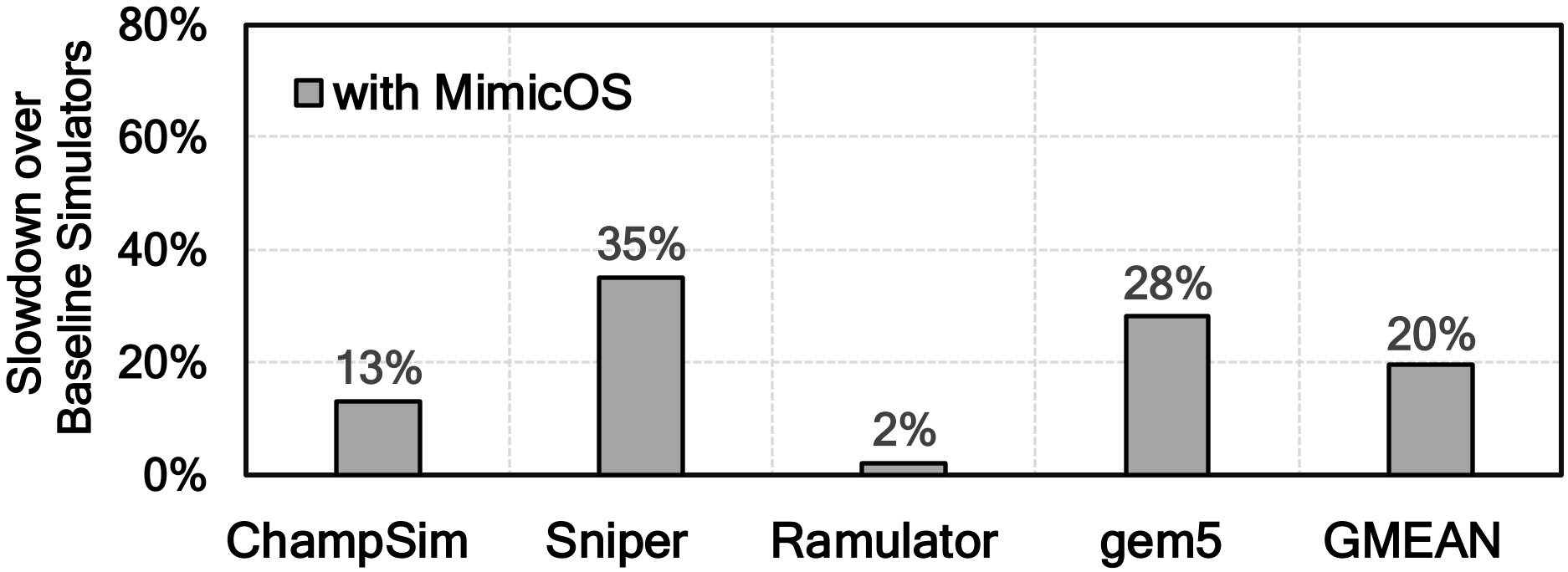
Virtuoso integrated with Sniper achieves IPC modeling accuracy within 9% of gem5-FS

Validation: Page Fault Handling Latency



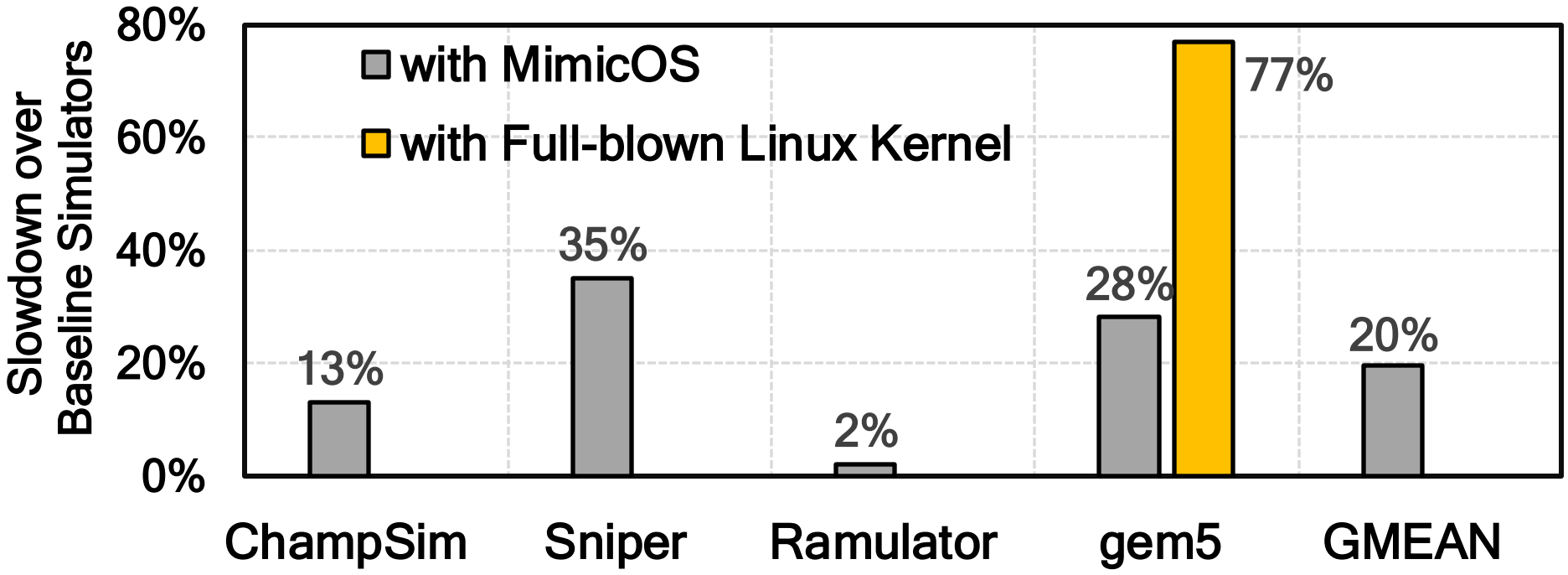
Virtuoso integrated with Sniper models the page fault handling latency with 66% accuracy, within 15% of gem5-FS

Simulation Speed Comparison



MimicOS leads to an average 20% simulation time overhead over the baseline version of the simulator

Simulation Speed Comparison



Enabling full-system execution mode in gem5 leads to 77% simulation time overhead

Virtuoso's Versatility

We showcase Virtuoso's versatility by evaluating multiple different use cases

Demonstrate new insights on state-of-the-art VM techniques

5 use cases in the paper

https://arxiv.org/pdf/2403.04635

Virtuoso Example Use Cases

Evaluating Different Page Table Designs

Evaluating Physical Memory Allocation Policies

Virtuoso Example Use Cases

Evaluating Different Page Table Designs

Evaluating Physical Memory Allocation Policies

Evaluation Methodology

Radix: Baseline radix-based page table

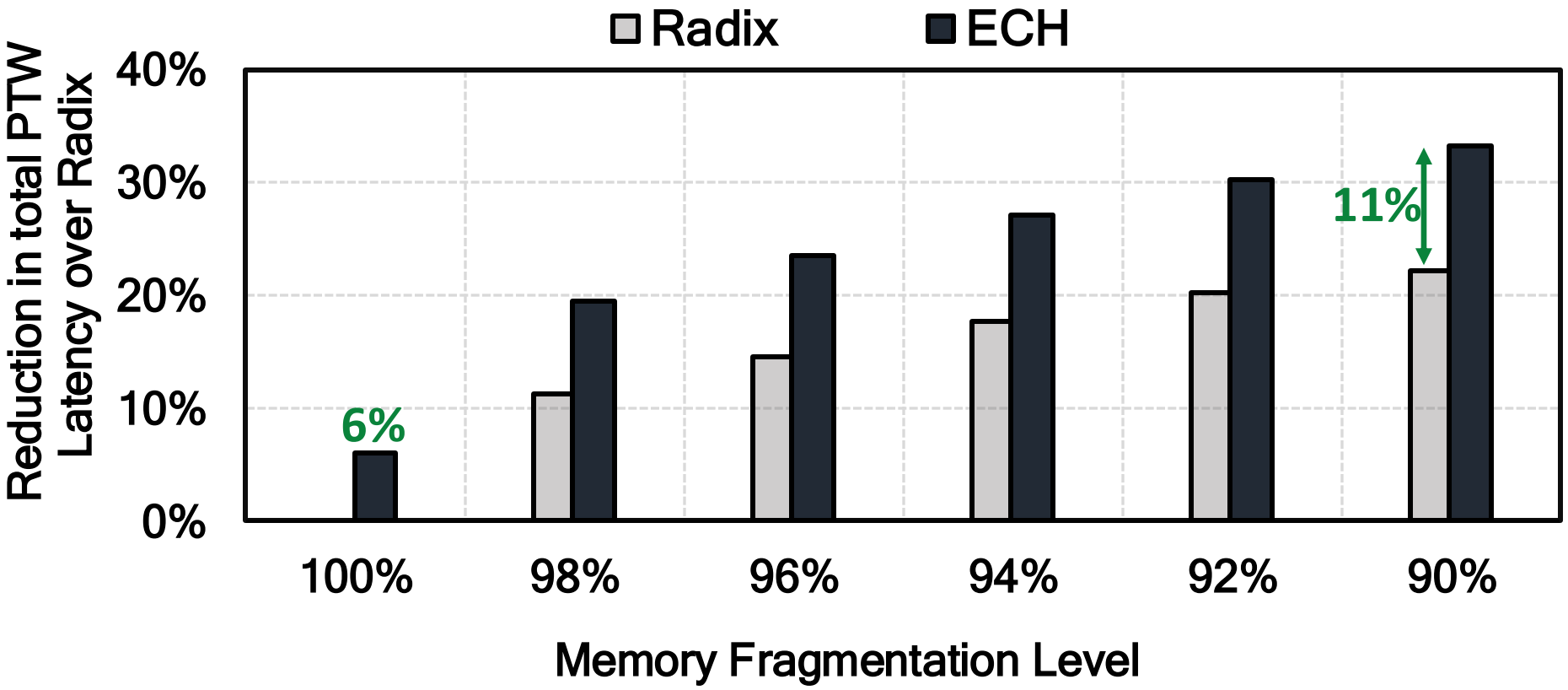
ECH[1]: Elastic Cuckoo Hash-based Page Table

Fragmentation: Number of available 2MB pages compared to the total number of 2MB pages

Workloads: **GraphBIG, XSBench, HPCC**

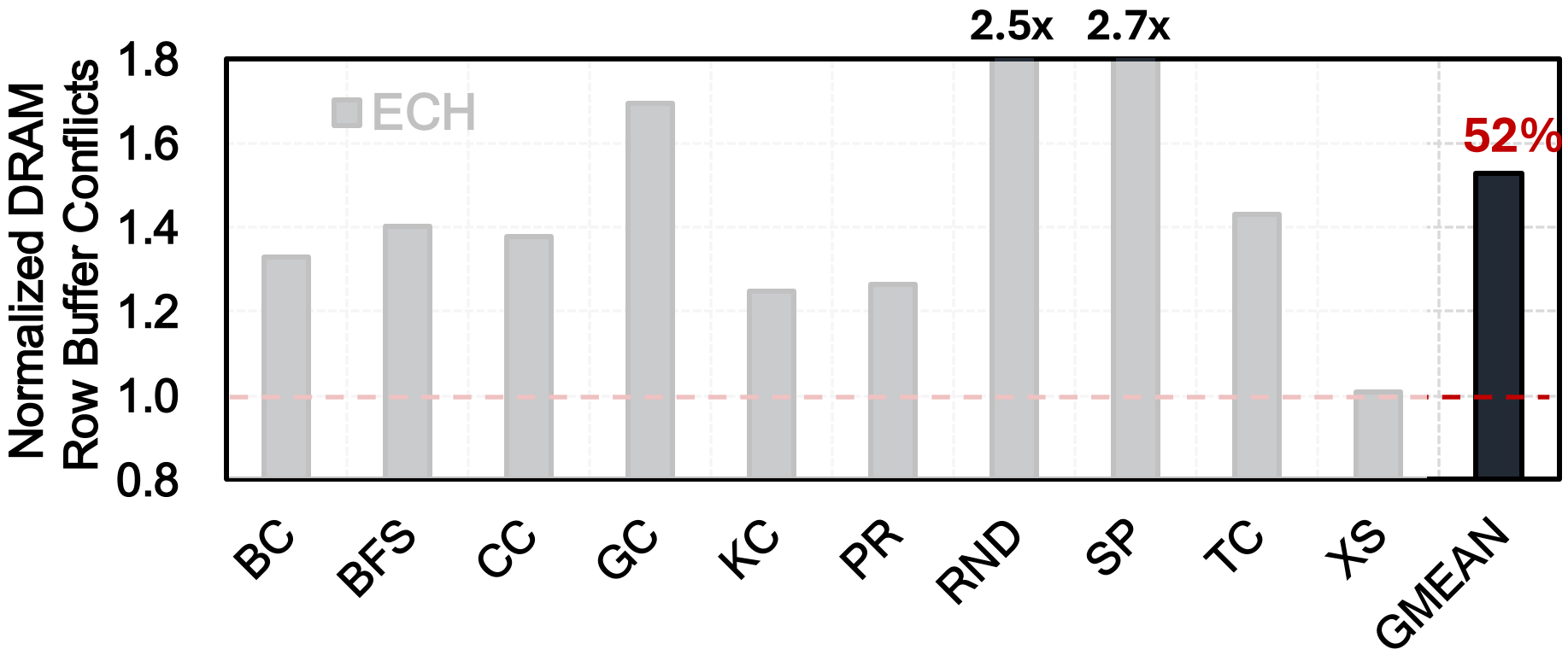
[1] *Skarlatos et al. "Elastic Cuckoo Page Tables: Rethinking Virtual Memory Translation for Parallelism" ASPLOS 2020*

Reduction in PTW Latency



As memory fragmentation decreases, the PTW latency reduction of ECH compared to Radix increases

Main Memory Interference



ECH leads to an average 52% increase in DRAM row buffer conflicts over Radix

Virtuoso Example Use Cases

Evaluating Different Page Table Designs

Evaluating Physical Memory Allocation Policies

Evaluation Methodology

Buddy: Baseline allocator with the buddy system

Utopia:[1] Part of memory is organized using a hash-based mapping

Workloads: **Short-input Short-output LLM inference**



Mistral-7B



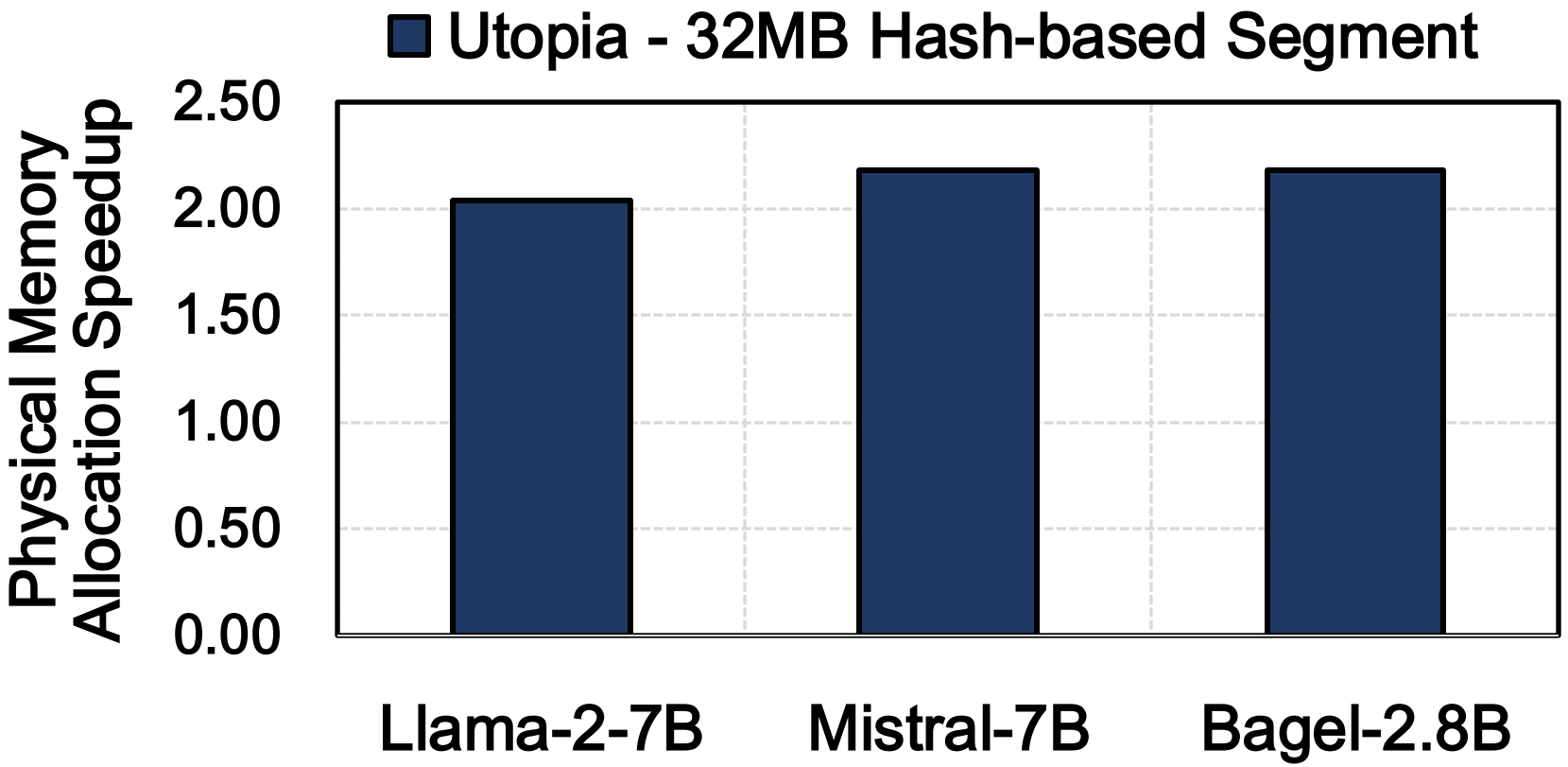
Llama-2-7B



Bagel-2.8B

[1] *Kanellopoulos et al. "Utopia: Fast and Efficient Address Translation via Hybrid Restrictive & Flexible Virtual-to-Physical Address Mappings" MICRO 2023*

Accelerating Memory Allocation



Restricting the virtual-to-physical mapping speeds up memory allocation by up to 2.17x

More Details in the Paper

Virtuoso: Enabling Fast and Accurate Virtual Memory Research via an Imitation-based Operating System Simulation Methodology

Konstantinos Kanellopoulos[1] Konstantinos Sgouras[1] F. Nisa Bostanci[1]
Andreas Kosmas Kakolyris [1] Berkin Kerim Konar [1] Rahul Bera[1]
Mohammad Sadrosadati[1] Rakesh Kumar [2] Nandita Vijaykumar [3] Onur Mutlu[1]

[1]ETH Zürich [2]Norwegian University of Science and Technology [3]University of Toronto

Abstract

The unprecedented growth in data demand from emerging applications has turned virtual memory (VM) into a major performance bottleneck. VM's overheads are expected to persist as memory requirements continue to increase. Researchers explore new hardware/OS co-designs to optimize VM across diverse applications and systems. To evaluate such designs, researchers rely on various simulation methodologies to model VM components. Unfortunately, current simulation tools (i) either lack the desired accuracy in modeling VM's software components or (ii) are too slow and complex to prototype and evaluate schemes that span across the hardware/software boundary.

We introduce Virtuoso, a new simulation framework that enables quick and accurate prototyping and evaluation of the *software and hardware components* of the VM subsystem. The key idea of Virtuoso is to employ a *lightweight userspace OS kernel*, called MimicOS, that (i) accelerates simulation time by imitating *only* the desired kernel functionalities, (ii) facilitates the development of new OS routines that imitate

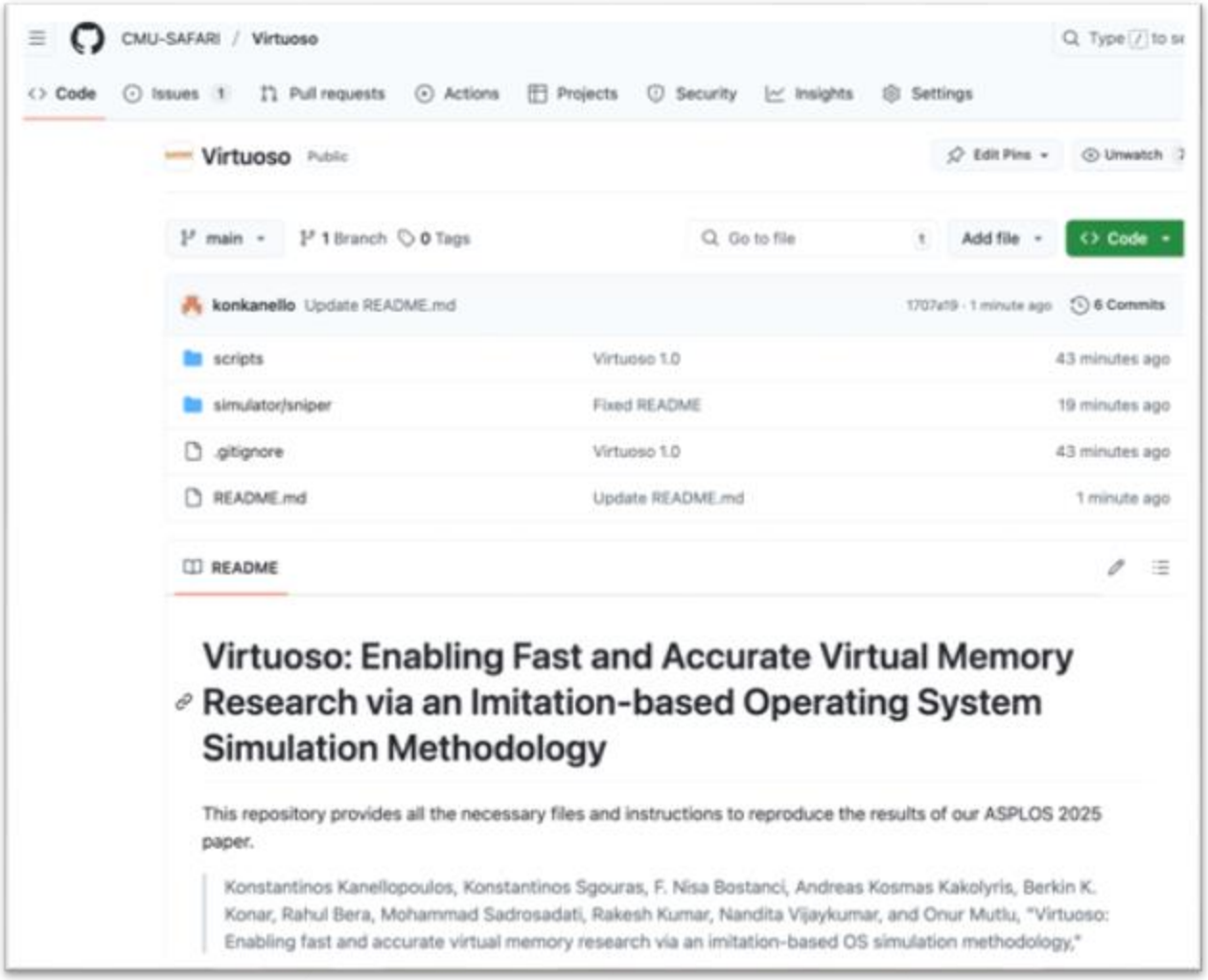
simulation time overhead of only 20%, on top of four baseline architectural simulators. The source code of Virtuoso is freely available at https://github.com/CMU-SAFARI/Virtuoso.

1 Introduction

Virtual memory (VM) [1–23] is a cornerstone of modern computing systems, enabling application-transparent physical memory management, isolation and data sharing. Contemporary applications (e.g., [24–45]) exhibit different characteristics that stress the VM subsystem. We classify these workloads into two broad categories: (i) long-running workloads (i.e., execution time larger than 100s of seconds) [24, 28–31, 33–35] with large data footprints and irregular memory access patterns, that exhibit high address translation overheads, and (ii) short-running workloads (i.e., execution time often lower than 1 second) [36–45] whose execution time does not amortize the overheads of system software operations (e.g., physical memory allocation). Multiple prior works and industrial studies [46–57] have shown that address trans-

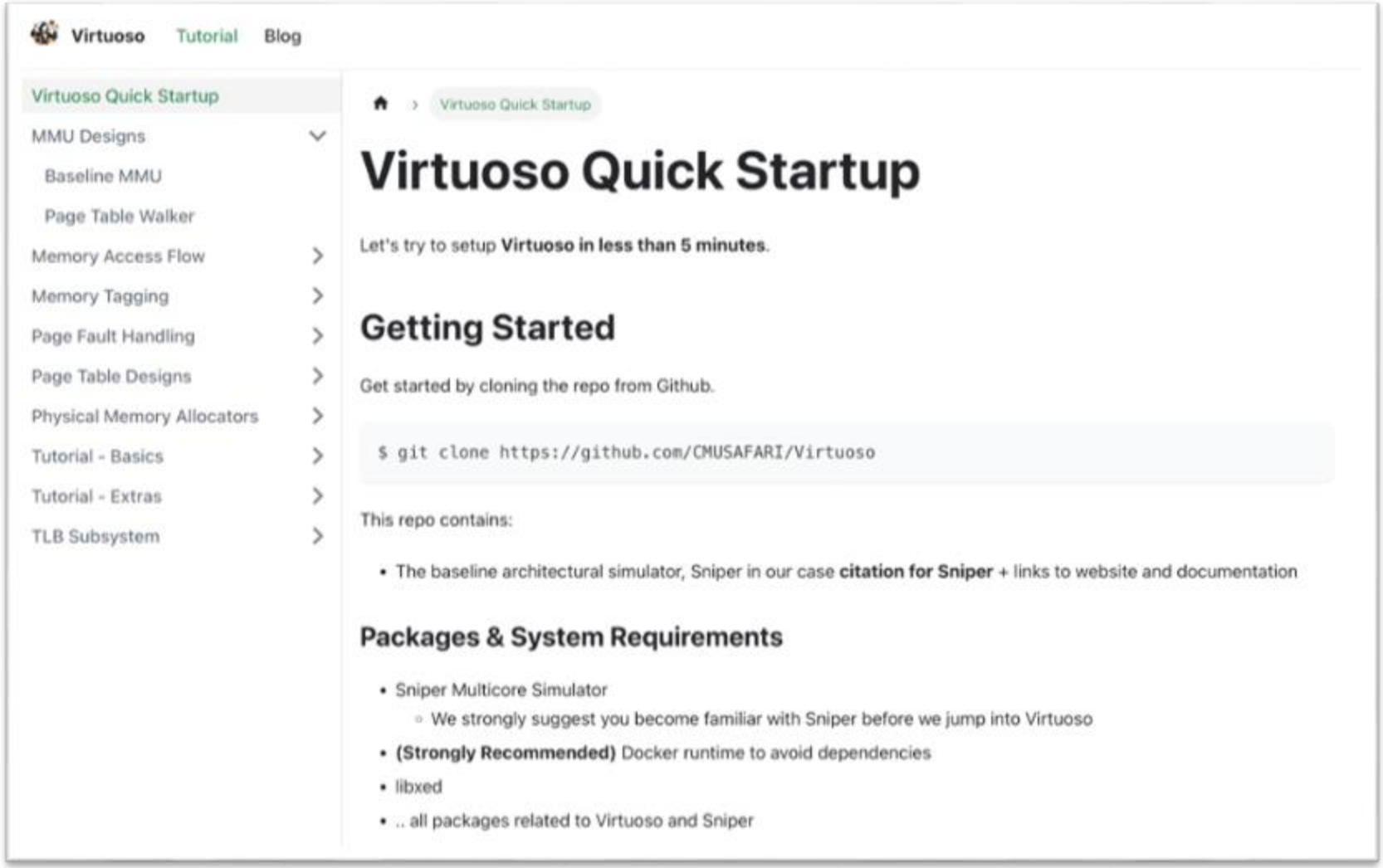
https://arxiv.org/pdf/2403.04635

Virtuoso is Open-Source



https://github.com/CMU-SAFARI/Virtuoso

Virtuoso Website - Documentation



https://safari.ethz.ch/virtuoso

Virtuoso Workshop & Tutorial – MICRO 2025



https://www.youtube.com/watch?v=fi6ttrmu_OM&t=30119s

We hope Virtuoso establishes a common ground for VM research

Virtual Memory Summary

Virtual Memory Summary

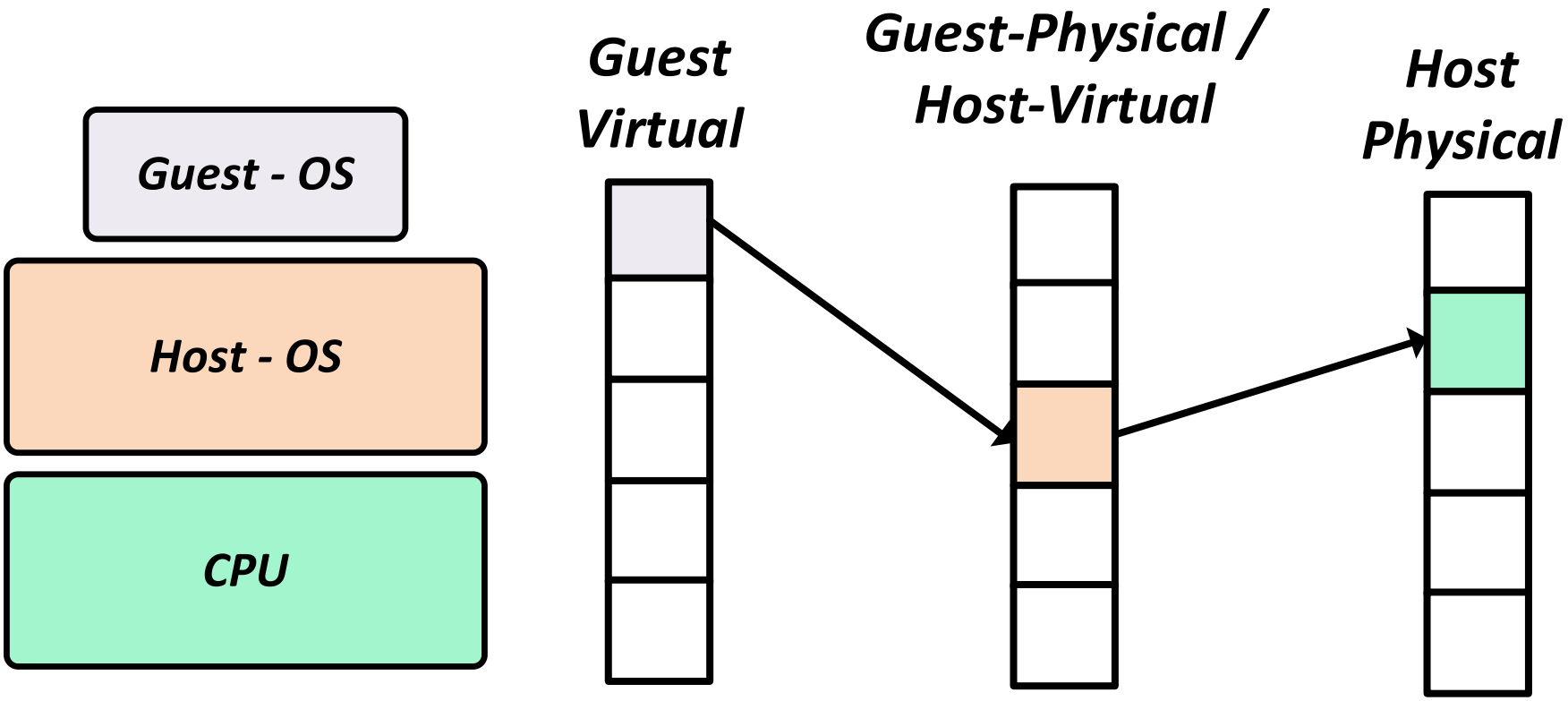
- Virtual memory gives the illusion of "infinite" capacity
- A subset of virtual pages are located in physical memory
- A page table maps virtual pages to physical pages – this is called address translation
- A TLB speeds up address translation
- Multi-level page tables keep the page table size in check
- Using different page tables for different programs provides memory protection

There is More… We Will Not Cover…

- How to handle virtualized systems?
  - Virtual machines running programs
  - Hypervisors
- …

Virtual Memory in Virtualized Environments

- Virtualized environments (e.g., Virtual Machines) need to have an additional level of address translation



Rethinking Virtual Memory

Nastaran Hajinazar, Pratyush Patel, Minesh Patel, Konstantinos Kanellopoulos, Saugata Ghose, Rachata Ausavarungnirun, Geraldo Francisco de Oliveira Jr., Jonathan Appavoo, Vivek Seshadri, and Onur Mutlu,
"The Virtual Block Interface: A Flexible Alternative to the Conventional Virtual Memory Framework"
*Proceedings of the 47th International Symposium on Computer Architecture (**ISCA**)*, Virtual, June 2020.
[Slides (pptx) (pdf)]
[Lightning Talk Slides (pptx) (pdf)]
[ARM Research Summit Poster (pptx) (pdf)]
[Talk Video (26 minutes)]
[Lightning Talk Video (3 minutes)]
[Lecture Video (43 minutes)]

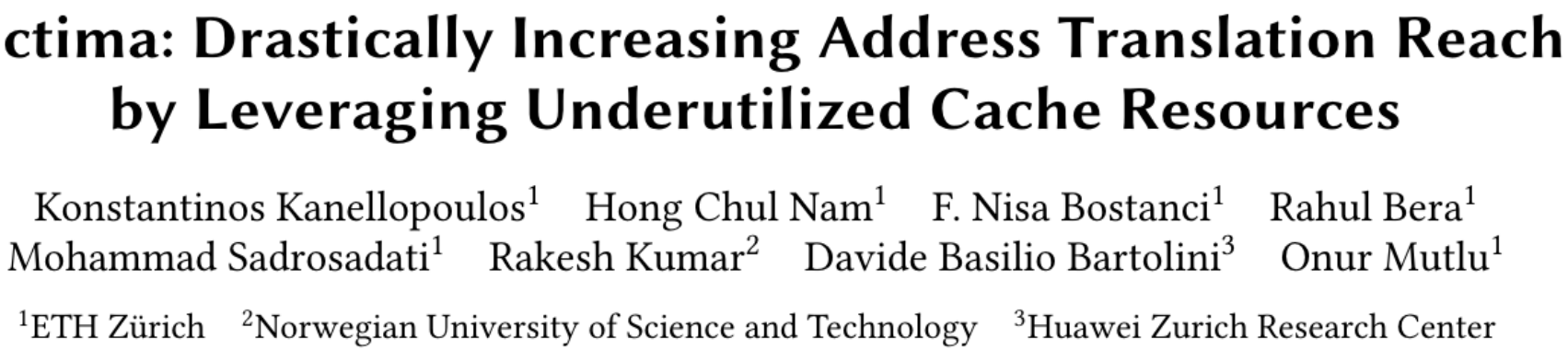
The Virtual Block Interface: A Flexible Alternative to the Conventional Virtual Memory Framework

Nastaran Hajinazar[⋆†] Pratyush Patel[⋈] Minesh Patel[⋆] Konstantinos Kanellopoulos[⋆] Saugata Ghose[‡]
Rachata Ausavarungnirun[⊙] Geraldo F. Oliveira[⋆] Jonathan Appavoo[◇] Vivek Seshadri[▽] Onur Mutlu[⋆‡]

[⋆]*ETH Zürich* [†]*Simon Fraser University* [⋈]*University of Washington* [‡]*Carnegie Mellon University*
[⊙]*King Mongkut's University of Technology North Bangkok* [◇]*Boston University* [▽]*Microsoft Research India*

Rethinking Virtual Memory (I)

Konstantinos Kanellopoulos, Hong Chul Nam, F. Nisa Bostanci, Rahul Bera, Mohammad Sadrosadati, Rakesh Kumar, Davide Basilio Bartolini, and Onur Mutlu,
"Victima: Drastically Increasing Address Translation Reach by Leveraging Underutilized Cache Resources"
Proceedings of the 56th International Symposium on Microarchitecture (***MICRO***),
Toronto, ON, Canada, November 2023.
[Slides (pptx) (pdf)]
[arXiv version] [Victima Source Code (Officially Artifact Evaluated with All Badges)]
Officially artifact evaluated as available, functional, reusable and reproducible.
Distinguished artifact award at MICRO 2023



Victima: Drastically Increasing Address Translation Reach by Leveraging Underutilized Cache Resources

Konstantinos Kanellopoulos[1] Hong Chul Nam[1] F. Nisa Bostanci[1] Rahul Bera[1]
Mohammad Sadrosadati[1] Rakesh Kumar[2] Davide Basilio Bartolini[3] Onur Mutlu[1]

[1]ETH Zürich [2]Norwegian University of Science and Technology [3]Huawei Zurich Research Center

Virtual Memory: Parting Thoughts

- VM is one of the most successful examples of
  - architectural support for programmers
  - how to partition work between hardware and software
  - hardware/software cooperation
  - programmer/architect tradeoff

- Going forward: How does virtual memory scale into the future? Four key trends:
  - Increasing, huge physical memory sizes (local & remote)
  - Hybrid physical memory systems (DRAM + NVM + SSD)
  - Many accelerators in the system addressing physical memory
  - Virtualized systems (hypervisors, software virtualization, local and remote memories)

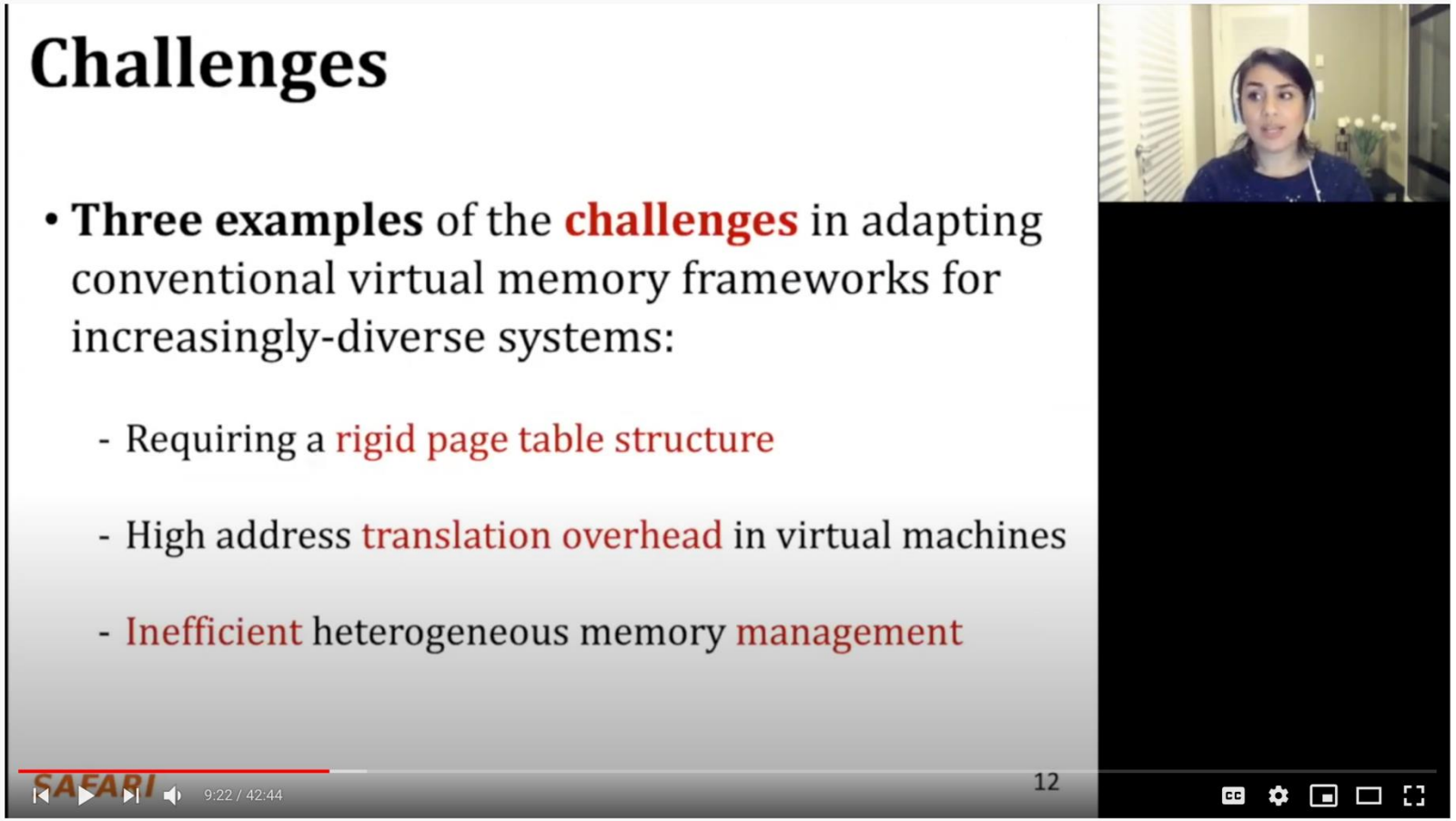
Virtual Memory: Parting Thoughts

"It's the Memory, Stupid!"
(Richard Sites, MPR, 1996)

What did we learn today?

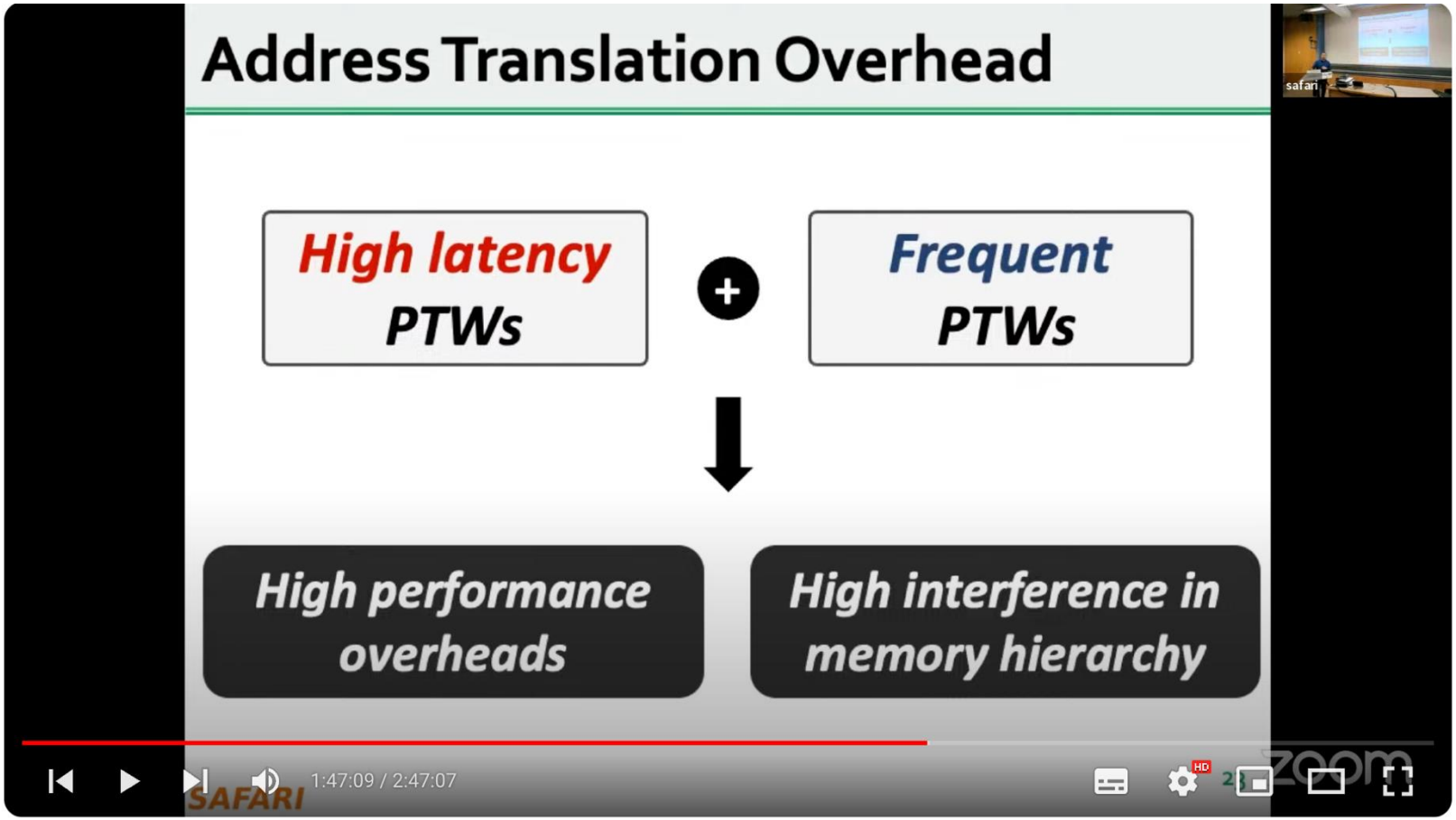
"It's also the (Virtual) Memory, Stupid!"

Lectures on Virtual Memory



https://www.youtube.com/watch?v=2RhGMpY18zw&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=22

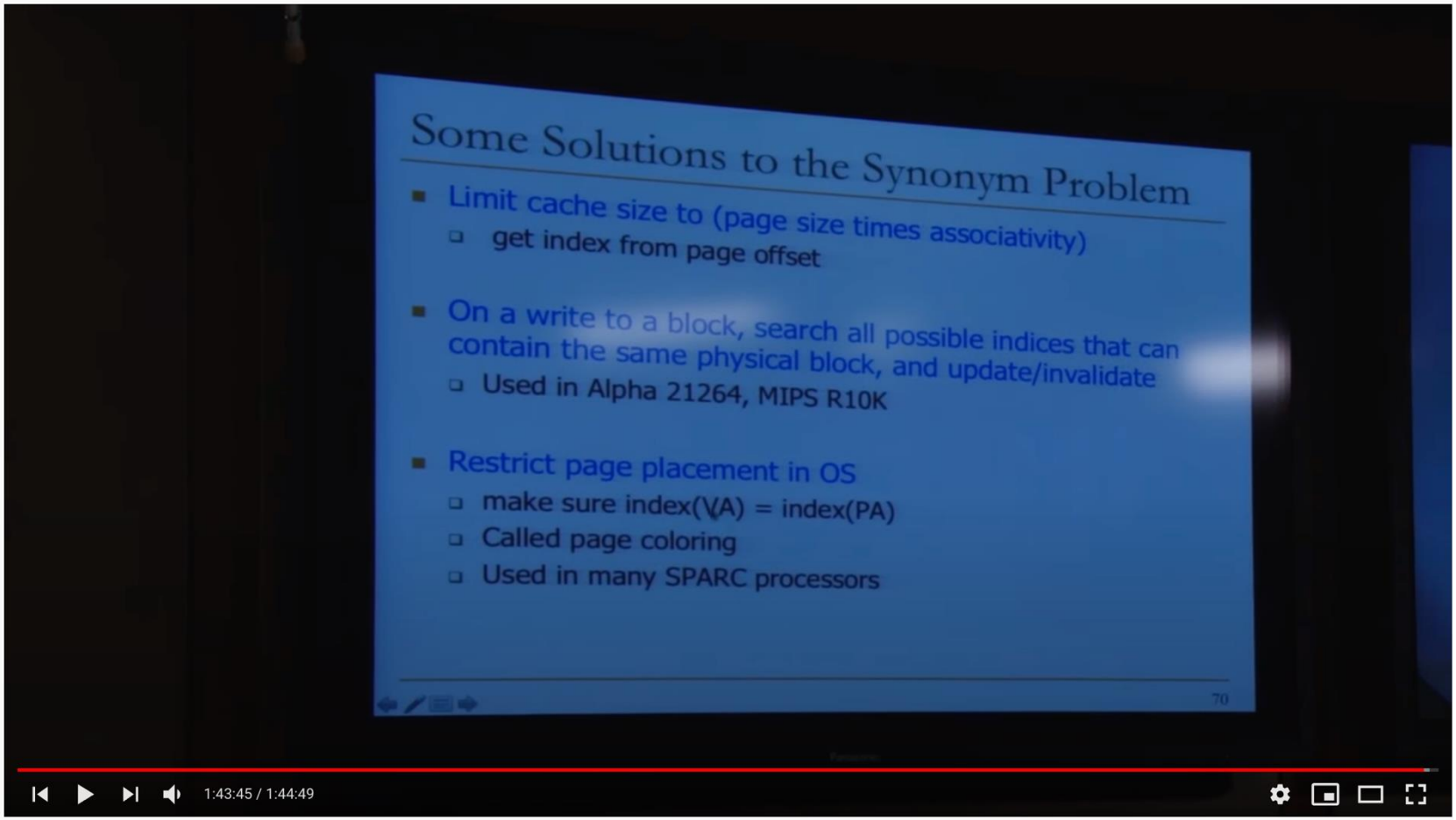
Lectures on Virtual Memory



Computer Architecture - Lecture 23: Cutting-Edge Research in Computer Architecture (Fall 2023)

Onur Mutlu Lectures
41,8 χιλ. εγγεγραμμένοι
Εγγραφήκατε
23
Κοινοποίηση
Λήψη

Lectures on Virtual Memory



https://www.youtube.com/watch?v=PPR7YrBi7IQ&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=24

Lectures on Virtual Memory

- Computer Architecture, Spring 2015, Lecture 20
  - Virtual Memory (CMU, Spring 2015)
  - https://www.youtube.com/watch?v=2RhGMpY18zw&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=22

- Computer Architecture, Fall 2020, Lecture 12c
  - The Virtual Block Interface (ETH, Fall 2020)
  - https://www.youtube.com/watch?v=PPR7YrBi7IQ&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=24

- Computer Architecture, Fall 2024, Lecture 25
  - Utopia, Victima, Virtuoso (ETH, Fall 2024)
  - https://www.youtube.com/live/dx1xkbb5QDc?si=mU-rmX7oNcrAxqCb

Research Opportunities

Research Opportunities

- If you are interested in doing research in Computer Architecture, Security, Systems & Bioinformatics:
  - Email Mohammad and Prof. Mutlu with your interest
  - **Talk with us**

- There are many exciting projects and research positions, e.g.:
  - Novel memory/storage/computation/communication systems
  - New execution paradigms (e.g., in-memory computing)
  - Hardware security, safety, reliability, predictability
  - GPUs, TPUs, FPGAs, PIM, heterogeneous systems, …
  - Security-architecture-reliability-energy-performance interactions
  - Architectures for genomics/proteomics/medical/health/AI/ML
  - A limited list is here: https://safari.ethz.ch/theses/

Computer Architecture

Lecture 22: Virtual Memory

Konstantinos Kanellopoulos

Prof. Onur Mutlu

ETH Zürich

Fall 2025

5 December 2025

Three Major Issues in Virtual Memory

1. How large is the page table and how do we store and access it?

2. How can we speed up translation & access control check?

3. When do we do the translation in relation to cache access?

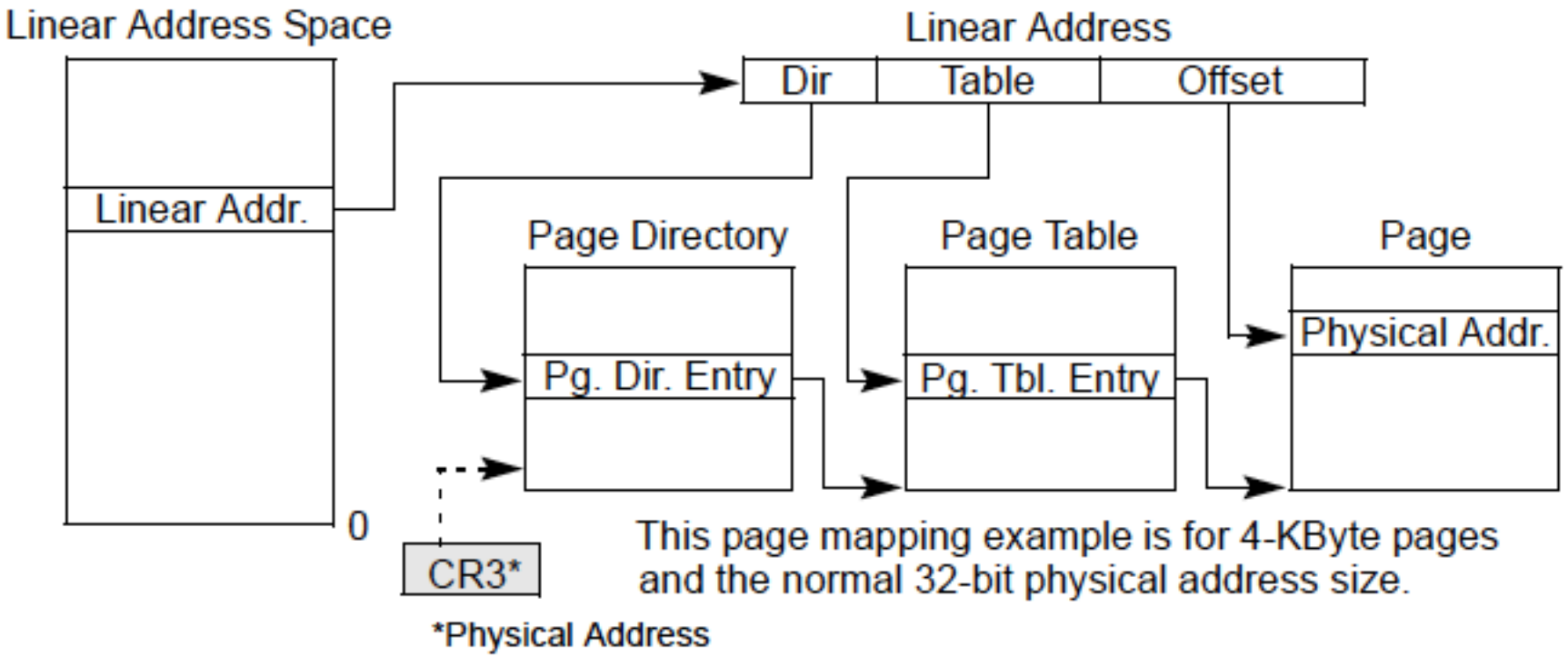
- There are many other issues we will not cover in detail
  - What happens on a context switch?
  - How can you handle multiple page sizes?
  - …

Virtual Memory Issue I

- How large is the page table?
- Where do we store it?
  - In hardware?
  - In physical memory? (Where is the PTBR?)
  - In virtual memory? (Where is the PTBR?)
- How can we store it efficiently without requiring physical memory that can store all page tables?
  - Idea: multi-level page tables
  - Only the first-level page table has to be in physical memory
  - Remaining levels are in virtual memory (but get cached in physical memory when accessed)

Recall: Solution: Multi-Level Page Tables

Example from the x86 architecture



Page Table Access

- How do we access the Page Table?

- Page Table Base Register (CR3 in x86)
- Page Table Limit Register

- If VPN is out of the bounds (exceeds PTLR) then the process did not allocate the virtual page → access control exception

- Page Table Base Register is part of a process's context
  - Just like PC, status registers, general purpose registers
  - Needs to be loaded when the process is context-switched in

More on x86 Page Tables (I): Small Pages

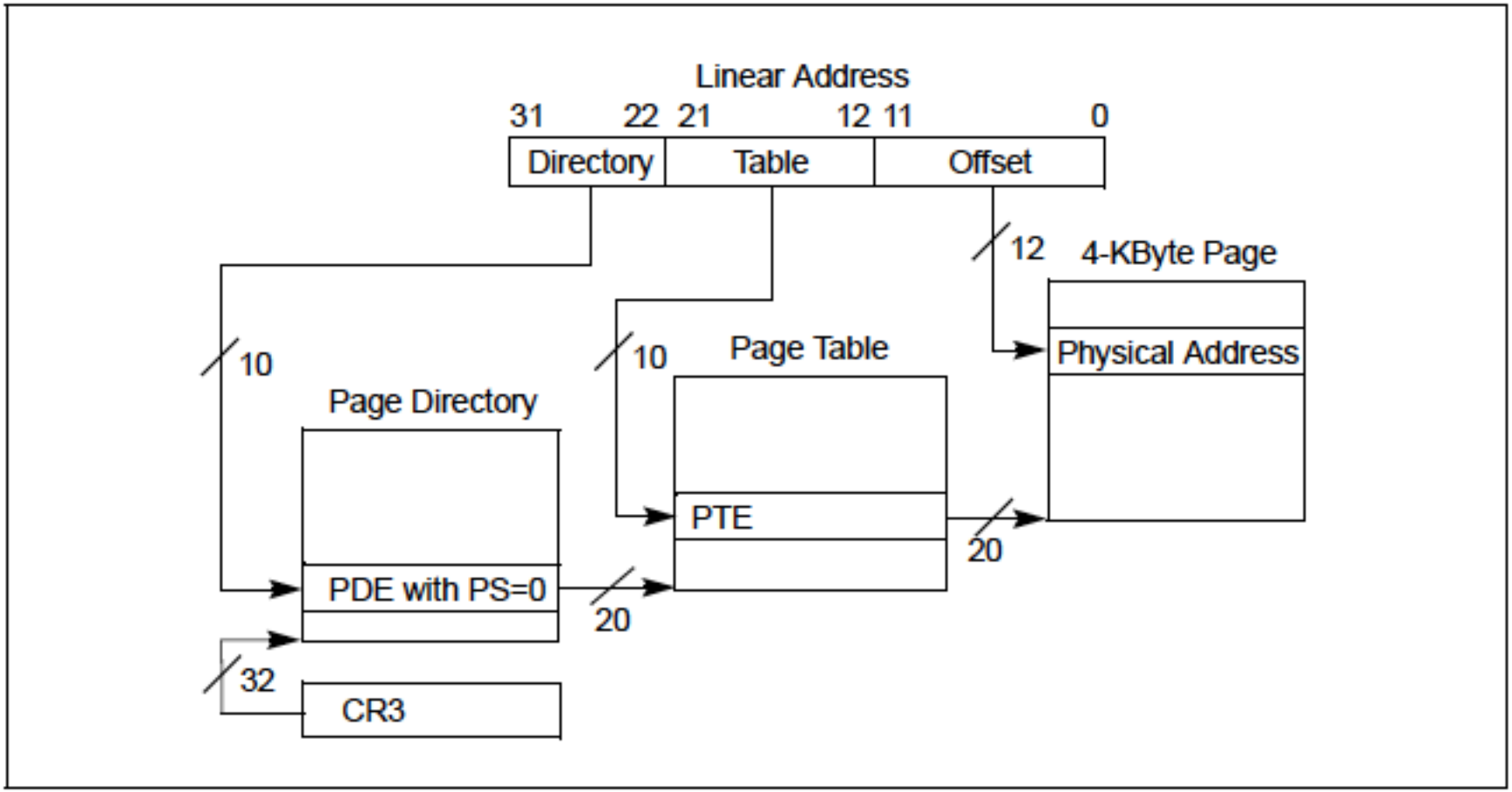


Figure 4-2. Linear-Address Translation to a 4-KByte Page using 32-Bit Paging

More on x86 Page Tables (II): Large Pages

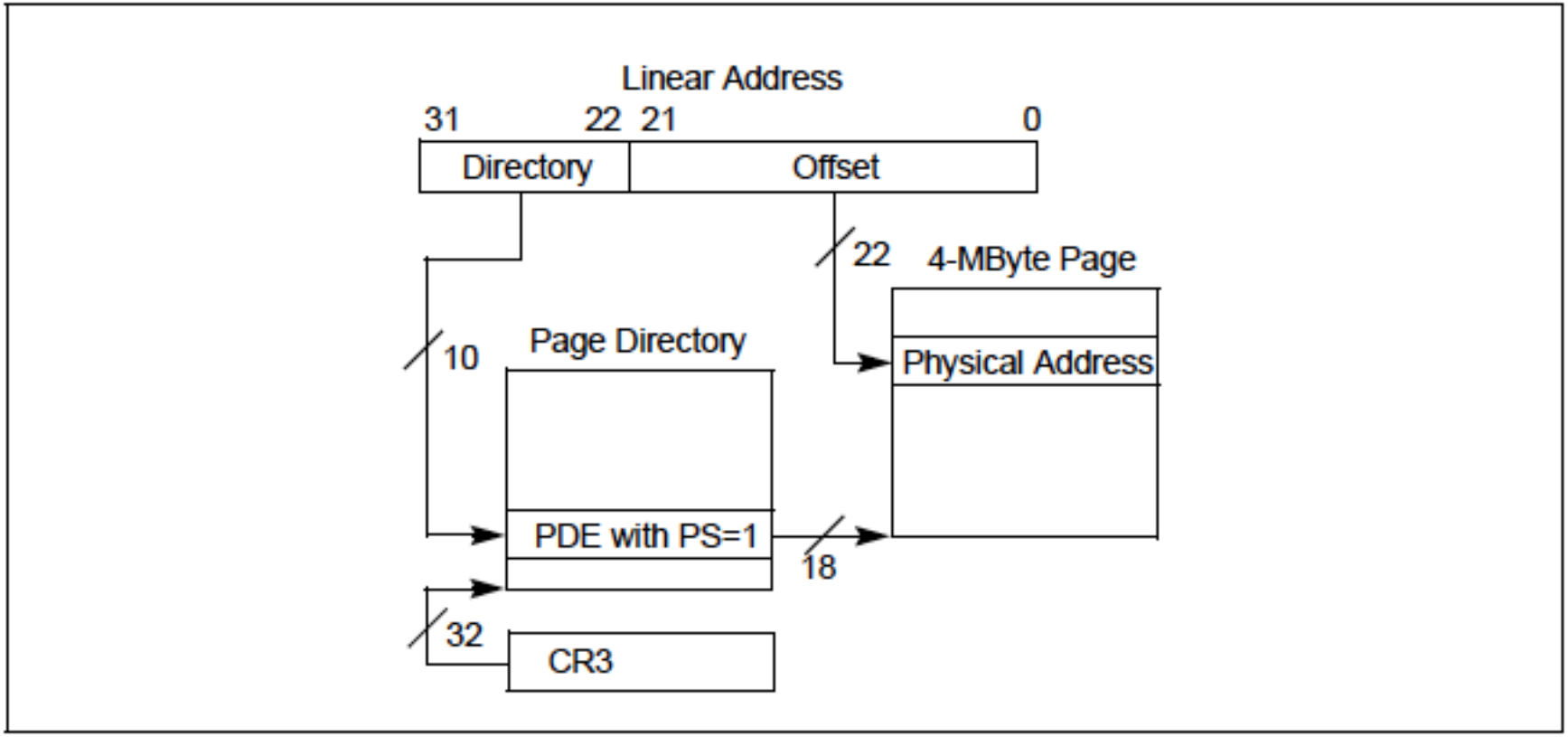


Figure 4-3. Linear-Address Translation to a 4-MByte Page using 32-Bit Paging

x86 Page Table Entries

Figure 4-4 gives a summary of the formats of CR3 and the paging-structure entries with 32-bit paging. For the paging structure entries, it identifies separately the format of entries that map pages, those that reference other paging structures, and those that do neither because they are "not present"; bit 0 (P) and bit 7 (PS) are highlighted because they determine how such an entry is used.

31–12	11–9	8	7	6	5	4	3	2	1	0	
Address of page directory[1]	Ignored					PCD	PWT	Ignored			CR3
Bits 31:22 of address of 2MB page frame	Reserved (must be 0)	Bits 39:32 of address[2]	PAT	Ignored	G	1	D	A	PCD	PWT	U/S	R/W	1	PDE: 4MB page
Address of page table	Ignored			0	Ign	A	PCD	PWT	U/S	R/W	1	PDE: page table
Ignored										0	PDE: not present
Address of 4KB page frame	Ignored	G	PAT	D	A	PCD	PWT	U/S	R/W	1	PTE: 4KB page
Ignored										0	PTE: not present

Figure 4-4. Formats of CR3 and Paging-Structure Entries with 32-Bit Paging

x86 PTE (4KB page)

Table 4-6. Format of a 32-Bit Page-Table Entry that Maps a 4-KByte Page

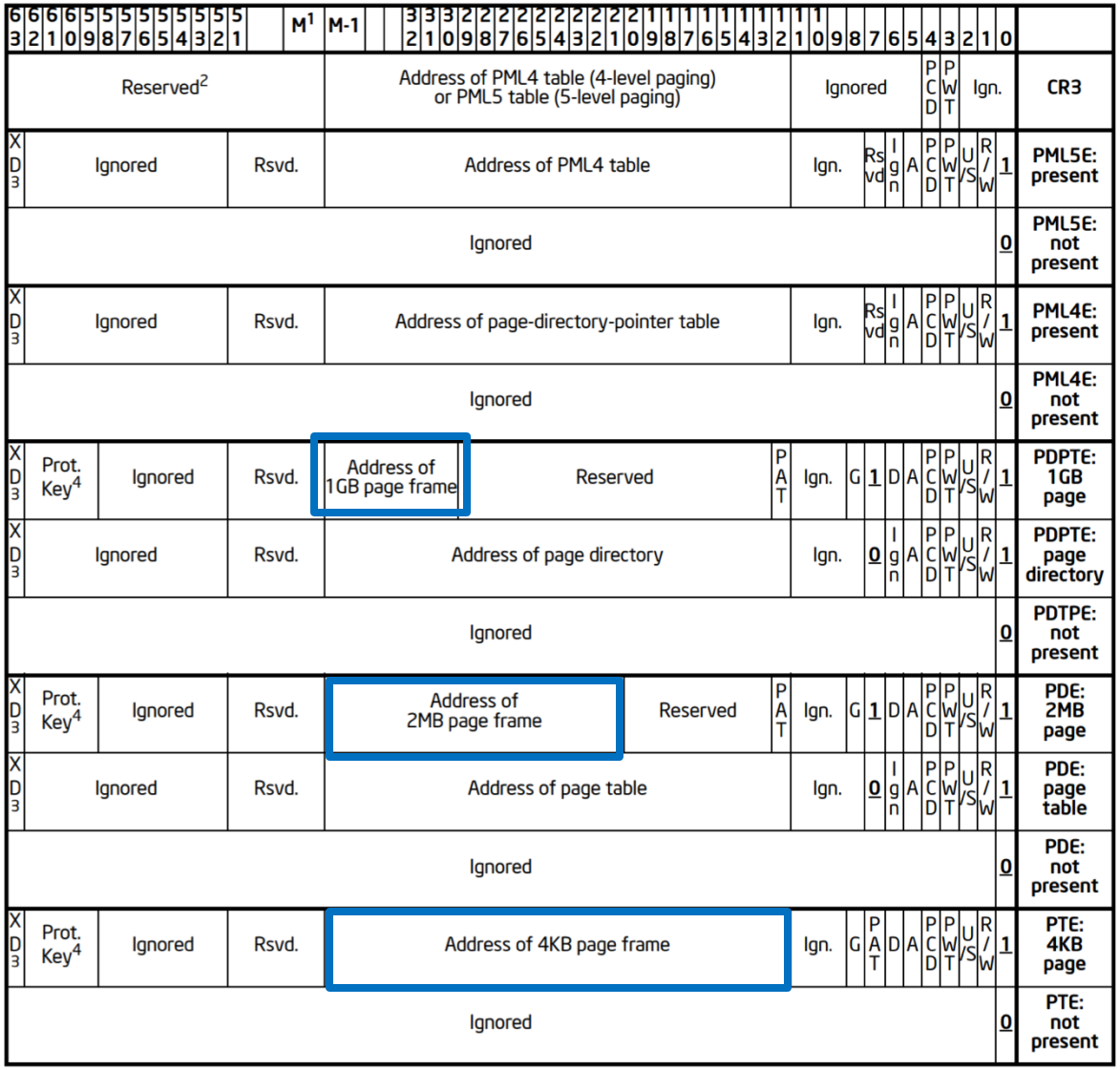
Bit Position(s)	Contents
0 (P)	Present; must be 1 to map a 4-KByte page
1 (R/W)	Read/write; if 0, writes may not be allowed to the 4-KByte page referenced by this entry (depends on CPL and CR0.WP; see Section 4.6)
2 (U/S)	User/supervisor; if 0, accesses with CPL=3 are not allowed to the 4-KByte page referenced by this entry (see Section 4.6)
3 (PWT)	Page-level write-through; indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9)
4 (PCD)	Page-level cache disable; indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9)
5 (A)	Accessed; indicates whether software has accessed the 4-KByte page referenced by this entry (see Section 4.8)
6 (D)	Dirty; indicates whether software has written to the 4-KByte page referenced by this entry (see Section 4.8)
7 (PAT)	If the PAT is supported, indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9.2); otherwise, reserved (must be 0)[1]
8 (G)	Global; if CR4.PGE = 1, determines whether the translation is global (see Section 4.10); ignored otherwise
11:9	Ignored
31:12	Physical address of the 4-KByte page referenced by this entry

x86 Page Directory Entry (PDE)

Table 4-5. Format of a 32-Bit Page-Directory Entry that References a Page Table

Bit Position(s)	Contents
0 (P)	Present; must be 1 to reference a page table
1 (R/W)	Read/write; if 0, writes may not be allowed to the 4-MByte region controlled by this entry (depends on CPL and CR0.WP; see Section 4.6)
2 (U/S)	User/supervisor; if 0, accesses with CPL=3 are not allowed to the 4-MByte region controlled by this entry (see Section 4.6)
3 (PWT)	Page-level write-through; indirectly determines the memory type used to access the page table referenced by this entry (see Section 4.9)
4 (PCD)	Page-level cache disable; indirectly determines the memory type used to access the page table referenced by this entry (see Section 4.9)
5 (A)	Accessed; indicates whether this entry has been used for linear-address translation (see Section 4.8)

X86-64 Page Table Entry Structure



63	62–M[1]	M-1–32	31–12	11–9	8	7	6	5	4	3	2	1	0	
Reserved[2]			Address of PML4 table (4-level paging) or PML5 table (5-level paging)	Ignored					PCD	PWT	Ign.			CR3
XD	Ignored		Rsvd.	Address of PML4 table	Ign.		Rsvd	Ign	A	PCD	PWT	U/S	R/W	1	PML5E: present
Ignored														0	PML5E: not present
XD	Ignored		Rsvd.	Address of page-directory-pointer table	Ign.		Rsvd	Ign	A	PCD	PWT	U/S	R/W	1	PML4E: present
Ignored														0	PML4E: not present
XD	Prot. Key[4]	Ignored	Rsvd.	Address of 1GB page frame	Reserved	PAT	Ign.	G	1	D	A	PCD	PWT	U/S	R/W	1	PDPTE: 1GB page
XD	Ignored		Rsvd.	Address of page directory	Ign.		0	Ign	A	PCD	PWT	U/S	R/W	1	PDPTE: page directory
Ignored														0	PDTPE: not present
XD	Prot. Key[4]	Ignored	Rsvd.	Address of 2MB page frame	Reserved	PAT	Ign.	G	1	D	A	PCD	PWT	U/S	R/W	1	PDE: 2MB page
XD	Ignored		Rsvd.	Address of page table	Ign.		0	Ign	A	PCD	PWT	U/S	R/W	1	PDE: page table
Ignored														0	PDE: not present
XD	Prot. Key[4]	Ignored	Rsvd.	Address of 4KB page frame	Ign.		G	PAT	D	A	PCD	PWT	U/S	R/W	1	PTE: 4KB page
Ignored														0	PTE: not present

Figure 4-11. Formats of CR3 and Paging-Structure Entries with 4-Level Paging and 5-Level Paging

X86-64 Page Table: Accessing 4KB pages

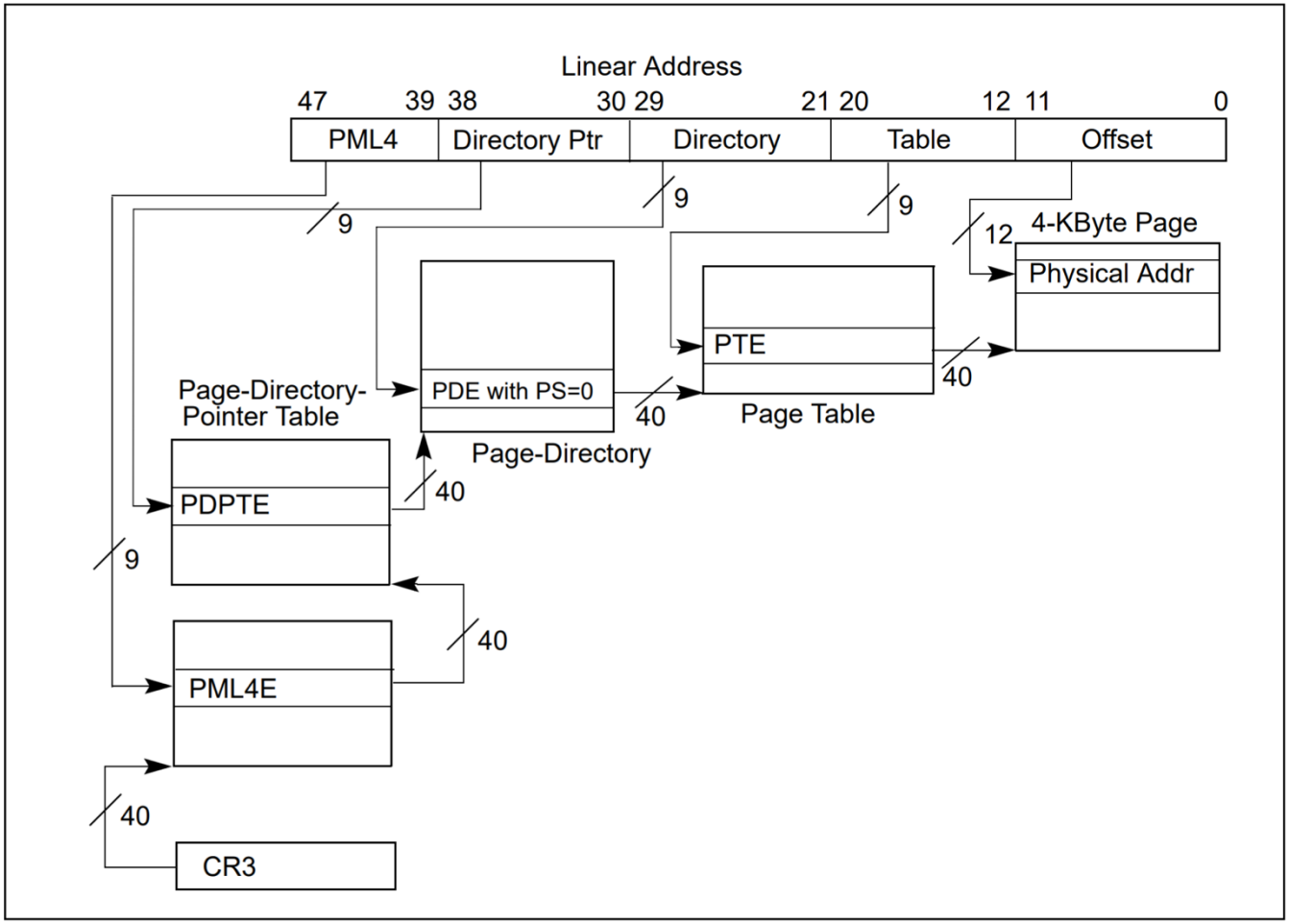


Figure 4-8. Linear-Address Translation to a 4-KByte Page using 4-Level Paging

X86-64 Page Table: Accessing 2MB pages

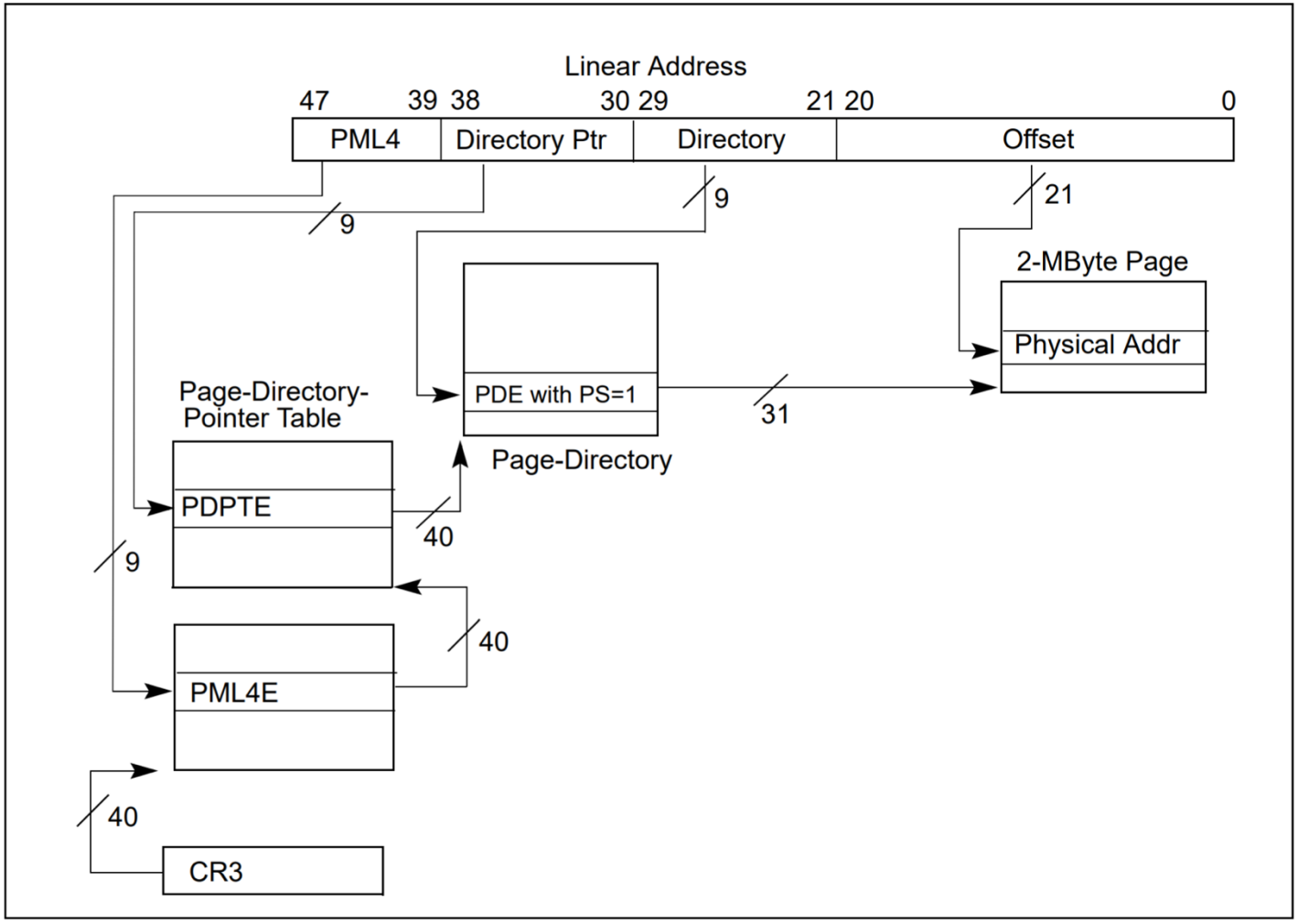


Figure 4-9. Linear-Address Translation to a 2-MByte Page using 4-Level Paging

X86-64 Page Table: Accessing 1GB pages

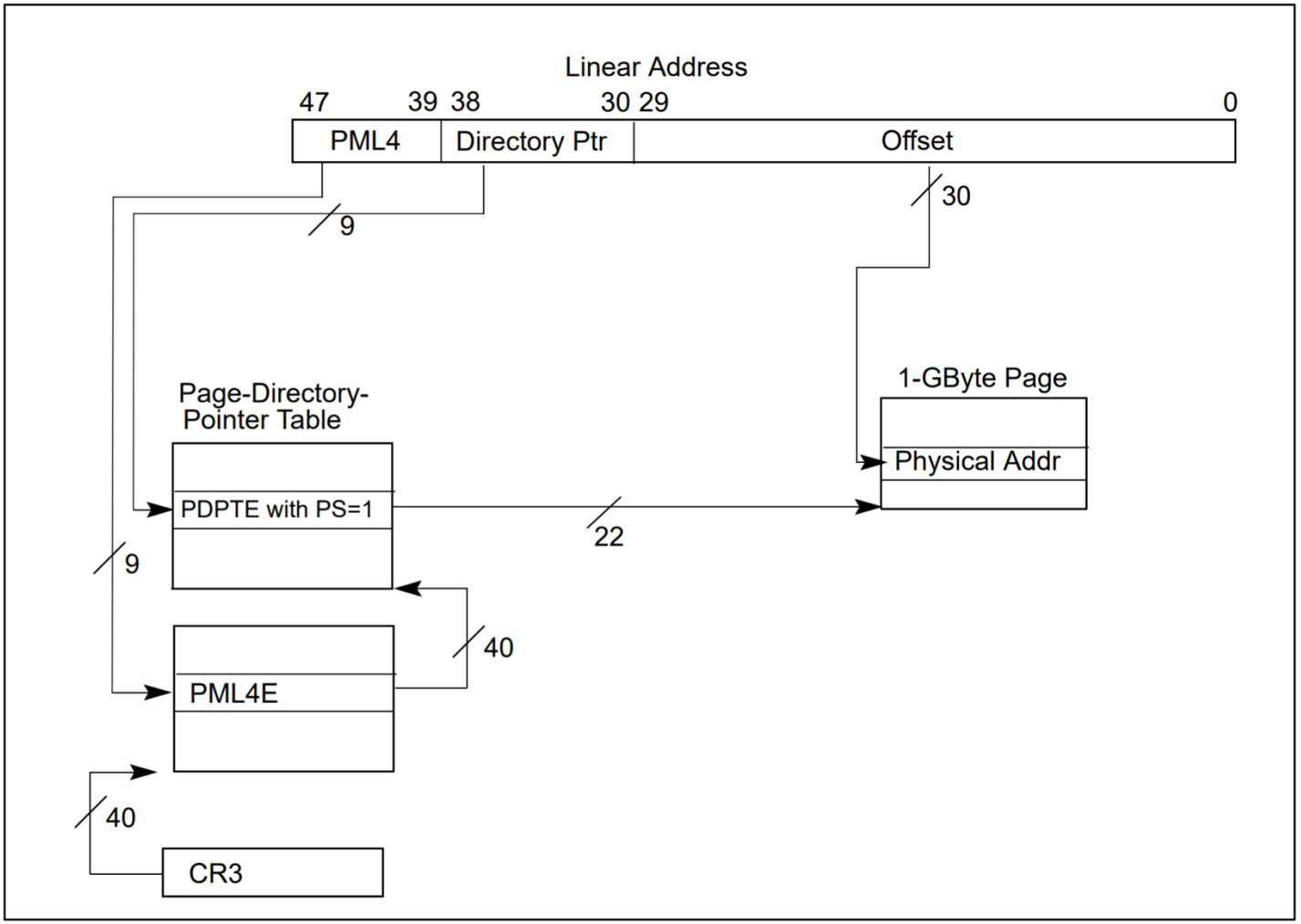


Figure 4-10. Linear-Address Translation to a 1-GByte Page using 4-Level Paging

Example: Transparent Huge Pages

Advantages

– The size of the page tables decreases, as does the number of page faults

– A single TLB entry maps a much larger amount of virtual memory in turn reducing the number of TLB misses.

– The TLB miss (page table walk) latency will significantly reduce

Disadvantages

– The amount of wasted memory will increase as a result of internal fragmentation.

– Larger pages take longer to transfer from secondary storage, increasing page fault latency (while decreasing page fault counts).

– The time required to simply clear very large pages can create significant kernel latencies.